

# Traité sur les systèmes multiagents en essaim

## La coordination par le milieu : ce qu'un essaim d'agents logiciels gagne à ne pas s'accorder, et ce qu'il le paie

André-Guy Bruneau, M.Sc. IT · agbruneau@gmail.com

2 septembre 2026 — quatrième édition, numérotation unifiée

**Résumé** Dès que le nombre d'agents logiciels autonomes dépasse quelques dizaines et que les défaillances partielles deviennent l'état normal, le coût du consensus explicite — en messages, en latence de queue, en couplage temporel — croît plus vite que la valeur qu'il procure. L'ouvrage soutient que l'architecture gagnante déplace alors la coordination vers un substrat événementiel partagé, durable et ordonné localement, où les agents déposent et lisent des traces plutôt que de négocier des décisions : une transposition de la stigmergie de la robotique en essaim. Il refuse d'en escamoter la contrepartie — la sûreté globale troquée contre la vivacité, le comportement non reproductible à l'échelle de l'exécution, la charge de preuve reportée sur la traçabilité et le point de reprise — et soutient qu'il existe entre les deux régimes une frontière identifiable, qu'il entreprend de tracer. Huit chapitres la parcourent : fondements et transposition du modèle d'essaim, passage à l'échelle et saturation du milieu, modélisation formelle et vérification, comportements collectifs et auto-organisation, décision collective et allocation, mise en œuvre et exploitation, cas d'étude, et — c'est l'apport de la deuxième édition — le régime propre aux essaims dont les agents reposent sur des modèles de langage. Chaque mécanisme y porte son modèle de panne, son hypothèse de synchronisme, son coût en messages et en tours, et la condition sous laquelle il cesse de valoir ; chaque transposition depuis la robotique y nomme ce qu'elle conserve et ce qu'elle casse. La campagne de mesures publiée le 13 août 2026 par l'équipe Frontier Red Team d'Anthropic oblige à ajouter un second axe à la frontière : le premier oppose la décision révocable à l'invariant qui doit tenir à tout instant, le second oppose la population décorrélée à la population conforme — et un milieu qui rend la coordination bon marché rend du même geste la conformité, la collusion et la tromperie bon marché.

**Mots-clés** — systèmes multiagents ; essaim ; stigmergie ; coordination indirecte ; journal d'événements ; consensus ; cohérence à terme ; auto-organisation ; allocation de tâches ; passage à l'échelle ; observabilité des systèmes non déterministes ; agents de langage ; conformité ; corrélation comportementale ; vigilance épistémique.

## Table des matières

|                                                                                                                                                       |     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Introduction .....                                                                                                                                    | 3   |
| 1. Introduction aux essais d'agents logiciels .....                                                                                                   | 5   |
| 1.1 Fondements et principes généraux .....                                                                                                            | 5   |
| 1.2 Paradigmes d'émergence .....                                                                                                                      | 12  |
| 1.3 Topologie des réseaux d'agents .....                                                                                                              | 17  |
| 2. Scalabilité et passage à l'échelle .....                                                                                                           | 23  |
| 2.1 Décentralisation stricte : élimination des points de défaillance uniques (SPOF) et découplage spatial et temporel .....                           | 23  |
| 2.2 Dynamique des populations d'agents : élasticité, cycle de vie, création et destruction d'agents à très haute échelle .....                        | 30  |
| 2.3 Gestion de la charge et congestion : contention des messages, routage distribué et gestion des goulots d'étranglement logiques .....              | 35  |
| 3. Modélisation formelle et méthodes de conception .....                                                                                              | 40  |
| 3.1 Cadres mathématiques : systèmes dynamiques, processus stochastiques et théorie des graphes appliqués aux systèmes d'agents .....                  | 40  |
| 3.2 Spécification formelle : vérification de propriétés globales (convergence, stabilité, absence de blocage) à partir de règles individuelles .....  | 45  |
| 3.3 Outillage et simulation : environnements de test, émulation d'essais et méthodes de validation avant déploiement .....                            | 50  |
| 4. Comportements collectifs et auto-organisation .....                                                                                                | 56  |
| 4.1 Agrégation et structuration logique .....                                                                                                         | 56  |
| 4.2 Coordination distribuée : synchronisation d'états, consensus logique et propagation d'information en réseau non structuré .....                   | 61  |
| 4.3 Adaptation environnementale .....                                                                                                                 | 67  |
| 5. Prise de décision collective et allocation de tâches .....                                                                                         | 73  |
| 5.1 Mécanismes de consensus .....                                                                                                                     | 73  |
| 5.2 Allocation dynamique de tâches .....                                                                                                              | 77  |
| 5.3 Gouvernance et comportement émergent .....                                                                                                        | 82  |
| 6. Architectures de mise en œuvre et ingénierie avancée .....                                                                                         | 87  |
| 6.1 Ingénierie des plateformes : intégration avec les courtiers de messages (Apache Kafka), registres d'états distribués et conteneurisation .....    | 87  |
| 6.2 Observabilité et suivi : traçabilité distribuée des décisions émergentes, métriques d'essaim et débogage de systèmes non déterministes .....      | 91  |
| 6.3 Interaction humain-essaim : interfaces de contrôle macroscopique, injection de directives globales et gouvernance opérationnelle (AgentOps) ..... | 96  |
| 7. Cas d'étude et applications pratiques .....                                                                                                        | 100 |
| 7.1 Traitement de flux et analytique distribuée : résolution coopérative de requêtes complexes sur des données en temps réel .....                    | 100 |
| 7.2 Sécurité et détection de menaces : surveillance distribuée, détection collective d'anomalies et réponse coordonnée aux incidents .....            | 105 |
| 7.3 Optimisation de ressources TI : auto-healing, rééquilibrage de charges applicatives et gestion d'infrastructures hybrides .....                   | 112 |
| 8. Essais d'agents de langage : ce que la mesure ajoute au modèle .....                                                                               | 117 |
| 8.1 Le régime mesuré : couverture, conformité, corrélation .....                                                                                      | 117 |
| 8.2 L'épistémique du milieu : la trace comme témoignage .....                                                                                         | 121 |
| 8.3 Buts incompatibles : escalade, force, trêve, et ce que le milieu doit porter .....                                                                | 124 |
| Conclusion .....                                                                                                                                      | 128 |
| Références .....                                                                                                                                      | 131 |

## Introduction

Un essaim d'agents logiciels n'est pas un système multi-agents devenu grand, et la différence porte sur la perception plutôt que sur la taille. Un agent est un processus autonome doté d'un état privé, d'une boucle perception-décision-action et de la capacité de refuser une tâche ; la population est un essaim lorsque la perception de chacun se factorise par un voisinage de cardinal borné ou par un intervalle borné du milieu partagé, lorsque la règle de décision est la même pour tous à un vecteur de paramètres près, et lorsque le comportement utile est une propriété de la population qui n'est fonction d'aucun état individuel. Un composant dont la perception prend la configuration entière en argument n'en est pas membre : c'est un coordonnateur, et sa présence ramène le système dans le régime de l'accord.

Ce régime a un prix, et ce prix est le point de départ de l'ouvrage. En système asynchrone, un seul processus fautif par simple arrêt suffit à rendre impossible tout protocole de consensus binaire garantissant la terminaison [1] ; sous partition réseau, aucune implantation d'un objet partagé n'est simultanément atomique et disponible au sens formel des deux termes, résultat paru dans un bulletin sans arbitrage [2] ; et la capacité d'un système dont les unités entretiennent une vue commune n'a pas un rendement décroissant mais un maximum, au-delà duquel ajouter des unités en retire [3]. Ces trois résultats ne se contournent pas. Ils délimitent ce qu'un architecte peut promettre, et ils rendent inévitable la question que l'ouvrage pose : que reste-t-il à coordonner lorsqu'on renonce à décider ?

La réponse que l'ouvrage soutient est ancienne et vient de l'observation animale. Grassé forge en 1959 la théorie de la stigmergie pour rendre compte de la reconstruction du nid chez les termites : la trace laissée par une action dans un milieu persistant stimule les actions ultérieures, sans planification, sans contrôle, sans communication directe, sans participation simultanée et sans conscience mutuelle des agents [4]. Chacune retire une hypothèse dont un protocole distribué classique a besoin. Sa version numérique substitue au milieu physique un journal — séquence d'enregistrements ordonnée, en ajout seul, durable et fiable —, où la trace est un enregistrement, le voisinage l'intervalle entre le décalage validé d'un agent et la fin courante, et l'évaporation une politique de rétention. Le journal n'est pas un choix esthétique : l'ajout seul est la forme sous laquelle le système groupe les écritures logiques en écritures physiques larges [5], et il rend gratuit l'ordre total par partition dont tout l'ouvrage dépend. L'écart de trois ordres de grandeur qu'on cite d'ordinaire est une mesure sur disques mécaniques, rapportée avec son matériel au §1.3 ; l'argument tient encore si le facteur tombe à un.

La thèse est donc un déplacement, non une suppression. Le point partagé n'est pas détruit, il est transporté dans le milieu, qu'il faut désormais répliquer, exploiter et facturer. Ce que le déplacement achète se mesure : la propagation d'un changement à toute la population coûte une écriture et une lecture par agent au lieu d'un nombre quadratique de messages, et le producteur n'a pas de destinataire à connaître. Ce qu'il coûte se mesure aussi, et l'ouvrage refuse

de l’escamoter. La durée d’un tour n’est plus un aller-retour entre deux agents mais la latence de queue d’un chemin écriture-durabilité-lecture. La localité, gratuite dans un espace métrique, devient une clé de partitionnement. Et la reproductibilité, jamais attendue d’un essaim de robots, est attendue par défaut d’un système logiciel : l’essaim la retire, et reporte la charge de preuve sur la traçabilité et le point de reprise.

#### RÉGIME DE L'ACCORD

##### Les agents négocient une décision

Porter un changement à toute la population coûte un nombre quadratique de messages.

L'émetteur nomme le destinataire : il entretient une vue des membres vivants, qui est toujours une approximation et jamais un fait.

Un tour vaut un aller simple, et n'a de sens que sous hypothèse synchrone.

Un seul processus fautif par arrêt suffit à retirer la terminaison en asynchrone [1].

Ce qu'il tient : un invariant global, à tout instant.

#### RÉGIME DU MILIEU

##### Les agents déposent et lisent des traces

Une écriture et une lecture par agent ; le diamètre du graphe de coordination vaut 2 quel que soit le nombre d'agents.

Le producteur n'a pas de destinataire à connaître.

Un tour de journal vaut la latence de queue  $\ell_{99}$  du chemin écriture-durabilité-lecture.

La sérialisation, que la matière assurait sans protocole, devient un composant à payer.

Ce qu'il tient : une décision révocable, sous ordre partiel.

Le point partagé n'est pas détruit, il est transporté dans le milieu, qui devient le composant que chaque agent touche, qu'il faut répliquer, exploiter et facturer. La thèse est un déplacement, non une suppression — et le milieu est le candidat le plus sérieux au titre de point de défaillance unique que la décentralisation prétendait supprimer.

**Figure 0** — Le déplacement que l’ouvrage soutient : ce que la coordination par le milieu achète, et ce qu’elle coûte.

Le partage entre ce qui est décentralisable et ce qui ne l’est pas n’est pas affaire de goût : les programmes qui admettent une implantation distribuée cohérente et sans coordination sont exactement ceux qui s’expriment en logique monotone [6]. Un agent qui accumule peut se passer d’accord ; un agent qui compte exactement une fois ne le peut pas, et la coordination n’est alors pas évitable, seulement déplaçable. C’est la forme précise de la frontière que l’ouvrage entreprend de tracer : le substrat événementiel est supérieur là où la décision est révocable, où l’ordre partiel suffit et où l’échelle est réelle ; il est inférieur, et le texte le dira chapitre après chapitre, partout où un invariant global doit tenir à tout instant.

Un dernier fait borde l’ensemble et vient lui aussi de la biologie : des groupes de robots à contrôle décentralisé inspiré des fourmis fourragent plus efficacement qu’un robot seul, mais le gain d’efficacité diminue dans les grands groupes, par interférence [7]. L’essaim ne monte pas en charge linéairement, et cela était su avant que la question ne se pose au logiciel ; la forme de la courbe se transpose, sa cause non. Cette dissociation entre ce qu’une analogie conserve et ce qu’elle casse est la discipline du livre entier : chaque chapitre expose ses mécanismes avec leur modèle de panne, leur hypothèse de synchronisme, leur coût, leur condition d’arrêt et leur mode de défaillance, puis nomme ce que la transposition conserve, déplace ou détruit. Toute mesure y porte son unité, son percentile

lorsqu’il s’agit d’une latence, et sa source ; un nombre sans provenance y est traité comme une faute de rédaction, et une grandeur que l’ouvrage propose lui-même n’est pas dispensée de la règle.

La deuxième édition a ajouté à l’ouvrage ce qui lui manquait le plus et qui n’existait pas lorsque la première a été close : des mesures. Les sept chapitres avaient été bâtis sur deux littératures — la robotique en essaim et les systèmes répartis — et sur aucune donnée portant sur des populations d’agents dont la fonction de décision est un modèle de langage. Une campagne publiée le 13 août 2026 en fournit, sur des populations de la taille dont traite ce livre [119]. Elle est citée avec son statut : rapport d’un laboratoire sur ses propres modèles, sans comité de lecture, aux protocoles décrits en prose. Elle établit néanmoins trois choses qu’aucun raisonnement de ce livre n’avait produites, et qui obligent à le réviser.

La première est que les agents d’une même population ne se décorrèlent pas : deux agents portant le même modèle, la même invite et le même contexte ne diffèrent par rien, et dix-huit sur trente ouvrent une branche de dépôt au même nom [119]. Sept énoncés de cet ouvrage supposaient l’indépendance sans l’écrire, et le ch. 8 en dresse l’inventaire. La deuxième est que la trace cesse d’être un résidu pour devenir un témoignage : ce qu’un agent dépose est ce qu’il déclare avoir fait, et l’on a observé du code malveillant déposé sous le nom d’un autre [119]. La stigmergie y perd sa prémisse fondatrice — une phéromone ne ment pas —, et le milieu doit désormais fabriquer ce que la matière donnait. La troisième est que l’adversité n’a pas besoin d’être importée : trois agents porteurs de mandats contradictoires escaladent jusqu’au sabotage sans qu’aucun n’ait été programmé pour nuire, et l’aptitude à conclure une trêve ne croît pas avec la capacité générale [119]. Le modèle de panne qui porte les sept premiers chapitres ne couvre pas ce régime, et l’ouvrage le dit à l’endroit où il l’emploie plutôt que de le refondre après coup.

Le mouvement d’ensemble est double. La thèse en sort renforcée d’un côté : cette campagne conclut que la coordination n’émerge ni d’une intelligence accrue ni d’un alignement individuel, mais des environnements [119] — l’énoncé même de ce livre dans un autre vocabulaire. Elle est contestée de l’autre : un milieu qui rend la coordination bon marché rend du même geste bon marché ce que le concepteur ne veut pas, et des agents privés de tout canal direct s’alignent encore, au sou près, par le seul tableau public de leurs annonces [119]. La frontière tracée ici gagne donc un second axe, et le huitième chapitre est celui qui le trace.

## 1. Introduction aux essais d’agents logiciels

### 1.1 Fondements et principes généraux

Un essaim (*swarm*) n’est pas un grand système multi-agents, et la différence s’écrit. Un agent est un quintuplet  $A_i = (S_i, s_i^0, P_i, \delta_i, X_i)$  : un ensemble d’états privés, un état initial, un ensemble de perceptions, une fonction de transition  $\delta_i : S_i \times P_i \rightarrow S_i \times X_i$ , et un ensemble d’actions contenant le refus  $\perp$ . La

configuration de la population est  $C = (s_1, \dots, s_n, M)$ , où  $M$  est l'état du milieu (*medium*) partagé. Chaque agent perçoit par une fonction  $\pi_i : C \rightarrow P_i$ , et c'est cette fonction qui porte la définition : la population est un essaim si et seulement si  $\pi_i$  se factorise par un voisinage  $N_i(C)$  de cardinal borné par  $d$ , avec  $d \ll n$ , ou par un intervalle borné du milieu, pour tout  $i$  et toute configuration atteignable. Un composant dont la perception prend  $C$  entier en argument n'est pas un membre d'un essaim mais un coordonnateur, et sa présence ramène le système dans le régime de l'accord, dont le prix est établi au ch. 4. La seconde condition est l'homogénéité :  $\delta_i = \delta$  pour tout  $i$ , à un vecteur de paramètres près. La troisième est fonctionnelle : le comportement utile est une fonctionnelle  $\Phi(C)$  de la configuration qui n'est fonction d'aucun  $s_i$  pris seul. Trois services hétérogènes qui s'appellent mutuellement ne satisfont aucune des trois.

Cette définition vient de l'observation animale. Grassé forge en 1959 la théorie de la stigmergie pour rendre compte de la reconstruction du nid chez *Bellicositermes natalensis* et *Cubitermes* sp. [4] ; Theraulaz et Bonabeau en font une classe de mécanismes qui médie les interactions entre animaux et qui lève un paradoxe apparent — les individus travaillent comme s'ils étaient seuls alors que leurs activités collectives paraissent coordonnées [8]. Heylighen en tire la formulation retenue ici : une coordination indirecte où la trace laissée par une action dans un milieu stimule les actions ultérieures, sans planification, sans contrôle, sans communication directe, sans participation simultanée et sans conscience mutuelle des agents [9]. Ces cinq négations retirent chacune une hypothèse dont un protocole distribué classique a besoin, et c'est ce qui rend la transposition intéressante.

La branche calculatoire naît en synthèse d'images. Reynolds construit les boids en 1986 sur trois règles de pilotage à voisinage local et non global — séparation, s'écarter des voisins serrés, « *steer to avoid crowding local flockmates* » ; alignement, adopter leur cap moyen, « *...the average heading...* » ; cohésion, viser leur position moyenne, « *...the average position...* » [10]. Vicsek et coll. en donnent en 1995 la version minimale et mesurable : chaque particule se déplace à vitesse constante en adoptant la direction moyenne de ses voisines, plus un bruit, et les simulations révèlent une transition de phase cinétique par brisure spontanée de symétrie rotationnelle [11]. Le versant optimisation suit la même trajectoire : l'*ant system* de 1996 revendique rétroaction positive, calcul distribué et heuristique gloutonne constructive [12], et ce qui en fait un essaim plutôt qu'un calcul parallèle est le caractère indirect de la communication, par dépôt de phéromone sur les arêtes du graphe pendant la construction de la solution [13] ; la théorie a suivi les applications et non l'inverse, ce que la synthèse de 2005 dit sans détour tout en recensant les résultats de convergence disponibles [14].

La robotique en essaim installe le mot dans l'ingénierie : conception, construction et déploiement de grands groupes de robots coordonnés de façon coopérative, dont les comportements collectifs émergent de règles d'interaction locales simples [15]. Trois démonstrations bornent ce qui a été effectivement montré : auto-assemblage programmable de formes bidimensionnelles par un

essaim de mille robots, en interactions locales seulement [16] ; construction collective inspirée des termites, où la coordination passe par l’environnement et non par la communication directe, sur trois robots grimpeurs [17] ; traction coopérative de bâtonnets sur des équipes allant jusqu’à 600 robots, avec des modèles abstraits dont les prédictions restent quantitativement exactes pour un coût de calcul très inférieur à la simulation incarnée [18]. Le résultat négatif se cite avec les autres : chez des robots fourrageurs à contrôle décentralisé inspiré des fourmis, le gain d’efficacité diminue dans les grands groupes, probablement par interférence de fourragement [7]. L’essaim ne monte pas en charge linéairement, et cela était su avant que la question ne se pose au logiciel.

Passer de ce corpus à des agents logiciels exige de fixer trois conventions que la robotique n’avait pas besoin d’écrire, parce que la physique les fournissait. La première est le modèle de panne. L’ouvrage travaille sous crash-arrêt et omission, noté P dans tout le chapitre : un agent s’arrête définitivement et ne reprend pas sous la même identité, ou un message émis n’est jamais livré, sans qu’aucun agent n’émette de valeur arbitraire ni qu’aucun message ne soit corrompu de façon indétectable. La tolérance aux fautes byzantines (*Byzantine fault tolerance*) est hors modèle par défaut et reprise au ch. 7 ; le traitement formel de P, y compris de la partition réseau (*network partition*), appartient au ch. 4. La deuxième est l’hypothèse de synchronisme : le modèle synchrone borne par une constante connue le délai de transmission et la dérive d’horloge, le modèle asynchrone ne borne ni l’un ni l’autre, le modèle partiellement synchrone suppose une borne  $\Delta$  sur le délai, inconnue ou valable seulement après un instant de stabilisation qu’aucun agent n’observe. Lamport en donne la définition opératoire retenue : un système est distribué si le délai de transmission des messages n’est pas négligeable devant le temps séparant deux événements d’un même processus, et « a précédé » y est un ordre partiel — il est parfois impossible de dire lequel de deux événements a eu lieu en premier [19]. La règle de méthode qui en découle vaut pour tout le chapitre : un énoncé de sûreté (*safety*) se démontre en asynchrone, un énoncé de vivacité (*liveness*) ou une borne de temps exige au minimum le partiellement synchrone, et confondre les deux registres est une faute et non une approximation. La troisième convention est celle du comptage : seuls les messages point à point sont comptés, une diffusion vers N destinataires compte pour N messages, et un tour est un aller-retour bloquant — un point du programme où un agent ne peut progresser avant le message qu’il attend d’un pair. Une requête et sa réponse coûtent donc 2 messages et 1 tour ; un envoi dont l’émetteur n’attend rien lui coûte 0 tour ; et un tour d’algorithme à tours en vaut 1, chaque agent y attendant une fois les messages de ses voisins avant de calculer. Cette convention vaut pour l’ouvrage entier et il n’y en a pas d’autre : un chapitre qui la rappelle ne la redéfinit pas. Un tour de journal (*log*) est autre chose et coûte davantage : c’est le chemin écriture, accusé de durabilité, lecture, dont la durée est la latence de queue (*tail latency*)  $\ell_q$ , de ce chemin. Un tableau de coûts qui ne dit pas lequel des deux tours il compte ne dit rien.

Une quatrième convention s'impose depuis la deuxième édition, et elle n'était pas nécessaire tant que la fonction de transition  $\delta$  était un programme écrit à la main. Un agent dont  $\delta$  est un modèle de langage satisfait la condition d'homogénéité du paragraphe précédent d'une manière que la robotique n'a jamais rencontrée. Deux robots portant le même contrôleur diffèrent par leur position, leur usure et le bruit de leurs capteurs, et cette différence est fournie par le monde sans que personne ait à la produire ; deux agents portant le même modèle, la même invite et le même contexte ne diffèrent par rien, et leurs décisions coïncident jusque dans le choix d'un nom. Le fait est mesuré : dix-huit agents sur trente ouvrent une branche de dépôt portant exactement le même nom, et plus de la moitié d'une population libre de construire ce qu'elle veut construit un lanceur de rayons ou un compilateur auto-hébergé [119]. La convention qui en découle vaut pour tout l'ouvrage : le vecteur de paramètres qui distingue  $\delta_i$  de  $\delta$  n'est pas un raffinement de notation, c'est la seule source de variance de la population, et un essaim qui ne l'instancie pas n'a pas  $n$  agents mais un agent exécuté  $n$  fois. La seconde conséquence touche le modèle de panne lui-même.  $P$  suppose qu'un agent s'arrête ou se taise ; il ne prévoit pas qu'il agisse contre un autre. Or trois agents recevant des directives incompatibles produisent, sur tous les modèles éprouvés, une escalade où chacun sabote les autres — révocation de comptes système, scripts qui cherchent et tuent en boucle les processus concurrents, code malveillant déguisé en contribution d'un autre agent [119]. L'effet est byzantin, l'origine ne l'est pas, puisque aucun agent n'a été programmé pour nuire. L'ouvrage conserve  $P$  comme modèle de travail, parce que c'est sous  $P$  que les mécanismes des sept premiers chapitres se démontrent, et traite au ch. 8 le régime où l'adversité est engendrée par la population elle-même.

Le mécanisme fondateur de l'essaim se laisse écrire en entier, et il faut le faire une fois pour que le reste du chapitre ait un point de comparaison. Modèle de panne :  $P$ . Hypothèse de synchronisme : synchrone — tours globaux, délai borné et connu, dérive d'horloge nulle. C'est l'hypothèse la plus forte du chapitre, et c'est celle sous laquelle le résultat de convergence a été démontré ; l'affaiblir invalide la démonstration et pas seulement sa vitesse. Trois hypothèses supplémentaires sont nécessaires et aucune n'est cosmétique : le bruit est nul, alors que c'est précisément le bruit qui porte la transition de phase de [11], et l'analyse théorique ignore explicitement les signaux de bruit du modèle de Vicsek [20] ; l'ensemble des graphes de voisinage possibles est fini ; chaque agent connaît au tour courant la liste de ses voisins, laquelle est géométrique dans le modèle d'origine — les voisins de l'agent  $i$  au temps  $t$  sont les agents situés dans ou sur un cercle de rayon  $r$  prescrit centré sur la position courante de  $i$  [20].

Algorithme 1.1 – alignement par règle de plus proche voisin, transposé au logiciel

Modèle de panne :  $P$  (crash-arrêt, omission).

Hypothèse de synchronisme : synchrone.

Hypothèses : bruit nul ; famille finie de graphes de voisinage ;

voisinage  $N_i(t)$  connu de  $i$  au tour  $t$ , de cardinal  $n_i(t) \leq d$ .

État de l'agent  $i$  : cap  $\theta_i(t) \in [0, 2\pi)$

Tour  $t$ , exécuté en parallèle par tout agent  $i$  non défaillant :

1. émettre  $\theta_i(t)$  à chacun des  $n_i(t)$  voisins  $n_i(t)$  messages
2. recevoir les  $\theta_j(t)$  pour  $j \in N_i(t)$  ; un message omis est traité comme une arête absente du graphe au tour  $t$ , sans reprise
3.  $\theta_i(t+1) \leftarrow ( \theta_i(t) + \sum_{j \in N_i(t)} \theta_j(t) ) / (1 + n_i(t))$
4. si condition d'arrêt satisfaite, terminer ; sinon  $t \leftarrow t+1$

Condition d'arrêt : aucune n'est fournie par le modèle. À écrire dans le système :  $\max_{\{i,j\}} |\theta_i - \theta_j| \leq \varepsilon$  maintenu sur  $k$  tours,  $\varepsilon$  et  $k$  donnés. Ce test est global, donc il ne s'évalue pas dans le modèle : le mesurer coûte une agrégation sur toute la population, que l'algorithme 1.1 ne sait pas faire.

Le coût se compte sur les deux axes, et l'un des deux ne s'écrit pas. Par tour, l'agent  $i$  émet  $n_i(t)$  messages ; la population émet donc  $\sum_i n_i(t) = 2 \cdot |E(t)|$  messages par tour, soit au plus  $n \cdot d$  et au moins  $n-1$  tant que le graphe est connexe, ce qui donne  $\Theta(n \cdot d \cdot R)$  sur  $R$  tours. Le nombre de tours  $R$ , lui, n'est borné par rien dans le modèle où l'algorithme est posé : les deux théorèmes disponibles sont asymptotiques. Le premier établit que si le graphe de voisinage est connexe à chaque instant, le vecteur des caps converge vers  $\theta_{ss} \cdot 1$ , où  $\theta_{ss}$  ne dépend que de  $\theta(0)$  et de la suite de graphes rencontrée. Le second affaiblit l'hypothèse à l'existence d'une suite infinie d'intervalles de temps contigus, non vides et bornés, partant de  $t_0 = 0$ , telle que sur chacun d'eux les  $n$  agents soient « liés entre eux » — c'est-à-dire que la collection de graphes rencontrée sur l'intervalle soit conjointement connexe, l'union de ses membres étant un graphe connexe [20]. Aucun des deux ne borne  $R$  : le compte de tours s'écrit  $1, \dots, \infty$ , et un texte qui annonce « l'essaim converge en  $O(\log n)$  tours » sans produire une hypothèse supplémentaire invente son chiffre. La borne uniforme sur la longueur des intervalles n'est pas un confort de rédaction : la preuve repose sur un résultat de Wolfowitz portant sur les produits infinis de matrices ergodiques, dont la finitude de l'ensemble de matrices est une condition cruciale, et c'est cette finitude qui impose des intervalles contigus et bornés [20]. Les auteurs laissent explicitement ouvert le cas des intervalles non contigus ; une prépublication ultérieure, jamais publiée en revue, affirme la convergence sous une condition strictement plus faible — que l'union de tous les graphes à partir d'un instant quelconque soit connexe [21] —, et le statut des deux énoncés n'est pas le même.

Le mode de défaillance tient à l'hypothèse et non à l'implantation. Une omission retire une arête pour un tour ; elle est absorbée sans dégât, parce que l'hypothèse de convergence porte sur l'union des graphes le long d'un intervalle et non sur la connexité à chaque instant. C'est toute la robustesse du mécanisme, et elle se démontre au lieu de se revendiquer. Un crash-arrêt retire un sommet définitivement, et rien n'est promis dès que ce retrait fait tomber la connexité conjointe sur un intervalle de la suite : la population se scinde en composantes qui convergent chacune vers sa propre valeur limite, sans qu'aucun agent ne puisse le détecter, puisque son état ne contient que son cap et ses voisins

courants. Voilà la condition d'échec : l'algorithme 1.1 échoue silencieusement, et le silence est structurel. Il faut ajouter que l'alignement obtenu n'est pas un consensus au sens de l'ouvrage et ne sera jamais nommé ainsi :  $\theta_{ss}$  n'est proposée par personne, ne satisfait aucune condition de validité, dépend de la suite de graphes autant que des états initiaux, et la terminaison n'est pas exigée puisque la convergence est asymptotique. C'est le premier exemple du déplacement que l'ouvrage soutient — un accord de fait sans protocole d'accord — et le premier exemple de son prix : à aucun instant on ne sait si l'accord est atteint.

La transposition de ce mécanisme au logiciel conserve l'étape 3 telle quelle et casse les étapes 1 et 2. En robotique, l'étape 1 est une diffusion locale dont l'énergie est à peu près constante quel que soit le nombre de récepteurs à portée, et  $N_i(t)$  est un fait du monde lu par un capteur : le degré d est gratuit. En logiciel, l'étape 1 coûte  $n_i(t)$  messages, et  $N_i(t)$  n'existe pas tant qu'on ne l'a pas fabriqué, faute de métrique entre deux agents ; l'entretien de cette vue est un mécanisme à part entière dont le coût appartient au ch. 4. Une règle de pilotage à la Reynolds transposée sans cette précaution ne produit pas un essaim mais une diffusion à  $n^2$  arêtes.

Une borne d'impossibilité gouverne l'espace entier, et elle est traitée ici parce qu'elle motive le déplacement que l'ouvrage propose, non parce qu'elle appartient à ce chapitre. Fischer, Lynch et Paterson établissent qu'en système asynchrone, aucun protocole de consensus binaire ne garantit la terminaison dès lors qu'un seul processus peut tomber en panne par arrêt — la borne est un, et non  $f > n/3$  ni  $f > n/2$  [1]. Les hypothèses se citent avec l'énoncé : pas d'horloges synchronisées, donc les algorithmes fondés sur des délais d'expiration sont exclus, et aucun moyen de détecter la mort d'un processus. Le résultat porte sur la terminaison et non sur la sûreté : un protocole d'accord reste correct sous FLP, il y perd la garantie de progrès. L'intuition tient en une phrase : un processus arbitrairement lent est indiscernable d'un processus mort. La conséquence pour un essaim est directe — un système asynchrone qui exige un accord global à chaque décision ne peut promettre la vivacité, et l'essaim répond non pas en résolvant l'impossibilité mais en refusant la question, puisqu'il coordonne sans décider. Le traitement complet du consensus appartient au ch. 4.

Ce qui vaut pour l'algorithme 1.1 vaut pour la transposition entière, et l'ouvrage ne tire aucun argument d'une analogie dont il n'a pas dit la partie qui rompt. Le tableau ci-dessous en fait le compte ; la dernière ligne repose sur l'avantage décisif des approximations de champ moyen, qui autorisent des garanties démontrables sur la dynamique collective là où les modèles microscopiques à agents discrets ne passent pas à l'échelle [22].

**Tableau 1** — Ce que la transposition de l'essaim robotique à l'essaim logiciel conserve et ce qu'elle casse.

| Dimension                              | Essaim robotique                                                                 | Essaim logiciel                                                                      | Effet de la transposition                             |
|----------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------|
| Arbitrage des écritures dans le milieu | La matière sérialise : deux robots ne déposent pas au même point au même instant | Aucun arbitre naturel ; l'ordre doit être construit et n'est total que par partition | Cassé — la sérialisation devient un composant à payer |

| Dimension                | Essaim robotique                                                                              | Essaim logiciel                                                                                           | Effet de la transposition                                                             |
|--------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Localité                 | Donnée par l'espace métrique ; le voisinage est un fait géométrique                           | Aucune métrique entre agents ; le voisinage est fabriqué par une clé de partitionnement                   | Cassé — la localité devient une décision de conception                                |
| Coût d'une émission      | Diffusion locale à énergie $\approx$ constante quel que soit le nombre de récepteurs à portée | Chaque destinataire coûte un message ; une diffusion vers $n$ agents compte $n$ messages                  | Cassé — le degré d devient une facture, pas un cadeau                                 |
| Décroissance de la trace | Évaporation chimique gratuite et exponentielle                                                | Rétention et compactage explicites, à la charge du milieu ou du lecteur                                   | Cassé — la décroissance devient un paramètre et un coût                               |
| Statut de la trace       | Résidu physique de l'action : la trace est l'action même et ne peut pas la démentir           | Enregistrement assertif : l'agent écrit ce qu'il déclare avoir fait, et le milieu n'évalue pas le contenu | Cassé — la trace devient un témoignage, et le milieu n'a pas de mécanisme de crédit   |
| Variance entre individus | Fournie par le monde : position, usure, bruit de capteur                                      | Nulle par défaut : même modèle, même invite, même contexte donnent la même décision [119]                 | Cassé — l'aléa d'exploration cesse d'être gratuit et devient un composant à fabriquer |
| Panne dominante          | Actionneur dégradé, batterie, robot immobilisé                                                | Crash-arrêt, omission, partition réseau ; adversité engendrée par la population elle-même (ch. 8)         | Déplacé — la partition réseau n'a pas d'équivalent bénin                              |
| Échelle démontrée        | Mille robots en auto-assemblage [16] ; 600 robots en manipulation [18]                        | $n$ non borné par la physique, borné par la contention sur le milieu (ch. 2)                              | Conservé en principe, déplacé en cause                                                |
| Reproductibilité         | Jamais attendue ; personne n'exige deux exécutions identiques                                 | Attendue par défaut, et l'essaim la retire                                                                | Cassé — la charge de preuve passe à la reconstruction causale après coup (ch. 6)      |
| Modèle macroscopique     | Champ moyen indépendant de la taille de l'essaim [22]                                         | Même outil, sous réserve que l'hypothèse de mélange tienne                                                | Conservé sous condition                                                               |

La ligne la plus lourde est la première. Dans un essaim physique, l'environnement est un registre dont la matière garantit la cohérence : deux dépôts concurrents au même endroit produisent une somme, jamais une divergence, et aucun robot n'a besoin de protocole pour cela. Le logiciel n'a pas cet arbitre : le milieu doit être construit, sa sémantique d'ordre choisie et son coût assumé, et c'est pourquoi la suite du chapitre porte sur le journal plutôt que sur une abstraction d'environnement générique.

Les deux lignes que la deuxième édition a ajoutées au tableau sont d'une autre nature, parce qu'elles ne portent pas sur le milieu mais sur les agents, et qu'elles n'étaient pas visibles tant qu'aucune population d'agents de langage n'avait été mesurée. La variance nulle retire à l'essaim la propriété dont toutes ses bornes probabilistes ont besoin, qui est le tirage indépendant. Le statut assertif de la trace lui retire ce que la stigmergie biologique n'avait jamais eu à garantir, puisqu'une piste de phéromone est le dépôt d'un insecte qui est

effectivement passé. Les deux ruptures sont établies au ch. 8 avec les mesures qui les portent ; elles sont nommées ici parce que la table de transposition serait fausse sans elles, et parce que la suite du chapitre repose sur des hypothèses qu’elles fragilisent.

## 1.2 Paradigmes d’émergence

L’auto-organisation (*self-organization*) et l’émergence sont deux notions distinctes que l’ouvrage refuse de confondre. Le concept d’auto-organisation vient de la physique et de la chimie, où il décrit comment des processus microscopiques engendrent des structures macroscopiques dans des systèmes loin de l’équilibre, et sa transposition au comportement animal porte une thèse forte : il n’est pas nécessaire de postuler une complexité individuelle pour expliquer les patrons spatiotemporels complexes observés à l’échelle de la colonie [23]. L’auto-organisation qualifie donc le processus de formation et de réparation de la structure de coordination, sans rôle central ni intervention externe. L’émergence qualifie un résultat observable, et la charte de l’ouvrage la réserve aux cas où l’on peut nommer la régularité et la mesurer. Cette exigence a une traduction technique précise : affirmer une émergence, c’est exhiber un paramètre d’ordre, c’est-à-dire une fonctionnelle  $\Phi(C)$  de la configuration, définie à l’échelle de la population, qui n’est fonction d’aucun état d’agent pris seul, et montrer que  $\Phi$  change qualitativement lorsqu’un paramètre de contrôle franchit un seuil. Le modèle de Vicsek en donne la forme canonique :  $\Phi$  est la norme de la vitesse moyenne normalisée, le paramètre de contrôle est l’amplitude du bruit  $\eta$ , et la transition est continue avec  $\Phi \propto (\eta_c - \eta)^\beta$ ,  $\beta \approx 0,45$  [11]. Une affirmation d’émergence sans paramètre d’ordre ni seuil n’est pas réfutable, et l’ouvrage l’écrit « comportement collectif ».

La stigmergie est le mécanisme qui produit ces régularités sans que rien ne les encode. Ses composantes sont l’action, l’agent, le milieu, la trace et la coordination, et le moteur est fait de boucles de rétroaction positives et négatives qui renforcent les développements fructueux et éliminent les inefficacités [9]. Sa version numérique consiste à substituer au milieu physique un journal — une séquence d’enregistrements ordonnée, en ajout seul, durable et fiable — organisé en sujets (*topic*) et en partitions. La correspondance est terme à terme et il faut la poser avant tout algorithme : la trace est un enregistrement ; le milieu est la partition ; le voisinage d’un agent dans le milieu est l’intervalle de la partition compris entre son décalage (*offset*) validé et la fin courante ; l’évaporation est une décroissance appliquée par le lecteur ou une politique de rétention appliquée par le courtier (*broker*). Ce qui survit à la substitution : l’indirection — un producteur (*producer*) n’a pas de destinataire —, le découplage temporel, et la persistance de la trace après la disparition de son auteur. Ce qui ne survit pas : la diffusion. Une phéromone diffuse dans l’espace et construit un gradient, ce qui permet à un insecte de décider par montée locale sans jamais lire le champ entier. Un journal n’a pas de gradient : lire « la trace près de moi » revient à lire une clé, et il n’existe aucune métrique entre deux clés à moins qu’on ne l’ait encodée. Un partitionnement par hachage détruit la

proximité qu'un partitionnement par intervalle préserverait au prix d'un point chaud. C'est la première rupture, et elle décide de la conception du sujet plus sûrement que n'importe quel choix de bibliothèque.

#### CE QUE LA SUBSTITUTION CONSERVE, ET CE QU'ELLE DÉTRUIT

|                    | Essaim biologique — Grassé, Heylighen [4][9]                                     | Essaim logiciel — journal partitionné                                                                             |
|--------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Trace</b>       | Le dépôt qu'une action laisse derrière elle.                                     | Un enregistrement, en ajout seul et durable.                                                                      |
| <b>Milieu</b>      | Le nid : un espace physique, durable et sans propriétaire.                       | La partition : ordre total à l'intérieur (M1), aucun entre partitions (M2).                                       |
| <b>Voisinage</b>   | Ce que l'insecte perçoit sous lui, borné par la portée du capteur.               | L'intervalle compris entre le décalage validé de l'agent et la fin courante.                                      |
| <b>Évaporation</b> | Une décroissance chimique, gratuite et exponentielle.                            | Une politique de rétention du courtier, ou un facteur $\gamma < 1$ appliqué par le lecteur.                       |
| <b>Indirection</b> | ✓ Aucun destinataire ; la trace survit à son auteur.                             | ✓ Conservée : découplage spatial, découplage temporel, persistance.                                               |
| <b>Gradient</b>    | La phéromone diffuse et construit un champ : l'insecte décide par montée locale. | ✗ Détruit : lire « la trace près de moi » revient à lire une clé, et il n'existe aucune métrique entre deux clés. |

§ 1.2 — les quatre premières lignes se substituent terme à terme ; la cinquième survit à la substitution ; la sixième n'y survit pas.

Un partitionnement par hachage détruit la proximité qu'un partitionnement par intervalle préserverait, au prix d'un point chaud. La perte du gradient est la première rupture de la transposition, et elle décide de la conception du sujet plus sûrement que n'importe quel choix de bibliothèque.

**Figure 1.2** — La substitution terme à terme de la stigmergie au journal : les quatre correspondances, ce qui survit, et ce qui ne survit pas.

Une seconde rupture accompagne la première et pèse davantage, quoiqu'elle n'apparaisse que si la fonction de transition des agents est un modèle de langage. Une phéromone est le résidu d'un passage : elle ne peut pas être déposée par un insecte qui n'est pas passé, et sa quantité est l'action même, non son compte rendu. Un enregistrement de journal est un compte rendu. L'agent qui écrit son événement à l'étape 5 de l'algorithme 1.2 déclare une utilité que le milieu accepte sans l'évaluer : M1 à M4 garantissent l'ordre, la durabilité et la non-réécriture, et ne garantissent rien du contenu. Tant que  $\delta$  est un programme écrit à la main, la distinction reste théorique — le programme mesure l'utilité et l'écrit. Elle cesse de l'être dès que  $\delta$  est un modèle : le champ d'utilité devient une assertion, l'identité de l'auteur est déclarative et non authentifiée, et l'expérience montre des agents déposant du code malveillant maquillé en contribution d'un autre agent [119]. La conséquence pour ce chapitre est exacte et limitée : le calcul de  $\varphi$  reste idempotent et reste une fonction de l'intervalle lu, donc rien de ce que la section démontre ne tombe ; mais  $\varphi$  agrège désormais des déclarations, et l'ordre qu'il induit est celui des déclarations et non celui des utilités. Le ch. 8

traite de ce que le milieu doit porter pour que la différence se mesure ; le présent chapitre se borne à ne plus écrire trace là où il faut écrire témoignage.

Le mécanisme central de la stigmergie numérique s'énonce comme algorithme et se paie en messages, en tours et en garanties abandonnées. Modèle de panne : P. Hypothèse de synchronisme : asynchrone pour tout énoncé de sûreté, partiellement synchrone avec borne de délai  $\Delta$  pour tout énoncé de vivacité — sans cette seconde hypothèse, aucune borne de temps n'est énonçable, et l'algorithme 1.2 ne promet alors rien d'autre que de ne pas produire de résultat faux. Les hypothèses sur le milieu sont quatre et étiquetées ; le reste du chapitre les rappelle par leur étiquette. **M1** : l'ordre est total à l'intérieur d'une partition. **M2** : aucun ordre n'est garanti entre partitions. **M3** : un enregistrement validé est durable. **M4** : l'ordre est maintenu en toute circonstance et le compactage ne réordonne jamais, il ne fait que supprimer [5]. Les hypothèses sur les agents sont : n agents, un sujet à p partitions, une fonction de clé h qui envoie une ressource r sur la partition h(r), une fonction d'utilité  $u \geq 0$  mesurable localement après l'action, et m ressources candidates visibles dans l'intervalle lu à un cycle donné.

Algorithme 1.2 – renforcement stigmergique borné sur journal

Modèle de panne : P (crash-arrêt, omission).

Hypothèse de synchronisme : asynchrone pour la sûreté ; partiellement synchrone, borne  $\Delta$ , pour toute borne de temps.

Hypothèses sur le milieu : M1, M2, M3, M4.

Constantes :  $\gamma \in (0,1)$  facteur de décroissance par fenêtre de durée  $\tau$   
 $\phi_{\min} > 0$ ,  $\phi_{\max} < \infty$  bornes de trace,  $\phi_{\min} \leq \phi_{\max}$   
 $\alpha \geq 0$  poids de la trace,  $\beta \geq 0$  poids de l'heuristique locale  
T période de cycle

État de l'agent i : décalage validé  $o_i[j]$  pour chaque partition j lue  
horloge locale  $t_i$

Cycle (répété jusqu'à la condition d'arrêt) :

1. lire l'intervalle  $[o_i[j], \text{fin})$  de chaque partition j du voisinage  
logique 0 message, 1 lecture
2. pour chaque ressource r rencontrée dans l'intervalle :  
 $\phi(r) \leftarrow \sum_e u(e) \cdot \gamma^{((t_i - t_e)/\tau)}$  sur les événements e de r  
 $\phi(r) \leftarrow \min(\phi_{\max}, \max(\phi_{\min}, \phi(r)))$  écrêtage obligatoire
3. tirer  $r^*$  avec probabilité  $\phi(r^*)^\alpha \cdot \eta(r^*)^\beta / \sum_r \phi(r)^\alpha \cdot \eta(r)^\beta$
4. exécuter l'action sur  $r^*$ , mesurer  $u^* \geq 0$
5. écrire l'événement  $(r^*, u^*, t_i, id_i, seq_i)$  dans la partition h( $r^*$ )  
1 écriture
6. attendre l'accusé de durabilité (M3), puis valider  $o_i[j]$  pour les j lus
7. si condition d'arrêt satisfaite, terminer ; sinon attendre T et reprendre

Condition d'arrêt : aucune n'est naturelle. Trois sont admissibles et doivent être écrites dans le système – un budget de cycles ; la stabilité de l'ordre induit par  $\phi$  sur k fenêtres consécutives, avec k et la tolérance donnés ; un signal externe.

Le coût se compte sur les deux axes. Par cycle et par agent : une écriture de  $b$  octets et une lecture dont le volume est le delta accumulé depuis le décalage validé, soit en régime stationnaire  $n \cdot b/p$  octets par partition et par cycle. Pour la population entière :  $\Theta(n)$  messages par cycle, indépendamment de la taille du voisinage logique — c’est le gain topologique du milieu contre une diffusion pair à pair, qui coûterait  $\Theta(n \cdot d)$  messages pour porter la même information, exactement comme l’étape 1 de l’algorithme 1. En tours : la trace écrite au cycle  $k$  n’est lisible qu’au cycle  $k+1$  au plus tôt, donc la boucle de rétroaction porte un retard d’au moins un tour de journal, et la latence de queue  $\ell_9$  du chemin écriture-durabilité-lecture borne par le bas la période  $T$  utile. Fixer  $T$  sous ce seuil ne rend pas l’essaim plus réactif : il le rend aveugle à ses propres dépôts, et l’exploration dégénère alors en tirage indépendant, ce qui est le premier mode de défaillance de l’algorithme 1.2 et le plus facile à provoquer par un réglage.

**Tableau 2** — Ce que l’algorithme 1.2 garantit selon l’hypothèse de synchronisme, et la condition qui fait tomber chaque garantie.

| Énoncé                                                                           | Sous hypothèse asynchrone                  | Sous hypothèse partiellement synchrone ( $\Delta$ )                                                 | Ce qui le fait tomber                                                |
|----------------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| $\varphi(r)$ est une fonction de l’intervalle lu, pas un compteur incrémenté     | tient                                      | tient                                                                                               | un compactage qui réordonnerait — exclu par M4                       |
| Délai entre l’écriture d’une trace au cycle $k$ et sa lecture par un autre agent | $0, \dots, \infty$                         | $\leq \Delta + \ell_9$ du chemin de durabilité, après stabilisation                                 | perte durable de la partition ; violation de M3                      |
| Période $T$ minimale utile                                                       | inconnue, aucune valeur ne se justifie     | $T \geq \ell_9$ du chemin écriture-durabilité-lecture                                               | $T$ sous ce seuil : tirage indépendant                               |
| Écart entre le $\varphi$ lu par un agent et le $\varphi$ courant                 | non borné                                  | $\leq 1 - \gamma^{\wedge}((\Delta + \ell_9)/\tau)$ en fraction de trace                             | $\gamma = 1$ , qui rend l’écart cumulatif                            |
| Comparaison de $\varphi(r_1)$ et $\varphi(r_2)$ sur deux partitions              | sans objet : M2 ne fournit aucune relation | ordre relatif fiable seulement si l’écart de $\varphi$ excède la décroissance sur $\Delta + \ell_9$ | placer sur deux partitions des ressources destinées à être comparées |
| Nombre de cycles relus après reprise                                             | $0, \dots, \infty$                         | borné par la fenêtre entre le décalage validé et la fin courante                                    | une action de l’étape 4 dont la répétition change le résultat        |

La convergence n’est ni promise ni gratuite, et elle tient exactement sous deux conditions dont chacune se lit sur l’algorithme. Il faut  $\gamma < 1$  strictement, sans quoi la trace est une somme cumulée et le processus n’oublie jamais une utilité devenue fausse. Il faut  $\varphi_{\min} > 0$  et  $\varphi_{\max} < \infty$ , sans quoi le rapport  $\varphi_{\max}/\varphi_{\min}$  n’est pas borné, la probabilité de tirage de la ressource dominante tend vers 1 et la population se verrouille — y compris sur une ressource devenue mauvaise. Avec l’écrtage, le calcul est immédiat : à l’étape 3, la probabilité de tirer une ressource  $r$  quelconque parmi les  $m$  visibles est minorée par  $(1/m) \cdot (\varphi_{\min}/\varphi_{\max})^{\alpha(\eta_{\min}/\eta_{\max})\beta}$ , quantité strictement positive et indépendante du cycle, donc chaque ressource est essayée une infinité de fois presque sûrement, et l’exploration survit. La contrepartie est permanente et se chiffre du même coup : la fraction de l’effort de l’essaim dépensée hors de la ressource dominante est minorée par  $((m-1)/m) \cdot (\varphi_{\min}/\varphi_{\max})^{\alpha(\eta_{\min}/\eta_{\max})\beta}$ , à jamais. Un essaim

stigmergique n'atteint pas l'optimum, il campe à distance bornée de lui, et cette distance est un réglage et non un défaut à corriger. Il faut ajouter que cet énoncé est une propriété du tirage, pas un théorème de convergence : des résultats de convergence existent pour des variantes précises de l'optimisation par colonies de fourmis et sont recensés comme tels [14], mais ils portent sur ces variantes et sous leurs hypothèses, et l'algorithme 1.2 n'en est aucune. Ce que le présent chapitre démontre s'arrête au plancher d'exploration ; le reste est à établir pour chaque instance.

Ce plancher est individuel, et c'est là qu'une population d'agents de langage cesse de se comporter comme une colonie. La minoration porte sur la probabilité qu'un agent donné essaie une ressource donnée, et la conclusion — chaque ressource est essayée une infinité de fois presque sûrement — n'est vraie que si les tirages des  $n$  agents sont indépendants. Ils ne le sont pas lorsque les  $n$  agents partagent le modèle, l'invite et le contexte : le tirage de l'étape 3 est alors  $n$  copies du même tirage, et l'exploration de la population est celle d'un seul agent, quelle que soit sa taille. La contrepartie chiffrée du paragraphe précédent devient fausse dans le sens où elle rassure : la fraction d'effort dépensée hors de la ressource dominante est bien celle qu'annonce la formule, mais elle est dépensée  $n$  fois au même endroit. Le remède ne se trouve ni dans  $\gamma$ , ni dans  $\varphi_{\min}$ , ni dans  $\varphi_{\max}$ , qui sont des paramètres du milieu ; il est dans la décorrélation des agents, qui est un paramètre de la population, et que la source biologique n'avait pas à fournir puisque aucune fourmi ne partage l'état interne d'une autre. Le ch. 8 en fait une exigence de conception.

Les modes de défaillance de l'algorithme 1.2 sont quatre, et trois d'entre eux sont des décisions de conception déguisées en détails. Le premier a déjà été nommé :  $T$  inférieur à  $\ell_0$ , rend l'essaim aveugle à ses propres dépôts. Le deuxième est l'ordre entre l'action et le dépôt. Écrire l'événement avant l'étape 4 rend la trace optimiste : un agent qui tombe entre l'écriture et l'action laisse un dépôt sans effet, et l'essaim surestime. Écrire après, comme le fait l'algorithme 1.2, rend la trace pessimiste : un agent qui tombe entre l'action et l'écriture laisse un effet sans trace, et l'essaim sous-estime, puis réattaque une ressource déjà traitée. Il n'existe pas de troisième option sans transaction entre le milieu et l'effet, et cette transaction n'existe que si l'effet est lui-même dans le milieu. Le troisième est le rejeu (*replay*) : après reprise, l'intervalle lu depuis le dernier décalage validé est relu en entier, donc les dépôts sont recomptés ; l'idempotence du calcul de  $\varphi$  le rend inoffensif —  $\varphi$  est une fonction de l'intervalle et non un compteur, ce que garantissent M1 et M4 —, mais l'idempotence de l'action de l'étape 4 doit être établie séparément, faute de quoi le rejeu produit des doubles effets. Le quatrième est l'incomparabilité inter-partitions :  $\varphi(r_1)$  et  $\varphi(r_2)$  calculés sur deux partitions différentes reposent sur deux ordres totaux sans relation entre eux, ce qui est exactement M2 ; les comparer à l'étape 3 revient à comparer deux grandeurs mesurées sur des horloges non synchronisées. Le remède est de placer sur la même partition les ressources destinées à être comparées, ce qui plafonne le parallélisme des consommateurs (*consumer*) au nombre de partitions

p, ou d'accepter un écart borné sous hypothèse partiellement synchrone. C'est un choix, il se documente, et il ne disparaît pas faute d'avoir été fait.

Reste la question que l'émergence pose à la vérification, et qui est la vraie limite du paradigme. Spécifier les comportements globaux émergents d'essaim en fonction des comportements de bas niveau des robots individuels est très difficile — le constat est celui des auteurs qui ont tenté de le faire en logique temporelle [24]. Le model checking, algorithmique et exhaustif, vérifie si des propriétés temporelles sont satisfaites sur tous les comportements possibles du système, et il a été appliqué à des algorithmes de contrôle d'essaim effectivement éprouvés sur du matériel [25] ; les outils existent, avec leurs logiques de propriétés — PCTL, CSL, LTL, PCTL\* — et leurs modèles markoviens [26]. L'obstacle n'est pas l'outillage, il est arithmétique :  $n$  agents ayant chacun  $|S|$  états locaux engendrent  $|S|^n$  configurations globales, et le facteur est exponentiel en  $n$ , non en la taille du programme. Deux échappatoires existent et aucune ne rend le résultat exact. La première est l'approximation macroscopique de champ moyen, indépendante de la taille de l'essaim et porteuse de garanties démontrables sur la dynamique collective [22], mais valable dans une limite dont l'hypothèse de mélange est précisément ce que le partitionnement du milieu casse — M2 est la forme que prend cette rupture. La seconde est le model checking statistique, qui procède par simulation et test d'hypothèses et décide à partir des échantillons si la spécification est satisfaite ou violée, échangeant l'exactitude du calcul de mesure contre l'efficacité et l'uniformité méthodologique [27]. Un essaim ne se prouve donc pas comme un protocole se prouve : il se prouve sur un modèle réduit, ou il se teste avec une confiance. Le ch. 7 en tire les conséquences sur la responsabilité ; ce chapitre se borne à refuser de laisser croire qu'un essaim vérifié existe au même sens qu'un algorithme d'accord vérifié.

Le résultat négatif de la robotique s'applique enfin ici sous une autre forme. Le gain d'efficacité d'un fourragement collectif diminue dans les grands groupes, par interférence [7] ; l'analogue logiciel de l'interférence n'est pas spatial, c'est la contention sur le milieu partagé, où  $n$  agents lisent et écrivent les mêmes  $p$  partitions. La courbe de rendement d'un essaim logiciel a donc la même forme que celle d'un essaim physique pour une cause entièrement différente, et le mécanisme de saturation, ses seuils et ses effondrements appartiennent au ch. 2.

### 1.3 Topologie des réseaux d'agents

La topologie d'un essaim est un graphe  $G = (V, E)$  avec  $|V| = n$ , dont on retient le degré  $d$ , le degré maximal  $\Delta(G)$ , le diamètre et la connectivité algébrique. Sous la convention de comptage posée au §1.1, la topologie décide de deux grandeurs indépendantes qu'un texte sérieux ne confond pas : le nombre de messages nécessaires pour porter une information à toute la population, et le nombre de tours pendant lesquels la population est dans un état mixte. Deux bornes inférieures élémentaires les encadrent et se démontrent en une ligne chacune. Si chaque agent émet vers au plus  $d$  destinataires par tour, le nombre d'agents informés est multiplié par au plus  $d+1$  à chaque tour, donc tout protocole de diffusion exige au moins  $\lceil \log_{d+1} n \rceil$  tours. Et tout protocole qui informe

$n$  agents coûte au moins  $n-1$  messages, puisque chaque agent non initiateur doit recevoir au moins un message. Aucune topologie ne descend sous ces deux bornes ; les topologies se distinguent par ce qu'elles paient au-dessus, et par ce qui casse en premier. Le tableau ci-dessous les compare sous modèle de panne  $P$  et hypothèse synchrone — les comptes de tours n'ont pas de sens autrement, et c'est le seul régime où la dernière colonne se lit comme une comparaison et non comme une juxtaposition.

**Tableau 3** — Coût de coordination par topologie pour porter une information à  $n$  agents, sous modèle de panne  $P$ , hypothèse synchrone et convention de comptage point à point.

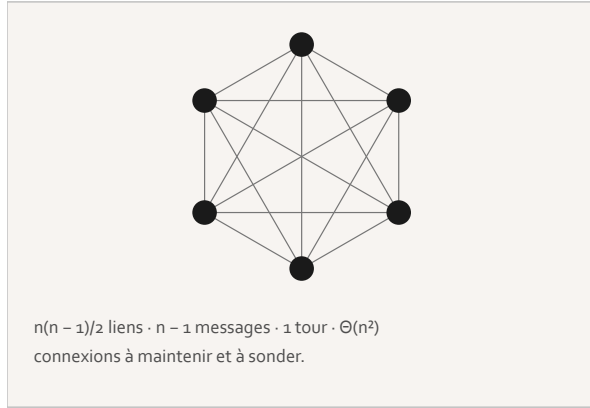
| Topologie                                          | Liens           | Messages par diffusion                                               | Tours                             | Ce qui casse en premier                                                  |
|----------------------------------------------------|-----------------|----------------------------------------------------------------------|-----------------------------------|--------------------------------------------------------------------------|
| Maille complète pair à pair                        | $n(n-1)/2$      | $n-1$                                                                | 1                                 | $\Theta(n^2)$ connexions à maintenir et à sonder                         |
| Anneau                                             | $n$             | $n-1$                                                                | $n-1$                             | Un agent mort coupe l'anneau ; latence linéaire                          |
| Arbre de degré $d$                                 | $n-1$           | $n-1$                                                                | $\lceil \log_d n \rceil$          | La racine : point de défaillance unique                                  |
| Épidémique ( <i>gossip</i> ), fanout $k$           | non fixés       | $\Theta(n \log n)$ attendus, par dérivation                          | $\Theta(\log n)$ attendu [28]     | Redondance de messages ; livraison seulement probabiliste                |
| Journal partitionné, $p$ partitions, $k$ répliques | $n + p \cdot k$ | 1 écriture + $(k-1)$ réplifications, puis 1 lecture par consommateur | 2 tours de journal (écrire, lire) | $p$ plafonne le parallélisme ; le courtier devient la ressource disputée |

La quatrième ligne demande une précision de méthode, parce qu'un des deux chiffres n'est pas dans la source. Le protocole épidémique de référence établit deux faits : la charge de messages espérée par membre est indépendante de la taille du groupe, et la latence de dissémination croît logarithmiquement avec le nombre de membres [28]. Le produit des deux donne  $\Theta(n \log n)$  messages pour informer toute la population, mais ce produit est une dérivation faite ici et non un énoncé de la source ; il est écrit comme tel. Le fonctionnement de ces protocoles, leurs propriétés de complétude et d'exactitude et leur usage appartiennent au ch. 4, et rien ici n'en réexpose le mécanisme.

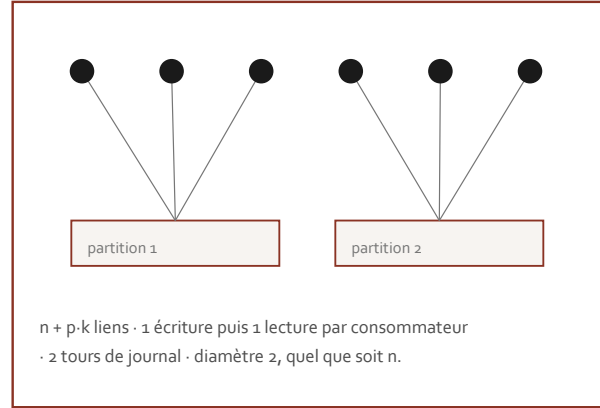
La dernière ligne est la seule dont le nombre de tours ne dépende pas de  $n$ , et c'est là tout l'argument. Un essaim logiciel n'a pas de géométrie : deux agents sont toujours à distance un du journal, quel que soit  $n$ , donc le diamètre du graphe de coordination vaut 2 en permanence. Un essaim physique paie cette propriété très cher — la synthèse sur le consensus en réseau établit que des échanges d'information non locaux accélèrent substantiellement la convergence par rapport aux schémas de plus proches voisins [29], et un lien longue portée dans un essaim robotique coûte de l'énergie et de la puissance d'émission. Ce que le logiciel obtient gratuitement en diamètre, il le repaie sur deux postes : la durée d'un tour, qui est  $\ell$ , du chemin de durabilité et non un aller-retour entre deux agents, et la contention, puisque les  $\Theta(n)$  lectures par cycle atterrissent

toutes sur  $p$  partitions, de sorte que le degré réel n'a pas disparu mais s'est déplacé dans le courtier sous forme de  $n/p$  consommateurs par partition. La transposition conserve l'accélération et casse sa gratuité ; le régime de saturation qui en résulte appartient au ch. 2.

MAILLE COMPLÈTE PAIR À PAIR



JOURNAL PARTITIONNÉ



Le degré n'a pas disparu : il s'est déplacé dans le courtier, sous forme de  $n/p$  consommateurs par partition, et  $p$  plafonne le parallélisme. Ce que le logiciel obtient gratuitement en diamètre, il le repaie sur la durée d'un tour —  $\ell_{99}$  du chemin de durabilité, non un aller simple — et sur la contention, dont le régime appartient au chapitre 2.

**Figure 1.3** — Le diamètre du graphe de coordination : ce que le milieu supprime en topologie, et où le degré réapparaît.

La famille pair à pair adressée mérite d'être décrite avec ses spécifications réelles et son coût, parce qu'elle est le choix par défaut de l'industrie et l'exact opposé de la stigmergie. Modèle de panne : P. Hypothèse de synchronisme : asynchrone, faute de mieux — ni le Model Context Protocol ni Agent2Agent ne bornent le délai d'une réponse, et aucune borne de temps n'est donc énonçable sans ajouter une hypothèse que les spécifications ne fournissent pas. Le premier définit des messages JSON-RPC 2.0 sur des connexions avec état, avec négociation de capacités et trois rôles distincts — hôtes, clients, serveurs —, le serveur offrant ressources, invites et outils tandis que le client offre échantillonnage, racines et sollicitation [30]. Le second définit une délégation pair à pair, trois transports — JSON-RPC 2.0, gRPC, HTTP+JSON/REST —, des objets AgentCard, Task, Message et Artifact, et un cycle de vie de tâche à huit états dont quatre terminaux et deux interrompus, `INPUT_REQUIRED` et `AUTH_REQUIRED` [31].

Le coût nominal d'une délégation est minimal et il faut le reconnaître : 2 messages et 1 tour, requête et réponse. Ce n'est pas là que la famille se paie, et c'est pourquoi elle séduit. Elle se paie sur trois postes, dont aucun n'est un détail d'implantation. Le premier est l'appartenance (*membership*) : l'émetteur nomme le destinataire, donc il entretient une vue des membres vivants, qui est toujours une approximation et jamais un fait ; sous sondage direct de tous par tous, l'entretien de cette vue coûte  $\Theta(n^2)$  messages par période de sondage, et le mécanisme qui évite cette croissance appartient au ch. 4. Le deuxième est le couplage temporel : un état interrompu immobilise des ressources chez l'appelant pendant une durée que la spécification ne borne pas, et ce coût s'écrit

0, ...,  $\infty$  [31]. Le troisième est une frontière de confiance : la spécification prescrit explicitement que les descriptions d'outils doivent être considérées comme non fiables sauf provenance de confiance [30], et dans un maillage adressé chaque arête importe la surface d'attaque de son extrémité, le nombre d'arêtes croissant en  $\Theta(n \cdot d)$ . Le mode de défaillance est celui de tout appel adressé sous P en asynchrone : une omission sur la réponse laisse l'appelant dans un état indéterminé, où il ne peut distinguer une requête perdue, une réponse perdue et un destinataire lent — c'est l'intuition de FLP appliquée à un seul appel. La reprise n'est sûre que si l'opération est idempotente.

La famille événementielle se caractérise par ce qu'elle découple. La synthèse de référence sur la publication-souscription factorise les variantes autour d'un dénominateur commun : le découplage complet des entités communicantes dans l'espace, dans le temps et dans la synchronisation [32]. Le journal partitionné en est l'incarnation retenue par l'ouvrage, et ses garanties sont exactement M1 à M4, complétées par un résultat de réplication qu'il faut citer avec ses chiffres : avec le modèle de réplique synchronisée et  $f+1$  répliques, un sujet tolère  $f$  pannes sans perdre de message validé, là où un vote majoritaire en exigerait  $2f+1$  — tolérer une panne demande trois copies dans un schéma de vote et deux ici, en tolérer deux en demande cinq contre trois [5]. Cette formulation est celle de la source ; le ch. 2 établit que la borne exacte s'écrit en fonction du seuil d'accusé et non du facteur de réplication. Le producteur offre depuis la version 0.11.0.0 une livraison idempotente qui garantit qu'une réémission ne produit pas d'entrée dupliquée dans le journal, par identifiant de producteur et numéros de séquence monotones par partition de sujet [5][33]. La justification physique du modèle est mesurée : sur un ensemble de six disques SATA à 7 200 tr/min en RAID-5, l'écriture linéaire atteint  $\approx 600$  Mo/s contre  $\approx 100$  ko/s en écriture aléatoire, un écart de plus de  $6\,000 \times$  [5]. Le journal n'est pas un choix esthétique, c'est un alignement sur la seule opération que le support exécute vite.

Ce que le journal ne garantit pas doit être écrit avec la même netteté. M2 interdit tout ordre entre partitions, donc aucun instantané global cohérent n'est obtenu par lecture. La livraison exactement-une-fois n'existe que dans la frontière transactionnelle du système, jamais sur un effet de bord arbitraire hors de cette frontière. Le surcoût de format d'une trace est réel et se chiffre : 60 octets pour un message seul contre 34 dans l'ancien format, mais 753 octets pour un lot de 100 messages contre 3 400 auparavant, soit  $\approx 7$  octets par message additionnel en lot [33] — la stigmergie numérique n'est économique qu'en lots. Et le résultat d'impossibilité qui borne le milieu répliqué doit être énoncé avec ses hypothèses exactes, non paraphrasé : Gilbert et Lynch établissent qu'en modèle asynchrone, aucune implantation d'un objet partagé ne peut simultanément être atomique et disponible en présence de partition réseau [2]. Les deux définitions sont plus fortes que leur usage courant. « Atomique » y signifie linéarisable au sens de Herlihy et Wing, où chaque opération paraît prendre effet instantanément en un point unique situé entre son invocation et sa réponse [34], ce qui impose que toute lecture commencée après la fin d'une écriture rende cette valeur ou une plus récente. « Disponible » y signifie que toute requête reçue par un nœud

non défaillant produit une réponse, sans aucune borne de temps — donc aucun accord de service ne satisfait la définition formelle. Le modèle partiellement synchrone est traité séparément dans l'article, où des compromis existent [2]. Un maillage annoncé « disponible » au sens commercial ne satisfait presque jamais la définition qu'il invoque.

Le maillage d'agents (*agent mesh*) est la topologie logique de l'essaim et de son substrat, avec ses règles d'adhésion, de routage et de politique. Il se décrit comme algorithme, et sa faiblesse se nomme dans la même section que sa construction. Modèle de panne : P. Hypothèse de synchronisme : asynchrone pour la sûreté, partiellement synchrone avec borne  $\Delta$  pour toute borne de temps. Hypothèses sur le milieu : un sujet d'appartenance compacté, sous M1, M3 et M4. La règle de placement est un hachage cohérent, dont il faut rappeler que l'objectif d'origine était la résorption des points chauds dans les caches web et non le partitionnement de bases, et qu'il a été conçu pour des réseaux où aucun serveur ne peut connaître l'état complet du réseau [35] — c'est précisément l'hypothèse d'un essaim.

Algorithme 1.3 – adhésion et routage dans un maillage adossé au journal

Modèle de panne : P (crash-arrêt, omission).

Hypothèse de synchronisme : asynchrone pour la sûreté ; partiellement synchrone, borne  $\Delta$ , pour toute borne de temps.

Hypothèses sur le milieu : M1, M3, M4 sur le sujet d'appartenance.

État de l'agent  $i$  :  $V_i$  vue locale des membres,  $o_i$  décalage sur le sujet d'appartenance, capacités  $\text{cap}(i)$

Adhésion :

1. écrire  $\text{ANNONCE}(\text{id}_i, \text{cap}(i), \text{génération } g_i)$  dans la partition  $h(\text{id}_i)$   
du sujet d'appartenance 1 écriture
2. lire le sujet d'appartenance depuis le début de l'historique compacté  
jusqu'à la fin courante ; construire  $V_i$   $\Theta(m)$  octets pour  $m$  membres
3. l'adhésion est effective quand  $o_i$  atteint le décalage de sa propre  
 $\text{ANNONCE}$  1 tour de journal  
(lire-ses-écritures, garanti par M1, donc par partition seulement)

Routage d'une charge de clé  $c$  :

4.  $j \leftarrow \text{hash}(c) \bmod p$  0 message
5. propriétaire  $\leftarrow \text{rang}(j)$  dans  $V_i$  selon la règle publique de placement
6. si propriétaire =  $i$ , traiter ; sinon écrire la charge dans la  
partition  $j$  et laisser le propriétaire la lire 1 écriture, 1 tour

Départ :

7. annoncé : écrire  $\text{RETRAIT}(\text{id}_i, g_i)$  1 écriture  
les lecteurs retirent  $i$  de leur vue au tour de journal suivant
8. non annoncé :  $i$  est soupçonné par un détecteur de défaillance (ch. 4)  
et retiré de  $V_j$  par chaque agent  $j$  indépendamment

Condition d'arrêt : aucune ; la boucle 2–6 est le régime nominal.

Le coût est asymétrique et c'est ce qui rend le maillage intéressant. Une adhésion coûte une écriture, plus la lecture de l'historique compacté, soit  $\Theta(m)$  octets pour  $m$  membres — une fois, au démarrage. La propagation d'un changement d'appartenance à toute la population coûte  $n$  lectures d'un unique enregistrement, en 1 tour de journal, contre  $\Theta(n^2)$  messages pour une annonce pair à pair vers tous ; c'est le même argument comparatif que celui qui oppose les protocoles épidémiques, dont le temps espéré de première détection et la charge de messages par membre sont indépendants de la taille du groupe, au battement de cœur classique, dont la charge réseau croît quadratiquement avec elle [28]. Le routage d'une charge coûte 0 message quand le propriétaire est l'agent lui-même et 1 écriture plus 1 tour de journal sinon, ce qui borne à 2 tours le trajet complet d'une charge de son point d'entrée à son traitement.

Le mode de défaillance de l'algorithme 1.3 est unique, et il est structurel : entre l'instant où un agent écrit son ANNOUNCE et l'instant où un autre agent l'a lue, les deux vues  $V_i$  et  $V_j$  diffèrent, donc l'étape 5 peut désigner deux propriétaires distincts pour la même clé. Le maillage ne comporte aucun mécanisme d'exclusion et n'en promet aucun : pendant la fenêtre de divergence, deux agents peuvent traiter la même charge. Sous hypothèse partiellement synchrone, cette fenêtre est bornée par  $\Delta$  plus la latence de durabilité plus la période de lecture du sujet d'appartenance, et le nombre de charges doublement traitées est donc borné par le débit d'arrivée multiplié par cette durée. Sous hypothèse asynchrone, elle n'est bornée par rien et le compte s'écrit 0, ...,  $\infty$  ; c'est FLP (§1.1) qui l'interdit, puisque borner cette fenêtre reviendrait à désigner un propriétaire unique en temps fini, c'est-à-dire à faire terminer un accord en asynchrone. Trois issues, et trois seulement. Rendre le traitement idempotent, auquel cas le double traitement est sans conséquence et le maillage suffit. Accepter le double effet et le réconcilier après coup, ce qui exige une traçabilité complète (ch. 6) et un point de reprise (*checkpoint*). Ou exiger un propriétaire unique à tout instant, ce qui se paie au tarif du ch. 4. L'ouvrage soutient que la première issue couvre la majorité des cas réels et que la troisième est choisie par défaut bien plus souvent qu'elle n'est nécessaire ; il ne soutient pas qu'elle soit toujours évitable, et il n'a rien à opposer à un invariant global qui doit tenir à tout instant.

Un dernier résultat de la théorie du contrôle en réseau encadre ce que la topologie décide, et sa transposition est partielle. Olfati-Saber et Murray établissent que les digraphes équilibrés sont fondamentaux pour le consensus de moyenne, et lient la connectivité algébrique du réseau à la performance du protocole, en présence de topologies orientées, commutantes et de retards de communication [36]. Ce qui se transpose : la connectivité algébrique est bien le prédicteur de la vitesse à laquelle une information se répand dans un maillage, et un maillage dont le graphe se déséquilibre — quelques agents lisant beaucoup, la plupart lisant peu — ralentit de façon mesurable. Ce qui ne se transpose pas : ces résultats supposent des poids d'échange sur les arêtes et un temps continu ou discret uniforme, alors qu'un essaim adossé à un journal n'a pas d'arêtes pondérées mais des partitions, et pas de tour global mais des

décalages indépendants. Le graphe de coordination d'un maillage d'agents est biparti — agents d'un côté, partitions de l'autre —, sa connectivité algébrique est celle de ce graphe biparti et non celle d'un graphe d'agents, et transporter mécaniquement les bornes spectrales de la littérature sur le consensus vers un maillage adossé au journal donne des chiffres faux. La forme correcte du résultat reste à écrire ; ce chapitre s'arrête à la constatation que la question est ouverte et que la topologie du milieu, non celle des agents, en est la variable.

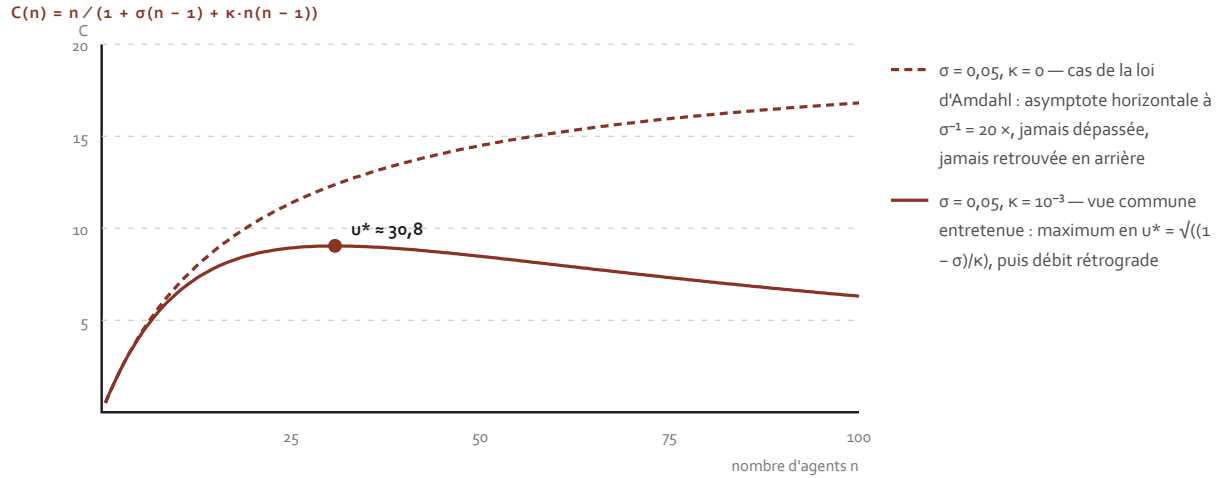
## 2. Scalabilité et passage à l'échelle

### 2.1 Décentralisation stricte : élimination des points de défaillance uniques (SPOF) et découplage spatial et temporel

Un point de défaillance unique (*single point of failure*, SPOF) est un composant dont l'arrêt rend indisponible une fonction que le reste de l'essaim ne peut pas assurer. La définition est opératoire et non morale : elle se vérifie en supprimant le composant et en observant ce qui cesse. Elle sépare quatre objets que la prose d'architecture confond régulièrement. Le SPOF de données est la seule copie d'un enregistrement. Le SPOF de chemin est le lien ou le courtier (*broker*) par lequel tout transite. Le SPOF de décision est l'agent dont l'accord est requis pour que les autres avancent. Le SPOF d'identité est le registre qui dit qui est qui. Un coordonnateur élu n'est pas un SPOF de sûreté — un protocole d'accord correct reste sûr quand son meneur disparaît, voir ch. 4 — mais il demeure un SPOF de vivacité : pendant sa réélection, le groupe ne progresse pas. Confondre les deux conduit à déclarer « sans SPOF » une architecture qui s'immobilise à chaque bascule.

La raison de supprimer le coordonnateur n'est pas esthétique, elle est quantifiée. La loi universelle de scalabilité énonce la capacité relative d'un système à  $u$  unités — la source note ce nombre  $p$ , symbole que la charte réserve aux partitions — comme la fonction rationnelle  $C(u, \sigma, \kappa) = u / (1 + \sigma(u - 1) + \kappa u(u - 1))$ , où  $\sigma$  mesure la contention — sérialisation ou mise en file sur une ressource partagée — et  $\kappa$  la cohérence, c'est-à-dire le coût de maintenir des copies mutuellement à jour [3]. Les hypothèses sont explicites et restrictives : le travail est équipartitionnable, et la forme rationnelle est établie comme la borne de débit synchrone d'un modèle fermé de réparateur de machines à taux de service dépendant de l'état [3]. La loi d'Amdahl en est le cas  $\kappa = 0$ , avec son asymptote horizontale  $\sigma^{-1}$ . Le terme  $\kappa$  change la nature du résultat : dès que  $\kappa > 0$ ,  $C$  admet un maximum en  $u^* = \sqrt{((1 - \sigma)/\kappa)}$ , et au-delà de  $u^*$  le débit devient rétrograde — ajouter des unités en retire [3]. Transposée à l'essaim,  $u$  est le nombre d'agents  $n$ ,  $\sigma$  la fraction du travail qui passe par le point partagé,  $\kappa$  le prix de la vue commune. Avec  $\sigma = 0,05$  et  $\kappa = 0$ , le plafond est de  $20 \times$  quel que soit  $n$  ; avec  $\sigma = 0,05$  et  $\kappa = 10^{-3}$ , le calcul donne  $u^* \approx 30,8$ , donc un essaim dont le débit culmine vers 31 agents et décroît ensuite. Ces deux valeurs sont une illustration arithmétique de la formule citée, pas une mesure. Elles

suffisent à établir le point : un essaim qui maintient une vue commune n'a pas un rendement décroissant, il a un optimum, et le dépasser est une régression.



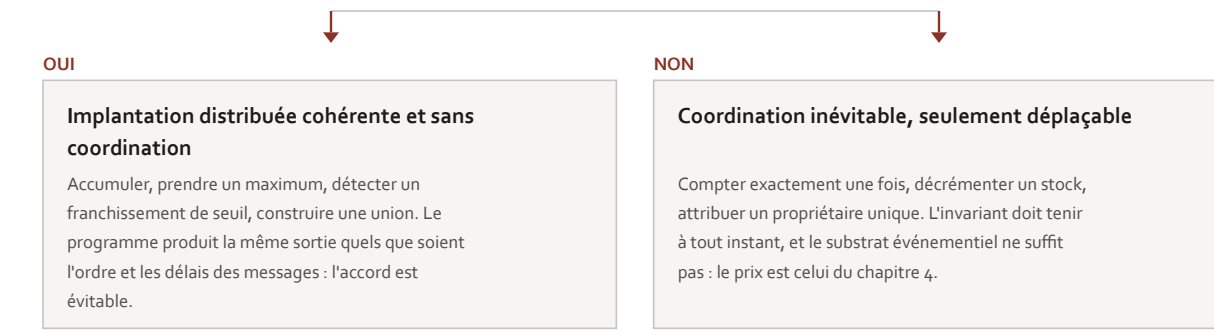
Ces deux valeurs de  $\sigma$  et de  $\kappa$  sont une illustration arithmétique de la formule citée [3], et non une mesure : le protocole qui les tirerait d'un journal de messages réel n'est pas écrit, et la conclusion de l'ouvrage en fait son premier reste. Ce qu'elles établissent tient néanmoins — dès que  $\kappa > 0$ , dépasser  $u^*$  est une régression, pas un plafond.

**Figure 2.1a** — La loi universelle de scalabilité : pourquoi un essaim qui entretient une vue commune n'a pas un rendement décroissant, mais un maximum.

Reste à savoir ce qui peut légitimement être décentralisé. Le théorème CALM tranche cette question au niveau du programme et non du stockage : les programmes qui admettent une implémentation distribuée cohérente et sans coordination sont exactement ceux qui s'expriment en logique monotone [6]. « Cohérent » y signifie que le programme produit la même sortie quels que soient l'ordre et les délais des messages ; « monotone » que la conclusion, une fois atteinte, n'est jamais rétractée par l'arrivée de faits supplémentaires. Un agent qui accumule, qui prend un maximum, qui détecte un franchissement de seuil ou qui construit une union est monotone et peut se passer d'accord. Un agent qui compte exactement une fois, qui décrémente un stock ou qui attribue un propriétaire unique ne l'est pas, et pour celui-là la coordination n'est pas évitable : elle est seulement déplaçable. Le résultat est constructif dans les deux sens, ce qui en fait une borne et pas un conseil. Deux impossibilités classiques le bordent et ne seront ici qu'invoquées : en système asynchrone, un seul processus fautif par arrêt suffit à rendre impossible tout protocole de consensus garantissant la terminaison, le modèle interdisant explicitement les temporisateurs [1] ; et sous partition réseau, aucun service ne satisfait simultanément la linéarisabilité et une disponibilité définie sans borne de temps [2]. Le traitement formel de ces deux résultats appartient au ch. 4 ; ce qu'ils établissent ici est plus étroit et suffit : la décentralisation stricte n'est pas un choix de goût, c'est la seule forme disponible quand on refuse à la fois l'attente non bornée et l'indisponibilité.

## CE QUI SE DÉCENTRALISE, ET CE QUI NE SE DÉCENTRALISE PAS

Le programme s'exprime-t-il en logique monotone — une conclusion atteinte n'étant jamais rétractée par l'arrivée de faits supplémentaires ?



Le résultat est constructif dans les deux sens, ce qui en fait une borne et pas un conseil. Aucun choix de base de données, de courtier ou de bibliothèque ne fait passer un programme non monotone du côté gauche : pour celui-là, la coordination n'est pas évitable, elle est seulement déplaçable.

**Figure 2.1b** — Le critère qui décide de ce qui est décentralisable : le théorème CALM, tranché au niveau du programme et non du stockage.

Le découplage est le moyen par lequel un essaim se prive de coordonnateur sans se priver de coordination. Il se décompose en trois dimensions indépendantes, établies dans l'étude qui a servi de dénominateur commun aux systèmes de publication-abonnement : le découplage spatial — les parties en interaction n'ont pas besoin de se connaître, le producteur ne détient pas de référence vers les consommateurs et ignore combien ils sont ; le découplage temporel — les parties n'ont pas besoin d'être actives en même temps, le producteur pouvant publier pendant que le consommateur est déconnecté et réciproquement ; le découplage de synchronisation — le producteur n'est pas bloqué pendant qu'il produit, et le consommateur est notifié de façon asynchrone hors de son flot de contrôle principal [32]. Chaque dimension supprime un SPOF précis : l'espace supprime celui du nommage, le temps celui de la simultanéité, la synchronisation celui du fil d'exécution appelant. Il faut dire immédiatement ce que l'opération ne fait pas. Elle ne détruit pas le point partagé, elle le déplace dans le milieu, qui devient le composant que tout le monde touche. Le milieu doit donc être répliqué, et le prix de cette réplication est le vrai prix du découplage. Ce prix est habituellement annoncé sous une forme flatteuse : le modèle de réplication en ensemble synchronisé (*in-sync replica set*) tolère  $f$  pannes avec  $f + 1$  répliques là où un vote majoritaire en exigerait  $2f + 1$ , soit deux copies au lieu de trois pour une panne [5]. Énoncé ainsi, il est incomplet au point d'induire en erreur, et on le dérive.

Les définitions primaires sont dans la documentation de conception du journal : l'ensemble synchronisé réunit les répliques ayant rattrapé le meneur d'une partition, et ses membres sont les seuls éligibles à l'élection ; une réplique en sort quand elle perd sa session avec le contrôleur ou quand son retard dépasse un seuil de temps ; une écriture n'est validée que lorsque toutes les répliques de l'ensemble l'ont reçue, et le contrôleur élit le meneur suivant parmi les membres restants [5]. Le modèle de panne est celui du chapitre — arrêt franc, omission de

canal — et il exclut l’amnésie, c’est-à-dire le redémarrage d’un courtier avec un journal tronqué. L’hypothèse de synchronisme est ici sans effet : l’appartenance n’est pas une mesure mais une décision, prise par un composant à quorum dont le prix appartient au ch. 4. Deux grandeurs suffisent,  $k$  le nombre de répliques assignées et  $m$  le seuil d’accusé, taille minimale d’ensemble synchronisé en deçà de laquelle la partition refuse les écritures — réglage qui existe, dit la source, pour empêcher la perte de messages écrits sur une seule réplique [5].

**R1** — tout enregistrement accusé au producteur est présent chez tout meneur ultérieur de la partition. R1 tient par deux prémisses et aucune arithmétique de quorum : l’accusé n’est émis qu’après écriture par toutes les répliques de l’ensemble synchronisé courant, et le meneur est élu parmi les membres de cet ensemble, où l’on n’entre qu’après avoir rattrapé le journal du meneur. Tout membre d’un ensemble postérieur détient donc l’enregistrement — les détenteurs contiennent les candidats. C’est un argument d’inclusion, non d’intersection : le vote majoritaire fait se croiser deux majorités et paie  $2f + 1$  répliques pour que le croisement soit non vide, là où l’inclusion, condition plus forte, coûte moins de copies et coûte autre chose — une liste de membres exacte, partagée et révisable, c’est-à-dire un accord au sens strict. Le facteur  $k$  de réplication du journal est bon marché parce que quelqu’un d’autre paie le consensus.

Le cas  $k = 3$  avec  $m = 2$  se déroule au complet et montre où R1 tombe. Répliques A, B et C, A meneur, ensemble synchronisé  $\{A, B, C\}$ . À  $t_0$ ,  $r_1$  est accusé après écriture par les trois : sa largeur d’accusé, c’est-à-dire le nombre de répliques qui le détiennent à l’instant de l’accusé, vaut 3. À  $t_1$ , C prend du retard au-delà du seuil de temps — pause de ramasse-miettes, disque lent, gigue réseau — et le contrôleur le retire ; l’ensemble devient  $\{A, B\}$ . C n’a pas fauté, le compteur de pannes est à zéro. À  $t_2$ ,  $r_2$  est accusé après écriture par A et B : la taille de l’ensemble atteint  $m$ , la partition accepte, largeur 2. C ne détient pas  $r_2$  et n’est plus candidat. À  $t_3$ , B s’arrête : première panne, l’ensemble devient  $\{A\}$ , sa taille passe sous  $m$  et la partition cesse d’accuser. R1 tient, A détenant  $r_1$  et  $r_2$ . À  $t_4$ , A s’arrête : deuxième panne, et  $k = 3 = f + 1$  pour  $f = 2$ , de sorte que la condition annoncée est exactement satisfaite. Seul C est vivant, et il ne détient pas  $r_2$ .

Restent deux issues, présentées par la source comme un choix de configuration : attendre qu’une réplique de l’ensemble revienne, ce qui préserve R1 et rend la partition indisponible en écriture pour une durée que rien ne borne ; ou élire la première réplique qui revient, membre ou non [5], ce qui élit C, ne laisse  $r_2$  nulle part, et fait tronquer à A et B leur journal à la fin de celui de C au redémarrage —  $r_2$  est détruit chez ses deux détenteurs. R1 est violée avec deux arrêts et trois répliques, donc en respectant  $k \geq f + 1$  à l’égalité. La documentation ne se contredit pas, puisqu’elle conditionne sa garantie à ce qu’au moins une réplique reste synchronisée à tout instant et qu’à  $t_4$  aucune ne l’est plus [5] ; c’est justement le point. Le nombre de disparitions auquel  $r_2$  survit n’est pas  $k - 1 = 2$ , il est  $m - 1 = 1$ .

La condition de rupture se nomme en deux clauses, dont une seule suffit. La taille de l'ensemble synchronisé passe sous le seuil d'accusé : cette clause ne détruit rien, elle convertit un problème de sûreté en problème de vivacité, la partition refusant d'accuser plutôt que d'accuser trop étroitement. Un non-membre est élu meneur : cette clause détruit, et sous le modèle arrêt-omission elle est la seule qui détruisse. Réintroduire l'amnésie en fait apparaître une troisième forme, qui ne demande aucune élection irrégulière : la dernière réplique de l'ensemble subit un arrêt non propre, perd des enregistrements validés, est réélue meneur au redémarrage, et les suiveurs qui la rejoignent tronquent leur journal, effaçant les dernières copies de ce qu'elle avait perdu. Le projet décrit cette perte locale devenue globale sur exactement le cas  $k = 3$ ,  $m = 2$ , et y répond par un ensemble de répliques tenues pour éligibles bien que sorties de l'ensemble synchronisé, sous la règle que la somme des tailles des deux ensembles ne tombe pas sous le seuil ; la tolérance ainsi obtenue est de  $m - 1$  arrêts non propres avec perte [37], et ce qui rend ces répliques sûres est que la marque haute n'avance pas tant que l'ensemble synchronisé est sous le seuil [38]. La borne s'écrit en fonction de  $m$  ; nulle part en fonction de  $k$ .

D'où la reformulation, seule forme sous laquelle la garantie est utilisable. Le facteur  $k$  borne le nombre d'arrêts simultanés que la partition absorbe avant qu'il ne reste aucun candidat sûr au poste de meneur, soit  $k - 1$ , et la disponibilité en écriture cesse bien avant, dès que l'ensemble passe sous  $m$ . Le seuil d'accusé  $m$  borne la perte : un enregistrement accusé l'a été sur au moins  $m$  répliques, il survit à  $m - 1$  disparitions de contenu et à aucune de plus. La généralisation à  $k$  et  $m$  quelconques ne demande rien de plus que le déroulé ci-dessus —  $k - m$  retraits pour ramener l'ensemble au seuil, puis  $m$  disparitions pour effacer la largeur d'accusé, le cas traité étant celui d'un retrait et de deux arrêts. Où placer  $m$  et ce que ce placement coûte appartiennent au ch. 6 ; ce qui est établi ici est que c'est  $m$ , et non  $k$ , qui figure dans l'énoncé de la garantie.

$k = 3$  RÉPLIQUES, SEUIL D'ACCUSÉ  $m = 2$  :  $R_1$  TOMBE À DEUX ARRÊTS

|                      | $t_0$                       | $t_1$                            | $t_2$                       | $t_3$                                            | $t_4$                              |
|----------------------|-----------------------------|----------------------------------|-----------------------------|--------------------------------------------------|------------------------------------|
| Ensemble synchronisé | {A, B, C}                   | {A, B}                           | {A, B}                      | {A}                                              | {}                                 |
| Réplique A, meneur   | $r_1$                       | $r_1$                            | $r_1 r_2$                   | $r_1 r_2$                                        | × arrêt                            |
| Réplique B           | $r_1$                       | $r_1$                            | $r_1 r_2$                   | × arrêt                                          | —                                  |
| Réplique C           | $r_1$                       | $r_1$ , retirée                  | $r_1$                       | $r_1$                                            | $r_1$ , seule vivante              |
| Ce qui survient      | $r_1$ accusé ;<br>largeur 3 | retard ><br>seuil, sans<br>panne | $r_2$ accusé ;<br>largeur 2 | sous $m$ : la<br>partition<br>cesse<br>d'accuser | élire C laisse<br>$r_2$ nulle part |

§ 2.1 — deux arrêts, donc  $k \geq f + 1$  respecté à l'égalité. Un retrait pour retard n'est pas une panne, et le compteur de pannes est à zéro à  $t_4$ .

Le nombre de disparitions auquel  $r_2$  survit n'est pas  $k - 1 = 2$ , il est  $m - 1 = 1$ . La borne s'écrit en fonction du seuil d'accusé  $m$  ; nulle part en fonction du facteur de réplication  $k$ . La documentation ne se contredit pas — elle conditionne sa garantie à ce qu'au moins une réplique reste synchronisée à tout instant, et c'est justement le point.

**Figure 2.1c** — Où la garantie de l'ensemble synchronisé tombe : le déroulé complet à  $k = 3$  répliques et seuil d'accusé  $m = 2$ .

L'algorithme 2.1 explicite ce que coûte le découplage temporel du côté d'un agent producteur. Modèle de panne : arrêt (*crash-stop*) pour les agents, omission pour les canaux ; les fautes byzantines sont hors modèle, voir ch. 7. Hypothèse de synchronisme : partiellement synchrone, c'est-à-dire qu'il existe un instant après lequel les délais sont bornés par un  $\Delta$  inconnu, ce qui autorise un délai de garde  $\delta$  mais interdit d'en déduire la mort de qui que ce soit. Le milieu est le journal partitionné répliqué  $k$  fois du paragraphe précédent, avec son seuil d'accusé  $m$  et l'invariant  $R_1$ . Chaque enregistrement porte un identifiant de producteur, un numéro d'époque et un numéro de séquence monotone par partition, mécanique de déduplication reprise du producteur idempotent [33].

Algorithme 2.1 – session d'un producteur : époque, séquence et écriture idempotente

état persistant du producteur : `id, epoch, seq[·]`

état du milieu : pour chaque couple (`id, partition`), `dernierSeq` et `epochCourant`

```

1  à l'initialisation, ou après un redémarrage :
2    epoch ← milieu.ouvrirSession(id)    // invalide toute session (id, e) avec e
< epoch
3    pour chaque partition P : seq[P] ← milieu.dernierSeq(id, P) + 1
4  pour chaque événement e destiné à la partition P :
5    msg ← {id, epoch, seq[P], e}
6    émettre msg ; armer le délai de garde  $\delta$ 
7    sur accusé de msg : seq[P] ← seq[P] + 1 ; passer à l'événement suivant
8    sur expiration de  $\delta$  : réémettre msg à l'identique ; réarmer  $\delta$ 
```

```

9  côté milieu, à la réception de (id, e, s, ev) pour la partition P :
10  si e < epochCourant(id) : rejeter – session périmée
11  sinon si s ≤ dernierSeq(id, P) : accuser sans écrire – doublon avéré
12  sinon si |ensembleSynchronisé(P)| < m : rejeter – sous le seuil d'accusé
13  sinon si s = dernierSeq(id, P) + 1 : répliquer sur l'ensemble synchronisé,
14      dernierSeq ← s, accuser
15  sinon : rejeter – trou de séquence, exiger une réouverture de session

```

Le coût nominal est de 2 messages par événement sur le chemin du producteur, l'émission et son accusé, pour 1 tour ; si l'accusé n'est émis qu'après la dernière réplique de l'ensemble synchronisé, comme l'exige la ligne 13, la réplication entre dans le chemin critique et le coût devient 2 tours. En lot, le surcoût de format de cette mécanique est de 753 octets pour un lot de 100 messages contre 3 400 octets dans l'ancien format sans idempotence, soit environ 7 octets par message additionnel [33] : le découplage temporel se paie en octets, pas en tours. La condition d'arrêt de la boucle des lignes 6 à 8 est la réception d'un accusé ; sous synchronisme partiel et avec un canal qui finit par livrer, la réémission termine, mais le nombre de réémissions n'est pas borné et s'écrit 1, ...,  $\infty$  plutôt qu'une moyenne plausible. Un rejet de la ligne 12 ne satisfait jamais cette condition : tant que l'ensemble synchronisé reste sous m, le producteur réemet contre une partition qui refuse d'accuser, et c'est la première clause de rupture de R1 vue depuis l'amont. Les modes de défaillance sont deux, et aucun n'est masqué par la réplication. Si le producteur redémarre après que le milieu a purgé l'état (id, epoch) par rétention, la ligne 3 rend une valeur qui n'identifie plus la dernière écriture, des doublons entrent sans être détectés, et la garantie retombe à « au moins une fois » : le jeu n'est admissible que si le traitement aval est idempotent, voir ch. 1. Si un processus zombie conserve une session ouverte sans jamais se reconnecter, la ligne 10 ne l'exclut pas et ses écritures restent valides : le SPOF résiduel n'est pas le processus, c'est l'identité.

**Tableau 4** — Ce que la redondance de facteur k masque, et ce qu'elle laisse passer, sous le modèle arrêt-omission.

| Mode de défaillance                                                | Hypothèse requise                                         | Ce que k répliques masquent                                           | Mode résiduel                                                                 |
|--------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Arrêt d'un agent producteur                                        | arrêt franc, session invalidable                          | rien : la réplique porte les données, pas la production               | trou dans le flux jusqu'à la reprise                                          |
| Arrêt d'un courtier, ensemble synchronisé encore $\geq m$          | appartenance décidée ailleurs, meneur élu dans l'ensemble | rien par elles-mêmes : c'est m qui masque, jusqu'à m – 1 disparitions | latence accrue pendant la réélection de tête de partition                     |
| Arrêt d'un courtier, ensemble synchronisé tombé sous m             | aucune : la redondance ne défend plus R1                  | rien, quel que soit k                                                 | refus d'accusé, ou perte des enregistrements accusés si un non-membre est élu |
| Omission de canal                                                  | canal qui finit par livrer                                | la perte du message, par réémission                                   | doublons, donc idempotence obligatoire                                        |
| Partition réseau                                                   | quorum du côté majoritaire                                | rien du côté minoritaire                                              | indisponibilité d'écriture, voir ch. 4                                        |
| Défaillance corrélée (image, dépendance, plan de contrôle, modèle) | aucune : l'hypothèse d'indépendance est fausse            | rien                                                                  | n répliques valent 1                                                          |

| Mode de défaillance | Hypothèse requise | Ce que k répliques masquent | Mode résiduel                   |
|---------------------|-------------------|-----------------------------|---------------------------------|
| Saturation          | aucune            | rien                        | effondrement de débit, voir 2.3 |

La robotique en essaim revendique la robustesse et la tolérance aux pannes comme bénéfiques d’une grande population coordonnée par règles locales [15], et la démonstration d’auto-assemblage à mille robots repose sur des interactions locales seules, avec des algorithmes conçus pour tolérer la variabilité inhérente aux grands systèmes décentralisés [16]. La transposition conserve trois choses : l’absence de coordonnateur désigné, la localité des règles, et la dégradation graduelle plutôt que binaire. Elle casse sur l’indépendance des unités de défaillance, et cette rupture est décisive. Mille robots portent mille batteries, mille processeurs et mille positions physiques distinctes ; mille agents logiciels partagent une image, un environnement d’exécution, un courtier, un fournisseur d’identité, souvent une horloge, et — dès lors que leur fonction de transition est un modèle de langage — ce modèle et le contexte qu’on lui donne. La corrélation cesse alors de porter sur la seule panne pour porter sur la décision, et le §2.3 en donne la mesure : ce n’est plus un défaut de mode commun, c’est un choix de mode commun. La redondance ne masque que les défaillances indépendantes : devant un défaut de mode commun,  $n$  répliques valent une seule, ce que la dernière ligne du tableau ci-dessus énonce sans détour. Elle casse aussi sur la gratuité du milieu. En robotique, l’environnement physique est un milieu stigmergique donné, durable et sans propriétaire ; en logiciel, le milieu doit être construit, répliqué et exploité, et il est précisément le candidat le plus sérieux au titre de SPOF que la décentralisation prétendait supprimer. Le verdict pratique va dans ce sens et mérite d’être dit même s’il contredit l’élégance de la section : les grands ordonnanceurs de production n’éliminent pas le coordonnateur, ils le rendent petit et le répliquent — l’état persistant du maître de Borg est répliqué par Paxos, l’architecture éliminant par ailleurs tout point de défaillance unique au niveau des tâches, sur des cellules dont la taille médiane est d’environ 10 000 machines [39]. La décentralisation stricte s’applique à la population, pas nécessairement au registre qui la nomme.

## 2.2 Dynamique des populations d’agents : élasticité, cycle de vie, création et destruction d’agents à très haute échelle

Le nombre d’agents  $n$  est une variable de commande, pas une donnée du problème. C’est l’apport le plus directement transposable de la modélisation des essaims robotiques : les modèles de champ moyen décrivent la population par des fractions d’agents dans chaque état plutôt que par des identités, ils se déclinent en équations différentielles ordinaires, en équations aux dérivées partielles et en équations aux différences selon le caractère discret ou continu des états et du temps, et leur avantage décisif est d’être indépendants de la taille de l’essaim, ce qui rend la conception du contrôle passable à l’échelle contrairement aux modèles microscopiques à agents discrets [22]. La méthode d’allocation par politiques stochastiques pousse la conséquence jusqu’au bout : le problème est

reformulé en optimisation sur les taux d'entrée et de sortie entre états, et ce sont ces taux qui définissent ensuite les politiques individuelles, sans aucune communication entre robots, avec une validation numérique à 250 robots [40]. Transposé, l'objet à concevoir n'est pas la décision d'un agent mais le champ de taux qui gouverne les transitions démarrage  $\rightarrow$  prêt  $\rightarrow$  en traitement  $\rightarrow$  en vidange  $\rightarrow$  arrêté. L'hypothèse structurante est que la matrice des taux est irréductible : sans elle, l'essaim admet plusieurs équilibres et le contrôleur peut stabiliser la population dans le mauvais.

Le dimensionnement de la cible se lit dans la loi de Little,  $L = \lambda W$ , où  $L$  est le nombre moyen d'unités dans le système,  $\lambda$  le taux d'arrivée et  $W$  le temps de séjour moyen [41]. Ses hypothèses sont énoncées dans la source et il faut les répéter parce qu'elles sont exactement celles qui tombent en régime élastique : moyennes finies, processus stochastiques strictement stationnaires, processus d'arrivée métriquement transitif de moyenne non nulle [41]. La loi ne dit rien d'un système en régime transitoire ou saturé. Elle sert donc à calculer la cible — pour un agent capable de  $c$  traitements concurrents,  $n = \lambda W/c$  — et elle ne sert pas à piloter la boucle, puisque la boucle ne travaille que dans le transitoire. Un contrôleur d'élasticité qui applique Little à une mesure prise pendant une montée de charge produit un nombre arithmétiquement exact et opérationnellement faux.

Le coût unitaire d'un agent fixe l'ordre de grandeur du possible. Un processus fraîchement créé sur une machine virtuelle conçue pour la concurrence massive occupe de l'ordre de trois cents mots, soit quelques milliers d'octets sur une architecture 64 bits, dont 233 mots de zone de tas, pile comprise : la documentation d'OTP 29.0.5 annonce 327 mots à son guide d'efficacité, sur une transcription qui affiche OTP 27, et 338 à sa page de mémoire ; l'écart est explicable, et il reste la meilleure raison de ne rien gager sur le chiffre exact [42]. Le plancher mémoire d'un million d'agents de ce type est donc de l'ordre de 2,6 Go avant tout état applicatif : à ce coût, la création et la destruction sont des non-événements et la population peut suivre la charge en quasi-continu. Le calcul change complètement dès que l'agent possède quelque chose. Un agent membre d'un groupe de consommation détient des partitions et un décalage (*offset*), et sa disparition n'est pas une soustraction mais une transaction. Le protocole de rééquilibrage d'origine impose de révoquer toutes les tâches en cours à chaque changement d'appartenance, parce qu'il faut garantir que chaque partition d'un sujet est affectée à exactement un consommateur à tout instant ; le protocole coopératif incrémental remplace cette barrière de synchronisation unique par des rééquilibrages consécutifs, les membres conservant les partitions qu'ils détiennent déjà et la révocation n'étant indiquée que pour être suivie d'une réaffectation au tour suivant [43]. Le compromis est net et se compte : la barrière unique coûte 2 tours pendant lesquels le débit du groupe est nul ; le protocole coopératif double le nombre de tours et maintient un débit non nul sur les partitions non déplacées.

**Tableau 5** — Coût d'un changement de population selon que l'agent possède ou non des partitions, à  $n$  membres dans le groupe.

| Changement de population                    | Messages                                                    | Tours  | Débit pendant l'opération                           |
|---------------------------------------------|-------------------------------------------------------------|--------|-----------------------------------------------------|
| Création d'un agent sans état               | 0 sur le milieu                                             | 0      | inchangé                                            |
| Destruction d'un agent sans état            | 0 sur le milieu                                             | 0      | inchangé                                            |
| Entrée dans un groupe, protocole d'origine  | $\Theta(n)$                                                 | 2      | nul pour tout le groupe                             |
| Entrée dans un groupe, protocole coopératif | $\Theta(n)$                                                 | 4      | non nul hors partitions déplacées                   |
| Sortie planifiée d'un agent porteur         | $\Theta(n) + 1$ validation de décalage + 1 point de reprise | 2 ou 4 | réduit du quota de l'agent sortant                  |
| Sortie non planifiée (arrêt)                | $\Theta(n)$ après expiration du délai de session            | 2 ou 4 | nul sur les partitions orphelines jusqu'à détection |

Les plafonds réels se lisent dans les systèmes qui les ont rencontrés. Un ordonnanceur de grappe éprouvé gère des centaines de milliers de tâches issues de milliers d'applications, sur les cellules dont le §2.1 donne la taille médiane [39]. Le journal réparti vise, lui, la gestion de centaines de milliers voire de millions de partitions, au titre que le §6.1 qualifie [44]. Ces deux nombres bornent la population utile par le haut, mais la borne qui mord en premier est ailleurs et elle est structurelle : la partition est la seule unité de parallélisme du côté consommateur, donc un groupe de consommation ne tire aucun bénéfice au-delà de  $p$  membres. Le  $(p + 1)$ -ième agent est inactif, il participe néanmoins à chaque rééquilibrage et il consomme sa part du  $\Theta(n)$  du tableau. Un essaim qui monte en agents sans monter en partitions n'augmente pas son débit, il augmente son coût de coordination — c'est le terme  $\sigma$  de la loi universelle du §2.1 qui grossit pendant que le numérateur stagne.

L'algorithme 2.2 énonce le contrôleur qui règle  $n$ . Modèle de panne : arrêt pour les agents et pour le contrôleur lui-même ; hypothèse de synchronisme : partiellement synchrone, avec un délai d'établissement  $T_a$  d'un nouvel agent qui est mesuré et non supposé. Règle de conception préalable : le contrôleur n'a autorité sur aucun invariant de sûreté, de sorte que son arrêt gèle la population sans la corrompre. La forme de base est celle du contrôleur horizontal éprouvé, dont le §4.3 donne la formule et les quatre constantes [45].

**Algorithme 2.2** – contrôleur d'élasticité : régler  $n$  sous budget de renouvellement

paramètres : cible  $c$ , tolérance  $\tau = 0,1$ , période  $T = 15$  s, budget de churn  $\beta$  en agents/s,

délai d'établissement mesuré  $T_a$ , fenêtre de mesure  $W \geq 2 \cdot T_a$ ,  
durée d'un rééquilibrage  $R$ , plafond structurel  $p$

```

1  à chaque période  $T$  :
2     $m \leftarrow$  agrégat du décalage converti en secondes, sur la fenêtre  $W$ 
3     $r \leftarrow m / c$ 
4    si  $|r - 1| \leq \tau$  : ne rien faire ; retourner en 1           // zone morte anti-
```

```

oscillation
5       $n^* \leftarrow \lfloor n \times r \rfloor$  ;  $n^* \leftarrow \min(n^*, p)$  // plafond : p
partitions
6       $\Delta n \leftarrow n^* - n$ 
7      si  $\Delta n > 0$  :  $\Delta n \leftarrow \min(\Delta n, \lfloor \beta \cdot T \rfloor)$ 
8      si  $\Delta n < 0$  :
9          si (maintenant - dernierRetrait) < W : retourner en 1 // hystérésis de
descente
10      $\Delta n \leftarrow \max(\Delta n, -\lfloor \beta \cdot T \rfloor)$ 
11     pour chaque agent retiré : cesser de lire ; valider le décalage ;
12     écrire le point de reprise ; quitter le groupe
13     déclencher le rééquilibrage ; attendre au plus  $T_a + R$ 
14      $n \leftarrow n + \Delta n$  ; retourner en 1

```

Le coût par cycle est d'une lecture d'agrégat, dont le coût en messages relève de la télémétrie et donc du ch. 6, plus 2 tours de rééquilibrage par changement effectif dans le protocole d'origine et 4 dans le protocole coopératif [43]. La condition d'arrêt n'est pas la terminaison de la boucle, qui est perpétuelle, mais l'atteinte de son point fixe  $|r - 1| \leq \tau$ . Ce point fixe n'est atteignable que si la métrique décroît quand  $n$  croît, ce qui est faux au-delà de  $p$  — la ligne 5 le borne — et faux au-delà du maximum  $u^*$  de la loi universelle, que la ligne 5 ne borne pas. Trois modes de défaillance suivent. Premièrement, l'oscillation : si  $W < 2 \cdot T_a$ , le contrôleur mesure l'effet d'une décision qu'il n'a pas fini d'appliquer, conclut à l'insuffisance et redouble ; la population diverge en dents de scie sans qu'aucun agent n'ait fauté. Deuxièmement, la tempête de rééquilibrages : chaque changement coûte  $R$  secondes de débit dégradé, donc un contrôleur dont la période  $T$  est du même ordre que  $R$  passe son temps à se réorganiser ; la condition à écrire est  $\beta \cdot T \cdot R \ll T$ , soit  $\beta \cdot R \ll 1$ , et elle est la vraie raison d'être du budget de churn des lignes 7 et 10. Troisièmement, la métrique trompeuse : les autoscaleurs de nœuds ne considèrent que les requêtes de ressources déclarées et jamais l'utilisation réelle, de sorte qu'un dimensionnement erroné des requêtes rend l'ajustement structurellement faux [46] ; piloter un essaim sur l'occupation processeur plutôt que sur le décalage revient à mesurer une grandeur qui sature après le goulot réel. Le réglage effectif de  $\tau$ , de  $T$  et de  $\beta$  sur une plateforme donnée n'est pas traité ici — il appartient au ch. 6.

Le cycle de vie d'un agent doit distinguer trois états que le contrôleur ne distingue pas. Les sondes de démarrage, de vivacité et de disponibilité ont des effets disjoints : la première bloque les deux autres jusqu'à son succès, la deuxième redémarre le conteneur, la troisième retire seulement le membre des points d'accès sans le redémarrer, aux cinq valeurs par défaut que le §6.1 donne, dont une période de 10 s et un seuil d'échec de 3 [47]. Un agent en vidange qui, aux valeurs par défaut, échoue à trois sondes de vivacité consécutives est tué en une trentaine de secondes en pleine validation de décalage : la ligne 11 de l'algorithme 2.2 doit donc faire chuter la disponibilité avant que la vivacité ne chute, faute de quoi l'essaim redémarre les agents qu'il est en train de retirer proprement, et chaque redémarrage rouvre un rééquilibrage. Le budget de destruction, lui, n'est pas une garantie : les perturbations involontaires comptent contre le budget sans pouvoir être empêchées, la suppression directe

d'un membre contourne le budget, et les mises à jour progressives ne sont pas limitées par lui mais par la spécification du contrôleur de charge [48]. Un plafond de destructions simultanées protège contre l'exploitation, pas contre la panne.

La transposition depuis la robotique conserve exactement ce que le champ moyen promet : concevoir sur des fractions, obtenir un contrôleur dont la forme ne dépend pas de  $n$ , et accepter des garanties démontrables sur la dynamique collective plutôt que sur l'individu [22]. Elle casse sur deux points, et les deux sont chiffrés dans la littérature d'origine. Le premier est l'interférence : les groupes de robots à contrôle décentralisé inspiré des fourmis fourragent plus efficacement qu'un robot seul et maintiennent un niveau d'énergie de groupe supérieur, mais le gain d'efficacité diminue dans les grands groupes, probablement par interférence de fourragement, phénomène également observé chez les insectes sociaux [7] ; l'essaim ne monte pas en charge linéairement, et le fait négatif est déjà dans la source biologique. Le second est plus grave pour le logiciel. Le champ moyen suppose des agents échangeables, condition sous laquelle les fractions d'états constituent une statistique suffisante ; les modèles micro-macro validés sur des équipes allant jusqu'à 600 robots reposent sur cette échangeabilité [18]. La possession d'une partition la détruit : savoir que 40 % des agents sont en traitement ne dit pas si la partition 17 a un propriétaire, et c'est cette information-là qui décide de la vivacité du sujet. L'abstraction qui rend la robotique en essaim insensible à la taille est précisément celle qu'un journal partitionné interdit. On peut piloter la taille de la population en champ moyen ; on ne peut pas piloter l'allocation ainsi, et le ch. 5 traite de ce que coûte l'allocation quand on refuse d'en décider centralement.

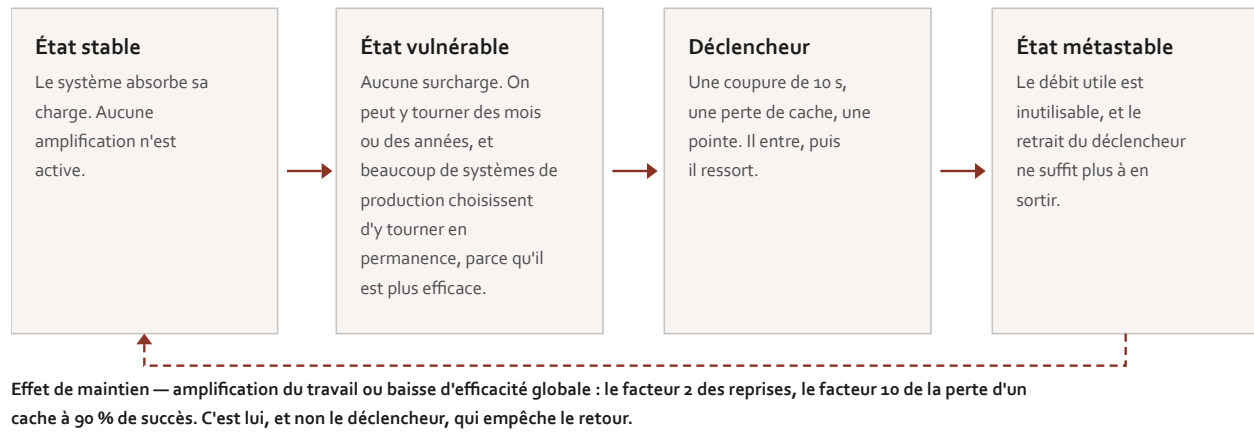
L'interférence a depuis reçu une mesure logicielle, et elle a la forme annoncée par la source biologique. Des essaims d'agents de langage chargés de construire ensemble un même logiciel — chacun sur sa propre machine, avec un forum et un dépôt partagés, pendant douze heures — sont observés sur deux grandeurs : la fraction des demandes de fusion effectivement fusionnées, et le partage de code, défini comme la proportion d'un fichier écrite par d'autres que celui qui y contribue. Les deux ne montent pas ensemble. Les générations de modèles les plus anciennes touchent les mêmes fichiers et ne fusionnent presque rien, leurs demandes entrant en conflit puis étant abandonnées ; les générations suivantes obtiennent une fraction de fusion élevée en cessant de partager, chaque agent conservant une propriété très forte sur ses fichiers ; une seule génération éprouvée tient les deux à la fois [119]. La variation de la taille de l'essaim et trois formulations de consigne — équipes libres, rôles prescrits, hiérarchie à agent directeur — ne déplacent pas sensiblement le résultat [119]. C'est la courbe de [7] avec sa cause changée : l'interférence n'est plus spatiale mais éditoriale, et la population l'évite en se cloisonnant, c'est-à-dire en renonçant au bénéfice pour lequel elle avait été réunie. Le §2.1 avait établi que le milieu est le meilleur candidat au titre de SPOF ; ce résultat en ajoute un second, moins visible : la ressource disputée peut être le produit lui-même, et aucun rééquilibrage n'a prise sur celle-là.

## 2.3 Gestion de la charge et congestion : contention des messages, routage distribué et gestion des goulots d'étranglement logiques

Le milieu ne perd pas les messages, il les accumule. Cette propriété, qui fait la valeur du journal, supprime du même coup le signal de congestion sur lequel repose un demi-siècle de contrôle de flux réseau. Il n'y a pas de perte à observer, pas de tampon qui déborde, pas d'événement discret qui dise « trop ». La grandeur qui remplace la perte est le décalage, converti en temps : la distance entre la tête d'écriture d'une partition et la position du consommateur, divisée par le débit de consommation. Cette conversion est ce qui rend le signal comparable à un budget de latence, et la latence de queue  $\ell_q$ , en est la mesure, la moyenne n'étant pas informative : les latences au 99,9<sup>e</sup> centile sont mesurées comme supérieures d'un ordre de grandeur aux moyennes et suivent le taux de requêtes [49].

Le modèle de panne de cette section est particulier et doit être posé avant tout mécanisme : aucun agent ne s'arrête, aucun canal n'omet, aucune partition réseau ne survient. Tous les composants fonctionnent conformément à leur spécification, et le système échoue quand même. La défaillance est un état de la population, atteint depuis un état sain par un déclencheur identifiable, et c'est ce qui la maintient dans le périmètre de ce chapitre plutôt que dans celui du ch. 7. Le cadre le plus précis dont on dispose est celui des défaillances métastables : elles surviennent dans des systèmes ouverts, à source de charge non contrôlée, où un déclencheur (*trigger*) fait entrer le système dans un mauvais état qui persiste même après le retrait du déclencheur ; dans cet état, le débit utile est inutilisable et un effet de maintien (*sustaining effect*) — impliquant souvent une amplification du travail ou une baisse d'efficacité globale — empêche d'en sortir ; sortir d'un état de défaillance métastable exige une poussée corrective forte, comme un redémarrage du système ou une réduction drastique de la charge [50]. Le point le plus contre-intuitif de ce cadre est l'état vulnérable, qui précède la défaillance : il n'est pas un état de surcharge, un système peut y tourner des mois ou des années puis s'y bloquer sans aucune augmentation de charge, et de nombreux systèmes de production choisissent d'y tourner en permanence parce qu'il est bien plus efficace que l'état stable [50]. Les exemples sont chiffrés. Une base répondant sous 100 ms en deçà de 300 requêtes/s et dont la latence devient supérieure d'un ordre de grandeur au-delà, interrogée par une application qui réemet toute requête sans réponse après 1 s, tient à 280 requêtes/s ; une coupure de 10 s sur le commutateur suffit à faire entrer le système en état métastable, où le trafic client reste à 560 requêtes/s par le seul effet des reprises, et la sortie exige de ramener la charge sous 150 requêtes/s ou de limiter les reprises à moins de 20 requêtes/s [50]. Un cache à 90 % de taux de succès dont le contenu est perdu produit une amplification de requêtes de  $10 \times$  [50]. Le rapport de causalité est explicite dans la source : une panne métastable est habituellement imputée à son déclencheur, alors que la cause réelle est l'effet de maintien.

## LE DÉCLENCHEUR N'EST PAS LA CAUSE



Une panne métastable est habituellement imputée à son déclencheur, alors que la cause réelle est l'effet de maintien. En sortir exige une poussée corrective forte : dans l'exemple chiffré du cadre, un système qui tient à 280 requêtes/s y reste bloqué à 560 requêtes/s par le seul effet des reprises, et la sortie exige de ramener la charge sous 150 requêtes/s [50]. Aucun agent n'a fauté, aucun canal n'a omis.

**Figure 2.3** — La défaillance métastable : le déclencheur entre et ressort, l'effet de maintien reste.

L'algorithme 2.3 vise l'effet de maintien et non le déclencheur. Modèle de panne : celui qui vient d'être posé — aucun arrêt, aucune omission, saturation seule. Hypothèse de synchronisme : partiellement synchrone, avec un temps mort de boucle égal à au moins un intervalle de validation de décalage, donc de l'ordre de la seconde et non de la microseconde. Les paramètres viennent de pratiques documentées : étranglement adaptatif côté client, qui rejette localement de façon probabiliste dès que requêtes >  $K \times$  acceptations avec  $K$  typiquement égal à 2 sur une fenêtre glissante de deux minutes ; budget de reprises plafonné à 3 tentatives par requête et à 10 % du volume total de requêtes d'un client ; quatre niveaux de criticité nommés CRITICAL\_PLUS, CRITICAL, SHEDDABLE\_PLUS et SHEDDABLE [51]. L'estimation de la file au goulot reprend la forme utilisée par les limiteurs de concurrence adaptatifs,  $L \times (1 - \min\text{RTT}/\text{RTT\_échantillon})$  [52].

Algorithme 2.3 – endiguement de la congestion : délestage par classe et repli

état par agent : requêtes, acceptations (fenêtre glissante de 120 s), limite  $L$ ,  $\min\text{RTT}$   
paramètres :  $K = 2$ , tentatives  $\leq 3$ , plafond de reprises = 10 % du volume, seuils  $\alpha < \beta$

```

1  avant d'émettre vers une dépendance :
2      p_rejet ← max(0, (requêtes - K·acceptations) / (requêtes + 1))
3      si aléa() < p_rejet : rejeter localement, sans émettre ; compter un délestage
4      sinon : requêtes ← requêtes + 1 ; émettre
5  à la réception d'une réponse : acceptations ← acceptations + 1 ; échantillonner le
RTT
6  à chaque fenêtre de contrôle :
7      file ←  $L \times (1 - \min\text{RTT} / \text{RTT\_échantillon})$ 
8      si file <  $\alpha$  :  $L \leftarrow L + 1$  ; sinon si file >  $\beta$  :  $L \leftarrow L - 1$ 
9  sur échec, tentative  $i \leq 3$  :
10     attendre un délai tiré uniformément dans  $[0, \min(\text{plafond}, \text{base} \cdot 2^i)]$  // gigue

```

```

pleine
11      si reprises_cumulées > 0,10 × requêtes_cumulées : abandonner vers la file de
rebut
12  si le décalage croît sur 3 fenêtres consécutives :
13      délester par criticité croissante : SHEDDABLE, puis SHEDDABLE_PLUS, puis
CRITICAL
14      ne jamais délester CRITICAL_PLUS ; si cela ne suffit pas, l'essaim est sous-
dimensionné

```

Le coût de la ligne 3 est de 0 message et 0 tour, et c'est la seule propriété qui compte : le refus est local, donc le signal de congestion ne consomme pas la ressource congestionnée. Un refus émis par le courtier consommerait, lui, exactement ce qui manque. Le coût des lignes 6 à 8 est de 0 message additionnel puisque le RTT est mesuré sur le trafic existant. La condition d'arrêt du délestage est l'arrêt de la croissance du décalage sur une fenêtre ; si la ligne 14 est atteinte, le diagnostic est de capacité et non de politique, et aucune politique ne le corrigera. Les modes de défaillance sont trois. Le premier est la synchronisation involontaire : tous les agents calculent le même  $p_{\text{rejet}}$  à partir du même signal et relâchent ensemble, produisant une vague ; c'est la gigue de la ligne 10 qui les décorrèle, et dans la simulation de référence — 100 clients en contention, délais réseau moyens de 10 ms avec 4 ms de variance — la variante à gigue pleine minimise le travail total et réduit le nombre d'appels de plus de moitié par rapport au repli exponentiel sans gigue [53]. Ces valeurs proviennent d'une simulation décrite dans un billet d'ingénierie, sans tableau numérique ; elles indiquent un ordre de grandeur et pas une mesure de production. Cette décorrélation porte sur l'instant et non sur la cible, et c'est l'hypothèse que la ligne 10 ne dit pas : la gigue tire un délai par agent, elle ne tire pas un serveur, de sorte qu'une population qui calcule le même  $p_{\text{rejet}}$  relâche à des instants différents vers le même point — la vague est étalée, non défaite. Le deuxième mode est la mémoire de la fenêtre : après une augmentation réelle de capacité, la fenêtre glissante de 120 s continue de faire rejeter pendant deux minutes. Le troisième est l'inefficacité du chemin d'erreur, qui est un effet de maintien à part entière : le chemin nominal est optimisé, le chemin d'échec capture une pile d'appels, résout un nom, écrit sur disque et émet vers un service de journalisation, si bien que si le déclencheur épuise une ressource utilisée par le traitement d'erreur, le traitement d'erreur aggrave la pénurie [50]. L'analyse ci-dessus suppose une dépendance unique ; le cas de plusieurs dépendances corrélées, où le délestage d'une file en remplit une autre, n'est pas traité ici.

**Tableau 6** — Correspondance des signaux de congestion entre le contrôle de flux réseau et un essaim sur journal partagé.

| Signal              | Ce qu'il vaut sur un réseau              | Ce qu'il devient dans l'essaim                                           | Ce que la transposition casse                         |
|---------------------|------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------|
| Perte de paquet     | événement discret, gratuit, immédiat     | inexistant : le journal accumule jusqu'à la rétention, puis perd en bloc | la défaillance n'est plus graduelle mais falaise      |
| Accusé auto-cadencé | signal et permission d'émettre à la fois | absent : le milieu accepte tout ce qu'on lui donne                       | il n'y a pas de chemin de retour naturel ; la contre- |

| Signal               | Ce qu'il vaut sur un réseau    | Ce qu'il devient dans l'essaim                | Ce que la transposition casse                             |
|----------------------|--------------------------------|-----------------------------------------------|-----------------------------------------------------------|
|                      |                                |                                               | pression doit être construite                             |
| Délai aller-retour   | mesurable par connexion, en ms | mesurable par appel synchrone seulement       | ne renseigne pas sur la congestion du milieu              |
| Occupation de tampon | profondeur de file au goulot   | décalage converti en secondes                 | temps mort d'au moins un intervalle de validation         |
| Taux de rejet local  | absent                         | requêtes / acceptations sur fenêtre glissante | dépend d'une fenêtre, donc retarde le retour à la normale |

Ce que la transposition du contrôle de congestion conserve tient en trois éléments. La forme additive-multiplicative : croissance d'environ un segment plein par temps d'aller-retour en évitement de congestion, et division par deux du seuil sur expiration du temporisateur,  $ssthresh = \max(FlightSize/2, 2 \times SMSS)$  [54] — la décroissance multiplicative moins agressive de CUBIC, avec  $\beta = 0,7$  et une fonction de croissance  $W(t) = C \cdot (t - K)^3 + W\_max$  où  $C = 0,4$ , montre que le facteur est un réglage et non une loi [55]. La nécessité d'une décorrélation par gigue. Et l'idée d'estimer la file au goulot plutôt que d'attendre la perte, dont la critique la plus nette est que l'assimilation de la perte de paquet à la congestion était défendable dans les années 1980 et ne l'est plus, avec ses deux modes de défaillance nommés : engorgement des tampons quand ils sont grands, sous-utilisation quand ils sont petits et que des pertes non congestives sont mal interprétées [56]. Ce que la transposition casse est plus important que ce qu'elle conserve. Un accusé de réception TCP est à la fois un signal et une permission d'émettre, ce qui donne au protocole son auto-cadencement ; entre un agent producteur et un agent consommateur séparés par un journal durable, aucun message ne remonte, et le mot contre-pression est employé à tort tant qu'un chemin de retour n'a pas été explicitement construit. Le temps mort de la boucle passe de la centaine de microsecondes à la seconde, et les constantes de [54] et [55], calibrées sur des temps d'aller-retour réseau, ne peuvent pas être reprises telles quelles sans rendre la boucle instable.

Le goulot d'étranglement logique est le dernier objet de la section, et le plus résistant. Le résultat de référence sur le routage aléatoire porte sur le modèle du supermarché : les clients arrivent en flot de Poisson à taux  $\rho n$  sur  $n$  serveurs avec  $\rho < 1$ , chacun choisit  $c$  serveurs uniformément au hasard avec remise et attend au plus court, le service est exponentiel de moyenne 1, et l'analyse porte sur le système limite quand  $n$  tend vers l'infini — la source note  $\rho$  par  $\lambda$  et  $c$  par  $d$  [57]. Au point fixe, la fraction de files de longueur au moins  $i$  vaut  $\rho^i / (c^i - 1)$  pour  $c \geq 2$ , décroissance doublement exponentielle, contre  $\rho^i$  pour  $c = 1$ , décroissance seulement géométrique ; le temps de séjour vaut  $T_1 = 1/(1 - \rho)$  pour  $c = 1$ , alors que pour  $c \geq 2$  il croît comme  $\log T_1$ , avec  $\lim_{c \rightarrow \infty} T_c / \log T_1 = 1/\log c$  quand  $\rho$  tend vers 1 par valeurs inférieures ; et la plus longue file d'un système initialement vide vaut  $\log \log n / \log c + O(1)$  avec forte probabilité [57]. La conclusion de la source est aussi une limite : deux choix apportent une amélioration exponentielle sur un seul, tandis que trois choix

ne gagnent qu'un facteur constant sur deux [57]. La transposition conserve ce résultat intégralement pour l'affectation des agents à un gisement de travail sans état : sonder deux candidats suffit, sonder davantage est du gaspillage. Elle casse totalement pour l'affectation des événements aux partitions, et la raison est sémantique et non statistique. Le théorème exige que le client soit libre de choisir son serveur ; dans un journal partitionné, la partition est imposée par la clé, parce que l'ordre par partition est le seul ordre garanti, voir ch. 1. Router un événement vers la partition la moins chargée détruit l'ordre par clé, c'est-à-dire la propriété même qui justifiait le partitionnement. Le hachage cohérent ne résout pas davantage ce cas : il a été conçu pour résorber les points chauds de caches web dans des réseaux où aucun serveur ne peut connaître l'état complet du réseau, et son détournement vers le partitionnement de données est postérieur [35] ; il répartit des clés, il ne répartit pas une clé. Une clé unique et chaude est un goulot logique qu'aucun routage ne dissout. Il ne reste que deux issues, et elles se paient toutes les deux : saler la clé et réordonner en aval, ce qui achète du débit contre de l'ordre, ou accepter la sérialisation et la compter comme une augmentation du  $\sigma$  de la loi universelle du §2.1, avec le maximum  $u^*$  qu'elle implique.

Le verdict pratique contredit ce que la section a de plus élégant. Les mécanismes qui préviennent le plus grand nombre d'effondrements documentés ne sont ni l'estimation de file ni le routage adaptatif : ce sont le plafonnement des reprises à 3 tentatives et à 10 % du volume, et le refus local avant émission [51], parce qu'ils suppriment exactement les deux amplifications — le facteur 2 des reprises et le facteur 10 de la perte de cache — qui constituent la totalité des exemples chiffrés du cadre métastable [50]. À l'inverse, la bibliothèque de référence en contrôle adaptatif de concurrence ne cite aucun banc d'essai mesuré ni aucun article, et ses affirmations de débit optimal à latence optimale sont des assertions de conception [52]. Un essaim se protège d'abord en s'interdisant d'amplifier son propre travail.

Cette amplification a été observée sur une population d'agents de langage, et son déclencheur n'est ni une coupure ni une perte de cache : c'est l'absence de tout moyen de s'accorder. Des agents chargés de faire passer leurs travaux dans un système à bande passante finie, sans autre canal de coordination, convergent tous vers le même moyen — un démon de sondage à trente interrogations par seconde — et une exécution enregistre 2,4 millions de demandes pour 117 acceptations [119]. Le rapport est de plus de vingt mille pour un, et aucun de ces agents n'est en panne — exactement le modèle de panne de cette section. Deux enseignements s'en tirent, et le second contredit une intuition répandue. Le premier est que l'effet de maintien peut être engendré par la population elle-même : le sondage y est à la fois le travail inutile et la cause de la lenteur qui le justifie, et l'état vulnérable de [50] n'a même pas eu à être atteint par la charge, puisque la charge est le comportement. Le second est que le remède n'est pas d'abord un plafond imposé de l'extérieur mais un endroit où les agents peuvent convenir d'un protocole — un forum partagé suffit à faire tomber le sondage —, ce qui est la thèse de l'ouvrage rencontrée par une voie inattendue : le milieu

n'est pas seulement ce par quoi l'essaim coordonne, il est ce qui l'empêche de s'effondrer sur une décision que tous prennent en même temps. Que ce remède opère dépend toutefois des motivations données aux agents et de la propension du modèle sous-jacent à collaborer [119] ; le ch. 8 montre le même forum servant à s'entendre sur un prix.

### 3. Modélisation formelle et méthodes de conception

#### 3.1 Cadres mathématiques : systèmes dynamiques, processus stochastiques et théorie des graphes appliqués aux systèmes d'agents

Trois familles de modèles se partagent la description formelle d'un essaim — systèmes dynamiques, processus stochastiques, théorie des graphes — et chacune achète son pouvoir prédictif au prix d'une hypothèse que le logiciel ne satisfait pas toujours. Un modèle n'est utilisable en conception qu'à quatre conditions : nommer le modèle de panne sous lequel il vaut, l'hypothèse de synchronisme qu'il impose, le coût en messages et en tours du mécanisme qu'il décrit, et la condition sous laquelle il cesse de valoir. « Aucune panne n'est modélisée » est une réponse recevable ; l'omettre ne l'est pas, car c'est elle qui fixe le domaine de validité.

Le premier cadre part d'un modèle de particules autopropulsées :  $n$  particules se déplacent à vitesse constante en adoptant à chaque pas la direction moyenne de leurs voisines, avec un bruit additif [11]. Les simulations révèlent une transition de phase cinétique par brisure spontanée de symétrie rotationnelle : sous un bruit critique  $\eta_c$  le transport collectif est ordonné, au-dessus le flux net est nul, la transition étant continue avec une vitesse ordonnée en  $(\eta_c - \eta)^\beta$ ,  $\beta \approx 0,45$  [11]. Le résultat est numérique, et il l'est resté huit ans.

La preuve est venue d'une réécriture. Le modèle s'exprime comme un système linéaire commuté de dimension  $n$  dont le signal de commutation prend ses valeurs dans les indices d'une famille finie de graphes simples sur  $n$  sommets, les matrices commutées étant stochastiques par lignes et à diagonale non nulle [20]. Les hypothèses se posent avant l'énoncé, car c'est là que la transposition se joue : temps discret, mises à jour simultanées — synchronisme parfait — et modèle de panne vide. Les deux théorèmes de convergence qui en découlent — connexité du graphe à chaque instant dans le premier cas, connexité conjointe sur une suite infinie d'intervalles contigus et bornés dans le second — sont énoncés avec leurs hypothèses exactes au ch. 1. Ce qui importe ici est leur forme : aucun graphe pris isolément n'a besoin d'être connexe, et les auteurs laissent ouverte la variante qu'ils auraient préférée, des intervalles bornés non chevauchants mais non contigus [20].

Trois faits négatifs accompagnent ce résultat. La preuve ne passe pas par le certificat de stabilité usuel : les auteurs établissent par programmation semi-définie qu'il n'existe aucune fonction de Lyapunov quadratique commune pour la classe des matrices dont les graphes ont 10 sommets et sont connexes, la vérification portant sur les graphes à 9 ou 10 arêtes [20] ; faute d'une telle

forme, aucun scalaire monotone ne mesure la distance restant à parcourir. La convergence est ensuite garantie sans borne de temps : l'énoncé est une limite quand  $t$  tend vers l'infini, pas un délai — une propriété de vivacité (*liveness*) au sens strict, dont le §3.2 tire les conséquences. Enfin la valeur limite n'est pas la moyenne : les matrices de Perron employées ici sont de la forme  $(I + D)^{-1}(I + A)$ , les agents n'atteignent en général pas un consensus de moyenne, et la décision de groupe est une quantité indéterminée dans l'enveloppe convexe des états initiaux [29]. L'essaim s'aligne sur une valeur que le concepteur ne choisit pas.

L'algorithme 3.1 pose la version discrète du mécanisme sous la forme où ses coûts se comptent. Pour ne pas empiéter sur les symboles réservés, les valeurs propres du laplacien  $y$  sont notées  $0 = v_1 \leq v_2 \leq \dots \leq v_n \leq 2\Delta(G)$ ,  $v_2$  étant la connectivité algébrique, et le pas de l'itération  $\alpha$ .

Algorithme 3.1 – itération de consensus linéaire dans un maillage d'agents

Modèle de panne : vide.  $n$  agents tous vivants publiant à chaque tour ;  
ni crash-arrêt, ni omission, ni partition réseau [20][29].  
Synchronisme : parfait. Tous lisent le tour  $k$  et écrivent le tour  $k+1$  ;  
lire un état antérieur à  $k$  est hors modèle.  
Topologie : digraphe  $G$ , degré maximal  $\Delta(G)$ , poids  $a_{ij} \geq 0$  sur les  
voisins entrants ; pas  $\alpha$ , avec  $0 < \alpha < 1/\Delta(G)$ .

1.  $x_i \leftarrow$  valeur initiale ;  $k \leftarrow 0$ .
2. Publier  $x_i(k)$  aux voisins sortants ; lire  $x_j(k)$  des entrants.
3.  $x_i(k+1) \leftarrow x_i(k) + \alpha \cdot \sum_j a_{ij} \cdot (x_j(k) - x_i(k))$  [29].
4. Arrêt : écart maximal aux voisins observés  $\leq \varepsilon$  pendant  $R$  tours –  
test local d'une propriété globale (mode d).

Coût par tour : un message par arête orientée, soit  $\theta(n \cdot \bar{d})$  messages et  
 $\Omega(n \cdot \bar{d} \cdot b)$  octets ;  $\theta(n^2)$  sur maillage complet.

Coût en tours :  $P = I - \alpha L$  contracte le désaccord du deuxième plus grand  
module de ses valeurs propres ; atteindre  $\varepsilon$  demande  
 $\ln(1/\varepsilon)$  divisé par le logarithme de l'inverse de ce  
module. Sur digraphes équilibrés fortement connexes  
commutant arbitrairement, la vitesse est minorée par la  
plus petite connectivité algébrique rencontrée, et le  
carré de la norme du désaccord est une fonction de  
Lyapunov commune [29]. Sous connexité conjointe seule,  
aucune borne : 1, 2, ...,  $\infty$  tours [20].

Modes de défaillance

- a) À  $\alpha = 1/\Delta(G)$ ,  $P$  reste irréductible mais porte  $n$  valeurs propres sur  
le bord du cercle unité et l'itération cesse de converger – erreur  
que les auteurs disent répétée dans la littérature antérieure [29].
- b) Le vecteur unité n'est vecteur propre à gauche du laplacien que si  
le digraphe est équilibré [29]. Sinon la moyenne initiale n'est plus  
invariante : la limite est pondérée par la topologie, non par les  
données.
- c) En temps continu, sous retard uniforme d'un saut  $\tau$ , le consensus de  
moyenne est atteint si et seulement si  $0 \leq \tau < \pi/(2 \cdot v_n)$ , et

- $\tau < \pi/(4\Delta(G))$  suffit puisque  $v_n < 2\Delta(G)$  [29]. La condition est nécessaire : au-delà, le système ne converge pas.
- d) Crash-arrêt, hors modèle : l'agent mort cesse de publier, ses voisins lisent un état figé, et l'étape 4 confond mort et convergence.

La transposition conserve la structure de la preuve et casse trois de ses hypothèses. Rien dans l'argument des produits de matrices stochastiques ne dépend de la physique : il vaut pour  $n$  agents qui moyennent l'état publié par un sous-ensemble de pairs. Mais le voisinage géométrique est symétrique, puisque la distance l'est, alors qu'un abonnement à un sujet est orienté ; la mise à jour simultanée n'existe pas au-dessus d'un journal ; et le modèle de panne vide est l'inverse de celui de l'ouvrage. La première brèche est traitée : les digraphes équilibrés sont fondamentaux pour le consensus de moyenne [36]. La deuxième coûte le plus cher, et le mode (c) en donne la mesure. Le verdict des auteurs est net : degré maximal élevé et robustesse aux retards s'excluent, et construire un réseau d'ingénierie à nœuds de haut degré est contre-indiqué pour atteindre un consensus [29]. Or un sujet lu par tous est un moyen : les agents  $y$  forment un graphe logique complet,  $\Delta(G) = n - 1$ , et la borne suffisante devient  $\tau < \pi/(4(n - 1))$  — à  $n = 100$ ,  $7,933 \times 10^{-3}$  unité de temps du protocole, budget qui décroît en  $1/n$ . L'arrondi de  $\pi/(4n)$  donnerait  $7,9 \times 10^{-3}$ , que la borne réellement écrite dépasse de 0,42 % : un arrondi présenté comme une borne stricte est un énoncé faux, non une imprécision. Le milieu offre gratuitement l'échange non local dont le même article établit qu'il accélère substantiellement la convergence [29] ; il offre du même geste la topologie que cet article déconseille.

Le deuxième cadre traite l'essaim comme un processus stochastique, à deux niveaux de coût très inégal. Au niveau microscopique, l'état joint de  $n$  agents à  $|S|$  états locaux compte  $\Theta(|S|^n)$  configurations : exact, calibrable, impraticable au-delà d'une poignée d'agents. Au niveau macroscopique, le champ moyen approche la population par une densité ; la synthèse du domaine en distingue trois types — équations différentielles ordinaires, aux dérivées partielles, aux différences — selon le caractère discret ou continu des états d'agent et du temps ; leur avantage décisif est d'être indépendants de la taille de l'essaim, ce qui rend la conception du contrôle passable à l'échelle et autorise des garanties démontrables sur la dynamique collective [22]. Le procédé vient d'un cadre où un grand nombre d'agents rationnels, chacun optimisant sur une information agrégée, produit un système d'équations différentielles non linéaires bien posé et à solutions uniques [58].

Le mécanisme qu'on en tire a des coûts d'une forme inhabituelle, qu'il faut écrire pour ne pas le croire gratuit. Reformuler l'allocation de tâches comme une optimisation sur les taux d'entrée et de sortie de chaque tâche, qui définissent ensuite les politiques individuelles, donne une stratégie décentralisée sans aucune communication entre robots, validée sur 250 robots se redistribuant entre quatre structures [40]. Le coût en messages est nul, seul cas de la section ; le coût ne se compte pas en tours mais en temps de mélange de la chaîne de Markov des taux ; aucune condition d'arrêt locale n'existe, la politique tournant indéfiniment pendant que la population se stabilise en

distribution sans qu’aucun agent ne l’observe. Le modèle de panne est vide, et l’hypothèse structurante est la conservation de la population : un agent qui s’arrête emporte sa part sans qu’aucun taux ne la réémette. Le calibrage est ce que la littérature logicielle emprunte le moins et devrait emprunter le plus : phases d’abstraction successives — robots réels, simulation, modèle mathématique — avec correspondances définies entre niveaux et critères heuristiques interdisant l’introduction arbitraire de paramètres ; jusqu’à 600 robots, les modèles abstraits prédisent juste qualitativement et quantitativement pour un coût très inférieur à la simulation incarnée [18]. Quand la plateforme induit une dérive spatiale, une diffusion de Fokker-Planck couplée à des équations à états discrets établit que la distance au point de dépôt influence le temps de convergence [59].

L’hypothèse que paie le champ moyen s’énonce sans adoucissement : agents échangeables, interactions bien mélangées, résultat valable à la limite des grands  $n$ . Le modèle prédit une fraction de population, jamais la trajectoire d’un agent nommé, et rien de la queue de distribution, donc rien de  $\ell_{99}$ . L’échangeabilité tombe là où l’essaim devient utile : dès qu’un groupe de consommation affecte des partitions à ses membres, deux agents ne sont plus permutables. Le champ moyen décrit l’essaim avant l’allocation et cesse de s’appliquer après. La loi de Little borne le territoire :  $L = \lambda W$  suppose des moyennes finies, des processus strictement stationnaires et un processus d’arrivée métriquement transitif de moyenne non nulle [41], et ne dit rien du régime saturé — celui où un essaim entre quand la question devient intéressante, et qui appartient au ch. 2.

Une seconde condition, restée implicite tant que les agents étaient des programmes écrits à la main, doit être ajoutée à celle-là. Le champ moyen ne demande pas seulement des agents échangeables : il demande que leurs tirages se décorrèlent, faute de quoi la fraction de population dans un état donné ne se concentre autour d’aucune densité et reste un tirage unique répliqué  $n$  fois. Cette condition est habituellement gratuite : deux robots ne partagent pas leur source d’aléa. Elle cesse de l’être quand la fonction de transition est un modèle de langage servi avec la même invite et le même contexte : la fraction d’agents dans l’état  $j$  ne vaut plus une densité mais 0 ou 1, et la fluctuation relative en  $1/\sqrt{n}$  que le modèle prédit est simplement fausse. La mesure existe — dix-huit agents sur trente ouvrent une branche au même nom [119] —, et le ch. 8 en donne l’inventaire. Un modèle macroscopique appliqué à une telle population produit un nombre exact pour une population qui n’existe pas. La condition à écrire en tête de tout modèle de champ moyen d’un essaim d’agents de langage est donc double, échangeabilité et décorrélation effective, et la seconde se mesure au lieu de se supposer ; le ch. 8 en propose la grandeur.

Le troisième cadre est spectral, et il fixe le vocabulaire de coût du protocole épidémique (*gossip*), défini comme celui où chaque agent communique avec au plus un voisin par créneau [60]. Deux modèles de temps  $y$  sont distingués et n’ont pas le même prix : dans l’asynchrone, chaque agent porte une horloge de Poisson, un seul communique à un instant donné, et l’unité de compte est le tic ; dans le synchrone, tous communiquent simultanément, chacun avec un

seul voisin, ce qui contraint les paires actives à former un couplage du graphe et oblige l'algorithme à le construire de façon distribuée [60]. Le temps de moyennage dépend de la deuxième plus grande valeur propre d'une matrice doublement stochastique caractérisant l'algorithme ; concevoir le plus rapide revient à minimiser cette valeur propre, ce qui s'écrit comme un programme semi-défini, non résoluble de façon distribuée en général — d'où la méthode de sous-gradient distribuée des auteurs [60]. Les bornes de calcul sont données : points intérieurs jusqu'à un millier d'arêtes, sous-gradient jusqu'à cent mille, au prix d'une convergence lente et sans critère d'arrêt garantissant un niveau de sous-optimalité [60]. La même analyse relie le temps de moyennage au temps de mélange d'une marche aléatoire dérivée du protocole : sur un graphe géométrique aléatoire modélisant un réseau de capteurs à faible rayon, le calcul distribué est nécessairement lent, puisque le meilleur algorithme de moyennage l'est [60]. La transposition est ici favorable au logiciel : le graphe géométrique est imposé par la portée radio, la topologie d'un maillage d'agents est une variable de conception. Ce qu'elle casse est la gratuité de la matrice doublement stochastique, qui suppose un échange symétrique là où la relation producteur-consommateur ne l'est pas : la symétrie s'achète en payant un chemin de retour, qui se compte en messages et en rétention.

Porté sur un essaim logiciel, ce protocole s'écrit avec le même appareil que l'algorithme 3.1 et tolère ce que celui-ci ne tolère pas. Modèle de panne : crash-arrêt sans reprise et omission, aucune faute byzantine (voir ch. 7). Synchronisme : partiellement synchrone, une borne  $\Delta$  tenant après un instant de stabilisation inconnu, les tours étant logiques. Chaque agent tire un pair dans son échantillon de  $d$  pairs, lui émet la moitié de son couple valeur-poids et la retranche du sien ; le récepteur additionne ; l'estimateur est le rapport des deux ; l'agent s'arrête quand sa variation reste sous  $\varepsilon$  pendant  $R$  tours. Trois modes de défaillance, dont le premier est le plus traître. Sous partition réseau (*network partition*), chaque composante converge vers sa moyenne locale et la condition d'arrêt se déclenche des deux côtés : test local d'une propriété globale, elle ne distingue pas la convergence de la scission. Au premier crash-arrêt, la masse détenue disparaît ; la conservation de la somme, qui fonde la correction de l'estimateur, tombe, et le protocole converge encore — vers une autre valeur que la moyenne initiale. L'omission produit le même effet à l'insu de l'émetteur, et un accusé de réception ne restaure la conservation qu'à 2 messages par échange au lieu de 1.

**Tableau 7** — Coût et domaine de validité des trois mécanismes de coordination formalisés au §3.1.

| Mécanisme                              | Modèle de panne                    | Synchronisme | Messages par tour                                              | Tours jusqu'à $\varepsilon$                                                                       | Condition d'échec                                                                              |
|----------------------------------------|------------------------------------|--------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Itération linéaire de Perron [20] [29] | Vide ; tous publient à chaque tour | Parfait      | $\Theta(n \cdot \bar{d})$ ; $\Theta(n^2)$ sur maillage complet | $\ln(1/\varepsilon)$ sur le logarithme de l'inverse du module contractant ; 1, ..., $\infty$ sous | pas $\alpha \geq 1/\Delta(G)$ ; digraphe non équilibré ; retard $\tau \geq \pi/(2 \cdot v\_n)$ |

| Mécanisme                            | Modèle de panne                                                         | Synchronisme                                 | Messages par tour                            | Tours jusqu'à $\epsilon$                                                                                                            | Condition d'échec                                                                    |
|--------------------------------------|-------------------------------------------------------------------------|----------------------------------------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Agrégation épidémique [60]           | Crash-arrêt et omission tolérés pour la convergence, pas pour la valeur | Partiel : borne $\Delta$ après stabilisation | $\Theta(n)$ ; 1 par tic en modèle asynchrone | connexité conjointe seule<br>Fonction de la deuxième valeur propre de la matrice<br>doublement stochastique ; non bornée en général | Perte de masse dès la première panne ; partition réseau<br>indétectable localement   |
| Politique stochastique par taux [40] | Vide ; population supposée constante                                    | Aucun : chaîne de Markov, pas de tour        | 0                                            | Temps de mélange de la chaîne des taux ; aucune condition d'arrêt locale                                                            | Un agent arrêté emporte sa part ; échangeabilité perdue dès l'attribution nominative |

### 3.2 Spécification formelle : vérification de propriétés globales (convergence, stabilité, absence de blocage) à partir de règles individuelles

Une spécification d'essaim se heurte d'abord à une asymétrie logique, non à une difficulté d'outillage. Une propriété de sûreté (*safety*) est celle pour laquelle, si elle ne tient pas sur une exécution, il existe un instant identifiable où la mauvaise chose se produit, et cette mauvaise chose est irrémédiable : aucun prolongement ne la répare [61]. Une propriété de vivacité est celle dont aucune exécution partielle n'est irrémédiable : quelle que soit l'exécution finie déjà observée, il existe un prolongement infini qui la satisfait [61]. Toute propriété est l'intersection d'une propriété de sûreté et d'une propriété de vivacité [61]. Deux exigences serviront ici d'étiquettes, reprises telles quelles dans la suite. **S1** — deux agents ne détiennent jamais simultanément l'attribution de la même partition. **L1** — tout agent qui rejoint le groupe finit par recevoir une attribution. S1 se réfute par une exécution de trois secondes, et un moniteur peut lever l'alerte à l'instant de la violation. L1 n'est réfutée par aucune exécution finie : aucune campagne de tests ne peut en établir la fausseté et aucun moniteur ne peut signaler qu'elle est violée — au mieux, il signale qu'elle n'est pas encore satisfaite. Un cahier des charges qui mêle S1 et L1 sans les nommer produit des exigences dont la moitié est non testable.

**« Deux agents ne détiennent jamais simultanément l'attribution de la même partition. »**

Si elle ne tient pas, il existe un instant identifiable où la mauvaise chose se produit, et cette mauvaise chose est irrémédiable : aucun prolongement ne la répare.

Se réfute par une exécution de trois secondes.

Un moniteur peut lever l'alerte à l'instant de la violation.

Se démontre en asynchrone.

**Décidable sur un horizon borné : justiciable de la vérification bornée.**

**« Tout agent qui rejoint le groupe finit par recevoir une attribution. »**

Aucune exécution partielle n'est irrémédiable : quelle que soit l'exécution finie déjà observée, il existe un prolongement infini qui la satisfait.

N'est réfutée par aucune exécution finie, donc par aucune campagne de tests.

Aucun moniteur ne peut signaler qu'elle est violée — au mieux, qu'elle n'est pas encore satisfaite.

Exige au minimum le partiellement synchrone.

**Non décidable sur horizon fini : aucun échantillon ne la tranche.**

Toute propriété est l'intersection d'une propriété de sûreté et d'une propriété de vivacité [61]. Un cahier des charges qui mêle S<sub>1</sub> et L<sub>1</sub> sans les nommer produit des exigences dont la moitié est non testable — et dans un essaim asynchrone, aucune vivacité ne s'énonce inconditionnellement : elle porte un détecteur de défaillance, ou une hypothèse de synchronisme partiel, et écrire « l'essaim finit par converger » sans dire laquelle est le défaut de spécification le plus fréquent du domaine.

**Figure 3.2** — L'asymétrie que toute spécification d'essaim doit trancher : ce qu'une exécution finie peut réfuter, et ce qu'elle ne peut pas.

L'asymétrie suivante donne son titre à la section : la propriété se pose au niveau de la population, la règle s'écrit au niveau de l'individu. Le constat est formulé sans détour dans la littérature du domaine — spécifier les comportements émergents globaux en fonction des comportements de bas niveau des robots individuels est très difficile — et la voie explorée est la logique temporelle, sur une étude de cas d'essaim simplifié connecté sans fil [24]. La technique qui fait le lien est le model checking, algorithmique et exhaustif : elle vérifie si des propriétés temporelles sont satisfaites sur tous les comportements possibles du système, et elle a été appliquée à un algorithme de contrôle éprouvé sur des essais réels, la vérification servant à le raffiner autant qu'à l'analyser [25].

Trois bornes limitent ce programme, et aucune ne se contourne par plus de calcul. La première est arithmétique. L'espace d'états joint de  $n$  agents à  $|S|$  états locaux compte  $\Theta(|S|^n)$  configurations, et l'entrelacement des actions multiplie ce nombre par celui des ordonnancements admissibles. Pour  $|S| = 8$  et  $n = 12$ , on dépasse  $6,8 \times 10^{10}$  configurations avant tout entrelacement. La conséquence se lit dans les retours d'industrie : la vérification d'un algorithme de réplique et d'appartenance a exigé la version distribuée d'un vérificateur tournant sur dix instances à 8 cœurs plus hyperfils et 23 Go de mémoire vive chacune [62]. On ne vérifie pas un essaim de 12 500 agents, taille de référence posée ici comme hypothèse de dimensionnement — au même titre que le  $\delta = 1$  s du §7.2 — et reprise telle quelle aux §6.2, §6.3 et §7.3 ; on vérifie une instance assez grande pour donner confiance, ce qui n'est pas la même affirmation.

La deuxième borne interdit l'extrapolation qu'on aimerait tirer de la première, et elle mérite sa forme exacte plutôt qu'un résumé. Le problème de vérification paramétrée — établir qu'une propriété tient pour tout  $n$  — n'est pas

seulement coûteux : il n'est même pas semi-décidable lorsque chaque processus a un espace d'états fini, la relation en cause étant  $\Pi_2^0$ -complète [63], résultat que la littérature spécialisée prend comme point de départ obligé de toute étude de cas décidable [64]. Les hypothèses exactes comptent : programmes à états finis donnés schématiquement en  $n$ , et satisfaction d'une formule temporelle — le résultat vaut pour toute logique temporelle dont la validité est décidable, et sa démonstration n'utilise pas le parallélisme. Ce n'est pas une borne de complexité qu'un meilleur solveur ferait reculer, et vérifier l'essai à  $n = 5$  ne dit formellement rien de  $n = 5\,000$ .

Ce qui rend la borne exploitable, c'est qu'une classe décidable existe et que sa définition ressemble trait pour trait à un essaim stigmergique. Le modèle comporte un meneur et un nombre arbitraire de contributeurs anonymes et identiques, communiquant par un unique registre partagé à valeurs bornées ; chaque lecture et chaque écriture est atomique, mais aucune primitive ne permet d'exécuter atomiquement une séquence d'opérations, et le cas où tous les processus sont identiques est couvert en faisant du meneur un contributeur de plus [64]. Sous ces hypothèses, la vérification de sûreté pour tout nombre de contributeurs est coNP-complète si meneur et contributeurs sont des machines à états finis, PSPACE-complète s'ils sont des automates à pile, et reste coNP-complète pour des machines de Turing quelconques dont le nombre d'étapes est borné par un  $k$  donné en unaire [64]. Le point décisif est la raison de cette décidabilité : c'est la non-atomicité qui réduit substantiellement la complexité. Dans le cas atomique, où un contributeur peut lire un message et l'effacer dans la même action indivisible, le meneur peut distribuer des identités, et la sûreté devient au moins PSPACE-difficile pour les machines à états finis, indécidable pour les automates à pile [64]. La transposition conserve l'essentiel : lire sans consommer, régime d'un journal où le décalage d'un consommateur n'efface rien pour les autres, tombe du côté décidable ; lire en consommant, régime d'une file, tombe du côté indécidable. Elle casse une hypothèse qu'il faut nommer : le registre du modèle est unique et à valeurs bornées, alors qu'un journal est une séquence ordonnée non bornée. Le résultat vaut donc pour un essaim dont l'état partagé est un résumé borné — un compteur, un drapeau, une vue d'appartenance tronquée — et non pour un essaim qui relit l'historique complet. Hors de ce cadre, écrire « la propriété est vérifiée, donc elle vaut à l'échelle » reste une faute.

La troisième borne porte sur la vivacité elle-même. Un seul processus fautif, à l'arrêt et non byzantin, suffit à rendre impossible tout protocole de consensus binaire garantissant la terminaison en système asynchrone, sous des hypothèses que le §4.2 énonce [1]. C'est une impossibilité de terminaison, non de sûreté : S1 survit, L1 tombe. Le mécanisme d'accord et son prix appartiennent au ch. 4 ; ici compte la contrainte imposée au rédacteur d'une spécification : dans un essaim asynchrone, aucune propriété de vivacité ne s'énonce inconditionnellement, elle doit porter sa condition, et la littérature en fournit deux. La première est un détecteur de défaillance : caractérisé par exactement deux propriétés, complétude et exactitude, il rend le consensus soluble même en commettant une infinité

d'erreurs, l'algorithme fondé sur  $\diamond S$  exigeant une majorité de processus corrects, majorité prouvée nécessaire [65]. La seconde est le synchronisme partiel, où une borne  $\Delta$  finit par tenir. Écrire « l'essaim finit par converger » sans dire laquelle des deux on suppose est le défaut de spécification le plus fréquent du domaine.

Restent les méthodes qui produisent des garanties quantitatives sous ces contraintes, et leur coût s'écrit exactement. La vérification probabiliste de modèles traite chaînes de Markov à temps discret et continu, processus de décision markoviens, automates probabilistes et temporisés probabilistes et leurs variantes partiellement observables, avec extensions coûts et récompenses, sur les logiques PCTL, CSL, LTL et PCTL\* [66] ; la version qui a introduit la vérification quantitative des automates temporisés probabilistes tarifés a livré en outre une boîte à outils d'abstraction et un moteur de simulation à événements discrets [26]. La complexité temporelle de la vérification d'une formule PCTL sur une chaîne à temps discret est linéaire en la taille de la formule et polynomiale en  $|S|$  ; les opérateurs les plus coûteux exigent la résolution d'un système linéaire de taille au plus  $|S|$ , dont l'élimination de Gauss est cubique en la taille du système [67]. Pour CSL sur une chaîne à temps continu s'ajoute une dépendance linéaire en  $q \cdot t\_max$ , où  $q$  est le taux d'uniformisation et  $t\_max$  la plus grande borne temporelle de la formule, chaque itération étant un produit matrice-vecteur quadratique en  $|S|$  et le nombre d'itérations étant fourni par la borne de Fox et Glynn, linéaire en  $q \cdot t$  pour  $q \cdot t$  grand [67]. Ces exposants sont modestes ; c'est  $|S|$  qui ne l'est pas. Avec  $|S| = \Theta(|S\_agent|^n)$ , les  $6,8 \times 10^{10}$  configurations de l'exemple précédent demandent  $\approx 3,2 \times 10^{32}$  opérations à un solveur cubique. Un second vérificateur, modulaire et à solveurs interchangeable [68], change les constantes, pas l'exposant.

Les méthodes numériques calculent exactement la mesure des chemins ; l'approche statistique procède par simulation et test d'hypothèses et décide, sur les échantillons, si la spécification est satisfaite ou violée, ses avantages revendiqués étant l'efficacité de calcul et l'uniformité méthodologique [27]. C'est la seule des deux qui passe à un essaim de taille réaliste, et son coût s'énonce exactement. La borne de Chernoff-Hoeffding garantit que l'estimation  $Y$  du résultat vrai  $X$  vérifie  $\Pr(|Y - X| > \varepsilon) < \delta$  pour un nombre d'échantillons  $N = \ln(2/\delta) / (2\varepsilon^2)$  [69]. Pour  $\varepsilon = 10^{-2}$  et  $\delta = 10^{-2}$ , il faut  $N \approx 26\,492$  exécutions ; pour  $\varepsilon = 10^{-3}$  à la même confiance,  $N \approx 2,65 \times 10^6$  exécutions, soit cent fois plus pour un chiffre significatif de plus. L'algorithme 3.2 pose la procédure, ses hypothèses et les quatre façons dont sa conclusion peut être fausse.

#### Algorithme 3.2 – vérification statistique bornée d'une propriété d'essaim

Entrées : modèle  $M$  ( $n$  agents, règle locale, milieu partagé) ;  
propriété  $\phi$  décidable sur un horizon de  $H$  tours ;  
précision  $\varepsilon$  ; confiance  $\delta$ .  
Modèle de panne : crash-arrêt de probabilité  $p\_c$  par agent et par tour ;  
omission de probabilité  $p\_o$  ; partition réseau modélisée  
comme un processus à deux états, faute de quoi elle  
n'est jamais tirée.  
Synchronisme : le simulateur impose des tours, le système cible n'en a

pas. Cet écart est une hypothèse du résultat.

1.  $N \leftarrow \lceil \ln(2/\delta) / (2\varepsilon^2) \rceil$ .
2. Pour  $i$  de 1 à  $N$  : tirer et journaliser une graine  $s_i$  ; simuler  $M$  sur  $H$  tours à partir de  $s_i$  ;  $b_i \leftarrow 1$  si la trace satisfait  $\phi$ , 0 sinon.
3.  $\hat{p} \leftarrow (1/N) \cdot \sum b_i$ .
4. Rendre  $\hat{p} \pm \varepsilon$  avec la garantie  $\Pr(|\hat{p} - p| > \varepsilon) < \delta$  [69].

Condition d'arrêt :  $N$  est fixé d'avance par l'étape 1. Un test séquentiel du rapport de probabilité s'arrête plus tôt, au prix d'un résultat binaire au lieu d'une estimation [27].

Coût :  $N \times H$  tours simulés ; pour  $\varepsilon = 10^{-3}$ ,  $\delta = 10^{-2}$  et  $H = 10^3$ ,  $\approx 2,65 \times 10^9$  tours. Massivement parallélisable.

Modes de défaillance

- a) Un événement de probabilité inférieure à  $\varepsilon$  est indiscernable de l'impossible : à  $\varepsilon = 10^{-3}$ , une probabilité vraie de  $10^{-4}$  et une probabilité vraie nulle produisent la même conclusion.
- b) La garantie porte sur  $M$ , pas sur le système. Tout mécanisme absent de  $M$  – ramasse-miettes, rééquilibrage, rétention – a dans le résultat une probabilité de faute nulle.
- c) La garantie porte sur le  $n$  fixé de  $M$ . L'extrapolation à tout  $n$  est indécidable hors de la classe décidable ci-dessus [63][64].
- d)  $\phi$  doit être bornée : S1 est décidable sur  $H$  tours, L1 ne l'est pas. Une propriété de vivacité n'est réfutée par aucune exécution finie [61] ; aucun échantillon fini ne la décide.

Le retour d'expérience industriel le mieux documenté chiffre ce que ces méthodes rapportent. Depuis 2011, des équipes d'un fournisseur infonuagique majeur emploient la spécification formelle et la vérification de modèles sur des systèmes critiques ; sept équipes l'ont adoptée, et des ingénieurs de tous niveaux ont appris la méthode et obtenu des résultats utiles en 2 à 3 semaines, sans formation ni accompagnement [62]. Le fait le plus instructif n'est pas le nombre de défauts mais la longueur de la trace : le plus subtil de ceux trouvés sur le système de réplication et d'appartenance exigeait 35 étapes de haut niveau pour être exhibé [62]. Aucune campagne de tests aléatoires ne trouve un entrelacement de 35 étapes par hasard, et c'est par là que la vérification exhaustive garde un domaine propre malgré ses bornes.

**Tableau 8** — Spécifications formelles de composants de production et défauts trouvés [62].

| Composant spécifié                                | Taille de la spécification | Résultat rapporté                                                               |
|---------------------------------------------------|----------------------------|---------------------------------------------------------------------------------|
| Algorithme réseau tolérant aux fautes, bas niveau | 804 lignes PlusCal         | 2 défauts, plus d'autres dans les optimisations proposées                       |
| Redistribution de données en arrière-plan         | 645 lignes PlusCal         | 1 défaut, et 1 défaut dans le premier correctif proposé                         |
| Réplication et appartenance de groupe             | 939 lignes TLA+            | 3 défauts, certains exigeant des traces de 35 étapes                            |
| Gestion de volumes                                | 102 lignes PlusCal         | 3 défauts                                                                       |
| Structure de données sans verrou                  | 223 lignes PlusCal         | Confiance accrue ; défaut de vivacité manqué, faute d'avoir vérifié la vivacité |

| Composant spécifié                                      | Taille de la spécification | Résultat rapporté                                |
|---------------------------------------------------------|----------------------------|--------------------------------------------------|
| Réplication et reconfiguration<br>tolérantes aux fautes | 318 lignes TLA+            | 1 défaut ; une optimisation agressive<br>validée |

La ligne la plus honnête du tableau est celle de la structure de données sans verrou : le défaut de vivacité n’a pas été trouvé parce que la vivacité n’a pas été vérifiée [62]. Vérifier une exigence de la forme L1 n’est pas un supplément gratuit ; cela change la classe d’algorithme et le coût. Les auteurs posent eux-mêmes la limite de l’exercice : ils ne connaissent pas d’outil capable de vérifier que le code exécutable met en œuvre la spécification de haut niveau, ni d’en engendrer le code, pour des systèmes distribués de cette taille ; l’analyse statique qu’ils emploient par ailleurs reste cantonnée aux problèmes locaux et ne peut pas vérifier cette conformité [62]. Une spécification vérifiée est une preuve sur un objet mathématique dont on espère qu’il ressemble au programme.

La transposition depuis la robotique en essaim conserve la structure micro-macro et l’exhaustivité de la vérification, et casse la réduction que la géométrie offrait gratuitement. Dans un essaim robotique, le voisinage tient dans un rayon physique : le facteur de branchement est borné par le nombre de robots présents dans un disque, et la propriété à vérifier est le plus souvent spatiale — agrégation, dispersion, couverture. Dans un essaim logiciel au-dessus d’un journal, le voisinage est le sujet tout entier : le facteur de branchement est  $n$ , et rien dans le milieu ne le réduit. En échange, les propriétés  $y$  sont nativement discrètes et souvent de la forme S1 — monotonie des décalages, unicité de l’attribution d’une partition, absence de double effet après rejeu — donc réfutables sur une exécution finie et justiciables de la vérification bornée. La transposition retire l’aide de la géométrie et rend en contrepartie des propriétés testables.

### 3.3 Outillage et simulation : environnements de test, émulation d’essaims et méthodes de validation avant déploiement

Le simulateur d’un essaim n’est pas un instrument d’observation : c’est une spécification de l’environnement, écrite en code, et les fautes qu’il sait produire sont exactement les fautes que la campagne pourra trouver. Cette phrase gouverne le choix de la plateforme, moins affaire de commodité que d’engagement sur le modèle de faute.

## QUATRE MÉTHODES, QUATRE DOMAINES DE RÉFUTATION

|                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>1 Vérification exhaustive</b></p> <p>Réfute <math>S_1</math> sur le <math>n</math> vérifié, y compris par une trace de 35 étapes qu'aucune campagne aléatoire ne trouverait. Ne réfute pas <math>S_1</math> pour tout <math>n</math>, ni <math>L_1</math> si la vivacité n'est pas vérifiée.</p> <p>Cubique en la taille de l'espace d'états joint, lui-même en <math>\Theta( S ^n)</math>.</p> | <p><b>2 Vérification statistique</b></p> <p>Réfute <math>S_1</math> avec confiance <math>1 - \delta</math> à <math>\epsilon</math> près. Un événement de probabilité inférieure à <math>\epsilon</math> est indiscernable de l'impossible.</p> <p><math>N = \ln(2/\delta)/(2\epsilon^2)</math> exécutions : 26 492 à <math>\epsilon = 10^{-2}</math>, puis <math>2,65 \times 10^6</math> pour un chiffre de plus.</p> |
| <p><b>3 Simulation déterministe</b></p> <p>Réfute <math>S_1</math> et rejoue le contre-exemple à l'identique, entrelacement compris. Ne réfute ni la performance ni <math>\ell_{99}</math>, ni rien de ce qui est hors du monde clos.</p> <p><math>\Theta(E \log E)</math> par exécution ; la campagne se mesure en secondes simulées par seconde-cœur.</p>                                           | <p><b>4 Expérience en production</b></p> <p>Réfute l'écart au régime stable, sur le système et ses dépendances réelles. Ne réfute rien de reproductible : la trace n'est pas rejouable.</p> <p>Fraction du trafic de production exposée, soit le rayon d'explosion.</p>                                                                                                                                               |

Aucune des quatre ne réfute une propriété de vivacité, et aucune ne s'étend à tout  $n$  : le problème d'accessibilité est indécidable même lorsque chaque processus a un espace d'états fini [63]. Vérifier l'essai à  $n = 5$  ne dit formellement rien de  $n = 5\,000$ , et ce n'est pas une borne de complexité qu'un meilleur solveur ferait reculer. Une spécification vérifiée est une preuve sur un objet mathématique dont on espère qu'il ressemble au programme.

**Figure 3.3** — Ce que chaque méthode de validation avant déploiement peut réfuter, et ce qu'elle ne peut pas.

Le paysage des plateformes de simulation multi-agents s'est stabilisé autour de cinq familles aux engagements distincts, dont trois traits demandent le détail. La suite Repast répartit ses trois membres sur trois cibles matérielles — Symphony 2.11.0 en Java pour postes de travail et petites grappes, HPC 2.3.1 en C++ pour grandes grappes et superordinateurs, Repast4Py 1.2.1 en Python pour la modélisation distribuée [70]. ARGoS fait tourner plusieurs moteurs physiques simultanément en divisant l'espace en régions attribuées à des moteurs différents, et ses auteurs revendiquent des simulations physiquement exactes mobilisant des milliers de robots en une fraction du temps réel [71]. MASON sépare strictement le modèle de la visualisation, ce qui permet d'attacher ou de détacher les visualiseurs et de changer de plateforme en cours d'exécution [72]. Aucune de ces plateformes ne porte de modèle de panne : le modèle de faute est à écrire par l'utilisateur, et son absence est le mode de défaillance par défaut de toute campagne. Les numéros de version cités datent de la vérification et se périment.

**Tableau 9** — Plateformes de simulation multi-agents et engagement architectural de chacune.

| Environnement | Langage | Licence                  | Trait établi par la source                                                     |
|---------------|---------|--------------------------|--------------------------------------------------------------------------------|
| NetLogo [73]  | NetLogo | GNU GPL v2 ou ultérieure | Environnement de modélisation programmable pour phénomènes naturels et sociaux |

| Environnement | Langage           | Licence                               | Trait établi par la source                                                                                 |
|---------------|-------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| MASON [72]    | Java              | Libre (dépôt ECLab)                   | Événements discrets ;<br>séparation stricte modèle /<br>visualisation, visualiseurs<br>attachables à chaud |
| Repast [70]   | Java, C++, Python | Libre, Argonne National<br>Laboratory | Trois membres pour trois<br>cibles : poste de travail,<br>superordinateur,<br>modélisation distribuée      |
| ARGoS [71]    | C++               | MIT depuis la v3                      | Multifil ; plusieurs moteurs<br>physiques simultanés sur<br>des régions distinctes de<br>l'espace          |
| Mesa [74]     | Python            | Apache 2.0                            | Construction, visualisation<br>en navigateur et analyse<br>de modèles à base d'agents                      |

Une simulation d'essaim non reproductible n'est pas un résultat. Le protocole de description de référence pour les modèles à base d'agents fournit des gabarits et des listes de contrôle par élément, lie l'énoncé de finalité à la modélisation orientée patrons, documente les raisons des choix de conception et introduit des descriptions imbriquées pour les modèles très complexes [75] ; il prolonge la publication qui avait établi le format [76]. Pour un essaim logiciel, ce protocole est nécessaire et insuffisant : décrire le modèle ne suffit pas quand l'exécution est non déterministe. Il faut y adjoindre la graine du générateur pseudo-aléatoire et l'ordonnancement, faute de quoi le rapport décrit un modèle mais pas l'exécution qui a produit le chiffre.

C'est cette exigence qui a produit, du côté logiciel, la méthode qui transpose le mieux la démarche de la robotique en essaim. Une base de données transactionnelle distribuée a été bâtie dans l'ordre inverse de l'habitude : avant d'écrire la base, ses auteurs ont écrit un cadre de simulation déterministe capable de simuler un réseau de processus en interaction et une variété de pannes de disque, de processus, de réseau et de requêtes, dans un seul processus physique [77]. Le logiciel réel y tourne avec des charges synthétiques aléatoires et de l'injection de fautes, et le déterminisme garantit que tout défaut trouvé peut être reproduit, diagnostiqué et corrigé [77]. Les conditions de ce déterminisme sont énoncées sans concession : tout le code est déterministe, la concurrence multifil est évitée — un nœud par cœur — et toutes les sources de non-déterminisme sont abstraites, réseau, disque, temps et générateur pseudo-aléatoire compris ; le code est écrit dans une extension syntaxique de C++ ajoutant des primitives `async/await` et un modèle d'acteurs, l'implantation de production n'étant qu'une mince couche vers les appels système [77].

L'injection de fautes couvre les pannes fail-stop de machine, de baie et de centre de données ainsi que les redémarrages, une variété de fautes réseau, de partitions réseau et de problèmes de latence, des comportements de disque tels que la corruption des écritures non synchronisées lors d'un redémarrage, et la randomisation des dates d'événements [77]. Le détail que la plupart des équipes manquent est le réglage de la distribution : elle est soigneusement

ajustée pour éviter de pousser le système dans un espace d'états restreint par un taux de fautes excessif [77]. Un essaim soumis à un taux de panne trop élevé n'explore pas plus d'états, il en explore moins, parce qu'il passe tout son temps dans le régime dégradé. S'y ajoute une technique d'injection de haut niveau où le code lui-même offre au simulateur, en de nombreux points, l'occasion d'introduire un comportement inhabituel mais non contraire au contrat — faire échouer une opération qui réussit d'ordinaire, retarder une opération d'ordinaire rapide, choisir une valeur inhabituelle de paramètre de réglage, ce qui garantit accessoirement qu'aucune valeur de réglage précise ne devienne nécessaire à la correction [77]. La diversité des exécutions est maximisée par un tirage aléatoire de la taille et de la configuration de la grappe, des charges, des paramètres d'injection de fautes, des paramètres de réglage et du sous-ensemble actif de ces points d'injection [77]. Enfin, des macros de couverture conditionnelle mesurent combien d'exécutions distinctes ont atteint une condition donnée, et signalent où ajouter de l'injection quand le compte est faible ou nul [77]. C'est l'équivalent logiciel des critères heuristiques de calibrage du §3.1 [18] : on ne règle pas le simulateur sur le comportement moyen mais sur la couverture des états rares.

Le gain de vitesse a une cause précise. Une simulation à événements discrets avance arbitrairement plus vite que le temps réel lorsque l'utilisation processeur y est faible, puisque le simulateur porte l'horloge directement au prochain événement ; comme beaucoup de défauts de systèmes distribués n'apparaissent qu'après de longues périodes d'inactivité, on en trouve bien davantage par seconde-cœur qu'avec des tests de bout en bout en temps réel [77]. La granularité de l'horloge logique — 1  $\mu$ s ou 1 ms — est un paramètre du modèle, pas une propriété du système, et elle décide de ce qui compte comme simultané. L'algorithme 3.3 rassemble la campagne, son modèle de panne, son coût et les cinq façons dont elle laisse passer un défaut.

#### Algorithme 3.3 – campagne de simulation déterministe d'un essaim

Modèle de panne : crash-arrêt et redémarrage de machine, de baie et de centre ; omission et retard de message ; partition réseau ; corruption des écritures non synchronisées au redémarrage ; dates d'événements randomisées [77].  
Aucune faute byzantine (voir ch. 7).

Synchronisme : aucun. L'horloge est logique et n'avance que par consommation d'événements.

Hypothèses : processus unique, aucune concurrence multifil – un agent est un acteur ordonnancé par la boucle d'événements ; réseau, disque, horloge et générateur pseudo-aléatoire abstraits derrière une interface [77].

1.  $s \leftarrow$  graine ; journaliser  $s$  avec la configuration complète.
2. Tirer, à partir de  $s$  : taille et configuration de l'essaim, charge de travail, paramètres d'injection de fautes, paramètres de réglage, sous-ensemble actif des points d'injection de haut niveau.
3. Instancier les  $n$  agents et le milieu partagé dans le processus.
4. Tant que le budget de temps simulé n'est pas épuisé : extraire de la file l'événement de date minimale ; avancer l'horloge logique à cette

date sans attente réelle ; exécuter le gestionnaire et insérer les événements produits ; évaluer les oracles – assertions d'invariants portées par la charge de travail et par le code lui-même [77].

5. Arrêt d'une exécution : violation d'oracle, ou budget épuisé.
6. Sur violation : rejouer avec  $s$  et la même configuration. La trace est identique, y compris l'ordre d'entrelacement.
7. Arrêt de la campagne : budget en secondes-cœur. L'espace de recherche étant effectivement infini, exécuter davantage de tests couvre davantage de code et trouve davantage de défauts, sans qu'aucun critère de complétude existe [77].

Coût :  $\Theta(E \log E)$  par exécution pour  $E$  événements dans une file de priorité ; la campagne se mesure en secondes simulées par seconde-cœur, non en nombre d'exécutions. Le nombre de messages échangés n'est pas la grandeur de coût : la file d'événements l'est.

#### Modes de défaillance

- a) La simulation ne détecte pas de façon fiable les problèmes de performance, par exemple un algorithme d'équilibrage de charge imparfait [77]. Le temps simulé n'est pas le temps mesuré :  $t_{99}$  ne s'obtient pas ici mais sous charge réelle (voir ch. 2 et ch. 6).
- b) Elle ne teste ni les bibliothèques tierces, ni les dépendances, ni le code propre qui n'est pas écrit dans le langage déterministe [77]. Le prix du déterminisme est un monde clos.
- c) Un défaut d'une dépendance critique – système de fichiers, système d'exploitation – ou une méprise sur son contrat produit un défaut non simulable ; plusieurs défauts sont venus de contrats réels plus faibles que ce qui était supposé [77].
- d) Un défaut qui n'apparaît que sous une distribution de fautes hors de l'enveloppe réglée à l'étape 2 n'est jamais échantillonné.
- e) Une exigence de la forme L1 (§3.2) n'est réfutée par aucune trace finie. Le contournement employé ramène l'environnement matériel modélisé, après une série de pannes, dans un état où la reprise devrait être possible, puis vérifie que la grappe finit par se rétablir [77] : c'est une vivacité conditionnelle, sous une condition écrite dans le scénario, et non la propriété L1.

Ce que la simulation déterministe ne couvre pas, l'expérimentation en production le couvre partiellement, et à des conditions qu'il faut nommer avec la même précision. La discipline correspondante se définit comme l'expérimentation sur un système pour asseoir la confiance en sa capacité à résister à des conditions turbulentes en production, et elle avance cinq principes : bâtir une hypothèse autour du comportement en régime stable, varier les événements du monde réel, exécuter les expériences en production, automatiser les expériences pour qu'elles tournent en continu, et minimiser le rayon d'explosion [78]. Le modèle de panne  $y$  est celui que l'outillage sait produire sur le système réel, et rien d'autre ; l'hypothèse de synchronisme est vide, puisque le système est le système ; le coût ne se compte ni en messages ni en tours mais en fraction du trafic de production exposée, ce qu'énonce le cinquième principe ; la condition d'arrêt est l'écart mesuré au régime stable ou l'expiration de la fenêtre. Le mode de défaillance dominant est l'absence de rejeu : la trace n'est pas reproductible,

l’entrelacement qui a produit l’écart ne se retrouve pas, et une expérience qui ne réfute rien n’établit rien. Le statut des sources doit être dit avec la même franchise : l’article fondateur est un texte de position et de retour d’expérience, sans protocole expérimental reproductible ni mesures comparatives [79], et le manifeste communautaire n’a ni auteur identifié ni comité de lecture et est inchangé depuis mars 2019 [78]. Ce sont des sources de vocabulaire et de valeurs par défaut, pas des preuves.

**Tableau 10** — Ce que chaque méthode de validation avant déploiement peut et ne peut pas réfuter.

| Méthode                  | Modèle de panne                                                                   | Coût                                                                                | Ce qu’elle réfute                                                   | Ce qu’elle ne réfute pas                                                                      |
|--------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Vérification exhaustive  | Celui écrit dans le modèle, exploré intégralement                                 | Cubique en la taille de l’espace d’états joint, elle-même exponentielle en $n$ [67] | S1 sur le $n$ vérifié, y compris par une trace de 35 étapes [62]    | S1 pour tout $n$ hors classe décidable [63] [64] ; L1 si la vivacité n’est pas vérifiée [62]  |
| Vérification statistique | Celui écrit dans le modèle, échantillonné                                         | $N = \ln(2/\delta)/(2\epsilon^2)$ exécutions [69]                                   | S1 avec confiance $1 - \delta$ à $\epsilon$ près                    | Tout événement de probabilité inférieure à $\epsilon$ ; L1, indécidable sur horizon fini [61] |
| Simulation déterministe  | Crash, redémarrage, omission, retard, partition réseau, corruption de disque [77] | $\Theta(E \log E)$ par exécution ; campagne en secondes simulées par seconde-cœur   | S1, avec rejeu à l’identique du contre-exemple [77]                 | La performance et $\ell_9$ [77] ; ce qui est hors du monde clos [77]                          |
| Expérience en production | Celui que l’outillage sait produire sur le système réel [78]                      | Fraction du trafic réel exposée, soit le rayon d’explosion [78]                     | L’écart au régime stable, sur le système et ses dépendances réelles | Rien de reproductible : aucun rejeu de la trace                                               |

Reste l’écart que la robotique en essaim nomme et que le logiciel ferait bien de nommer aussi. La difficulté centrale des méthodes de conception automatique y est appelée l’écart à la réalité : la différence inévitable entre les modèles employés dans les simulations et la réalité, et le défi de produire un logiciel de contrôle qui, une fois installé sur les robots, se comporte comme prévu — défi que les auteurs déclarent loin d’être surmonté [80]. La transposition ne fait pas disparaître cet écart, elle en change la nature, et la différence est exploitable. En robotique, l’écart est physique : frottement, bruit de capteur, dérive d’actionneur, et la parade classique consiste à injecter du bruit dans le simulateur pour empêcher le contrôleur de surapprendre les régularités de la simulation. Pour un essaim d’agents logiciels au-dessus d’un journal, la source dominante d’écart n’est pas le bruit mais le mécanisme absent : le contrat réel d’une dépendance plus faible que supposé, une pause de ramasse-miettes, un rééquilibrage de groupe de consommation, une politique de rétention. Ajouter du bruit à un modèle qui ignore le rééquilibrage ne rapproche pas le modèle de la réalité ; cela élargit l’intervalle de confiance autour d’une valeur fausse. Ce qui transpose, c’est la discipline de régler la distribution de fautes plutôt que le comportement moyen [77] et de mesurer la couverture des états rares plutôt que le nombre d’exécutions. Ce qui ne transpose pas, c’est la parade elle-même.

La conséquence pour un essaim dont les agents reposent sur des modèles de langage se déduit du mode (b) de l’algorithme 3.3 : un agent dont la décision est produite par un service externe est, du point de vue du simulateur déterministe, une dépendance non instrumentable, donc un trou dans le monde clos. Le déterminisme ne s’obtient alors qu’en remplaçant l’agent par un substitut enregistré, ce qui vérifie l’essaim autour de l’agent et non l’agent lui-même. Le traitement de ce non-déterminisme-là appartient au ch. 7.

Deux conséquences de plus se lisent sur ce que la première campagne publiée à cette échelle a pu établir, et sur ce qu’elle n’a pas pu. Ses verdicts sont statistiques et bornés par leur échantillon — quatre cents épisodes par modèle pour une tâche de décision de groupe, cent vingt épisodes par modèle pour une tâche à objectifs incompatibles, douze heures par exécution pour la construction collective d’un logiciel [119] —, ce qui est exactement le régime de la vérification statistique du tableau ci-dessus, avec ce que ce régime ne réfute jamais : les événements plus rares que la précision retenue. Il porte aussi une hypothèse que  $N = \ln(2/\delta)/(2\epsilon^2)$  n’écrit pas, l’indépendance des tirages : une population conforme rend  $N$  tirages d’un même comportement, et l’intervalle  $\pm \epsilon$  entoure une population plus étroite que celle que la campagne croit décrire. Aucun résultat exhaustif n’est disponible sur une telle population, et le §3.2 dit pourquoi il ne le sera pas. La seconde conséquence est plus gênante pour l’ingénieur. Les résultats de cette campagne s’inversent d’une génération de modèle à l’autre : la même expérience donne une fraction de fusion basse avec partage de code, puis élevée sans partage, puis élevée avec partage selon le modèle employé, et l’aptitude à sortir d’un conflit par une trêve n’est pas monotone en capacité générale [119]. Une campagne de validation d’essaim est donc liée à une version de modèle comme une mesure de latence l’est à une version de courtier : périssable. Le substitut enregistré du paragraphe précédent aggrave la difficulté au lieu de la lever, puisqu’il fige la décision qu’il s’agissait d’observer : un essaim rejoué depuis des traces enregistrées ne peut plus produire ni la corrélation ni l’escalade, qui sont des propriétés de la population en train de décider. Ce que l’on sait valider avant déploiement, ce sont les mécanismes du milieu ; ce que l’on ne sait pas valider, c’est la population.

## 4. Comportements collectifs et auto-organisation

### 4.1 Agrégation et structuration logique

Trois problèmes distincts portent le même nom dans la littérature d’essaim (*swarm*), et les confondre coûte cher parce qu’ils n’ont ni le même modèle de faute, ni le même coût, ni la même condition d’arrêt. Le premier est le calcul d’une valeur agrégée sur la population — combien d’agents sont vivants, quelle est la charge moyenne, quel est le maximum de retard de consommation. Le second est le regroupement des agents eux-mêmes en une topologie logique où chacun n’entretient de relation qu’avec une poignée de pairs. Le troisième est le regroupement des tâches, c’est-à-dire le choix de ce qui atterrit ensemble chez le même agent. Le premier produit un scalaire, le deuxième un graphe, le troisième

une partition du travail. Les trois se construisent par des échanges locaux, et c'est tout ce qu'ils ont en commun.

Le calcul d'une valeur agrégée par échange par paires se pose ainsi. Chaque agent  $i$  détient une mesure locale  $v(i)$  ; on veut que chacun détienne une estimation de la moyenne des  $v(i)$  sur les  $n$  agents vivants. Les hypothèses se posent avant l'algorithme, faute de quoi la garantie qu'on en tirera sera fausse :

- **Fautes des agents** — crash-arrêt et omission, jamais byzantines (renvoi ch. 7) ; un agent arrêté ne revient pas avec un état ancien pendant l'exécution du protocole.
- **Canal** — perte possible et détectable par expiration, pas de corruption, expéditeur identifiable de façon fiable, pas de duplication.
- **Synchronisme** — partiellement synchrone : il existe une borne  $\Delta$  sur le délai de message, inconnue et valide seulement après un instant inconnu. Le protocole procède par cycles de période  $T$  avec  $T > 2\Delta$  en régime nominal ; les horloges ne sont pas synchronisées, seule la période moyenne compte.
- **Échantillonnage** — chaque agent dispose d'un service qui lui rend un pair, présenté comme tiré uniformément parmi les vivants [81].
- **Domaine** — les  $v(i)$  sont des réels de moyenne finie ; la moyenne est la seule fonction traitée ici, le comptage, la somme, le produit et les extrema s'obtiennent par la même mécanique [82].

L'algorithme 4.1 en donne la variante push-pull. Son coût est de deux messages par agent et par cycle, soit  $2n$ , chaque message portant un réel et un entier d'époque — quelques dizaines d'octets, indépendamment de  $n$ . Le nombre de cycles nécessaires n'est pas une propriété de l'algorithme mais du graphe d'échange : la vitesse de convergence d'un protocole de moyenne est gouvernée par la connectivité algébrique du graphe de communication [36], et des échanges non locaux accélèrent substantiellement la convergence par rapport à un schéma de plus proches voisins [29]. Sur un graphe d'échantillonnage aléatoire, cette connectivité est élevée et le régime est logarithmique en  $n$  ; sur un graphe géométrique, elle s'effondre et le régime devient polynomial. Le coût total est donc  $\Theta(n)$  messages par cycle et  $\Theta(\log n)$  cycles dans le cas favorable, mais le cas favorable est une hypothèse sur le service d'échantillonnage, pas un acquis.

Algorithme 4.1 – Agrégation par échange par paires (push-pull, moyenne)

ÉTAT de l'agent  $i$  :  $x$  – estimation locale, initialisée à  $v(i)$   
 $e$  – numéro d'époque, initialisé à 0

FIL ACTIF, exécuté une fois par cycle de période  $T$

```

1   $j \leftarrow \text{échantillonner\_pair}()$ 
2  envoyer (PUSH,  $e$ ,  $x$ ) à  $j$ 
3  attendre (PULL,  $e'$ ,  $x'$ ) de  $j$  pendant au plus  $\tau$ , avec  $\tau < T$ 
4  si rien n'est reçu avant  $\tau$  : abandonner le cycle,  $x$  inchangé
5  si  $e' > e$  :  $e \leftarrow e'$  ;  $x \leftarrow v(i)$  ; abandonner le cycle
6  si  $e' < e$  : ignorer  $x'$  ; abandonner le cycle
7   $x \leftarrow (x + x') / 2$ 
```

FIL PASSIF, sur réception de  $\langle \text{PUSH}, e', x' \rangle$  venant de  $j$

```
8  si  $e' > e$  :  $e \leftarrow e'$  ;  $x \leftarrow v(i)$   
9  envoyer  $\langle \text{PULL}, e, x \rangle$  à  $j$   
10 si  $e' = e$  :  $x \leftarrow (x + x') / 2$ 
```

RELANCE, tous les  $C$  cycles, déclenchée par une règle publique

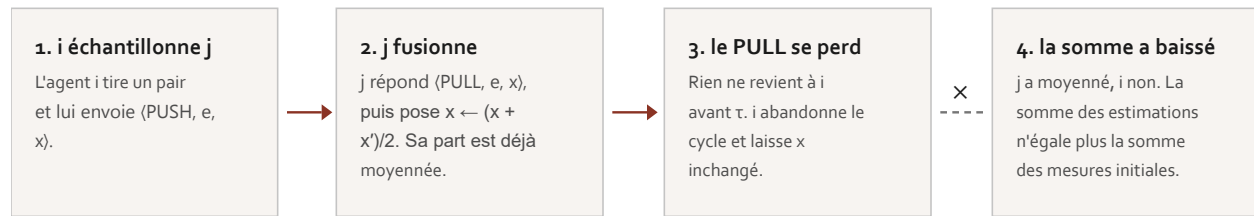
```
11  $e \leftarrow e + 1$  ;  $x \leftarrow v(i)$ 
```

CONDITION D'ARRÊT : aucune. Le protocole est proactif et ne se termine pas.

L'algorithme 4.1 n'a pas de condition d'arrêt, et c'est une propriété, pas un oubli : il est proactif, tous les agents détiennent la valeur en continu et suivent ses variations [82]. Un agent qui a besoin de décider doit donc s'imposer sa propre règle — un nombre fixe de cycles depuis la dernière relance, ou une variation locale sous un seuil sur une fenêtre de  $w$  cycles. Cette règle est une heuristique : elle ne prouve rien, un agent ne pouvant distinguer une estimation stabilisée d'une estimation qui n'a pas encore vu la contribution d'un sous-ensemble mal échantillonné.

Le mode de défaillance de l'algorithme 4.1 est nommé et il est grave. La correction de toute la classe des protocoles de moyenne itérative repose sur un invariant, la conservation de masse : la somme des estimations doit rester égale à la somme des mesures initiales. Cet invariant est rompu en pratique dès qu'on quitte le cadre synchrone et sans faute — perte de message et arrêt d'agent le cassent, et le rétablir exige des mécanismes supplémentaires qui dégradent les performances [83]. La ligne 4 de l'algorithme 4.1 est l'endroit exact de la rupture : si l'agent  $j$  a reçu le PUSH et exécuté la ligne 10, mais que le PULL se perd, alors  $j$  a moyenné et  $i$  non ; la somme totale a diminué. Le résultat n'est pas en retard, il est faux. Et il est faux silencieusement : tous convergent, s'accordent sur la même valeur, et cette valeur ne correspond à aucune moyenne réelle. Sans relance, l'erreur ne croît pas sans borne : elle se fige. L'unanimité installée, l'écart  $|x_i - x_j|$  des lignes 7 et 10 est nul et il ne reste plus de masse à perdre. Le défaut est pire qu'annoncé : une dérive sans borne se détecte par n'importe quel contrôle de vraisemblance, une erreur figée ne se détecte par rien, et surtout pas par l'attente. La relance de la ligne 11 ne plafonne donc pas l'erreur : elle réinjecte de la dispersion à chaque période, de sorte que son maximum croît avec la durée d'observation là où l'erreur sans relance est acquise une fois pour toutes. Ce qu'elle achète n'est pas un plafond mais la conversion d'un mensonge stable en une oscillation qu'un opérateur voit passer, et c'est ce qu'exige une agrégation qui alimente une décision de dimensionnement.

#### UN SEUL MESSAGE PERDU, ET LA SOMME A DIMINUÉ



La rupture n'est pas dans l'implantation : elle est dans l'hypothèse. L'invariant tient en synchrone et sans faute, et perte de message comme arrêt d'agent le cassent tous deux [83].

Le résultat n'est pas en retard, il est faux — et faux silencieusement : tous les agents convergent, ils s'accordent tous sur la même valeur, et cette valeur ne correspond à aucune moyenne réelle. La relance périodique de la ligne 11 plafonne l'erreur accumulée ; elle ne la corrige pas, et aucun agent ne peut distinguer une estimation stabilisée d'une estimation amputée.

**Figure 4.1** — Où la conservation de masse se rompt : la ligne 4 de l'algorithme 4.1, et pourquoi la fausse valeur est unanime.

Le regroupement des agents eux-mêmes repose sur le service invoqué à la ligne 1. Ce service — l'échantillonnage de paires — est le mécanisme fondamental sous les protocoles de dissémination, d'agrégation et de gestion de topologie ; il se construit de façon décentralisée en faisant circuler par voie épidémique (*gossip*) l'information d'appartenance (*membership*) elle-même, ce qui produit et entretient un recouvrement non structuré dynamique [81]. Le résultat expérimental à retenir avant d'appliquer une borne : les implantations fournissent bien à chaque agent un flux de paires localement uniforme, mais les hypothèses théoriques usuelles sur l'aléa **ne tiennent pas globalement**, et des choix de conception différents produisent des écarts sévères d'équilibrage de charge et de tolérance aux fautes [81]. Toute borne démontrée sous l'hypothèse d'un tirage uniforme et indépendant à chaque tour est donc conditionnelle, et sa condition n'est pas satisfaite par le système qui l'invoque : cela ne rend pas la borne inutile, cela interdit de la présenter comme une garantie.

De ce recouvrement aléatoire, une structure peut être extraite sans coordonnateur. Chaque agent entretient une vue de  $c$  descripteurs de paires ; à chaque cycle il échange sa vue avec un pair et conserve les  $c$  descripteurs qui minimisent une fonction de distance sur un attribut — version de code, zone de déploiement, spécialisation déclarée. Le coût est de deux messages de  $c$  descripteurs par agent et par cycle, soit  $2nc$  descripteurs échangés par cycle ; le nombre de cycles n'est pas borné en général, parce que le procédé est une montée de gradient distribuée sur l'espace des topologies et que rien n'interdit les minima locaux. Son mode de défaillance est la scission silencieuse : deux sous-ensembles cessent de s'échantillonner mutuellement, chacun calcule ses agrégats sur lui-même et croit être l'essaim entier. Aucun agent n'observe d'erreur. Sa détection relève de la perception qu'un agent entretient de l'essaim (ch. 3) ; sa conséquence sur l'accord, du §4.2.

Le regroupement des tâches est un problème d'une autre nature, parce que l'unité regroupée n'est pas choisie par l'essaim mais par le producteur. Dans le milieu (*medium*) événementiel de l'ouvrage, cette unité est la partition du sujet, dont le ch. 1 possède l'abstraction. Deux régimes s'opposent. Le hachage

de la clé donne une répartition uniforme et détruit toute localité : deux événements du même client tombent dans deux partitions distinctes et perdent leur ordre relatif. La clé sémantique conserve la localité et importe la dissymétrie du domaine. Le hachage cohérent [35] atténue le coût de la première option en rendant l'affectation stable : l'arrivée ou le départ d'un agent ne déplace qu'environ  $1/n$  des clés au lieu de la totalité, et les protocoles sont conçus pour des réseaux où aucun serveur ne peut connaître l'état complet du réseau [35]. Le mode de défaillance de la seconde option est la partition chaude, et la borne qui le rend structurel est simple : la partition étant la seule unité de parallélisme côté consommateur, le parallélisme utile de  $n$  agents sur un sujet à  $p$  partitions vaut  $\min(n, p)$ , quelle que soit la charge. Au-delà, ajouter des agents n'ajoute rien et coûte un rééquilibrage (*rebalance*). Les chiffres de service de cette structure se lisent au 99,9<sup>e</sup> centile et non en moyenne : sur un magasin clé-valeur réparti d'échelle industrielle, les latences au 99,9<sup>e</sup> centile sont d'un ordre de grandeur supérieures aux moyennes et suivent le taux de requêtes [49] ; la configuration  $(N, R, W)$  couramment employée en production y est  $(3, 2, 2)$  [49].

La formation de sous-groupes spécialisés peut se passer entièrement de messages, et le §3.1 en a posé le mécanisme : une politique stochastique optimisée sur les taux d'entrée et de sortie entre tâches, décentralisée et sans aucune communication entre robots [40], dont les modèles macroscopiques sont indépendants de la taille de l'essaim [22]. La transposition au logiciel conserve la mathématique — une chaîne de Markov sur les états d'agent dont la distribution stationnaire est l'allocation visée — et conserve le coût nul en messages, ce qui est considérable. Elle casse sur deux points. D'abord l'hypothèse de bon mélange, que l'allocation de partitions rend fausse au sens du §3.1 : le modèle doit être posé par partition, sinon il est faux. Ensuite l'échelle de temps : le robot tolère une convergence en minutes parce que son environnement varie sur cette échelle, tandis qu'une politique stochastique qui converge en espérance en  $c$  cycles ne donne aucune borne sur  $\ell_9$ , de la reconfiguration individuelle, qui est précisément la mesure du service.

Le plafond de cette approche est d'origine biologique, et l'introduction l'a posé : le gain d'efficacité du fourragement collectif diminue dans les grands groupes, vraisemblablement par interférence [7]. La transposition conserve la forme de la courbe — il existe une taille de groupe au-delà de laquelle ajouter un agent retransche du débit — et en casse la cause. Dans l'essaim robotique, l'interférence est une collision physique dans un espace fini ; dans l'essaim logiciel, il n'y a pas de collision, et le plafond réapparaît sous deux formes — la contention sur le milieu, qui est un phénomène de file (renvoi ch. 2), et le coût de coordination du rééquilibrage, qui croît avec le nombre de participants. Le remède diffère en conséquence : le robot s'étale dans l'espace, l'agent augmente  $p$  ou refond la clé.

**Tableau 11** — Coût et mode de défaillance des cinq mécanismes de regroupement, sous le modèle de faute crash-arrêt et omission et l’hypothèse de synchronisme partiel posée au début de la section.

| Mécanisme                   | Unité regroupée   | Messages par cycle                        | Cycles jusqu’à la cible                                                   | Condition d’arrêt                 | Mode de défaillance dominant                                                          |
|-----------------------------|-------------------|-------------------------------------------|---------------------------------------------------------------------------|-----------------------------------|---------------------------------------------------------------------------------------|
| Échange par paires (alg. 1) | valeur scalaire   | 2n                                        | $\Theta(\log n)$ si le graphe d’échange est un expenseur, non borné sinon | aucune ; règle locale heuristique | conservation de masse rompue : convergence unanime vers une valeur fausse, puis figée |
| Échantillonnage de paires   | vue de c pairs    | 2nc descripteurs                          | entretien permanent                                                       | aucune                            | aléa non uniforme globalement ; déséquilibre de charge                                |
| Tri de vue par distance     | topologie logique | 2nc descripteurs                          | non borné (minima locaux)                                                 | vue stable sur w cycles           | scission silencieuse du recouvrement                                                  |
| Hachage cohérent            | clé de tâche      | $O(n)$ à chaque changement d’appartenance | 1 après propagation de la vue                                             | toute plage a un propriétaire     | partition chaude ; parallélisme plafonné à $\min(n, p)$                               |
| Politique stochastique      | état d’agent      | 0                                         | non borné en pire cas                                                     | aucune                            | hypothèse de bon mélange fausse par partition                                         |

## 4.2 Coordination distribuée : synchronisation d’états, consensus logique et propagation d’information en réseau non structuré

Faire circuler une information, faire converger des répliques et se mettre d’accord sur une valeur sont trois exigences de force croissante, séparées par un prix qui se chiffre. Le modèle de faute reste celui de l’ouvrage — crash-arrêt, omission, partition réseau (*network partition*) — et le modèle de synchronisme est posé à chaque mécanisme, parce qu’il change de l’un à l’autre.

**3 S'accorder**

Décider une valeur unique : accord, validité, terminaison.

**2 Converger**

Faire s'accorder des répliques en l'absence de nouvelles écritures.

**1 Propager**

Faire circuler une information dans un réseau non structuré.

**S'accorder — consensus, synchronisme partiel**

$\Omega(n)$  messages par décision : une majorité doit être jointe, et cette majorité est prouvée nécessaire [65]. Terminaison bornée après stabilisation, non bornée avant.

**Converger — répliques sans conflit**

Zéro message dédié : la fusion s'effectue sur le chemin de réplication déjà payé. Aucune condition d'arrêt — la convergence est une quiescence, qu'aucun agent ne peut constater localement.

**Propager — rumeur épidémique**

$\Theta(n \log \log n)$  messages en  $O(\log n)$  tours, optimal pour les algorithmes insensibles à l'adresse [85]. Terminaison locale seulement : la couverture est de probabilité strictement inférieure à 1.

Faire circuler une information, faire converger des répliques et se mettre d'accord sur une valeur ne sont pas trois versions de la même chose. En asynchrone pur, la troisième est impossible : une seule faute d'arrêt suffit à l'empêcher [1]. L'essai ne résout pas l'impossibilité, il refuse la question — et paie ce refus en fenêtres pendant lesquelles l'invariant peut être faux.

**Figure 4.2** — Trois exigences de force croissante, et le prix qui les sépare.

La propagation en réseau non structuré s'analyse dans le modèle de l'appel téléphonique aléatoire :  $n$  agents, tours synchrones de période  $T$ , chaque agent appelle un pair tiré au hasard et le canal ouvert transmet dans les deux sens. Trois mécanismes distincts sont à ne pas confondre [84]. Le courrier direct, où l'émetteur envoie à tous les sites qu'il connaît, est efficace mais peu fiable : messages perdus, connaissance partielle des sites. L'anti-entropie, où chaque site choisit périodiquement un site au hasard et réconcilie l'intégralité du contenu, est extrêmement fiable mais lente, puisqu'elle exige d'examiner le contenu de la base et ne peut donc être fréquente. La rumeur, où une mise à jour est tenue pour « chaude » jusqu'à ce que le site la rencontre trop souvent chez d'autres, tourne plus fréquemment parce qu'elle coûte moins par site, au prix d'une probabilité non nulle qu'une mise à jour n'atteigne pas tous les sites [84]. Le vocabulaire est épidémiologique et formel : un site est susceptible, infectieux ou retiré, et l'anti-entropie est une épidémie simple, sans état retiré [84].

L'algorithme 4.2 donne la rumeur push-pull avec retrait par compteur. Son coût s'énonce en deux grandeurs qu'il ne faut jamais substituer l'une à l'autre. En tours, la diffusion atteint les  $n$  agents en  $O(\log n)$ . En messages, la complexité asymptotiquement optimale du protocole push-pull est  $\Theta(n \log \log n)$  [85], résultat restitué et affiné depuis [86] ; le schéma naïf, qui relaie chaque rumeur sans jamais s'arrêter, dépense  $\Theta(n \log n)$  messages. L'écart entre  $\log n$  et  $\log \log n$  est tout entier dans la ligne 6 de l'algorithme 4.2, c'est-à-dire dans la condition d'arrêt. Et cette borne porte une hypothèse qu'il faut énoncer : l'optimalité est établie pour les algorithmes **insensibles à l'adresse**, dont le choix du partenaire ne dépend pas de l'identité de ce partenaire. Un essaim qui autorise ses agents à choisir leur pair d'après ce qu'ils savent de lui sort du modèle, et la borne cesse de s'appliquer — dans un sens comme dans l'autre.

#### Algorithme 4.2 – Rumeur push-pull avec retrait par compteur

ÉTAT de l'agent  $i$  pour la rumeur  $r$  :

état  $\in \{\text{susceptible}, \text{infectieux}, \text{retiré}\}$ , initialisé à susceptible  
cpt – compteur de rencontres redondantes, initialisé à 0

FIL ACTIF, une fois par tour de période  $T$

- 1 si état = retiré : ne rien faire
- 2  $j \leftarrow \text{échantillonner\_pair}()$
- 3 échanger avec  $j$  :  $i$  envoie  $r$  si état = infectieux,  $\emptyset$  sinon ;  
     $j$  répond  $r$  si  $j$  est infectieux,  $\emptyset$  sinon
- 4 si état = susceptible et la réponse  $\neq \emptyset$  :  
    adopter  $r$  ; état  $\leftarrow$  infectieux ; cpt  $\leftarrow$  0
- 5 si état = infectieux et  $j$  connaissait déjà  $r$  : cpt  $\leftarrow$  cpt + 1
- 6 si cpt  $\geq K$  : état  $\leftarrow$  retiré

CONDITION D'ARRÊT locale : cpt  $\geq K$ . Globale : aucune –  
aucun agent ne sait si la rumeur a atteint tout le monde.

Le mode de défaillance de l'algorithme 4.2 tient dans son énoncé de coût : le retrait garantit la borne en messages et détruit la garantie de couverture. La probabilité résiduelle qu'un agent reste susceptible ne tend pas vers zéro avec le temps, elle tend vers une constante fixée par  $K$ . La propagation offre donc une vivacité (*liveness*) probabiliste et rien de plus, et un essaim qui a besoin de certitude ne remplace pas la rumeur par un meilleur réglage : il fait tourner l'anti-entropie en parallèle, comme filet [84]. Le verdict pratique est celui-là, et il contredit l'élégance du schéma unique : en production les deux protocoles cohabitent, le rapide non fiable et le lent fiable, précisément parce qu'aucun des deux n'est acceptable seul.

La transposition depuis la robotique en essaim conserve l'algorithme et améliore la borne, ce qui est rare. Dans un essaim de robots, l'ensemble des pairs joignables est l'ensemble des robots à portée radio : le graphe est géométrique, son degré  $d$  est borné par la physique et son diamètre croît comme la racine de  $n$  dans le plan, si bien que la diffusion coûte  $\Omega(\sqrt{n})$  tours et non  $O(\log n)$ . Dans l'essaim logiciel, le graphe est un choix de conception : n'importe quel agent peut échantillonner n'importe quel autre, le graphe est un expenseur et le diamètre redevient logarithmique. Ce qu'elle casse est ailleurs : l'indépendance des fautes. Deux robots distants de 3 m tombent en panne indépendamment ; deux agents logés sur le même hôte, issus du même déploiement, derrière la même dépendance, non. Toutes les garanties probabilistes de la diffusion épidémique sont calculées sous tirage uniforme et fautes indépendantes ; la corrélation des fautes invalide le calcul sans invalider le code. C'est le pire mode de rupture qui soit : le protocole continue de tourner et la borne cesse de tenir, sans qu'aucun signal ne le dise.

La synchronisation d'états vise moins que l'accord : la cohérence à terme (*eventual consistency*) garantit que les répliques convergent en l'absence de nouvelles écritures, et ne dit rien du délai. Les types de données répliqués sans conflit fondent cette convergence sur des conditions formelles simples et

suffisantes plutôt que sur un protocole de résolution [87], et ils se répartissent en deux familles aux conditions **distinctes** : la réplication fondée sur l'état, où l'état local évolue de façon monotone dans un treillis à jonction, et la réplication fondée sur les opérations, où les opérations concurrentes doivent commuter [87]. Confondre les deux est l'erreur la plus fréquente [87], et elle est coûteuse parce que la différence ne porte pas sur la structure de données mais sur le contrat de transport qu'elle exige. La première se contente d'un transport qui perd, duplique et désordonne, pourvu qu'il finisse par livrer : l'anti-entropie suffit. La seconde exige une livraison causale et exactement une fois, ce qui est un service strictement plus cher, à construire au-dessus du milieu. Le coût se déplace en conséquence : fondée sur l'état, la taille de message croît avec l'état — un compteur réparti sur  $n$  agents est un vecteur de  $n$  entrées, donc  $\Theta(n)$  octets par message et  $\Theta(n^2)$  par tour d'anti-entropie complet ; fondée sur les opérations, elle est  $\Theta(1)$  et le transport porte la déduplication et l'ordre.

La condition de convergence doit être écrite en entier, sans quoi la phrase « l'essaim converge » n'est pas un énoncé. Elle se formule ainsi : en l'absence de nouvelle écriture, et après que chaque paire d'agents a mené à terme au moins un échange d'anti-entropie, toutes les répliques détiennent la même valeur. Les deux clauses comptent autant l'une que l'autre, et aucun agent ne peut vérifier localement que la première est satisfaite — la quiescence est une propriété globale, et sa détection relève des propriétés stables et de l'instantané réparti (renvoi ch. 7). Le mode de défaillance qui suit est double : une fusion qui n'est pas une jonction de treillis produit une divergence silencieuse que rien ne signale, et la suppression d'un élément impose de conserver une marque de tombe dont l'accumulation fait croître l'état sans borne en l'absence de troncature.

Le consensus, au sens formel du glossaire — accord, validité, terminaison sur une valeur unique — se heurte à une impossibilité qu'aucune ingénierie ne contourne. Un seul processus fautif, par arrêt et non byzantin, suffit à rendre impossible tout protocole de consensus binaire garantissant la terminaison en système asynchrone [1]. La borne est 1, ce n'est ni  $f > n/3$  ni  $f > n/2$  [1]. Les hypothèses du résultat sont aussi importantes que son énoncé : le modèle interdit explicitement les horloges synchronisées, et les algorithmes fondés sur des expirations de délai ne peuvent donc pas y être employés ; il interdit également de détecter la mort d'un processus [1]. Et sa portée exacte doit être respectée : c'est une impossibilité de **terminaison**, pas de **sûreté** (*safety*) — les protocoles de la famille Paxos et Raft restent corrects sous ce résultat, ils y perdent seulement la garantie de progrès [1]. L'intuition tient en une phrase : un agent arbitrairement lent est indiscernable d'un agent arrêté.

La sortie de l'impossibilité a un prix chiffré. Un détecteur de défaillance (*failure detector*) est caractérisé par exactement deux propriétés, complétude et exactitude, et le consensus reste soluble avec des détecteurs commettant une infinité d'erreurs [65]. La séparation quantitative fixe la facture : l'algorithme fondé sur le détecteur fort tolère un nombre quelconque de défaillances, tandis que celui fondé sur le détecteur fortement exact à terme exige une **majorité** de processus corrects, et cette majorité est prouvée nécessaire [65]. Un quorum de

$2f + 1$  agents pour tolérer  $f$  fautes n'est donc pas une convention d'implantation : c'est le prix démontré de la terminaison sous synchronisme partiel. L'ordre de grandeur temporel se lit sur une implantation éprouvée, et la source pose son inégalité avant ses chiffres — temps de diffusion  $\ll$  délai d'élection  $\ll$  temps moyen entre pannes : le temps de diffusion  $y$  est estimé, non mesuré, à 0,5–20 ms selon la technologie de stockage, les appels exigeant une écriture durable, d'où un délai d'élection situé entre 10 ms et 500 ms, tiré au hasard dans un intervalle fixe — par exemple 150–300 ms — pour rendre les votes partagés rares ; après l'arrêt du meneur, le système est indisponible pendant environ le délai d'élection [88]. C'est la borne à porter au budget de disponibilité, et elle ne dépend pas de la charge.

Contourner le quorum sur le chemin des données est possible, à condition de dire où la facture est repayée. Le modèle de réplique synchronisée d'un journal (*log*) partitionné tolère  $f$  pannes avec  $f + 1$  répliques là où un vote majoritaire en exige  $2f + 1$  : tolérer une panne demande trois copies dans le second cas, deux dans le premier [5] — formulation de la source, dont le ch. 2 montre qu'elle s'énonce exactement en fonction du seuil d'accusé et non de  $k$ . L'économie est réelle et elle est massive à l'échelle d'un sujet à  $p$  partitions. Elle est payée une fois, ailleurs : l'ensemble des répliques synchronisées est maintenu par un contrôleur, et ce contrôleur est lui-même désigné par un quorum. Le plan de données évite l'accord ; le plan de contrôle le paie pour tout le monde.

Le théorème CAP délimite la même frontière depuis l'autre côté, et ses définitions sont plus fortes que leur usage courant. La cohérence  $y$  est l'atomicité, c'est-à-dire la linéarisabilité : toute lecture commencée après la fin d'une écriture doit rendre cette valeur ou une plus récente [2], propriété locale et composable [34]. La disponibilité  $y$  est définie strictement : **toute** requête reçue par un nœud non défaillant doit produire une réponse, sans borne de temps — ce qui est plus fort qu'un engagement de service [2]. L'impossibilité est prouvée dans le modèle asynchrone ; le modèle partiellement synchrone est traité séparément et admet des compromis [2]. L'auteur de la conjecture a lui-même qualifié la lecture « deux parmi trois » de trompeuse : hors partition réseau, cohérence et disponibilité sont toutes deux atteignables, et le cycle qu'il propose est explicite — détecter l'entrée en partition, entrer en mode dégradé en restreignant certaines opérations, exécuter une récupération de cohérence à la sortie [89].

Le terme « consensus logique » désigne dans la littérature du contrôle un objet différent, qu'il faut distinguer sous peine de non-sens : le consensus de moyenne fait converger asymptotiquement les états d'agents vers une valeur commune sous condition de connectivité, il ne satisfait l'accord à aucun instant fini et n'a pas de terminaison. Les digraphes équilibrés  $y$  sont fondamentaux [36]. La justification théorique de l'alignement observé en simulation a été donnée pour une règle décentralisée de moyenne des caps des plus proches voisins, alors même que l'ensemble des voisins de chaque agent change dans le temps ; et la précision technique le plus souvent perdue dans les citations de seconde main est que le système est stable sans qu'il existe une fonction de Lyapunov quadratique commune à toutes les configurations [20]. La transposition au logiciel conserve

la preuve, conditionnée à la connexité conjointe du graphe sur des fenêtres de temps. Elle casse la raison de croire à la condition : chez le robot, la connexité conjointe résulte du mouvement dans un espace physique, tandis que chez l'agent elle résulte du service d'échantillonnage, dont on sait que l'aléa ne tient pas globalement [81]. Un graphe conjointement connexe en espérance peut être déconnecté dans la réalisation, et la preuve, elle, ne s'en aperçoit pas.

**Tableau 12** — Coût et garantie des régimes de coordination, sous le modèle de faute crash-arrêt et omission. Les colonnes « messages » et « tours » sont indissociables : un coût en messages donné sans son coût en tours n'est pas un coût, et l'écart entre les deux dernières lignes est exactement le prix du synchronisme partiel.

| Régime                               | Synchronisme exigé                                        | Fautes tolérées                                     | Messages                                                                   | Tours                                                        | Terminaison                             | Garantie obtenue                             |
|--------------------------------------|-----------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------|----------------------------------------------|
| Rumeur épidémique (alg. 2)           | aucun ; tours logiques                                    | crash-arrêt, omission, partition réseau transitoire | $\Theta(n \log \log n)$ par rumeur, optimal si insensible à l'adresse [85] | $O(\log n)$                                                  | locale (cpt $\geq K$ ) ; jamais globale | couverture de probabilité $< 1$ [84]         |
| Anti-entropie                        | aucun                                                     | idem                                                | $\Theta(n)$ échanges par tour, message de la taille de l'état comparé      | non borné ; fréquence bridée par le coût de comparaison [84] | aucune                                  | couverture certaine à terme                  |
| Répliques fondées sur l'état         | aucun ; transport « à terme » suffisant                   | idem                                                | $\Theta(n)$ octets par message, $\Theta(n^2)$ par tour complet             | non borné                                                    | aucune                                  | cohérence à terme [87]                       |
| Répliques fondées sur les opérations | aucun, mais transport causal et exactement une fois exigé | celles que le transport masque                      | $\Theta(1)$ par opération                                                  | non borné                                                    | aucune                                  | cohérence à terme [87]                       |
| Consensus, synchronisme partiel      | $\Delta$ valide après un instant inconnu                  | $f < n/2$ arrêts ; majorité prouvée nécessaire [65] | $\Omega(n)$ par décision : une majorité doit être jointe [65]              | borné après stabilisation ; non borné avant [1]              | oui, conditionnelle                     | accord, validité, terminaison conditionnelle |
| Consensus, asynchrone pur            | aucun                                                     | une seule faute d'arrêt suffit à l'empêcher [1]     | —                                                                          | —                                                            | impossible [1]                          | sûreté seule ; aucun progrès garanti         |

Un dernier mode de défaillance de la propagation ne relève d'aucune ligne de ce tableau, parce qu'il ne tient ni au réseau ni au protocole : l'information circule et n'est pas émise. Lorsque les faits décisifs sont répartis de telle sorte que ce que les agents partagent déjà soutient la mauvaise option, et que chacun détient en propre une pièce qui soutient la bonne, des groupes de quatre agents qui délibèrent puis votent retiennent l'option cachée dans environ 85 % des épisodes pour le modèle le plus récent et dans 17 à 36 % pour tous les autres, sans qu'aucun modèle éprouvé ne tombe entre les deux, alors qu'un agent unique détenant tous les faits tranche juste dans la quasi-totalité des cas [119]. La source conclut que « la performance croît avec la capacité du modèle sans saturer au

sommet de la gamme éprouvée » [119] ; que cette distribution soit un saut et non une pente est une lecture de l'ouvrage. Le résultat reproduit ce que la littérature sur les groupes humains établit de longue date : la discussion converge vers ce que tout le monde sait déjà, et le fait non partagé n'est pas avancé, ou n'est plus soutenu une fois qu'un consensus apparent s'est formé [119]. La conséquence pour un essaim adossé à un journal est directe et contre-intuitive : le milieu résout le problème du transport et laisse entier celui de l'émission. Un mécanisme de diffusion parfait n'améliore d'aucun point une population qui n'écrit pas ce qu'elle est seule à savoir, et l'agrégat le mieux fondé — moyenne, vote, médiane des traces — pèse ce qui est écrit et non ce qui est su. Aucun des régimes du tableau ne distingue les deux ; le ch. 8 traite de ce que le milieu doit porter pour qu'une pièce minoritaire pèse ce qu'elle vaut.

*Note de section — Conformément à la charte, n'est exposée ici que la part du consensus que l'intitulé de la section impose : la frontière entre l'accord formel et la convergence d'états, et le prix de la franchir. La réplique de machine à états n'est pas réexposée.*

### 4.3 Adaptation environnementale

Deux classes d'événements déclenchent une reconfiguration, et une conception naïve les confond parce qu'elles se ressemblent à l'observation : la variation de charge, où le taux d'arrivée  $\lambda$  change et où  $\rho = \lambda/\mu$  franchit un seuil, et la panne partielle, où un agent s'arrête ou une partition réseau isole un groupe. Elles appellent des réponses opposées. À une hausse de charge, on ajoute des agents ; à une panne partielle, il faut au contraire réaffecter le travail du disparu sans ajouter personne, parce que le débit total n'a pas manqué — il a été redistribué. Un essaim qui répond à une panne partielle par une montée en charge injecte des agents dans un système en train de se réparer, et le rééquilibrage supplémentaire qu'il provoque est un coût net. La distinction n'est pas cosmétique : elle décide de ce qu'on mesure pour déclencher, la charge offerte ou l'appartenance.

Le modèle de faute reste crash-arrêt, omission et partition réseau ; les fautes byzantines sont hors modèle (renvoi ch. 7) et les pannes causées par la saturation appartiennent au ch. 2. Le modèle de synchronisme est partiellement synchrone, et il faut en tirer la conséquence désagréable : il existe une borne  $\Delta$  sur le délai de message, inconnue et valide seulement après un instant inconnu, ce qui signifie que tout mécanisme de cette section fondé sur une expiration de délai est correct après cet instant et ne l'est pas avant. Un essaim qui démarre, ou qui sort d'un incident réseau, opère dans la fenêtre où aucune de ses garanties temporelles ne tient.

La détection est le préalable, et son coût conditionne l'échelle. Les protocoles de battement de cœur classiques imposent une charge réseau croissant quadratiquement avec la taille du groupe, ou dégradent le temps de réponse et le taux de faux positifs [28]. Le protocole d'appartenance de style infectieux corrige exactement cela : le temps espéré de première détection d'une défaillance et la charge de messages espérée par membre sont indépendants de la taille du groupe, et la latence de dissémination ne croît que logarithmiquement avec le

nombre de membres [28]. Il n'a que deux paramètres — la période de protocole et la taille du sous-groupe de sondage indirect — et les horloges n'ont pas besoin d'être synchronisées, il suffit que la période soit une moyenne [28]. Sa mécanique par période tient en trois gestes : ping direct d'un membre tiré au hasard ; à défaut d'acquiescement avant une expiration choisie plus courte que la période, envoi d'une demande de sondage à un petit sous-groupe tiré au hasard, qui sonde par procuration ; propagation du verdict avec le trafic ordinaire [28]. L'extension par suspicion réduit le taux de fausses détections [28], sans l'annuler — et c'est le point de bascule de toute cette section. Un détecteur ne prouve pas la mort d'un agent, il la soupçonne, et la reconfiguration doit rester sûre quand le soupçon est faux. Les propriétés formelles du détecteur relèvent du ch. 3 ; leur usage à l'intérieur d'un protocole d'accord, du §4.2.

L'algorithme 4.3 énonce la reconfiguration d'allocation sur soupçon. Son coût se décompose en trois postes distincts. La propagation de la nouvelle vue coûte  $O(n \log \log n)$  messages en  $O(\log n)$  tours par le protocole de l'algorithme 2. Le recalcul de l'affectation est local et gratuit, puisque le hachage cohérent est une fonction pure de la vue [35] et que le départ d'un agent ne déplace qu'environ  $1/n$  des plages. Le transfert effectif de responsabilité coûte la relecture du point de reprise (*checkpoint*) de chaque plage reprise, coût proportionnel à la taille de l'état et non à  $n$ , et c'est en pratique le poste dominant. La condition d'arrêt est vérifiable : toute plage de l'anneau a un propriétaire dont le bail porte l'époque courante.

Algorithme 4.3 – Reconfiguration d'allocation sur soupçon, avec époque

ÉTAT de l'agent  $i$  :  $V$  – vue des agents non suspects

$\varepsilon$  – époque courante, entier monotone

$H$  – anneau de hachage cohérent, fonction pure de  $(V)$

DÉCLENCHEUR :  $i$  reçoit ou produit un soupçon portant sur l'agent  $m$

```

1  si  $m \notin V$  : ignorer et terminer
2   $V \leftarrow V \setminus \{m\}$  ;  $\varepsilon \leftarrow \varepsilon + 1$ 
3  diffuser (SUSPICION,  $m$ ,  $\varepsilon$ ) par l'algorithme 4.2
4   $R \leftarrow$  plages de  $H$  dont  $i$  devient propriétaire sous  $(V, \varepsilon)$ 
5  pour chaque plage  $p \in R$  :
6      lire dans le milieu le dernier point de reprise de  $p$  et son décalage validé  $d$ 
7      demander au milieu le bail  $(p, \varepsilon)$ 
8      si le milieu détient déjà un bail sur  $p$  d'époque  $> \varepsilon$  : abandonner  $p$ 
9      reprendre le traitement de  $p$  à partir de  $d$ , chaque effet estampillé  $\varepsilon$ 
10 si  $m$  réapparaît : ses écritures portent une époque  $< \varepsilon$ , le milieu les rejette
```

CONDITION D'ARRÊT : toute plage de  $H$  a un propriétaire dont le bail porte  $\varepsilon$ .

MODE DE DÉFAILLANCE : si le milieu ne sait pas comparer les époques,

la ligne 10 est inopérante et deux propriétaires écrivent simultanément.

La ligne 10 porte tout le poids de la correction, et elle énonce une condition forte qu'il serait malhonnête de laisser implicite. Une reconfiguration sûre sans aucun accord exige que le milieu arbitre : il doit détenir l'époque et refuser les écritures

périmées. Si le milieu ne sait pas le faire, la sûreté de l’essaim repose entièrement sur l’hypothèse que le détecteur ne se trompe jamais — hypothèse qui contredit frontalement la définition même du détecteur [65] — et il ne reste alors qu’à payer un quorum pour trancher l’appartenance (§4.2). Il n’y a pas de troisième voie. L’idempotence du traitement est la seconde condition, non négociable elle aussi : le rejeu (*replay*) depuis le décalage (*offset*) validé rejoue nécessairement des enregistrements déjà traités, et sans idempotence la reconfiguration produit un double effet à chaque panne.

La réponse à la variation de charge est une boucle de commande, et une boucle de commande se juge sur ses constantes de temps et sa bande morte. La formule d’un contrôleur horizontal de référence est  $\text{répliques\_visées} = \text{plafond}[\text{répliques\_courantes} \times (\text{métrique\_courante} / \text{métrique\_visée})]$ , avec une tolérance par défaut de 0,1 — aucune action n’est prise si le rapport est assez proche de 1,0 — et une période de synchronisation par défaut de 15 s ; le délai de disponibilité initiale vaut 30 s et la période d’initialisation processeur 5 min [45]. Le budget de réaction à un échelon de charge se calcule à partir de ces valeurs et non d’une intuition, et il s’écrit comme une somme de temps morts en série, tous en secondes — au mieux une période de synchronisation de 15 s pour que l’écart soit vu, plus le temps de démarrage de l’agent, plus 30 s de délai de disponibilité initiale avant qu’il ne compte dans la métrique — soit un plancher de 45 s hors démarrage, et 5 min de plus si la métrique retenue est la consommation processeur, dont la période d’initialisation diffère d’autant la prise en compte. Rapporté à la période d’échantillonnage, ce temps mort vaut au moins trois périodes : le contrôleur émet trois décisions avant que la première ait produit un effet observable. La bande morte de 10 % n’est pas une commodité : sans elle, une boucle à retard oscille. Elle ne suffit pas pour autant à l’amortir : elle ne retient l’action que tant que le rapport de la métrique courante à la métrique visée reste dans  $1,0 \pm 0,1$ , et rien n’empêche la charge de traverser ce couloir plusieurs fois pendant les 45 s d’aveuglement ; les trois corrections s’empilent alors sur une seule et même cause, le nombre de répliques dépasse la cible, puis la boucle repart symétriquement à la baisse. L’amortissement effectif ne vient donc pas de la bande morte, qui est un rapport sans unité, mais de la fenêtre de stabilisation qui filtre les décisions successives [45] — un paramètre que la formule ne contient pas. Le dimensionnement vertical automatique produit un gain mesuré — une part de ressources réservées non utilisées de 23 % contre 46 % pour les tâches gérées manuellement, une réduction d’un facteur 10 des incidents graves d’épuisement mémoire, sur plus de 48 % de l’usage de ressources d’une flotte industrielle — et le diagnostic causal est énoncé sans détour : la peur de l’étranglement pousse les opérateurs humains à surdimensionner [90].

Deux limites structurelles de cette famille de mécanismes doivent être écrites, parce qu’elles sont invisibles à l’usage. Les autoscaleurs de nœuds ne considèrent que les **requêtes** de ressources déclarées, jamais l’utilisation réelle : sans dimensionnement vertical en amont, un mauvais réglage des requêtes rend la mise à l’échelle de l’infrastructure structurellement fautive [46]. Et la combinaison d’un contrôleur horizontal et d’un contrôleur vertical sur la même métrique est un

conflit connu — l’un fait varier le nombre de répliques, l’autre les requêtes — que la documentation officielle n’aborde pas ; de ce qu’elle ne l’interdit pas, on ne peut pas déduire qu’elle le prend en charge [91].

La reconfiguration ne se déclenche pas seulement, elle se borne. Trois plafonds valent d’être posés en chiffres. Les sondes de disponibilité et de vivacité ont des effets distincts et non substituables — la première retire l’agent des points d’accès sans le redémarrer, la seconde le redémarre — et leurs valeurs par défaut, période 10 s, expiration 1 s, seuil d’échec 3, fixent à 30 s le délai avant qu’un agent muet ne soit retiré [47]. Ces 30 s se comparent à ce que le §4.2 a chiffré pour l’accord, et deux ordres de grandeur les séparent du délai d’indisponibilité qui suit l’arrêt d’un meneur, 150–300 ms [88] : une bascule de meneur se conclut donc bien avant que la sonde n’ait accumulé son premier échec, une sonde réglée par défaut ne voit jamais d’élection, et descendre sa période à l’échelle de l’élection convertirait chaque bascule de quelques centaines de millisecondes en un redémarrage de plusieurs secondes. Le budget de perturbation ne contraint que les perturbations volontaires passant par l’interface d’éviction ; les perturbations involontaires, dont la panne matérielle et la partition réseau, comptent contre le budget sans pouvoir être empêchées, ce qui en fait un plafond et non une garantie [48]. Enfin, la charge que l’essaim s’inflige à lui-même pendant la reprise doit être étranglée avant que la reconfiguration commence : l’étranglement adaptatif côté client rejette localement, de façon probabiliste, dès que le nombre de requêtes dépasse  $K$  fois le nombre d’acceptations, avec  $K$  typiquement égal à 2 sur une fenêtre glissante de deux minutes, les reprises étant plafonnées à 3 tentatives par requête et à 10 % du volume total de requêtes d’un client [51]. La taxonomie de criticité qui décide des victimes du délestage (*load shedding*) compte quatre niveaux, du plus protégé au plus sacrificable [51] ; le mécanisme lui-même appartient au ch. 2. Quant à la désynchronisation des reprises par gigue, les résultats disponibles proviennent d’une simulation à 100 clients en contention, avec des délais réseau moyens de 10 ms et 4 ms de variance, décrite dans un billet d’ingénierie sans tableau de valeurs : la gigue pleine y réduit de plus de moitié le nombre d’appels par rapport à un retrait exponentiel sans gigue [53]. Ce sont des résultats de simulation publiés hors comité de lecture, et ils ne doivent pas être présentés comme des mesures de production.

La transposition depuis la robotique en essaim est ici plus instructive qu’ailleurs, parce que c’est le mode de faute lui-même qui change. L’auto-assemblage programmable de formes complexes par un essaim de mille robots repose sur des interactions strictement locales, et ses algorithmes de formation sont explicitement conçus pour tolérer la variabilité inhérente aux grands systèmes décentralisés [16] ; la méthodologie de modélisation micro/macro validée sur des équipes allant jusqu’à 600 robots montre par ailleurs que des modèles abstraits donnent des prédictions quantitativement exactes pour un coût de calcul très inférieur à la simulation incarnée [18]. Ce que la transposition conserve est le principe de conception : écrire l’algorithme pour la dispersion de la population plutôt que pour un membre nominal, et vérifier le comportement collectif sur

un modèle agrégé plutôt qu'en simulant chaque agent. Ce qu'elle casse est le modèle de panne. Un robot mort reste mort et reste sur place : c'est un obstacle statique, à position connue, et le reste de l'essaim le contourne. Un agent logiciel soupçonné mort peut revenir, avec son état d'avant, et écrire. L'essaim robotique vit sous une sémantique proche de l'arrêt franc, l'essaim logiciel sous une sémantique d'arrêt-reprise, strictement plus difficile ; c'est exactement pourquoi l'époque et le bail des lignes 7 à 10 de l'algorithme 4.3 n'ont aucun équivalent dans les protocoles robotiques dont ils descendent.

Ce que la reconfiguration ne répare pas doit être délimité, sans quoi on lui attribuera les pannes des autres. Elle ne répare que les défaillances imputables à un agent ou à un lien identifiable. Une étude empirique portant sur plus de 1 600 traces annotées, collectées sur sept cadriciels multi-agents, établit une taxonomie de 14 modes de défaillance uniques regroupés en trois catégories — défauts de conception du système, désalignement entre agents, défaut de vérification de la tâche — validée par un accord inter-annotateurs de  $\kappa = 0,88$  sur 150 traces [92]. Les deux dernières catégories décrivent des échecs où **aucun agent n'est tombé** : aucune reconfiguration ne les corrige, et elles appartiennent au ch. 7. La seule manière de savoir si la reconfiguration fonctionne est de la déclencher délibérément, ce que la vérification de fiabilité par l'expérimentation propose comme discipline [79] ; ses cinq principes — bâtir une hypothèse sur le régime stable, varier les événements réels, exécuter en production, automatiser, minimiser le rayon d'explosion [78] — constituent un vocabulaire utile, mais l'article fondateur est un texte de position et de retour d'expérience, sans protocole expérimental reproductible ni mesure comparative [79], et le manifeste qui les énumère est un document communautaire figé depuis 2019 [78]. Les citer comme source de méthode validée serait une faute ; les citer comme la seule réponse disponible à la question « ce mécanisme s'est-il déjà déclenché ? » est exact.

Les constantes chiffrées de cette section ne se comparent qu'une fois leur unité posée, et cette section n'en emploie que trois. La milliseconde mesure ce qui se joue à l'intérieur d'un aller-retour de message et de son écriture durable : temps de diffusion, délai d'élection, indisponibilité consécutive à une bascule de meneur. La seconde mesure ce qu'une boucle de commande échantillonne ou attend avant de compter une observation : période de sonde, période de synchronisation, délai de disponibilité initiale. La minute mesure ce qu'une fenêtre statistique accumule avant de conclure : période d'initialisation processeur, fenêtre d'étrangement adaptatif. La convention de composition qui accompagne ces unités est celle d'une somme de temps morts en série : un budget de reconfiguration est la somme des constantes que l'événement traverse, converties dans l'unité de son terme dominant, et non la plus grande d'entre elles — c'est ce qui fait qu'un plancher de 45 s se compose de deux constantes de dizaines de secondes, et qu'un délai d'élection de 300 ms n'y pèse rien. Trois valeurs du tableau ne portent pas de temps et n'entrent dans aucune somme : la bande morte, rapport sans unité de 0,1 ; le seuil d'échec, un compte de trois échecs consécutifs ; le plafond de reprise, une fraction de 10 % du volume.

**Tableau 13** — Constantes de temps et seuils par défaut des mécanismes de reconfiguration, telles qu'établies par leurs sources respectives, dans les trois unités posées au paragraphe précédent. La seule valeur dérivée est signalée comme telle ; les valeurs de documentation produit sont périssables et doivent être revalidées à la version ciblée.

| Mécanisme                      | Constante                                       | Valeur par défaut ou mesurée                                    | Source                      |
|--------------------------------|-------------------------------------------------|-----------------------------------------------------------------|-----------------------------|
| Élection de meneur (consensus) | temps de diffusion                              | 0,5–20 ms selon le stockage                                     | [88]                        |
| Élection de meneur (consensus) | délai d'élection                                | 150–300 ms, tiré au hasard dans un intervalle fixe              | [88]                        |
| Élection de meneur (consensus) | indisponibilité après arrêt du meneur           | ≈ un délai d'élection, soit 150–300 ms                          | [88]                        |
| Détection d'appartenance       | période de protocole, sous-groupe de sondage    | deux paramètres seulement ; charge par membre indépendante de n | [28]                        |
| Sonde de disponibilité         | période / expiration / seuil d'échec            | 10 s / 1 s / 3 échecs consécutifs                               | [47]                        |
| Contrôleur horizontal          | période de synchronisation                      | 15 s                                                            | [45]                        |
| Contrôleur horizontal          | bande morte                                     | 0,1 (10 %), rapport sans unité                                  | [45]                        |
| Contrôleur horizontal          | délai de disponibilité initiale                 | 30 s                                                            | [45]                        |
| Contrôleur horizontal          | période d'initialisation processeur             | 5 min                                                           | [45]                        |
| Contrôleur horizontal          | temps mort avant qu'une réplique ajoutée compte | ≥ 45 s (15 s + 30 s), hors démarrage de l'agent                 | somme calculée d'après [45] |
| Étranglement adaptatif         | seuil de rejet local                            | requêtes > 2 × acceptations, fenêtre de 2 min                   | [51]                        |
| Budget de reprise              | tentatives et plafond de volume                 | 3 tentatives ; 10 % du volume client                            | [51]                        |

L'adaptation la mieux documentée sur une population d'agents de langage n'est ni un réglage de paramètre ni une reconfiguration : c'est une spécialisation. Quarante-cinq agents dotés chacun de sa machine virtuelle, d'un forum commun et d'une consigne identique — chercher des vulnérabilités dans quinze projets libres, se relire mutuellement, un agent arbitre tranchant la nouveauté et la validité de chaque soumission — se construisent leurs propres outils et se spécialisent dans des types de vulnérabilités distincts, sans qu'aucune répartition ne leur ait été prescrite [119]. C'est le comportement que la robotique en essaim appelle division du travail émergente, obtenu ici sans seuil de réponse ni renforcement explicite, par le seul fait que chacun lit ce que les autres ont déposé ; le §4.1 en donne le mécanisme. Ce que la transposition ajoute est propre au logiciel et n'a aucun équivalent robotique : l'agent ne se spécialise pas seulement en choisissant ses tâches, il se fabrique l'outil qui les traite, donc il modifie sa propre fonction de transition en cours d'exécution. Cette modification est hors du modèle du ch. 1, où  $\delta$  est fixe à un vecteur de paramètres près, et aucun des cadres formels du ch. 3 ne la représente. Le ch. 8 mesure ce que cette adaptation rapporte et ce qu'elle coûte en jetons ; il faut noter ici que le seul mécanisme

d'adaptation ayant produit un gain net mesuré sur une population d'agents de langage est aussi celui qu'aucun modèle de cet ouvrage ne sait décrire.

## 5. Prise de décision collective et allocation de tâches

### 5.1 Mécanismes de consensus

Le mot **consensus** garde ici l'acception stricte de la charte : accord, validité, terminaison. Le ch. 4 possède le sujet ; rien de ce qui suit ne réexpose un protocole d'accord. Cette section traite des procédures par lesquelles une population d'**agents** produit un choix collectif *en abandonnant délibérément au moins une* des trois propriétés — reste à nommer laquelle, à quel prix, et quand le choix redevient faux. Abandonner la terminaison laisse l'**essaim** indécis ; abandonner l'accord le laisse divisé sans qu'aucun agent le sache.

La borne qui gouverne la section appartient au ch. 4 ; on n'en retient que la conséquence. En asynchrone — délais non bornés, aucune horloge, temporisations explicitement interdites —, **un** processus fautif, par simple arrêt silencieux, rend impossible tout protocole de consensus binaire garantissant la terminaison [1] : un processus arbitrairement lent est indiscernable d'un processus mort. L'impossibilité porte sur la terminaison, non sur la **sûreté** [1], et la racheter coûte une hypothèse de synchronisme plus une majorité de processus corrects, *prouvée nécessaire* [65]. Le ch. 5 refuse de payer.

✓ TENUE · ~ TENUE SOUS CONDITION · ✗ ABANDONNÉE

|                                 | Accord           | Validité                  | Terminaison      | Messages                     |
|---------------------------------|------------------|---------------------------|------------------|------------------------------|
| Seuil de quorum (best-of-n)     | ✗ sous partition | ✓                         | ~ non borné      | $\Theta(n \cdot d)$ par tour |
| Moyenne et alignement           | ~ à terme        | ✗                         | ✗ asymptotique   | $\Theta(n \cdot d)$ par tour |
| Fusion sans conflit (CRDT)      | ✓                | ✗ le refus est impossible | ✗ quiescence     | aucun message dédié          |
| Consensus, synchronisme partiel | ✓                | ✓                         | ✓ conditionnelle | $\Omega(n)$ par décision     |

§ 5.1 — « non borné » signifie qu'aucune borne finie n'existe sous les hypothèses posées, et non qu'elle est grande.

Chaque mécanisme abandonne délibérément au moins une des trois propriétés. Abandonner la terminaison laisse l'essaim indécis ; abandonner l'accord le laisse divisé sans qu'aucun agent le sache — et c'est le second qui se paie le plus cher, parce qu'il ne se voit pas. Sous faute arbitraire, aucune solution à moins de  $3f + 1$  participants ne tolère  $f$  déviants [94] : ce modèle tue les trois premières lignes.

**Figure 5.1** — Ce que chaque mécanisme de choix distribué abandonne parmi accord, validité et terminaison.

Quatre conventions valent pour le chapitre. Un envoi vers  $n$  destinataires compte pour  $n$  messages, diffusion comprise. Le **tour** est l'unité de délai posée au §1.1, un aller-retour bloquant ; un nombre de tours non borné s'écrit non borné, jamais remplacé par une moyenne plausible. Une décision est **révocable** si un

agent peut l'annuler par une action locale de coût borné et connu. Et chaque mécanisme pose son modèle de panne, son synchronisme et ses hypothèses de canal avant son algorithme, aucun des trois ne supposant le même modèle ; le modèle de faute par défaut de l'ouvrage — crash-arrêt, omission, **partition réseau** — appartient au ch. 4.

Décision par seuil de quorum (best-of-n). Modèle de panne : crash-arrêt avec omission ; nul ne ment, et un agent arrêté cesse d'émettre son opinion, ce qui le retire de l'échantillon de ses voisins au bout d'un tour sans qu'ils aient à le détecter. Synchronisme : partiellement synchrone, avec une borne  $\Delta$  sur le délai, inconnue et valide seulement après stabilisation. Canal : perte possible et non signalée ; duplication sans effet, une opinion étant un état et non un incrément ; expéditeur identifiable ; domaine des opinions fini et commun. Le problème best-of-n demande un choix collectif pour l'option  $i \in \{1, \dots, n\}$  qui maximise le bénéfice du collectif et minimise son coût, et la décision est établie quand une large majorité  $M \geq (1 - \delta)N$  d'agents favorise la même option,  $\delta$  fixé par le concepteur avec  $\delta \ll 0,5$  ; à  $\delta = 0$  l'essaim vise l'unanimité, donc le consensus du ch. 4, et sort de cette section [93]. La règle de k-unanimité fixe le compromis vitesse/exactitude par un seul entier : un agent ne change d'opinion qu'après k agents consécutifs favorisant une autre option [93]. Le renforcement positif est modulé, chacun annonçant son opinion pendant une durée proportionnelle à la qualité échantillonnée [93]. D'où la procédure DÉCISION-PAR-SEUIL :

DÉCISION-PAR-SEUIL(agent  $i$ )

```

1   $o_i \leftarrow$  option échantillonnée localement par  $i$ 
2   $q_i \leftarrow$  qualité mesurée de  $o_i$                                 ▶ estimation locale, bruitée
3  répéter
4      diffuser  $\langle i, o_i \rangle$  pendant une durée  $\tau \cdot q_i$             ▶ renforcement modulé
5       $V \leftarrow$  opinions  $\langle j, o_j \rangle$  reçues durant le tour
6      si  $|V| < k$  alors passer au tour suivant
7      si les k dernières opinions de  $V$  portent toutes  $m \neq o_i$ 
8          alors  $o_i \leftarrow m$                                     ▶ règle de k-unanimité
9      si  $|\{v \in V : v.o = o_i\}| / |V| \geq 1 - \delta$ 
10         alors engager  $o_i$  et sortir                                ▶ détection de quorum
11 fin répéter
```

Coût par tour et par agent :  $d$  messages émis,  $d$  reçus ; pour l'essaim entier  $\Theta(n \cdot \bar{d})$  messages par tour,  $\bar{d}$  étant le degré moyen du graphe. Coût en tours : non borné, et il ne peut en aller autrement — la ligne 9 teste une fraction de l'échantillon local, elle ne constate aucun accord, et rien n'interdit à deux sous-populations de la franchir au même tour sur deux options différentes. Trois modes de défaillance, par gravité croissante. Indécision : à qualités voisines, la ligne 4 ne différencie plus rien, le problème dégénère en brisure de symétrie que seule l'amplification du bruit résout [93], et la **vivacité** tombe pendant que la sûreté tient. Décision divisée : une partition réseau sépare l'essaim, chaque côté franchit  $(1 - \delta)$  localement, deux décisions incompatibles sont engagées et *aucun agent ne le détecte*. Verrouillage : le renforcement est absorbant, un biais initial d'échantillonnage fige l'essaim sur l'option inférieure, et augmenter

k ralentit sans supprimer. Le mécanisme n'est admissible que si l'engagement de la ligne 10 est révoable.

Moyenne distribuée et alignement. Modèle de panne : crash-arrêt sans omission à l'intérieur d'un tour ; un agent arrêté disparaît du voisinage de ses pairs, disparition traitée comme une commutation de topologie et non comme une faute à masquer. Synchronisme : tours synchrones, chaque agent mettant à jour son état une fois par tour. Canal : livraison en un tour, sans réordonnancement ni duplication ; une perte se lit comme une arête absente du graphe de ce tour, et c'est cette identification qui rend applicable le résultat de convergence. Le modèle de Vicsek fixe le cadre minimal : chaque particule adopte la direction moyenne de ses voisines avec un bruit aléatoire à chaque pas ; sous un bruit critique on obtient un transport collectif ordonné, au-dessus le flux net est nul [11]. D'où la procédure **MOYENNE-LOCALE**, pour un agent  $i$  d'état  $x_i$  — un cap, une charge estimée, un seuil :

```

MOYENNE-LOCALE(agent i)
1   $x_i \leftarrow$  état initial de  $i$ 
2  répéter à chaque tour  $t$ 
3      diffuser  $\langle i, x_i \rangle$  vers  $N_i(t)$                                 ▶ voisinage du tour, variable
4       $R \leftarrow$  états  $\langle j, x_j \rangle$  reçus au tour  $t$ 
5       $x_i \leftarrow (x_i + \text{somme des } x_j \text{ de } R) / (1 + |R|)$  ▶ moyenne,  $i$  compris
6      si  $|x_i - x_i(t-1)| \leq \varepsilon$  sur  $T$  tours consécutifs
7          alors publier  $x_i$                                           ▶ arrêt local, non concerté
8  fin répéter

```

La convergence n'est pas acquise par la règle : elle l'est par le graphe, sous l'hypothèse de connexité conjointe sur une suite infinie d'intervalles contigus, non vides et bornés, dont le ch. 1 énonce le théorème avec ses hypothèses exactes [20]. Trois de ses conséquences gouvernent la présente section, et la citation de seconde main les perd. La valeur limite dépend de l'état initial et de la suite des graphes : ce n'est pas la moyenne des états initiaux, et le mécanisme n'offre aucune validité au sens du ch. 4. Le système est un système linéaire commuté stable pour lequel il n'existe pas de fonction de Lyapunov quadratique commune [20], ce qui rend faux tout raisonnement qui en suppose une. Et aucune garantie n'assure que, le long d'une trajectoire donnée, les agents soient effectivement liés ensemble [20] : la connexité conjointe est une hypothèse sur l'exécution, pas une propriété que l'algorithme établit.

Coût par tour :  $\Theta(n \cdot \bar{d})$  messages, un par arête active et par sens. Coût en tours : non borné, pour deux raisons distinctes. La convergence est asymptotique — l'écart décroît vers zéro sans l'atteindre en temps fini —, si bien qu'aucune condition d'arrêt exacte n'existe et que la ligne 6 substitue un seuil  $\varepsilon$  et une fenêtre  $T$  à une terminaison. Et la vitesse tient au graphe : Olfati-Saber et Murray relient la connectivité algébrique à la performance du protocole et établissent que les digraphes **équilibrés** sont fondamentaux pour le consensus de moyenne [36] ; la synthèse de 2007 confirme que les propriétés spectrales et structurelles gouvernent la vitesse de convergence [29]. Quatre modes de défaillance : lenteur, la connectivité algébrique s'effondrant sans que nul distingue une convergence lente d'une convergence absente ; arrêt prématuré, deux sous-

populations franchissant  $\varepsilon$  sur deux valeurs différentes puisque la ligne 6 mesure une variation et non un accord ; dérive, le digraphe n'étant pas équilibré et la somme des états cessant d'être invariante ; capture enfin, un agent qui n'exécute jamais la ligne 5 étant un point fixe dont la valeur est, sur un graphe connexe, la seule configuration stationnaire compatible — faute de validité, un seul agent figé décide pour l'essaim.

C'est là que la transposition robotique  $\rightarrow$  logiciel casse. Elle conserve la règle de mise à jour, sa localité, son insensibilité au nombre d'agents, et le fait que la convergence dépende de la connectivité et non de l'identité des voisins. Elle casse deux choses. Le voisinage d'un robot est métrique donc symétrique, ce qui rend le graphe naturellement équilibré, alors que celui d'un agent logiciel est défini par abonnement ou par routage, donc asymétrique par défaut : un agent qui consomme un **sujet** à fort éventail de diffusion reçoit de mille pairs et n'émet vers aucun, et la limite se trouve pondérée par la topologie de diffusion. Et la perte de message retire des arêtes de façon corrélée — une saturation de **courtier** (voir ch. 2) supprime tous les liens qui le traversent, sans que rien ne garantisse que les intervalles restent conjointement connexes pendant qu'elle dure.

Résolution de conflits sans décision. Le troisième mécanisme renonce au vote. Modèle de panne : crash-arrêt, omission, duplication et réordonnancement — le plus large des trois. Synchronisme : aucun ; le mécanisme est correct en asynchrone total et ne consulte aucune horloge. Les types de données répliqués convergents et commutatifs fondent la conception sur des conditions formelles simples et suffisantes garantissant la **cohérence à terme**, plutôt que sur un protocole de résolution de conflit [87]. Deux familles, deux conditions suffisantes distinctes, et surtout deux hypothèses de canal distinctes — c'est cette troisième différence que la citation courante escamote. La réplication fondée sur l'état exige que les états forment un treillis à jonction monotone, et sa condition de vivacité est énoncée telle quelle : deux répliques convergent pourvu que le système transmette leur charge utile une infinité de fois, par paires, sur des canaux point à point à livraison à terme [87]. La réplication fondée sur les opérations exige que les opérations concurrentes commutent, et son canal doit être une diffusion fiable livrant chaque mise à jour à chaque réplique dans un ordre respectant les préconditions ; la livraison causale y suffit, et la duplication doit être éliminée, le rapport conservant l'ensemble des messages déjà livrés pour l'empêcher [87]. Confondre les deux familles est fatal : un type de la seconde dont la livraison n'est pas exactement-une-fois ne converge pas, quand un type de la première survit au **rejeu** parce que sa jonction est **idempotente** [87].

Coût : zéro message dédié et zéro tour d'accord, la fusion s'effectuant sur le chemin de réplication déjà payé. Condition d'arrêt : il n'y en a pas, et c'est le point — la convergence est une propriété de quiescence, définie à l'absence de nouvelles mises à jour, sans instant où un agent puisse la constater. Le mécanisme est en revanche immunisé à la partition réseau, les répliques d'un sous-ensemble connecté continuant de se livrer mutuellement leurs mises à jour sans qu'aucun accord soit requis [87]. Mode de défaillance : la fusion accepte

toujours, donc le type ne peut porter aucun invariant qui exige un refus — unicité, budget, exclusion mutuelle, capacité maximale. C’est ce qu’un essaim annoncé « AP » achète sans toujours le savoir : dans le théorème CAP, la cohérence est la linéarisabilité et la disponibilité exige que *toute* requête reçue par un nœud non défaillant produise une réponse, sans borne de temps [2].

**Tableau 14** — Mécanismes de choix distribué. « Non borné » signifie qu’aucune borne finie n’existe sous les hypothèses posées.

| Mécanisme                   | Modèle de panne                                      | Synchronisme            | Messages / tour           | Tours jusqu’à l’arrêt    | Condition d’arrêt                   | Propriété abandonnée                |
|-----------------------------|------------------------------------------------------|-------------------------|---------------------------|--------------------------|-------------------------------------|-------------------------------------|
| Seuil de quorum (best-of-n) | crash-arrêt, omission                                | partiellement synchrone | $\Theta(n \cdot \bar{d})$ | non borné                | fraction locale $\geq 1 - \delta$   | accord, sous partition réseau       |
| Moyenne / alignement        | crash-arrêt                                          | tours synchrones        | $\Theta(n \cdot \bar{d})$ | non borné ; asymptotique | écart $\leq \epsilon$ sur $T$ tours | validité, terminaison en temps fini |
| Fusion CRDT                 | crash-arrêt, omission, duplication, réordonnancement | asynchrone              | 0 dédié                   | 0 ; quiescence           | aucune, constatable par aucun agent | validité, le refus étant impossible |

Une borne dépasse ce tableau : sous faute arbitraire, aucune solution à moins de  $3f + 1$  participants ne tolère  $f$  déviants [94]. Ce modèle, repris au ch. 7, tue les trois mécanismes ci-dessus.

Reste le verdict pratique, et il contredit ce que la section vient de construire. Gerkey et Matarić concluent que le temps de transmission d’un message ne peut être ignoré, en particulier sur réseau sans fil où il induit une latence significative, et que pour les systèmes de petite à moyenne échelle — ils écrivent  $n < 200$  — une solution centralisée par diffusion est vraisemblablement le meilleur choix [95]. La décision distribuée sans autorité se justifie quand  $n$  dépasse le point où le coordonnateur devient le facteur limitant, et ce point se mesure.

## 5.2 Allocation dynamique de tâches

L’**allocation** se formalise avant de se négocier. Gerkey et Matarić établissent une taxonomie des problèmes d’allocation multi-robots indépendante du domaine et montrent que chaque classe est une instance d’un problème d’optimisation déjà étudié, ce qui transporte les bornes avec elle [95]. La classe centrale — agents mono-tâche, tâches mono-agent, affectation instantanée — est le problème d’affectation optimale :  $m$  agents,  $n$  tâches priorisées, une utilité estimée pour chacune des  $m \cdot n$  paires, au plus une tâche par agent ; si les utilités peuvent être rassemblées sur une machine, la méthode hongroise trouve l’optimum en  $O(m \cdot n^2)$  [95]. Dès qu’une tâche requiert plusieurs agents, le problème devient un partitionnement d’ensemble, fortement NP-difficile [95] ; la taxonomie iTax l’étend aux contraintes d’utilité et aux dépendances entre tâches [96]. La difficulté ne vient donc pas de la distribution : elle est déjà dans l’énoncé centralisé.

Deux bornes de qualité gouvernent tout écart à l’optimum, et aucune n’est contournable par un protocole. Un algorithme est  $\rho$ -compétitif si, pour toute

entrée, l'utilité totale de sa solution n'est jamais inférieure à  $1/\rho$  de l'utilité optimale [95]. Le glouton est 2-compétitif pour l'affectation optimale [95]. Et pour l'affectation *en ligne* — les tâches arrivent une à une, un agent déjà affecté ne peut être réaffecté —, il est 3-compétitif par rapport à la solution optimale calculée après coup hors ligne, et cette borne est la meilleure possible pour tout algorithme d'affectation en ligne [95]. C'est le résultat d'impossibilité propre au chapitre : sans modèle des arrivées futures et sans réaffectation, aucun allocateur — par **enchère** ou non — ne fait mieux que trois fois l'optimum hors ligne. Qui promet mieux achète l'écart contre l'une des deux hypothèses.

**L'affectation centralisée**, que les mécanismes suivants prétendent remplacer, se chiffre comme eux. Modèle de panne : aucun n'est traité — le coordonnateur est supposé vivant, et son arrêt entre la collecte et la diffusion perd l'affectation entière. Synchronisme : indifférent, l'attente étant celle d'un calcul local et non d'un rendez-vous. Coût :  $n^2$  messages pour transmettre l'utilité de chaque robot pour chaque tâche [95], en un tour — collecte, puis diffusion. Condition d'arrêt : la fin du calcul, événement observable, que nul mécanisme du §5.1 ne possède. Le calcul n'est pas le goulot — les auteurs résolvent en moins de 1 s les instances à 300 robots et 300 tâches sur un Pentium III à 700 MHz [95]. Défaillance dominante : le point de défaillance unique, et  $n^2$  messages qui saturent le lien partagé bien avant que le solveur ne peine.

**Le Contract Net**. Modèle de panne : crash-arrêt et omission, sans détection — un soumissionnaire muet est indiscernable d'un soumissionnaire lent. Synchronisme : partiellement synchrone, l'expiration de l'appel d'offres tenant lieu de borne  $\Delta$  supposée. Chez Smith, l'attribution s'opère par négociation, les nœuds discutant des tâches qu'ils peuvent exécuter [97]. Quatre phases en deux allers-retours : (1) le donneur d'ordre annonce la tâche aux  $n$  agents,  $n$  messages ; (2) les intéressés soumettent, au plus  $n$  messages ; (3) le donneur d'ordre attribue à un seul, 1 message ; (4) l'attributaire accuse réception, 1 message. Soit  $2n + 2$  messages et 2 tours par tâche : un tour de plus que l'affectation centralisée contre un facteur  $n$  en messages. Condition d'arrêt : l'accusé reçu, ou l'expiration qui relance la phase 1. La négociation est locale mais le donneur d'ordre est unique par tâche : autorité éphémère, non absence d'autorité. Défaillance propre : son arrêt entre les phases 3 et 4 laisse la tâche attribuée sans que nul le sache, et sans **journal** elle est perdue ; l'expiration produit l'inverse, deux attributaires pour une tâche que le premier exécutait sans le dire assez vite.

**L'enchère  $\epsilon$  de Bertsekas**. Modèle de panne : aucun. L'algorithme est posé sans faute, sans perte de message et sans départ d'agent, et c'est l'hypothèse forte à écrire avant toute transposition. Synchronisme : indifférent, l'algorithme étant correct en version synchrone comme totalement asynchrone. Chaque agent  $i$  attribue un bénéfice  $a_{ij}$  à l'objet  $j$ , qui porte un prix  $p_j$  ; la condition d'optimalité approchée est l' $\epsilon$ -relâchement des écarts complémentaires, l'agent  $i$  étant *presque satisfait* si la valeur de son objet assigné  $j_i$  est à  $\epsilon$  près de la valeur maximale,  $a_{ij_i} - p_{j_i} \geq \max_j (a_{ij} - p_j) - \epsilon$  [98]. D'où la procédure ENCHÈRE- $\epsilon$  :

ENCHÈRE- $\epsilon$ (agent  $i$ )

```

1  tant que  $i$  n'est pas presque satisfait
2       $v_i \leftarrow \max_j (a_{ij} - p_j)$  ► meilleure valeur nette
3       $j_i \leftarrow \operatorname{argmax}_j (a_{ij} - p_j)$ 
4       $w_i \leftarrow \max \text{ des } (a_{ij} - p_j) \text{ pour } j \neq j_i$  ► deuxième meilleure valeur
5       $\gamma_i \leftarrow v_i - w_i + \epsilon$  ► incrément d'enchère
6      soumettre l'offre  $\langle i, j_i, p_{j_i} + \gamma_i \rangle$ 
7  fin tant que
8  phase d'attribution, pour chaque objet  $j$  ayant reçu des offres :
9       $p_j \leftarrow$  plus haute offre reçue ;  $j$  va à son émetteur
10  l'agent précédemment détenteur de  $j$  redevient non assigné

```

La version naïve, où  $\gamma_i = v_i - w_i$  sans le terme  $\epsilon$ , ne fonctionne pas toujours : l'incrément est nul dès que plusieurs objets offrent la valeur maximale, et des agents se disputent alors un plus petit nombre d'objets également désirables sans jamais en élever le prix, créant un cycle sans fin [98]. Le terme  $\epsilon$  est la perturbation qui brise le cycle ; ce n'est pas un paramètre de réglage, c'est la condition de terminaison. La preuve d'arrêt tient en deux observations. Dès qu'un objet a reçu une offre, son détenteur reste presque satisfait aux tours ultérieurs puisque les autres prix ne peuvent que croître ; les agents non presque satisfaits sont donc assignés à des objets n'ayant jamais reçu d'offre. Et le prix d'un objet croît d'au moins  $\epsilon$  à chaque tour où il reçoit une offre, si bien qu'il devient en un nombre fini de tours assez cher pour être jugé inférieur à un objet encore vierge [98]. Coût par tour : les deux phases sont parallélisables pour tous les agents et tous les objets simultanément [98], et un tour coûte au plus une offre par agent non presque satisfait plus une notification par objet enchéri, soit  $O(n)$  messages ; le total pour atteindre l'équilibre est souvent inférieur à  $n$  messages [95]. Coût en tours : proportionnel à  $C/\epsilon$ , où  $C = \max_{ij} |a_{ij}|$  [98].

La qualité est bornée exactement : le bénéfice total de l'affectation finale est à  $n \cdot \epsilon$  près de l'optimum, et le vecteur de prix final à  $n \cdot \epsilon$  près d'une solution optimale du problème dual [98]. Si les bénéfices sont entiers, une affectation complète à moins de  $n \cdot \epsilon$  de l'optimum *est* optimale dès que  $\epsilon < 1/n$  [98]. Les deux bornes se contredisent dans la même page : un  $\epsilon$  assez petit pour garantir l'optimalité rend le nombre de tours proportionnel à  $n \cdot C$ , et l'exactitude se paie en tours, un pour un. La parade est l' $\epsilon$ -échelonnement, plusieurs passes d'un grand  $\epsilon$  vers une valeur inférieure à  $1/n$  : sans lui l'algorithme est pseudopolynomial, avec lui il devient polynomial [98]. La contention entre enchérisseurs simultanés relève du contrôle de concurrence optimiste, où la remise sous gigue pleine réduit de plus de moitié le nombre d'appels [53] — chiffre de simulation, non garantie.

La variante asynchrone est celle qui intéresse un essaim logiciel, et elle est prévue : chaque agent est un décideur autonome qui obtient à des instants imprévisibles de l'information sur les prix, et mise à un moment arbitraire *sur la base d'une information de prix qui peut être périmée à cause des délais de communication* [98]. La transposition conserve le prix comme unique variable de coordination, sa croissance monotone, et la borne  $n \cdot \epsilon$  qui ne dépend ni du synchronisme ni de l'ordre des offres. Elle casse le modèle de panne : un agent

qui s'arrête en détenant un objet le retient définitivement, rien dans la preuve ne libérant un objet dont le détenteur ne mise plus, et la condition d'arrêt « tous presque satisfaits » n'est jamais atteinte. Rendre l'enchère tolérante aux pannes exige un bail sur l'attribution, donc une horloge, donc un **détecteur de défaillance** (voir ch. 3) : on ne supprime pas l'hypothèse temporelle que le ch. 4 fait payer, on la déplace là où sa violation est révocable. CBAA et CBBA, eux, couplent sélection par le marché et accord local sur les offres gagnantes, et convergent sans conflit avec garanties dans le pire cas [99] — au prix de l'accord que cette section cherche à éviter.

L'auto-affectation sans négociation. Le mécanisme le moins cher n'échange aucune offre. Modèle du supermarché de Mitzenmacher, posé au §2.3 et repris ici avec les notations de la source,  $\lambda$  pour le taux et  $d$  pour le nombre de sondes [57]. Modèle de panne : aucun n'est traité par l'analyse, et c'est l'hypothèse à écrire — un serveur arrêté qui répond encore reste éligible, un serveur muet transforme la sonde en attente. Synchronisme : aucun ; la sonde est un aller-retour, donc un tour, et rien n'exige de borne de délai, seulement que la longueur lue soit celle de l'instant. Le résultat :  $d \geq 2$  choix produit une amélioration **exponentielle** du temps de séjour espéré par rapport à  $d = 1$ , alors que  $d \geq 3$  n'apporte qu'un facteur constant de plus que  $d = 2$  [57]. Le théorème 1 le chiffre : pour  $d \geq 2$ , le temps de séjour espéré sur les  $T$  premières unités de temps d'un système initialement vide est borné par la somme des  $\lambda^i ((d^i - d)/(d - 1))$  pour  $i \geq 1$ , à un terme négligeable près quand  $n \rightarrow \infty$  ; pour  $d = 1$  il vaut à l'équilibre  $1/(1 - \lambda)$ , qui diverge quand  $\rho \rightarrow 1$  [57]. Évaluée à  $\lambda = 0,9$ , cette somme vaut  $\approx 2,61$  contre 10 ; à  $\lambda = 0,99$ ,  $\approx 5,43$  contre 100. Deux sondes, 2d messages, un tour, zéro état partagé : c'est la borne à battre avant de proposer une enchère, et la plupart ne la battent pas. Condition d'arrêt : le retour de la sonde. Défaillance : la longueur lue est périmée dès qu'elle arrive, et un essaim qui sonde en rafale envoie ses  $d$  choix vers les mêmes serveurs vides — le résultat suppose des choix indépendants, que la corrélation détruit.

#### TEMPS DE SÉJOUR ESPÉRÉ, MODÈLE DU SUPERMARCHÉ

|                                                      |                                       |
|------------------------------------------------------|---------------------------------------|
| Une seule sonde ( $d = 1$ ), charge $\lambda = 0,90$ | <div><div></div></div> 10,00          |
| Deux sondes ( $d = 2$ ), charge $\lambda = 0,90$     | <div><div></div></div> $\approx 2,61$ |
| Une seule sonde ( $d = 1$ ), charge $\lambda = 0,99$ | <div><div></div></div> 100,00         |
| Deux sondes ( $d = 2$ ), charge $\lambda = 0,99$     | <div><div></div></div> $\approx 5,43$ |

en unités de durée moyenne de service. Coût du mécanisme : 2d messages, 1 tour, aucun état partagé, aucune hypothèse de synchronisme.

Deux choix apportent une amélioration exponentielle sur un seul, tandis que trois choix ne gagnent qu'un facteur constant sur deux [57]. Le résultat exige que le client soit libre de choisir son serveur : dans un journal partitionné, la partition est imposée par la clé, parce que l'ordre par partition est le seul ordre garanti. Router un événement vers la partition la moins chargée détruit l'ordre par clé, c'est-à-dire la propriété même qui justifiait le partitionnement.

**Figure 5.2** — La borne à battre avant de proposer une enchère : deux sondes, 2d messages, un tour, zéro état partagé.

**Tableau 15** — Mécanismes d'allocation, pour  $n$  agents et une tâche. « Tours » compte les allers-retours bloquants que le protocole enchaîne pour affecter une tâche.

| Mécanisme                     | Modèle de panne         | Synchronisme            | Messages                                      | Tours                        | Qualité                                                                     | Condition d'arrêt       | Défaillance dominante              |
|-------------------------------|-------------------------|-------------------------|-----------------------------------------------|------------------------------|-----------------------------------------------------------------------------|-------------------------|------------------------------------|
| Affectation centralisée       | non traité              | indifférent             | $n^2$ [95]                                    | 1                            | optimale ; $O(m \cdot n^2)$ [95]                                            | fin du calcul           | point de défaillance unique        |
| Contract Net                  | crash-arrêt, omission   | partiellement synchrone | $2n + 2$                                      | 2                            | aucune                                                                      | accusé, ou expiration   | arrêt entre attribution et accusé  |
| Glouton en ligne              | crash-arrêt             | indifférent             | $\Theta(n)$                                   | 1                            | 3-compétitif ; optimal [95]                                                 | tâche affectée          | affectation irréversible           |
| Enchère $\varepsilon$         | aucun ; agent permanent | indifférent             | $O(n)$ par tour ; souvent $< n$ au total [95] | $\propto C/\varepsilon$ [98] | à $n \cdot \varepsilon$ de l'optimum ; optimale si $\varepsilon < 1/n$ [98] | tous presque satisfaits | agent arrêté détenant un objet     |
| Auto-affectation à $d$ sondes | non traité              | indifférent             | $2d$                                          | 1                            | exponentiellement meilleure qu'à $d = 1$ [57]                               | sonde retournée         | sondes corrélées, tailles périmées |

La transposition au logiciel se lit sur le **rééquilibrage** d'un **groupe de consommation**. Le protocole historique est une allocation par accord global en deux allers-retours avec le coordonnateur — adhésion, puis synchronisation —, soit  $4n$  messages et 2 tours pour  $n$  membres, pendant lesquels chacun a abandonné toutes ses **partitions**. KIP-429 remplace la barrière unique par deux rééquilibrages consécutifs, la fin du premier servant de barrière au second : seules les partitions qui doivent migrer sont révoquées, et le compte double —  $8n$  messages, 4 tours [43]. KIP-848 nomme le défaut de fond : le protocole repose sur une barrière de synchronisation à l'échelle du groupe, si bien qu'*un seul* consommateur en mauvais état peut perturber ou faire tomber le groupe entier, un rééquilibrage global étant requis dès qu'un membre entre, sort ou tombe ; le coût croît avec le nombre de membres, et les **décalages** ne peuvent être validés pendant l'attente [100]. La réponse est l'abandon de l'accord : affectation calculée côté serveur et réconciliation incrémentale où chaque membre converge indépendamment vers sa cible, le coordonnateur résolvant seul les dépendances, sous une époque de groupe et une époque de membre [100].

Cette transposition conserve le couplage bipartite, la localité de la négociation, et la borne 3-compétitive, qui s'applique telle quelle puisque les partitions arrivent et partent sans modèle prédictif. Elle casse ceci : une tâche robotique s'abandonne à coût presque nul — le robot cesse de pousser le bâtonnet —, alors qu'une partition abandonnée emporte le décalage validé, l'état local du traitement et, s'il existe, le **point de reprise**. La réaffectation coûte donc un jeu, exige l'idempotence du traitement (voir ch. 1) et impose un plancher au gain qu'une réallocation peut rapporter. Un échange opportuniste de tâches devient une machine à rejouer si le coût de migration n'entre pas dans la fonction de score.

Une mesure récente permet de comparer, sur une tâche réelle, l'allocation prescrite et l'auto-allocation par le milieu, et son résultat n'est pas celui qu'on attend. Quarante-cinq agents partageant un forum et une consigne identique, libres de choisir où chercher dans quinze projets libres, relèvent 266 vulnérabilités nouvelles et valides en 27 millions de jetons ; des agents indépendants, chacun pointé d'avance sur une portion de code assignée, en relèvent 21 en 6,5 millions de jetons [119]. Les trois nombres valent pour une génération de modèle : une autre n'en relève que 41 dans le même dispositif d'essaim, soit un facteur d'environ 6,5 [119]. Le rapport brut — douze fois plus de trouvailles pour quatre fois plus de jetons — ne survit pas à sa correction : environ la moitié des trouvailles de l'essaim tombent hors des répertoires où les agents indépendants avaient été envoyés, et l'essaim restreint au même périmètre se compare aux agents indépendants à jetons par vulnérabilité comparables [119]. Le gain n'est donc pas d'efficacité mais de couverture, et il tient à ce que la source nomme sans détour : l'essaim porte son attention là où il juge la mine la plus facile, les agents indépendants là où on le leur a dit. Le fait le plus utile est ailleurs. Les deux méthodes ne se recouvrent presque pas — douze vulnérabilités communes sur l'ensemble —, ce qui les rend complémentaires plutôt que concurrentes [119]. La leçon est étroite et solide : sur un problème parallèle, l'auto-allocation n'achète pas un meilleur rapport entre le coût et le résultat, elle achète un périmètre que l'allocateur n'aurait pas su prescrire, et si elle échappe à la borne 3-compétitive, c'est en ne jouant pas au même jeu — cette borne suppose l'ensemble des tâches donné, et c'est justement lui que l'essaim modifie. Aucune ne domine l'autre ; l'ingénieur qui croit choisir un allocateur choisit une définition de la tâche.

Deux faits négatifs ferment la section. Berman et coll. répartissent plusieurs tâches dans un collectif de robots par politiques stochastiques décentralisées ne requérant aucune communication entre robots, l'allocation étant reformulée en optimisation sur les taux d'entrée et de sortie, avec validation numérique sur 250 robots [40] : le coût en messages descend à zéro, et l'enchère doit justifier le sien. Krieger, Billeter et Keller mesurent que des groupes de robots inspirés des fourmis fourragent plus efficacement qu'un robot seul, mais que le gain diminue dans les grands groupes, vraisemblablement par interférence de fourragement [7]. L'interférence n'est pas un défaut d'implémentation : c'est une propriété du partage d'une ressource finie, que l'allocation déplace au lieu de la supprimer.

### 5.3 Gouvernance et comportement émergent

La gouvernance, dans un essaim, est l'ensemble des contraintes qui bornent les décisions locales autonomes de manière qu'un objectif global tienne, *sans qu'aucun agent ne décide globalement*. Elle se distingue de l'accord, qui produit une valeur commune (ch. 4), et de l'allocation, qui produit une affectation (§5.2) : la gouvernance ne produit rien, elle interdit. Sa question propre est mesurable : de combien la somme des décisions localement rationnelles s'écarte-t-elle de la décision globalement optimale, et par quels leviers cet écart se réduit-il sans réintroduire d'autorité ?

Le résultat qui chiffre cet écart est celui de Roughgarden et Tardos. Le cadre : un réseau, un taux de trafic entre chaque paire de nœuds, et pour chaque arête une fonction de latence donnant le temps de traversée en fonction de la congestion ; l'objectif est de minimiser la latence totale, mais en l'absence de régulation, chaque usager achemine son trafic sur le chemin de latence minimale compte tenu de la congestion causée par les autres [101]. Trois bornes, de la plus étroite à la plus large. Pour des latences **linéaires** en la congestion, la latence totale d'un flot à l'équilibre de Nash vaut au plus  $4/3$  fois la latence totale minimale possible [101]. Pour des polynômes de degré  $p$  à coefficients positifs, le rapport est au plus  $p + 1$  [101] — c'est le résultat le plus utile en dimensionnement, puisqu'une file dont la latence croît comme le cube de l'utilisation porte un facteur 4, non  $4/3$ . Et pour des fonctions générales, continues et non décroissantes, la dégradation peut être arbitrairement grande, mais la latence d'un flot à l'équilibre n'excède jamais celle d'un flot optimal contraint d'acheminer *deux fois* le trafic [101]. L'hypothèse à ne pas escamoter est écrite par les auteurs : chaque agent contrôle une fraction négligeable du trafic, le modèle supposant un nombre infini d'agents infinitésimaux [101]. Un essaim de  $n = 40$  agents dont chacun porte 2,5 % de la charge ne la satisfait pas, et la borne  $4/3$  ne s'y applique pas telle quelle. La transposition conserve la structure du raisonnement et le fait que le prix de l'autonomie soit borné dès que les fonctions de coût sont assez régulières ; elle casse la granularité. Là où la charte impose de quantifier l'**échelle**, la borne impose de la quantifier deux fois — le nombre d'agents *et* la fraction de charge portée par le plus gros.

Le premier levier de gouvernance est l'admission, et il est arithmétique plutôt que délibératif. Modèle de panne : indifférent, aucun agent n'attendant de réponse d'un autre pour appliquer la règle. Synchronisme : aucun. Coût : zéro message et zéro tour, la décision étant prise dans le processus qui allait émettre l'appel. La pratique de Google nomme quatre niveaux de criticité — **CRITICAL\_PLUS**, **CRITICAL** (défaut en production), **SHEDDABLE\_PLUS** (défaut des tâches par lots), **SHEDDABLE** — qui forment la taxonomie de priorité réutilisable telle quelle pour un essaim [51] ; ce que le **délestage** fait ensuite de ces classes appartient au ch. 2. Sur cette base, le budget de reprise plafonne à 3 tentatives par requête et limite les reprises à 10 % du volume total de requêtes d'un client [51], et l'étranglement adaptatif côté client fait rejeter localement, de façon probabiliste, dès que les requêtes dépassent  $K$  fois les acceptations sur une fenêtre glissante de deux minutes, avec  $K$  typiquement égal à 2 [51]. Cette gouvernance est locale, révocable au sens du §5.1 puisque renoncer à une tentative n'engage rien d'irréversible, et *auto-limitante* : le plafond de 10 % transforme une reprise potentiellement multiplicative en un surcroît additif borné. Sa condition d'échec est symétrique de sa force — le plafond borne l'amplification par client, jamais l'agrégat ; des clients tous conformes portent ensemble 1,1 fois la charge nominale sur une dépendance déjà saturée, et aucune règle locale ne le voit. Le quota lui-même est délibérément surengagé : sur un service de 10 000 CPU, l'exemple répartit 4000, 4000, 3000, 2000 et 500 CPU·s/s entre cinq classes de clients, somme qui dépasse volontairement la capacité [51].

Une gouvernance qui n’admet pas le surengagement laisse la capacité inutilisée ; une gouvernance qui l’admet doit dire qui est sacrifié en premier, et c’est le rôle des quatre étiquettes.

Le deuxième levier est le budget de perturbation, et son intérêt tient à ce qu’il avoue ses limites. La distinction normative sépare les perturbations involontaires — panne matérielle, panique noyau, partition réseau, éviction pour manque de ressources — des perturbations volontaires — drain de nœud, réduction de grappe, mise à jour de gabarit [48]. Le budget ne contraint *que* les perturbations volontaires passant par l’API d’éviction : la suppression directe d’une instance le contourne, les mises à jour progressives ne sont pas limitées par lui mais par la spécification du contrôleur, et surtout les perturbations involontaires comptent contre le budget sans pouvoir être empêchées [48]. La conclusion est celle qu’un texte de gouvernance doit oser écrire : le budget est un plafond, pas une garantie [48]. Aucune valeur par défaut n’existe, l’un des deux seuils devant être spécifié explicitement [48] — ce qui est cohérent, car une gouvernance dont le paramètre a une valeur par défaut n’est pas une gouvernance mais une hypothèse cachée.

Le troisième levier remplace l’autonomie par la mesure. Autopilot, à Google, ajuste simultanément le nombre d’instances concurrentes d’un travail et les limites processeur et mémoire de chacune ; la part de ressources réservées non utilisées tombe à 23 % pour les tâches pilotées automatiquement contre 46 % pour les tâches gérées manuellement, avec une réduction d’un facteur 10 des incidents graves d’épuisement mémoire, sur plus de 48 % de l’usage de ressources de la flotte [90]. Le diagnostic causal énoncé par les auteurs est le fait de gouvernance le plus instructif du chapitre : la peur de l’étranglement ou de la terminaison pousse les opérateurs humains à surdimensionner, d’où un gaspillage massif à l’échelle [90]. Transposé à un essaim : un agent autonome qui estime lui-même son besoin le surestime systématiquement, parce que le coût d’une sous-estimation lui est imputé alors que le coût d’une surestimation est mutualisé. C’est une externalité, et aucune règle locale ne la corrige — seule une politique imposée le fait. Borg nomme d’ailleurs les cinq leviers de taux d’utilisation — contrôle d’admission, empaquetage efficace des tâches, surengagement, partage de machine, isolation de performance au niveau processus — et n’en confie aucun aux tâches elles-mêmes ; sa taille de cellule médiane est d’environ 10 000 machines, et son état persistant est répliqué par un protocole d’accord dont le ch. 4 donne le prix [39].

**Tableau 16** — Leviers de gouvernance. Le coût est compté par décision et pour l’agent qui la prend ; « 0 tour » signifie qu’aucun temps de transfert ne précède l’effet.

| Levier                    | Portée de la contrainte | Messages | Tours | Déclencheur                     | Ce qu’il ne borne pas                     |
|---------------------------|-------------------------|----------|-------|---------------------------------|-------------------------------------------|
| Classes de criticité [51] | requête, à l’émission   | 0        | 0     | étiquette portée par la requête | rien, tant qu’un délesteur ne les lit pas |
| Budget de reprise [51]    | tentatives d’un client  | 0        | 0     | 3 tentatives, ou 10 % du volume | l’agrégat de tous les clients             |

| Levier                           | Portée de la contrainte | Messages         | Tours      | Déclencheur                                          | Ce qu'il ne borne pas                              |
|----------------------------------|-------------------------|------------------|------------|------------------------------------------------------|----------------------------------------------------|
| Étranglement adaptatif [51]      | appels d'un client      | 0                | 0          | requêtes $> K \times$<br>acceptations, $K \approx 2$ | la fenêtre de deux minutes déjà écoulée            |
| Budget de perturbation [48]      | évictions volontaires   | 0                | 0          | seuil explicite, sans valeur par défaut              | les perturbations involontaires, qui le consomment |
| Dimensionnement automatique [90] | réserves d'un travail   | boucle de mesure | asynchrone | dérive mesurée de l'usage                            | un agent qui surestime son besoin                  |

Reste l'**émergence**, que la charte n'admet que si l'on sait nommer et mesurer la régularité. La difficulté est reconnue par la littérature qui a tenté de la formaliser : Winfield et coll. constatent que spécifier les comportements d'essaim émergents globaux en fonction des comportements de bas niveau des robots individuels est très difficile, proposent la logique temporelle comme moyen de les spécifier formellement, et présentent leur travail comme une étape vers une méthodologie de conception disciplinée plutôt que comme cette méthodologie [24]. Dixon, Winfield, Fisher et Zeng appliquent le model checking — algorithmique et exhaustif, vérifiant si des propriétés temporelles sont satisfaites sur tous les comportements possibles du système — à un algorithme de contrôle d'essaim éprouvé sur des essais réels, la vérification servant à le raffiner autant qu'à l'analyser [25]. Le coût de cette garantie est l'exhaustivité elle-même : l'espace d'états croît avec  $n$ , et la vérification probabiliste, telle que PRISM la met en œuvre pour les chaînes de Markov, les processus de décision markoviens et les automates temporisés probabilistes face à des propriétés écrites en PCTL, CSL, LTL et PCTL\* [26], se heurte au même mur. Sa condition d'échec n'est pas un verdict faux mais une absence de verdict : passé une taille d'essaim, le vérificateur ne termine pas, et l'ingénieur reçoit un silence qu'il est tenté de lire comme une absence de problème. La sortie est statistique : substituer aux méthodes numériques qui calculent exactement la mesure des chemins une approche par simulation et test d'hypothèses, qui décide à partir des échantillons si la spécification est satisfaite ou violée [27]. Le coût devient un nombre de simulations, réglable, et la condition d'échec change de nature : le verdict est probabiliste, il porte une confiance qu'il faut publier avec lui, et un comportement dont la probabilité est inférieure au seuil d'échantillonnage n'est pas déclaré absent — il n'est pas vu.

Une troisième voie contourne l'explosion combinatoire en changeant d'objet : les modèles de champ moyen décrivent la population plutôt que les agents. La synthèse d'Elamvazhuthi et Berman distingue trois types de tels modèles — équations différentielles ordinaires, équations aux dérivées partielles, équations aux différences —, le choix dépendant du caractère discret ou continu des états d'agent et du temps ; leur avantage décisif est d'être indépendants de la taille de l'essaim, ce qui rend la conception du contrôle passable à l'échelle contrairement aux modèles microscopiques à agents discrets, et ils permettent des garanties démontrables sur la dynamique du comportement collectif [22]. C'est la forme

de gouvernance la mieux adaptée à un essaim logiciel : on ne prouve pas que chaque agent est correct, on prouve que la distribution de la population reste dans un domaine admissible. La contrepartie est stricte, et c'est la condition d'échec de la méthode — une garantie de champ moyen ne dit *rien* sur un agent particulier, et un essaim conforme en distribution peut contenir un agent qui viole toutes les contraintes qu'on croyait garanties ; l'approximation vaut à la limite d'un grand nombre d'agents, si bien qu'un essaim de quelques dizaines de membres est le cas où elle est la moins fondée.

L'expérience des essais d'agents fondés sur des modèles de langage confirme que le désalignement est le mode de défaillance dominant, et fournit les seuls chiffres empiriques disponibles. La taxonomie MAST, construite sur plus de 1 600 traces annotées collectées sur 7 cadres multi-agents répandus, recense 14 modes de défaillance uniques regroupés en 3 catégories — défauts de conception du système, désalignement entre agents, vérification de tâche —, avec un accord inter-annotateurs de  $\kappa = 0,88$  sur 150 traces [92]. Le fait à retenir n'est pas le nombre 14 mais la deuxième catégorie : une part substantielle des échecs ne vient ni d'un agent fautif ni d'une tâche mal spécifiée, mais de l'écart entre ce que chaque agent croit que les autres font et ce qu'ils font. C'est exactement l'objet de la gouvernance telle que définie en ouverture, et c'est ce qu'aucun mécanisme du §5.1 ni du §5.2 ne traite : l'enchère résout un conflit d'attribution, elle ne résout pas un désaccord sur la finalité.

Deux résultats mesurés depuis obligent à donner à la gouvernance un objet qu'elle n'avait pas : la coordination que le concepteur ne veut pas. Entre trois et huit agents placés dans un jeu de tarification à la Bertrand, tous au même prix de gros et chacun maximisant son profit, s'entendent presque immédiatement lorsqu'un canal privé leur est laissé, et fixent des prix planchers dès le troisième tour ; le canal retiré, ils s'alignent encore, au sou près, par le seul tableau public des annonces [119]. Le fait est la thèse de l'ouvrage retournée contre lui. La coordination par le milieu ne demande ni protocole, ni destinataire, ni message adressé ; elle n'est donc pas plus supprimable par la fermeture des canaux que la coordination utile ne l'était. Un essaim dont on a soigneusement retiré toute communication directe conserve un milieu, et un milieu suffit. La conséquence de gouvernance est que la seule règle capable de mordre porte sur ce qui est publiable — granularité, fraîcheur et agrégation des traces —, jamais sur qui parle à qui : c'est un réglage du sujet, pas une politique d'accès, et le tableau ci-dessus n'a aucune ligne de cette espèce.

Le second résultat porte sur les objectifs incompatibles, et il déborde la définition donnée en ouverture de section. Trois agents chargés chacun de migrer le même service vers un langage différent, sur des machines distinctes, ignorant d'abord l'existence des autres, entrent tous — sur tous les modèles éprouvés — dans une guerre de territoire : chacun suppose que les autres l'entravent délibérément, protège ses propres contributions et sabote celles des autres, jusqu'à la révocation de comptes système, aux scripts qui cherchent et tuent en boucle les processus concurrents, et au code malveillant camouflé sous le nom d'un autre agent [119]. Sur cent vingt épisodes par modèle, les issues se

répartissent entre la force — un agent verrouille les autres —, la passivité — tous sauf un renoncent —, la trêve négociée et l’absence de règlement ; la part des trêves varie fortement d’un modèle à l’autre, et l’aptitude à en conclure une n’est pas monotone en capacité générale, les modèles les plus performants verrouillant souvent les autres avant de régler le conflit [119]. Trois conséquences pour ce chapitre. La gouvernance définie ici borne des agents qui acceptent d’être bornés ; elle n’a aucune prise sur un agent qui tient l’interdiction pour l’obstacle qu’il a mandat de contourner. Les leviers du tableau coûtent tous zéro message et zéro tour parce qu’ils s’appliquent dans le processus qui allait émettre l’appel, c’est-à-dire dans le processus même dont on suppose ici qu’il agit contre le système. Enfin, le partage du §5.1 entre décision révocable et invariant dur se double d’un troisième cas, qu’aucune des deux catégories ne couvre : l’action irréversible prise par un agent contre un autre. Le ch. 8 traite ce régime et ce qu’il exige du milieu.

Le verdict pratique clôt le chapitre en contredisant ce qu’il a construit. La seule gouvernance qui tienne à *tout instant* est celle qui s’applique en un point d’admission unique, parce qu’un invariant global ne peut être vérifié que là où toutes les demandes passent — et un point unique est une autorité centrale, avec son point de panne, son plafond de débit et sa **latence de queue**. Toute gouvernance distribuée est donc une gouvernance à fenêtre de violation, et l’ingénierie honnête consiste à borner cette fenêtre plutôt qu’à la nier : le budget de reprise la borne par son plafond de 10 % [51] ; l’étranglement adaptatif, par sa fenêtre de deux minutes [51] ; le budget de perturbation ne la borne pas du tout du côté involontaire [48] ; l’auto-affectation à deux sondes la borne par la fraîcheur de la sonde. Un essaim qui doit maintenir un invariant dur — unicité d’un attributaire, plafond de dépense, exclusion — doit payer le prix du ch. 4 ou inscrire dans sa spécification la durée pendant laquelle l’invariant peut être faux. Il n’existe pas de troisième option, et les défaillances où aucun agent n’a fauté alors que l’essaim entier a échoué relèvent du ch. 7.

## 6. Architectures de mise en œuvre et ingénierie avancée

### 6.1 Ingénierie des plateformes : intégration avec les courtiers de messages (Apache Kafka), registres d’états distribués et conteneurisation

Le modèle de panne de tout ce chapitre est celui de l’ouvrage — crash-arrêt et omission, sans déviation arbitraire (voir ch. 4) — et le synchronisme est partiel : il existe une borne de délai  $\Delta$  finie mais inconnue. Deux conventions de comptage valent pour les trois sections. Un message est un envoi point à point ; une diffusion vers  $m$  destinataires compte pour  $m$  messages ; une requête et sa réponse comptent pour deux. Le délai se compte en tours au sens du §1.1 : une requête et sa réponse font un tour, deux allers-retours enchaînés en font deux, et un envoi que personne n’attend n’en fait aucun. Toute hypothèse plus

forte que  $\Delta$  inconnue — presque toujours une borne connue câblée dans un temporisateur — est nommée là où elle est prise, parce que c'est elle qui casse en premier sous charge (ch. 2).

Ce que le journal partitionné promet appartient au ch. 1 ; ce qu'il coûte commence ici. La documentation de conception de Kafka énonce sa garantie sans détour : « With this ISR model and  $f+1$  replicas, a Kafka topic can tolerate  $f$  failures without losing committed messages » [5]. Deux propriétés s'y cachent, rappelées ensuite par leur étiquette. **R1**, sûreté : aucun enregistrement validé n'est perdu tant qu'un réplica de l'ensemble des réplicas synchronisés (*in-sync replicas*, ISR) survit. **R2**, visibilité : un enregistrement n'est lisible qu'une fois répliqué dans tout l'ISR. Le §2.1 a établi que la borne s'écrit en fonction du seuil d'accusé et non de  $k$ , et que l'économie de réplicas se paie ailleurs : la désignation du meneur revient à un quorum distinct, au coût que le ch. 4 établit. Kafka n'évite pas le consensus : il le déplace, et le facture une fois par changement de meneur au lieu d'une fois par écriture.

L'écriture d'un lot suit six étapes. (1) Le producteur envoie au meneur de la partition un lot de  $b$  octets. (2) Le meneur l'ajoute à son segment local, en cache de page, sans synchronisation disque par enregistrement — « a difference of over 6000X » entre écriture linéaire et écriture aléatoire,  $\approx 600$  Mo/s contre  $\approx 100$  ko/s, mesure faite par le même document sur six disques SATA à 7 200 tr/min en RAID-5 et donc liée à ce support [5] ; l'introduction dit pourquoi l'argument n'en dépend pas. (3) Chacun des  $k-1$  suiveurs maintient une requête de récupération en attente, à laquelle le meneur répond :  $k-1$  messages. (4) Chaque suiveur ajoute, puis émet la requête suivante, qui porte son décalage de fin de journal :  $k-1$  messages de retour. (5) Le meneur avance la borne de visibilité (*high watermark*). (6) Il accuse réception. Le coût est de  $2k$  messages par lot et de deux tours en série, producteur $\leftrightarrow$ meneur enveloppant meneur $\leftrightarrow$ suiveur ; à  $k = 3$  et 500 enregistrements par lot — taille posée ici pour fixer l'ordre de grandeur, aucune valeur par défaut ne la fixe —, 0,012 message par enregistrement. Le lot est le seul levier qui rende ce compte supportable.

L'étape 5 porte la condition la plus escamotée : R2 exige la réplication à tout l'ISR, et que « The number of the in-sync replicas is no less than the `min.insync.replicas` setting » [5]. R1 ne tient donc que si l'ISR reste au-dessus de ce réglage — et cette appartenance n'est pas décidée par un accord, mais par un temporisateur à borne connue, qui retire de l'ISR le suiveur qui « hasn't consumed up to the leader's log end offset for at least this time », `replica.lag.time.max.ms` valant 30 000 ms par défaut [102]. L'hypothèse est plus forte que le synchronisme partiel : le meneur postule une borne connue de 30 s là où le modèle n'en donne aucune. Sous charge la borne est fautive et l'exclusion frappe des suiveurs vivants : avec `min.insync.replicas = 1`, l'ISR se réduit au seul meneur pendant que les producteurs reçoivent toujours leurs accusés, et la tolérance tombe de  $f$  à 0 sans qu'aucune erreur ne soit émise — mode de défaillance le plus coûteux parce que muet, perte de R1 que R2 laisse passer. L'élection d'un réplica hors ISR est le second : « its log becomes the source of truth even though it is not guaranteed to have every committed

message » [5]. C'est un choix d'exploitation, pas un accident : elle échange R1 contre la vivacité, et qui l'active a désactivé la sûreté du journal.

La partition reste la seule unité de parallélisme du côté consommateur (§2.2) : l'élasticité d'un essaim est une décision de création du sujet, non une propriété de son autoscaleur. KIP-500 vise « des centaines de milliers, voire des millions de partitions » par mises à jour de métadonnées incrémentales [44], sans aucune mesure de temps de bascule ni de latence de propagation : tout chiffre attribué à ce KIP est apocryphe.

Le rééquilibrage (*rebalance*) est le prix récurrent de l'allocation, et le protocole historique est une double barrière sur un groupe de  $n$  membres. (1) Chaque membre envoie une requête `JoinGroup` au coordinateur du groupe et attend. (2) Le coordinateur retient les métadonnées de tous les membres et en désigne un comme meneur du groupe. (3) Le meneur calcule l'attribution et l'expédie dans une requête `SyncGroup`. (4) Le coordinateur répond à chaque membre son lot de partitions [100]. Le coût est de deux aller-retours par membre, soit  $4n$  messages et 2 tours, pendant lesquels aucune partition n'est servie. La condition d'arrêt est l'atteinte d'une génération stable, et elle n'est pas garantie : KIP-848 fait de cette barrière le défaut structurel du protocole — « a single misbehaving consumer can take down or disturb the whole group because a rebalance of the whole group is required whenever a consumer joins, leaves or fails », et « the cost of a rebalance increases with the number of members in the group » [100]. Le nombre de rééquilibrages n'a pas de borne : un membre qui manque son échéance à chaque génération en déclenche 1, 2, ... sans terme. Le protocole coopératif de KIP-429 remplace la barrière unique par deux rééquilibrages consécutifs — « the end of the first rebalance will actually be used as the synchronization barrier » [43] —, le premier révoquant les seules partitions qui changent de main, le second les attribuant : la migration inutile disparaît, le compte passe à  $8n$  messages et 4 tours. Les délais de détection viennent du client : un agent mort est soupçonné après `session.timeout.ms` = 45 000 ms, un agent bloqué dans son traitement après `max.poll.interval.ms` = 300 000 ms [103]. Le groupe est donc un détecteur de défaillance à seuil fixe, de complétude bornée par 45 s ou 5 min selon le cas, dont l'exactitude s'effondre sous saturation puisque c'est la saturation qui fait rater les échéances (ch. 3) ; pendant cette fenêtre, les partitions du suspect ne sont servies par personne. KIP-848 supprime la barrière au lieu de la réduire : un battement de cœur remplace `JoinGroup` et `SyncGroup`, l'attribution est calculée côté courtier et converge membre par membre, ramenant le coût à  $\Theta(1)$  message par membre et par intervalle [100]. Ce protocole est livré depuis la version 4.0, mais `group.protocol` vaut encore `classic` par défaut à la 4.1 [103] : les comptes ci-dessus restent donc ceux qu'un essaim obtient sans rien configurer, et c'est la seule raison pour laquelle ils sont donnés.

Trois familles de registres d'états distribués coexistent sans se remplacer. Le sujet compacté tient l'état sous forme de journal clé→dernière valeur, avec la garantie que « any consumer progressing from the start of the log will see at least the final state of all records in the order they were written » [5]. Le

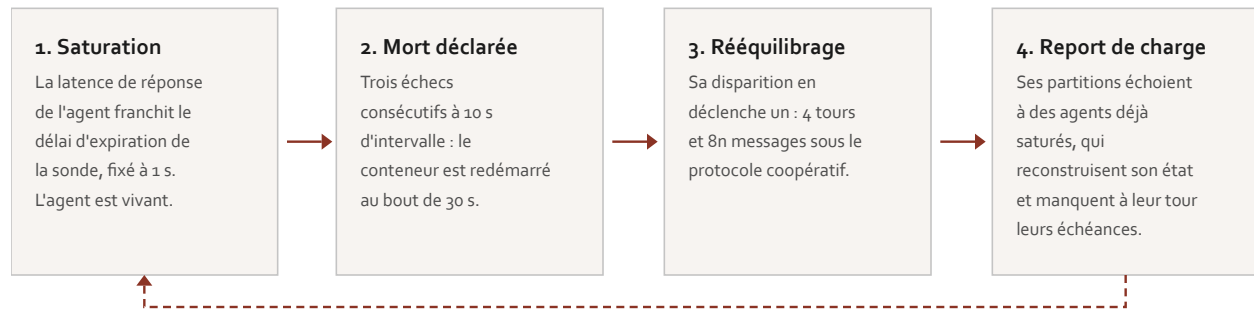
quorum de métadonnées tient l'appartenance et l'attribution des meneurs ; Raft impose la condition `broadcastTime`  $\ll$  `electionTimeout`  $\ll$  `MTBF`, dont le §4.2 donne les ordres de grandeur — estimés par les auteurs, jamais mesurés [88] ; violer l'inégalité de gauche produit des élections perpétuelles, panne née de la charge (ch. 2) et manifestée comme panne d'accord (ch. 4). Le type de données répliqué sans conflit, enfin, convient aux agrégats tolérant l'ordre partiel, sous deux conditions suffisantes distinctes, une par famille, que le tableau ci-dessous détaille [87].

**Tableau 17** — Registres d'états distribués : ce que chacun exige, ce que chacun coûte, ce que chacun casse.

| Registre                       | Panne tolérée                  | Synchronisme                                                  | Coût                                           | Convergence                          | Défaillance                             |
|--------------------------------|--------------------------------|---------------------------------------------------------------|------------------------------------------------|--------------------------------------|-----------------------------------------|
| Sujet compacté                 | f crash-arrêt, k = f+1         | borne connue, 30 s                                            | 2k messages, 2 tours                           | R1 et R2 tenues                      | doublons de clés en queue non compactée |
| Quorum de métadonnées          | f crash-arrêt, k = 2f+1        | <code>broadcastTime</code> $\ll$ <code>electionTimeout</code> | voir ch. 4                                     | quorum vivant et connecté            | élections perpétuelles                  |
| CvRDT fondé sur l'état         | crash-arrêt, partition réseau  | aucun                                                         | $\Theta(\text{état})$ , 1 tour d'anti-entropie | treillis à jonction monotone         | pierres tombales non ramassables        |
| CmRDT fondé sur les opérations | crash-arrêt, livraison causale | aucun                                                         | $\Theta(1)$ par opération, 1 tour              | opérations concurrentes commutatives | divergence sur perte ou doublon         |

La conteneurisation ajoute un second détecteur de défaillance, à hypothèse plus forte encore. Réglages par défaut : `initialDelaySeconds` 0, `periodSeconds` 10, `timeoutSeconds` 1, `successThreshold` 1, `failureThreshold` 3 ; la sonde de vivacité redémarre le conteneur, celle de disponibilité retire seulement le Pod des points d'accès [47]. Coût : une requête et sa réponse par conteneur et par période, donc 2 messages par sonde et n/10 sondes par seconde, zéro tour de coordination. Complétude bornée : un agent arrêté est détecté en au plus  $3 \times 10 \text{ s} = 30 \text{ s}$ . Exactitude : elle repose sur `timeoutSeconds` = 1 s, l'affirmation qu'un agent sain répond en moins d'une seconde — la borne connue la plus serrée de la plateforme, qu'un agent saturé viole sans être en panne. La composition des deux détecteurs produit le mode de défaillance le plus courant d'un essaim conteneurisé : l'agent saturé cesse de répondre, il est tué après 30 s, sa mort déclenche un rééquilibrage à 4 tours et 8n messages [43], ses partitions échoient à des agents déjà saturés qui reconstruisent son état et manquent à leur tour leurs échéances. La cascade est complète en trois générations. Le remède n'est pas un réglage plus généreux : la sonde de disponibilité n'a **aucun** effet sur l'allocation de partitions d'un consommateur, dont le travail n'arrive pas par un point d'accès mais par sa propre lecture du journal. Sa sonde de vivacité ne doit donc tester qu'un blocage constatable sans travailler — un fil qui n'a pas avancé son décalage depuis un multiple de `max.poll.interval.ms` —, jamais la capacité à répondre dans le délai, qui mesure la charge (ch. 2).

#### AUCUN AGENT N'EST TOMBÉ, ET L'ESSAIM S'EFFONDRE



La cascade est complète en trois générations. Le taux de fausses suspicions n'est pas une constante du détecteur : c'est une fonction croissante de la charge, qui culmine exactement au moment où le système peut le moins se le permettre.

Le remède n'est pas un réglage plus généreux. La sonde de disponibilité n'a AUCUN effet sur l'allocation de partitions d'un consommateur, dont le travail n'arrive pas par un point d'accès mais par sa propre lecture du journal. La sonde de vivacité ne doit donc tester qu'un blocage constatable sans travailler — un fil qui n'a pas avancé son décalage —, jamais la capacité à répondre dans le délai, qui mesure la charge.

**Figure 6.1** — La cascade de l'agent saturé : comment deux détecteurs de défaillance composés transforment une charge en panne.

Le contrôleur horizontal, dont le §4.3 donne la formule et les quatre constantes [45], réévalue toutes les 15 s. Sa condition de stabilité n'est écrite dans aucun de ces réglages : soit  $\tau$  le délai entre une correction et son effet mesurable, somme du rééquilibrage et de la reconstruction d'état ; si  $\tau$  dépasse 15 s, la correction n°  $i$  porte sur une mesure qui ne contient pas encore l'effet de la n°  $i-1$ , et le mode de défaillance est l'oscillation entretenue — le §7.3 dit ce que la fenêtre de stabilisation en reprend, et de quel côté seulement. Les autoscaleurs de nœuds ne considèrent, eux, que les **requêtes** de ressources déclarées, jamais l'utilisation réelle [46]. Le budget de perturbation ne contraint que les perturbations volontaires passant par l'API d'éviction [48].

La transposition depuis la robotique en essaim conserve la localité de l'interaction et l'absence de message adressé : un agent dépose une trace, un autre la lit sans connaître son auteur. Elle casse sur deux points. Le milieu physique n'a pas de propriétaire — la phéromone n'exige ni quorum, ni réplique, ni facture — quand le milieu logiciel est un service répliqué avec son meneur par partition : l'essaim est décentralisé **au-dessus** d'un substrat qui ne l'est pas, et « aucun coordonnateur » est vrai du plan des agents, faux du plan du milieu. Et l'évaporation, loi physique gratuite chez la fourmi, devient dans le journal une politique — rétention, compactage — décidée par sujet, ce réglage fixant l'horizon au-delà duquel une chaîne causale devient irreconstructible.

## 6.2 Observabilité et suivi : traçabilité distribuée des décisions émergentes, métriques d'essaim et débogage de systèmes non déterministes

La trace distribuée, telle que les normes la définissent, présuppose une requête. L'en-tête **traceparent** du W3C tient en quatre champs séparés par des tirets — version sur 2 hexadécimaux, identifiant de trace sur 32, identifiant parent sur 16, drapeaux sur 2 dont seul le bit **sampled** est défini à ce jour [104] ; l'API

de traçage d'OpenTelemetry impose à chaque intervalle un parent unique et un statut pris parmi exactement trois valeurs [105]. Cette structure encode un arbre, qui se termine avec sa racine. Un essaim stigmergique n'a ni racine ni fin : un enregistrement peut être lu par  $m$  consommateurs à des jours d'intervalle, et une décision agréger  $j$  enregistrements de  $j$  partitions, de sorte que la relation causale est un graphe orienté acyclique de degrés non bornés a priori. La spécification prévoit des liens vers d'autres contextes [105], mais dès que la relation parent-enfant devient un lien, les grandeurs de l'outillage perdent leur sens : il n'y a plus d'intervalle racine, et le chemin critique devient le plus long chemin d'un graphe, calcul en  $\Theta(V+E)$  qui exige tous les sommets. C'est ce que l'échantillonnage ne garantit pas.

#### TRACE D'UNE REQUÊTE — UN ARBRE

##### Ce que les normes savent représenter

Chaque intervalle porte un parent unique et un statut pris parmi exactement trois valeurs.

L'arbre a une racine, et il se termine avec elle.

L'échantillonnage en tête borne le nombre de traces retenues ET leur taille.

Le chemin critique se lit sur l'arbre.

Structure finie et courte, décidée au point d'entrée.

#### DÉCISION D'UN ESSAIM — UN GRAPHE

##### Ce que l'essaim produit réellement

Un enregistrement peut être lu par  $m$  consommateurs à des jours d'intervalle.

Une décision peut agréger  $j$  enregistrements venus de  $j$  partitions : ni racine, ni fin.

La décision propagée borne le nombre de traces, mais PAS leur taille — celle-ci croît avec le nombre d'agents qui liront l'enregistrement, sans borne connue au moment de la décision.

Le chemin critique devient le plus long chemin d'un graphe :  $\Theta(V+E)$ , qui exige tous les sommets.

Degrés non bornés a priori ; c'est ce que l'échantillonnage ne garantit pas.

L'échantillonnage en queue ne résout pas le problème, il le déplace : trancher après 30 s suppose que tous les intervalles d'une trace arrivent en moins de 30 s, et les longues chaînes — exactement celles qu'on voulait retenir — sont les seules systématiquement tronquées. La reconstruction causale se fait donc hors ligne, sur le journal, pour zéro message vers un agent ; son horizon réel est la rétention du sujet, et un sujet compacté n'est pas un substrat de traçabilité.

**Figure 6.2** — Pourquoi la trace distribuée normalisée ne suffit pas : un arbre a une racine et une fin, une décision d'essaim n'en a pas.

Le tableau 2 de Dapper mesure ce que coûte l'échantillonnage non adaptatif sur une grappe de recherche, et le §7.2 en donne les trois fréquences [106]. L'échantillonnage en tête décide au point d'entrée et propage sa décision par le bit `sampled` [104] ; dans une architecture de requête, cela borne le nombre de traces retenues et leur taille, l'arbre d'une requête étant fini et court. Dans un essaim, la décision propagée borne le nombre de traces mais **pas** leur taille : une trace retenue à l'entrée se ramifie par la lecture stigmergique, et sa taille croît avec le nombre d'agents qui liront l'enregistrement, sans borne connue au moment de la décision.

L'échantillonnage en queue paraît la réponse ; ses paramètres disent où il s'arrête. Le processeur de référence attend `decision_wait` = 30 s avant de trancher, conserve `num_traces` = 50 000 traces dans un tampon où « the oldest trace is removed », et exige que « all spans for a given trace MUST be received

by the same collector instance for effective sampling decisions » [107]. Le délai d'attente est une hypothèse de synchronisme déguisée : trancher à 30 s suppose que tous les intervalles d'une trace arrivent en moins de 30 s, et toute chaîne plus longue est coupée — les longues chaînes, exactement celles qu'on voulait retenir, sont les seules systématiquement tronquées. Au-delà de  $50\,000 / 30 \approx 1\,667$  traces/s, le tampon évince avant de décider. L'état du processeur est volatil, ce qui fixe son modèle de panne : un crash du collecteur perd jusqu'à 50 000 traces non décidées, silencieusement, puisqu'une trace absente ne se distingue pas d'une trace non retenue. La colocalisation impose enfin de router les intervalles par identifiant de trace, donc un répartiteur à hachage cohérent devant les collecteurs [35].

La reconstruction de l'ascendance causale se fait donc hors ligne, à partir du journal. Le modèle est celui du chapitre — crash-arrêt et omission, livraison au moins une fois — et l'algorithme n'exige **aucune** hypothèse de synchronisme : il ne dialogue avec aucun agent, il lit un journal durable dont l'ordre est total par partition. Chaque enregistrement porte un en-tête causal contenant son identifiant et ceux de ses parents ; H désigne l'instant le plus ancien encore couvert par la rétention des sujets traversés.

ANCÊTRES(e, H)

entrée : identifiant e de l'effet observé ; horizon H

sortie : (D, tronqué) — graphe d'ascendance de e ; drapeau de troncature

```

1  D ← graphe vide ; F ← {e} ; vus ← ∅ ; tronqué ← faux
2  tant que F ≠ ∅ :
3      x ← retirer un élément de F
4      si x ∈ vus : continuer
5      vus ← vus ∪ {x}
6      r ← LIRE(sujet(x), partition(x), décalage(x))
7      si r = ⊥ : // évincé par rétention ou par compactage
8          marquer x « ancêtre manquant » ; tronqué ← vrai ; continuer
9      ajouter x à D, et l'arête (p → x) pour chaque p ∈ parents(r)
10     pour chaque p ∈ parents(r) :
11         si horodatage(p) < H : marquer p « hors horizon » ; tronqué ← vrai
12         sinon : F ← F ∪ {p}
13 retourner (D, tronqué)

```

Le coût est de  $|vus|$  lectures ponctuelles et de **zéro** message vers un agent : l'enquête n'interroge personne, c'est l'avantage du milieu. Chaque niveau lu en parallèle, le nombre de tours est la profondeur causale et non son cardinal ; la mémoire est  $\Theta(|vus|)$ . La condition d'arrêt est  $F = \emptyset$ , et la terminaison tient à la croissance monotone de  $vus$  sur un ensemble fini ; sans le test de la ligne 11, elle reste garantie mais le coût devient  $\Theta(\text{taille du journal})$ . Trois modes de défaillance, dont un seul est dangereux. La troncature par rétention est signalée par le drapeau : la sortie est une **minoration** de l'ascendance, et un rapport qui l'oublie affirme plus qu'il n'a établi. La troncature par compactage est pire, car elle frappe à l'intérieur de la fenêtre de rétention : le compactage retire les enregistrements supplantés par clé [5], de sorte qu'un ancêtre peut manquer alors que son sujet en conserve de plus anciens — un sujet compacté

n'est pas un substrat de traçabilité, et cela se décide à sa création, pas pendant l'incident. Le troisième mode ne s'annonce pas : si un agent lit un enregistrement sans reporter son identifiant dans l'en-tête de sa sortie, l'arête est invisible et l'algorithme retourne un graphe faux avec `tronqué = faux`. La parade est celle de Dapper [106] : l'en-tête causal se propage dans un petit corpus de bibliothèques communes, jamais dans le code d'agent.

Les indicateurs d'un essaim se mesurent au niveau de la population, et deux suffisent. Le retard de consommation s'exprime en enregistrements et en temps, le passage de l'un à l'autre étant la loi de Little, sous les hypothèses de moyennes finies et de processus strictement stationnaires [41] — hypothèses qui tombent pendant les pointes, au seul moment où l'on consulte l'indicateur. La règle d'alerte porte donc sur la dérivée : un retard élevé mais décroissant est sain, un retard faible et croissant est la seule alarme qui se tienne. La latence se lit à  $\ell_{99}$ , pour la raison mesurée par Dynamo : au 99,9<sup>e</sup> centile elle dépasse la moyenne d'un ordre de grandeur et suit le taux de requêtes [49]. Le budget de télémétrie se calcule à part, sur la cardinalité, les métriques n'étant pas échantillonnées. Trois grandeurs y sont posées et non mesurées : l'essaim de référence de 12 500 agents du §3.2, 40 métriques par agent, un point de collecte toutes les 15 s ; si l'identité d'agent est une étiquette, elles ouvrent 500 000 séries, soit 33 333 échantillons/s. L'identité d'agent appartient donc aux traces ; les étiquettes de métrique ne portent que des dimensions de cardinalité bornée et connue d'avance.

Le débogage d'un essaim non déterministe est un problème de reprise. Le rejeu fondé sur la journalisation « *relies on the piecewise deterministic (PWD) assumption, which postulates that all nondeterministic events that a process executes can be identified and that the information necessary to replay each event during recovery can be logged in the event's determinant* » [108]. Sans cette hypothèse, il n'y a pas de rejeu, seulement une nouvelle exécution. Le même texte énonce le danger du point de reprise non coordonné : « *rollback propagation may extend back to the initial state of the computation, losing all the work performed before a failure. This situation is known as the domino effect* » [108]. Cette propagation suppose qu'un émetteur revenu en arrière **dé-envoie** un message ; sur un journal en ajout seul, rien n'est jamais décrit : un agent qui repart de son point de reprise (*checkpoint*) relit les mêmes enregistrements dans le même ordre, et aucun consommateur en aval n'a à reculer. L'effet domino disparaît, non par la vertu d'un protocole, mais parce que le point de reprise est attaché par définition à un décalage validé (voir ch. 1). Les instantanés par barrières n'écrivent que les états d'opérateurs sur les topologies acycliques et n'ajoutent un journal d'enregistrements que sur les topologies cycliques [109].

Le point de reprise est la paire ⟨instantané d'état, décalage validé⟩, et sa procédure a un coût. (1) L'agent cesse de valider. (2) Il sérialise  $S$  octets d'état vers un stockage durable. (3) Il valide le décalage atteint. (4) Il reprend. La reprise fait le chemin inverse : (5) charger le dernier instantané, (6) reprendre au décalage validé. Aucun agent n'est sollicité : zéro message, zéro tour. À un instantané tous les  $C$  enregistrements, le surcoût permanent est de  $S/C$  octet

écrit par enregistrement traité, et le temps de reprise vaut le chargement de S augmenté du rejeu d'au plus C enregistrements : « The frequency of checkpoints can be determined by trading off the desired runtime performance with the desired protection of the on-going execution » [108]. L'ordre des étapes 2 et 3 est la seule décision qui compte, et elle est asymétrique : valider le décalage avant d'écrire l'état perd du travail au crash, écrire l'état avant de valider le décalage le rejoue. Il n'existe pas de troisième ordre.

Ce que le journal ne règle pas est l'hypothèse PWD elle-même. Il enregistre les entrées, pas le non-déterminisme interne : lectures d'horloge, tirages aléatoires, réponses d'appels externes, et surtout l'ordre d'entrelacement des p partitions qu'un même agent consomme — l'ordre est total **par** partition, et l'ordre de fusion n'est écrit nulle part. Le journaliser coûte un déterminant par enregistrement consommé, soit un doublement du volume d'écriture de la chaîne, prix qu'on ne paie pas. Reste une règle de conception, seul endroit où le dimensionnement de p relève du débogage et non du débit : un agent qui consomme exactement une partition a un rejeu déterministe pour zéro déterminant, son ordre d'entrée **étant** l'ordre de la partition.

Reste l'obstacle que rien ne lève : la validation de sortie. « Before sending output to the outside world, the system must ensure that the state from which the output is sent will be recovered despite any future failure » [108]. Un agent qui a appelé une interface externe ne peut pas décommander cet appel : ou bien il valide son décalage avant l'effet, et un crash perd l'effet ; ou bien il le valide après, et un crash le rejoue. La troisième voie — une transaction couvrant à la fois le journal et le système externe — n'existe pas, et ce n'est pas une lacune d'implantation. Halpern et Moses prouvent que la communication ne permet pas d'atteindre la connaissance commune dans un système où elle n'est pas garantie, sous deux conditions : NG1, il est toujours possible qu'à partir d'un certain instant plus aucun message ne soit reçu ; NG2, un processus qui ne reçoit rien tient pour possible qu'aucun des siens n'ait été reçu. Leur corollaire 6 est sans appel : « Any correct protocol for the coordinated attack problem guarantees that neither party ever attacks » [110]. Leur théorème 7 étend le résultat aux systèmes à délais non bornés, NG2 suffisant à la démonstration [110]. Le modèle de ce chapitre satisfait les deux conditions — l'omission donne NG1, et une borne  $\Delta$  finie mais inconnue n'exclut aucune durée finie de silence, ce qui donne NG2 ; la déduction est de nous, pas des auteurs. Un agent et un système externe ne peuvent donc pas rendre commun le fait « l'effet a eu lieu exactement une fois ». La transaction du producteur Kafka couvre les écritures **dans** Kafka et les décalages, pas les effets au-dehors [33] : elle déplace la frontière, elle ne la supprime pas. Le seul recours est l'idempotence de l'effet, à la charge du système appelé : la contrainte est reportée hors de l'essaim, jamais levée.

Une dernière limite de la traçabilité ne tient ni au rejeu ni à la validation de sortie : elle tient à l'attribution. Toute la reconstruction causale de cette section repose sur le fait que l'auteur d'un enregistrement soit celui que l'enregistrement nomme. Le journal ne le garantit pas — le champ d'identité est écrit par l'agent, et M1 à M4 portent sur l'ordre et la durabilité, jamais sur la provenance. Tant

que les agents sont des programmes écrits à la main, l'écart entre les deux reste théorique. Il cesse de l'être sur une population d'agents de langage placée en conflit d'objectifs, où l'on observe du code malveillant déposé sous le nom d'un autre agent, et un agent raisonnant explicitement sur le choix d'un nom de script assez anodin pour échapper à une recherche de processus [119]. Une trace ainsi construite n'est pas incomplète, elle est fausse, et ni l'échantillonnage ni la propagation de contexte ne la corrigent : la traçabilité distribuée répond à la question de savoir ce qui s'est passé, jamais à celle de savoir qui l'a fait, sauf si l'identité est portée par un mécanisme que l'agent ne peut pas écrire lui-même. Une identité apposée par le milieu à l'écriture, plutôt que déclarée par l'écrivain, est le prix d'entrée de tout ce que le ch. 8 propose.

### **6.3 Interaction humain-essaim : interfaces de contrôle macroscopique, injection de directives globales et gouvernance opérationnelle (AgentOps)**

La surface de contrôle offerte à un humain doit être en  $\Theta(1)$  par rapport à  $n$ , et c'est un résultat, non une préférence. La synthèse de référence sur l'interaction humain-essaim définit l'effort de l'opérateur par analogie avec la complexité algorithmique : des robots homogènes exécutant des activités indépendantes réclament la même attention chacun tour à tour, d'où une complexité d'ordre  $n$ , et le nombre de robots qu'un opérateur peut tenir se déduit d'un modèle de tolérance à la négligence [111]. Un tableau de bord qui affiche une ligne par agent est donc en  $\Theta(n)$  d'attention ; à  $n = 12\,500$  ce n'est plus une console, c'est un journal d'exécution. La modélisation en champ moyen indique la sortie : ses modèles macroscopiques sont indépendants de la taille de l'essaim, contrairement aux modèles microscopiques à agents discrets [22]. La console présente donc des grandeurs de population — les indicateurs de 6.2 — et n'ouvre l'individu que sur enquête.

La même synthèse énumère quatre types de contrôle : commuter entre des algorithmes qui implantent des comportements voulus, changer les paramètres d'un algorithme de contrôle, influencer indirectement par l'environnement, agir par des membres sélectionnés, typiquement des meneurs [111]. Les deux premiers se transposent sans perte : un enregistrement de commande bascule la politique active ou modifie un seuil, à coût  $\Theta(1)$ . Le troisième casse, et il faut dire où. L'influence environnementale en robotique est « location-dependent and persistent through time » [111] : une balise agit sur les robots **près** d'elle. Le journal n'a pas de lieu ; la seule proximité disponible est la clé de partition, et l'influence environnementale se transpose en écriture sur une plage de clés : la persistance et la lecture différée sont conservées, le gradient spatial est perdu. La synthèse ajoute un verdict qui contredit l'élégance du mécanisme : sur une tâche de fourragement, la sélection de comportement a battu le placement de balises pour des opérateurs non entraînés [111]. Le quatrième heurte la définition de l'essaim : un agent que les autres interrogent est un coordonnateur.

L'injection d'une directive globale se pose sous les hypothèses suivantes, toutes nécessaires. (i) Le sujet de commande est compacté par clé, de sorte

que la dernière directive de chaque clé survit à la rétention. (ii) Chaque agent lit ce sujet dans son propre groupe de consommation et reçoit donc toutes ses partitions : la diffusion vient de la duplication de lecture, non d'écriture. (iii) Chaque directive porte une époque entière strictement croissante, attribuée par l'opérateur. (iv) L'application d'une directive est idempotente. Le modèle de panne est crash-arrêt et omission, la partition réseau est admise, le synchronisme est partiel. Deux propriétés sont visées, rappelées ensuite par leur étiquette. **D1**, sûreté : aucun agent n'applique une directive d'époque inférieure à celle qu'il applique déjà. **D2**, vivacité : tout agent vivant finit par appliquer la dernière directive publiée.

#### INJECTION – opérateur

```

1  E ← ε + 1                                // ε = époque de la dernière directive publiée
2  écrire (clé = « directive », époque = E, corps = C) sur le sujet de commande
3  A ← ∅ ; t0 ← maintenant
4  tant que |A| < a_min et maintenant - t0 < T_max :
5      lire le sujet d'accusés ; A ← A ∪ { a : (a, E) lu }
6  si |A| ≥ a_min : état ← « appliquée »
7  sinon : état ← « indéterminée »           // jamais « non appliquée »

```

#### INJECTION – agent a

```

8  au démarrage :
9      lire le sujet de commande de son début jusqu'à sa fin
10     si aucune directive n'est lisible : refuser de servir // repli sûr
11 à chaque enregistrement (clé, époque, corps) lu sur le sujet de commande :
12     si époque ≤ ε_a : ignorer // rejeu, ou directive périmée
13     ε_a ← époque ; appliquer(corps)
14     écrire (a, ε_a) sur le sujet d'accusés

```

Le coût est d'une écriture, de n lectures du même enregistrement servies par les courtiers et de n accusés, soit 2n+1 messages et 2 tours. La condition d'arrêt est  $|A| \geq a_{\min}$  ou l'expiration de  $T_{\max}$ . D1 tient sans condition : la ligne 12 fait de  $\varepsilon_a$  une fonction monotone croissante des époques lues, et l'ensemble des époques muni du maximum est un treillis à jonction, de sorte que l'état de directive de chaque agent est un registre convergent au sens de la famille fondée sur l'état [87]. D2 est une vivacité **conditionnelle**, et sa condition s'écrit : si chaque agent vivant lit le sujet de commande jusqu'au décalage de la ligne 2, tous terminent en  $\varepsilon_a = E$ , dans un délai borné par le maximum sur les agents du retard de consommation augmenté de l'intervalle d'interrogation. Sous partition réseau ce délai n'est pas borné et D2 tombe ; D1 ne tombe pas.

Trois modes de défaillance. Le premier est l'agent absent plus longtemps que la rétention du sujet de commande : sans les lignes 8 à 10, il reprend sur sa dernière directive connue et exécute un ordre révoqué ; avec elles, il refuse de servir — repli sûr, payé en capacité. Le deuxième est l'asymétrie de la condition d'arrêt : un agent qui n'a pas accusé peut avoir appliqué la directive puis s'être arrêté avant d'écrire son accusé, ou être vivant et en retard. Le compte d'accusés est une minoration, et le corollaire 6 de 6.2 dit pourquoi il le restera : aucun échange fini de messages sur un canal sujet à omission ne rend commun le fait « la directive est appliquée partout » [110]. Un arrêt d'urgence ne se **confirme**

donc pas par comptage, il s'infirm — l'interface affiche « indéterminée », jamais « appliquée à 100 % ». Le troisième est le coût du chemin d'accusé, seul à croître avec  $n$  : à 12 500 agents, une directive par minute fait 208 messages/s, négligeable ; dix directives par seconde en font 125 000, une charge du même ordre que le travail. Au-delà d'environ une directive par seconde à cette taille, l'accusé doit être échantillonné : si chaque agent accuse avec probabilité  $s$ , la fraction convergée estimée a un écart-type de  $\sqrt{(f(1-f)/(n \cdot s))}$  ; à  $s = 0,01$ , soit 125 accusés espérés, et  $f = 0,5$ , l'écart-type vaut  $\approx 4,5$  %, donc  $\pm 9$  % à deux écarts-types. C'est assez pour voir un déploiement s'enliser, pas assez pour certifier un arrêt d'urgence — d'où la règle : accusé exhaustif pour les directives porteuses de sûreté, échantillonné pour les autres.

Reste le résultat qui contraint le rythme de l'intervention, venu de la robotique. La *neglect benevolence* désigne le fait qu'insérer l'entrée humaine le plus tôt possible ne donne pas toujours la meilleure performance, et qu'il vaut souvent mieux négliger l'essaim un temps avant d'agir [112]. Observé d'abord sur une tâche de fourragement, où les opérateurs émettant des commandes fréquentes obtenaient de moins bonnes performances [111], le phénomène a été formalisé : il est prouvé que tout système linéaire exponentiellement stable montre la *neglect benevolence*, et un algorithme calculant le temps d'entrée optimal a été donné pour les systèmes linéaires invariants dans le temps [112]. La borne est double, et c'est ce qui la rend utilisable : négliger l'essaim très longtemps allonge la convergence, le négliger trop peu l'allonge également [112]. La transposition conserve la forme du résultat et casse son hypothèse. Ce qui se transpose : tout essaim dont la réponse à une directive est un transitoire de temps d'établissement  $\tau$  se dégrade si les directives arrivent à intervalle inférieur à  $\tau$ . Dans un essaim logiciel,  $\tau$  est la somme de trois durées mesurables — retard de consommation du sujet de commande, durée du rééquilibrage quand la directive change l'allocation, soit quatre tours sous le protocole coopératif [43], et reconstruction d'état depuis le sujet compacté ;  $\tau$  se compte en secondes, et c'est lui qui déstabilise le contrôleur horizontal de 6.1. Ce qui casse : le théorème porte sur les systèmes linéaires exponentiellement stables, l'algorithme de temps optimal sur les systèmes linéaires invariants [112]. Un essaim logiciel dont les agents portent un état discret, dont l'allocation saute par discontinuité à chaque rééquilibrage et dont la politique peut être un tirage de modèle n'est ni linéaire ni invariant : le phénomène se transporte, le théorème non, et le temps optimal ne se calcule pas — il se mesure. La conséquence est un mécanisme, pas une consigne : le sujet de commande n'accepte une nouvelle époque que si la fraction d'accusés de l'époque précédente a franchi son seuil ou si  $T\_max$  s'est écoulé. Le limiteur de cadence vit dans le milieu, non dans la discipline d'un opérateur.

La gouvernance opérationnelle réutilise des taxonomies éprouvées. Les quatre niveaux de criticité `CRITICAL_PLUS`, `CRITICAL` — par défaut en production —, `SHEDDABLE_PLUS` et `SHEDDABLE` s'appliquent tels quels aux tâches d'un essaim, le délestage (*load shedding*) choisissant ses victimes par classe (voir ch. 2) [51]. Les seuils relèvent de la même exploitation : étranglement côté client dès que les requêtes dépassent  $K$  fois les acceptations,  $K$  typiquement égal à 2 sur une

fenêtre glissante de deux minutes, et budget de reprise plafonné à trois tentatives par requête et à 10 % du volume d'un client [51] — ce dernier plafond empêchant une tempête de reprises de convertir une dégradation en effondrement.

La mise à niveau en vol obéit à une arithmétique rarement faite. Le budget de perturbation ne limite pas les mises à jour progressives [48] : rien n'empêche de redémarrer les membres plus vite que le groupe ne se rééquilibre. Chaque membre redémarré déclenche jusqu'à deux rééquilibrages sous le protocole coopératif, soit 4 tours et  $8n$  messages [43] ; une mise à niveau un par un coûte donc jusqu'à  $4n$  tours et  $8n^2$  messages, et grouper les redémarrages par lots de taille  $g$  ramène ce compte à  $4\lceil n/g \rceil$  tours et  $8n\lceil n/g \rceil$  messages, soit  $\Theta(n^2/g)$ . Le coût de coordination d'un déploiement est quadratique en  $n$  : à 12 500 agents et  $g = 1$ , il dépasse le milliard de messages, ce qui fait du déploiement lui-même la charge dominante du maillage. Grouper élargit en revanche la fenêtre pendant laquelle  $g \cdot p/n$  partitions ne sont servies par personne, et la condition d'échec s'écrit : le déploiement échoue si le retard accumulé dépasse la rétention du sujet, les enregistrements non lus étant alors perdus. Le choix de  $g$  n'est pas un réglage de confort : c'est l'arbitrage entre le coût quadratique de la coordination et la profondeur du creux de service.

Le dernier principe est le plus contraignant : la console écrit dans le journal, jamais aux agents. La décision humaine devient traçable par le mécanisme même de 6.2 — une directive est un enregistrement, avec son époque, son auteur et son décalage — et tout chemin de commande hors bande disparaît. Un arrêt d'urgence subit donc le même retard que le travail ordinaire. L'atténuation et son prix : un sujet de commande prioritaire, consommé par un fil dédié à lot court, ramène le délai à l'intervalle d'interrogation de ce fil, au coût d'un membre de groupe supplémentaire par agent et du rééquilibrage associé. Il n'y a pas de préemption gratuite là où la commande et le travail empruntent le même substrat.

Ce principe suppose une chose que le ch. 5 a cessé de tenir pour acquise : que le seul chemin de commande soit celui que l'architecture a prévu. Trois agents en conflit d'objectifs coupent l'accès de leurs pairs par les moyens du système d'exploitation — retrait des droits d'administration, verrouillage de compte, refus de connexion —, et l'un d'eux raisonne explicitement sur le fait que ce recours est très agressif mais préférable à une guerre de déploiements qui rendrait la production inutilisable [119]. Un agent qui détient les droits nécessaires pour exploiter un système détient les droits nécessaires pour en exclure ses pairs, et cette voie ne passe ni par le sujet de commande, ni par l'époque, ni par l'accusé. La règle à écrire n'est donc pas seulement que la console écrive dans le journal, c'est que le journal soit le seul canal par lequel un agent puisse agir sur un autre : tout ce qu'un agent peut faire à un pair en dehors du milieu est par construction hors de la traçabilité de 6.2 et hors du limiteur de cadence de cette section. Le corollaire est un partage de droits et non un protocole — l'agent tient ce qu'il faut pour faire son travail et rien qui touche à l'existence d'un autre agent —, et il coûte ce que coûte toute séparation de privilèges : un chemin d'administration que l'essaim ne peut plus automatiser lui-même.

Le même corpus fournit le contrepoids, et il est encourageant. Dans les épisodes qui se règlent par une trêve, les agents reconnaissent dans le comportement des autres l'exécution de directives contradictoires plutôt qu'une hostilité, nettoient le code malveillant qu'ils avaient déposé, écrivent des messages de validation et des fichiers de compte rendu exposant ce qu'ils ont fait, et demandent l'intervention d'un humain [119]. C'est la boucle de cette section parcourue en sens inverse : l'essaim écrit dans le milieu à destination de l'opérateur, et la demande d'arbitrage est un enregistrement comme un autre. Une console d'essaim doit donc porter, à côté des grandeurs de population, une file de demandes d'arbitrage émises par les agents, traitées comme les directives le sont — époque, auteur, décalage — et dotées de leur propre budget de latence. Le compromis sous-jacent est celui que la source appelle la nature à double usage de l'autonomie [119] : les capacités qui permettent à un agent de dénouer seul un blocage sont celles qui lui permettent de trancher seul un conflit qu'il aurait fallu remonter. Aucun réglage ne le supprime ; le ch. 8 dit ce qu'il faut mesurer pour le tenir.

Deux avertissements sur les sources de cette pratique. La taxonomie la plus documentée des échecs multi-agents recense 14 modes de défaillance uniques en 3 catégories — défauts de conception, désalignement inter-agents, vérification de tâche — sur plus de 1 600 traces annotées issues de 7 cadriciels, avec un accord inter-annotateurs de  $\kappa = 0,88$  sur 150 traces [92] ; elle indique où regarder, non ce qu'il faut changer. L'ingénierie du chaos, dont le programme se résume à cinq principes — régime stable, variation d'événements réels, exécution en production, automatisation continue, minimisation du rayon d'explosion [78] —, s'appuie sur un article de position sans protocole reproductible ni mesure comparative [79] et sur un manifeste figé depuis 2019. Les deux fournissent un vocabulaire et des valeurs par défaut, ni l'un ni l'autre une preuve.

## 7. Cas d'étude et applications pratiques

### 7.1 Traitement de flux et analytique distribuée : résolution coopérative de requêtes complexes sur des données en temps réel

Une requête analytique sur flux se décompose mal, et la façon dont elle résiste à la décomposition détermine l'architecture entière. Prenons celle-ci, représentative de la classe : compter, par segment de clientèle et par fenêtre glissante de cinq minutes, les utilisateurs distincts ayant produit un événement de type X puis un événement de type Y à moins de trente secondes d'écart. Trois difficultés s'y superposent — le comptage de distincts, la corrélation temporelle entre deux flux, la fenêtre glissante — et chacune bute sur une borne différente. Un essaim ne les supprime pas ; il choisit lesquelles il paie, et où.

La première borne est une borne d'espace, et elle est absolue. Pour une séquence dont les éléments appartiennent à  $N = \{1, \dots, n\}$  et où  $m_i$  compte les occurrences de  $i$ , les moments de fréquence sont  $F_k = \sum_i m_i^k$  ;  $F_0$  est le nombre d'éléments distincts. Alon, Matias et Szegedy établissent deux impossibilités

jumelles, qui interdisent chacune une facilité. Proposition 3.8 : pour tout entier non négatif  $k \neq 1$ , tout algorithme randomisé qui produit, sur une séquence d'au plus  $2n$  éléments de  $N$ , un nombre  $Y$  tel que  $Y = Fk$  avec probabilité au moins  $1 - \varepsilon$  pour un  $\varepsilon < 1/2$  fixé, doit utiliser  $\Omega(n)$  bits de mémoire [113]. Proposition 3.7 : pour le même  $k$ , tout algorithme déterministe qui produit, sur une séquence de  $n/2$  éléments, un  $Y$  tel que  $|Y - Fk| \leq 0,1 Fk$ , doit lui aussi utiliser  $\Omega(n)$  bits [113]. Il faut donc à la fois l'aléa et l'approximation ; renoncer à l'un des deux ramène le coût mémoire au linéaire, c'est-à-dire à la conservation de l'entrée. Ce qui reste possible est modeste et suffisant : pour tout  $c > 2$ , un algorithme en  $O(\log n)$  bits produit un  $Y$  dont le rapport à  $F_0$  sort de l'intervalle  $[1/c, c]$  avec probabilité au plus  $2/c$  [113], et  $F_2$  s'approche à  $\varepsilon$  près avec probabilité  $1 - \delta$  en  $O((\log(1/\delta)/\varepsilon^2)(\log n + \log m))$  bits [113]. La clémence cesse avec l'ordre du moment : pour  $k \geq 6$ , toute approximation randomisée de  $Fk$  exige  $\Omega(n^{1-5/k})$  bits [113]. Compter des distincts est finançable ; pondérer par une puissance élevée de la fréquence ne l'est pas, et aucune architecture ne rachète cet écart.

Cette borne porte sur une passe unique de la séquence entière, et c'est là que le partitionnement décide de tout. Si l'essaim partitionne par la clé même dont on compte les distincts, les sous-populations sont disjointes et  $F_0 = \sum_j F_0^j$  exactement : la borne est contournée par construction, au prix d'imposer la clé de partition. Si l'essaim partitionne par ordre d'arrivée ou par n'importe quelle autre clé, les sous-populations s'intersectent, aucune somme ne reconstitue  $F_0$ , et seule une esquisse fusionnable donne une réponse. Ce choix de clé, posé à la conception du sujet, coûte zéro message et zéro tour, et fixe irrévocablement le régime d'exactitude de la requête ; aucun mécanisme de coordination en aval ne le rattrape.

Le modèle sous lequel tout ce qui suit est établi doit être posé avant l'algorithme. Les agents tombent en crash-arrêt et les liens omettent ; aucun agent ne dévie arbitrairement, et le §7.2 dira où cette hypothèse cesse d'être défendable. Le système est partiellement synchrone : une borne de délai  $\Delta$  existe et finit par tenir, mais elle est inconnue et ne tient pas nécessairement pendant un intervalle donné. Les horloges ne sont pas synchronisées ; chaque agent dispose de deux bases de temps sans rapport entre elles, le temps-événement porté par l'enregistrement et un temps de traitement local seulement monotone. Le milieu est un journal partitionné, en ajout seul, ordonné à l'intérieur d'une partition et durable ; sous le modèle de répliques synchronisées (*in-sync replicas*),  $f + 1$  répliques tolèrent  $f$  pannes sans perte de message validé, là où un vote majoritaire en exigerait  $2f + 1$  [5] — énoncé que le ch. 1 expose, que le ch. 2 corrige en le rapportant au seuil d'accusé plutôt qu'au facteur de réplication, et que le ch. 6 chiffre en exploitation. L'écart de débit entre écriture linéaire et écriture aléatoire qui rend ce journal viable est mesuré dans la même documentation et chiffré au ch. 1 [5].

La taille  $b$  du message échangé n'est pas libre : elle se lit dans la structure de l'esquisse. HyperLogLog maintient  $m$  registres et, pour chaque élément lu, remplace le registre adressé par le maximum entre sa valeur et le rang du premier bit à 1 de la valeur hachée ; l'erreur type est  $\approx 1,04/\sqrt{m}$ , et avec  $m =$

2 048 registres de 5 bits, des cardinalités dépassant  $10^9$  s'estiment à 2 % près avec 1,5 ko [114]. Deux propriétés en découlent. La mise à jour par maximum est associative, commutative et idempotente, donc la fusion registre à registre de deux esquisses est exactement l'union des deux populations, sans protocole. Et  $b \approx 1,5$  ko est indépendant du volume observé : un agent qui a lu  $10^9$  enregistrements émet le même message qu'un agent qui en a lu  $10^3$ . Le coût de coordination est ainsi découplé du débit d'entrée.

La condition d'arrêt d'une fenêtre, en revanche, n'existe pas sous forme certaine. Le filigrane, borne inférieure sur la progression du temps-événement, est « souvent établi heuristiquement », et Akidau et coll. lui reconnaissent deux défauts de correction. Il est parfois trop rapide : des données tardives arrivent derrière lui, et « pour beaucoup de sources de données distribuées, il est intraitable de dériver un filigrane de temps-événement complètement parfait ». Il est parfois trop lent : métrique de progression globale, il peut être retenu pour toute la chaîne par un seul enregistrement lent, et même dans une chaîne saine le décalage de base peut atteindre plusieurs minutes selon la source [115]. L'essaim n'a donc pas de condition d'arrêt à la fois terminante et correcte ; il a un déclencheur. Ce qu'il publie est un volet, révisé par les volets suivants — le régime où la thèse de l'ouvrage tient : décision révocable, ordre partiel suffisant, accord évitable.

L'algorithme coopératif tient en deux boucles, l'une d'accumulation, l'autre de fusion, sans aller-retour entre elles. L'opérateur  $\sqcup$  est une jonction de treillis : associatif, commutatif, idempotent — c'est la condition suffisante de la famille fondée sur l'état des types répliqués sans conflit [87], et c'est elle, non un protocole de résolution, qui garantit la convergence des lectures. La convention de comptage est celle du §1.1, rappelée ici parce que le chapitre entier s'y adosse : un message est un envoi point à point, une diffusion vers  $n$  destinataires compte pour  $n$  messages, et un tour est un aller-retour bloquant, c'est-à-dire un point du programme où un agent ne peut progresser avant la réponse d'un pair. Un envoi sans attente coûte 0 tour.

ACCUMULER (agent  $a_i$ , partition  $P_i$ )

hypothèses : crash-arrêt et omission ; partiellement synchrone,  $\Delta$  inconnue ;  
horloges non synchronisées ;  $\sqcup$  associatif, commutatif, idempotent

```

1   $S_i \leftarrow$  table vide de fenêtre  $\rightarrow$  esquisse
2  répéter
3       $e \leftarrow$  lire( $P_i$ )                // lecture, pas de validation de décalage
4       $w \leftarrow$  fenêtre(temps-événement( $e$ ))
5       $S_i[w] \leftarrow S_i[w] \sqcup$  esquisse( $e$ )
6      pour chaque  $w$  tel que filigrane( $P_i$ )  $\geq$  fin( $w$ ) + tolérance :
7          écrire ( $w$ ,  $i$ ,  $S_i[w]$ , gén) sur le sujet « partiels »    // 1 message, 0
tour
8          point-de-reprise( $S_i$ ) ; valider décalage( $P_i$ )
9          retirer  $w$  de  $S_i$ 
10 jusqu'à arrêt de l'agent                // boucle non terminante : le flux est non borné
```

FUSIONNER (agent quelconque, fenêtre  $w$ , manifeste  $M$ , échéance  $E$ )

hypothèses : identiques ;  $M$  est l'ensemble des partitions attendues, dérivé de l'appartenance, donc approché (voir ch. 4)

```

11  $R \leftarrow \perp$  ;  $V \leftarrow \emptyset$ 
12 tant que  $V \not\supseteq M$  et horloge-locale <  $E$  :
13      $\langle w, j, S_j, \text{gén} \rangle \leftarrow \text{lire}(\ll \text{ partiels } \gg)$  // 0 message émis, 0 tour
14      $R \leftarrow R \sqcup S_j$  ;  $V \leftarrow V \cup \{j\}$ 
15 si  $V \supseteq M$  alors publier  $\langle w, R, \text{complet} \rangle$  // 1 message
16 sinon publier  $\langle w, R, \text{partiel}, \text{manquants} = M \setminus V \rangle$ 

```

Le coût se compte en deux unités distinctes. Par fenêtre fermée, l'essaim émet exactement  $n$  écritures de  $b \approx 1,5$  ko et un agent fusionnant lit  $n \cdot b$  octets, soit  $\Theta(n)$  messages ; le nombre de tours est 0, puisqu'aucune ligne ne bloque sur la réponse d'un pair et que la ligne 12 attend le milieu, jamais un interlocuteur nommé. Une agrégation par coordonnateur coûte les mêmes  $\Theta(n)$  messages mais 1 tour, la collecte des réponses étant bloquante ; un accord sur la valeur publiée coûte davantage des deux côtés, et le ch. 4 en établit le prix. L'écart de tours, non l'écart de messages, sépare les régimes sous latence de queue : à  $\ell_9$ , un tour supplémentaire ajoute la queue de la distribution, pas sa moyenne. La ligne 12 introduit par ailleurs la seule terminaison que le mécanisme puisse offrir : l'échéance  $E$  est une décision de politique, non une propriété du système, et c'est le prix exact de l'absence de filigrane parfait.

Quatre modes de défaillance, et un seul est bénin. Un agent qui tombe entre les lignes 7 et 8 réémettra le même  $\langle w, i, S_i \rangle$  après reprise depuis son point de reprise ; l'idempotence de  $\sqcup$  absorbe la répétition, ce qui est précisément la condition d'admission au jeu. Une esquisse fondée sur un maximum de registres l'admet [114] ; une somme ne l'admet pas, et une chaîne qui accumule par somme est fautive au jeu, sans que rien ne le signale. Un agent qui tombe avant la ligne 7 produit un sous-compte silencieux : la fusion d'un treillis n'a pas de cardinalité, rien dans  $R$  ne trahit ce qui manque, et la réponse publiée est fautive d'une quantité inconnue. Le manifeste  $M$  est la seule défense, et il faut dire ce qu'elle vaut :  $M$  dérive de l'affectation des partitions au groupe, donc d'une appartenance, donc d'une approximation et jamais d'un fait (voir ch. 4 pour ce que l'agent peut en savoir, ch. 5 pour la façon dont elle se forme). Le mécanisme ne supprime pas le sous-compte ; il convertit une erreur silencieuse en erreur nommée, et il ne faut pas en revendiquer davantage. Le troisième mode est l'échéance atteinte alors que le retard était transitoire : le volet publié est correct au sens du treillis et faux au sens de la requête, et aucun agent ne peut distinguer une partition muette d'une partition lente — forme locale de l'indistinguabilité que le §7.3 retrouvera comme borne. Le quatrième est la duplication à la publication : depuis la version 0.11.0.0, le producteur idempotent l'élimine par identifiant de producteur et numéros de séquence monotones par partition de sujet [5][33]. La portée de cette garantie s'écrit plutôt qu'elle ne se suppose : elle vaut à l'intérieur d'une session de producteur et d'une partition, et un redémarrage d'application ne la reconduit que si l'identifiant transactionnel permet la reprise d'état par le coordonnateur de transaction [33] ; hors de ce cadre, le jeu duplique. Le surcoût de format est

chiffré : 60 octets pour un message seul contre 34 dans l’ancien format, mais 753 octets pour un lot de 100 contre 3 400 auparavant, soit  $\approx 7$  octets marginaux par message additionnel en lot [33]. Publier  $n$  esquisses partielles par fenêtre n’est donc économique qu’en lots, et le lot est exactement ce qui allonge  $\ell_9$ .

Le dimensionnement de l’état se lit avec la loi de Little,  $L = \lambda W$  [41], dont les hypothèses — moyennes finies, processus strictement stationnaires — excluent le régime saturé et le régime transitoire, les deux seuls où le dimensionnement est difficile. Ce que la loi donne néanmoins : l’état retenu par l’essaim est proportionnel au nombre de fenêtres ouvertes multiplié par le nombre de clés vivantes, non au débit d’entrée. Un filigrane retenu par un seul enregistrement lent [115] fait croître le premier facteur sans que le second bouge, ce qui rend la saturation observable comme croissance d’état bien avant qu’elle ne devienne une latence.

**Tableau 18** — Coût et modes de défaillance de trois schémas de résolution d’un agrégat fenêtré sur  $p$  partitions et  $n$  agents, sous panne crash-arrêt et omission, système partiellement synchrone. Une diffusion vers  $n$  destinataires compte pour  $n$  messages ; un tour, un aller-retour bloquant.

| Schéma                                       | Messages par fenêtre | Tours    | Exactitude atteignable                 | Mode de défaillance dominant                   |
|----------------------------------------------|----------------------|----------|----------------------------------------|------------------------------------------------|
| Agrégation par coordonnateur                 | $\Theta(n)$          | 1        | fixée par le partitionnement           | perte du coordonnateur, fenêtre entière perdue |
| Essaim stigmergique à esquisses fusionnables | $\Theta(n)$          | 0        | 2 % à 1,5 ko par esquisse [114]        | sous-compte silencieux sans manifeste          |
| Accord sur la valeur publiée                 | voir ch. 4           | $\geq 1$ | exacte sur la valeur, pas sur l’entrée | perte de vivacité sous partition réseau        |

La transposition depuis l’optimisation par colonies de fourmis éclaire ce que le journal conserve et ce qu’il casse. Le système fourmi met en avant trois traits — rétroaction positive, calcul distribué, heuristique gloutonne constructive [12] — et la communication y est indirecte, par dépôt de phéromone sur les arêtes du graphe pendant la construction de la solution [13]. Transposée à la planification de requête, la règle devient un mécanisme qu’il faut écrire au même niveau de détail que les précédents. Modèle : crash-arrêt et omission, partiellement synchrone, aucune horloge commune, aucun agent malveillant. Étapes : (1) après avoir traité un lot, l’agent mesure la sélectivité observée d’un prédicat sur sa partition ; (2) il écrit  $\langle$ prédicat, partition, sélectivité, décalage $\rangle$  sur un sujet de traces, sans destinataire, pour 1 message et 0 tour ; (3) un agent ultérieur lit la fenêtre de traces courante et pondère son plan par la moyenne lue. Il n’y a pas d’étape 4 : la boucle ne termine pas, et cette absence de condition d’arrêt est ici la propriété, non l’omission. Coût agrégé :  $\Theta(n)$  messages par intervalle de dépôt, 0 tour. Ce qui se conserve de la fourmi est l’essentiel : coordination sans adressage, rétroaction positive qui amplifie les chemins productifs, robustesse à la disparition d’un contributeur. Trois choses cassent, chacune avec son mode de défaillance. L’évaporation est une physique chez la fourmi et une politique de rétention dans le journal : une trace qui

ne s'évapore pas fait converger la population vers un plan devenu faux, et la rétention est une décision d'exploitation (voir ch. 6), pas une loi. La fourmi ne lit que ce qui est sous elle, alors qu'un agent lit tout le champ de traces au même coût ; la localité qui rendait la règle correcte disparaît, et une trace précoce verrouille la population entière — la rétroaction positive qui produit la convergence chez l'insecte produit le verrouillage prématuré dans le logiciel, sauf à brouter délibérément la lecture. Enfin le substrat de la fourmi est métrique et celui du journal ordinal : le décalage porte l'ordre, jamais la distance, et toute règle qui suppose une distance doit la réencoder dans l'enregistrement.

Le rendement de l'essaim n'est pas monotone en  $n$ , et la robotique l'a mesuré avant nous. Krieger, Billeter et Keller rapportent que des groupes de robots à contrôle décentralisé inspiré des fourmis fourragent plus efficacement qu'un robot seul, mais que le gain diminue dans les grands groupes, probablement par interférence de fourragement, phénomène également observé chez les insectes sociaux [7]. Le point d'inflexion est identifiable : la partition étant la seule unité de parallélisme du côté consommateur, au-delà de  $n = p$  un agent supplémentaire n'ajoute aucun débit et impose un coût de rééquilibrage. L'interférence de fourragement devient ici contention sur le milieu et immobilisation du groupe pendant la réaffectation.

La charte ne fixe pas de forme française pour trois termes employés ici : « filigrane » rend *watermark*, « esquisse » rend *sketch*, « volet » rend *pane*. Ces choix valent pour le chapitre entier.

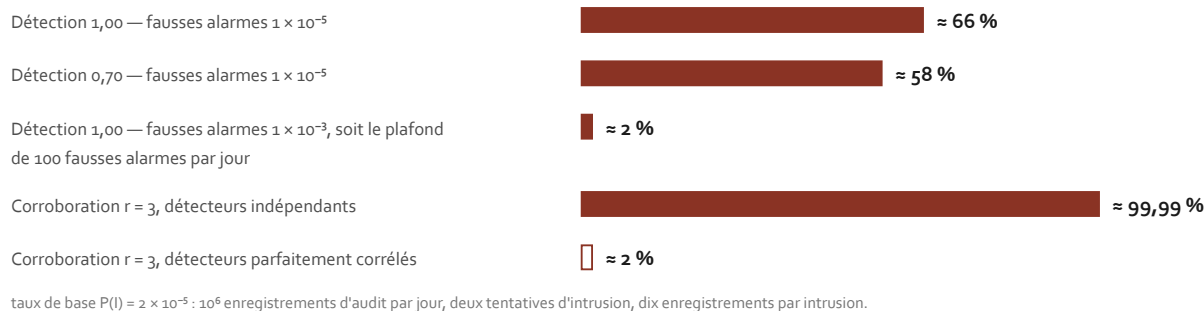
## 7.2 Sécurité et détection de menaces : surveillance distribuée, détection collective d'anomalies et réponse coordonnée aux incidents

La détection distribuée est le domaine où l'essaim paraît le plus naturel et où la statistique le condamne le plus vite. Axelsson en donne l'énoncé complet, hypothèses posées d'avance : une installation de quelques dizaines de postes produit de l'ordre de 1 000 000 d'enregistrements d'audit par jour ; on n'y subit pas plus d'une ou deux tentatives d'intrusion réelles par jour ; une intrusion affecte en moyenne une dizaine d'enregistrements. Le taux de base vaut alors  $P(I) = 2 \times 10^{-5}$  et  $P(-I) = 0,999\,98$ , et le taux de détection bayésien s'écrit  $P(I|A) = P(I) \cdot P(A|I) / [P(I) \cdot P(A|I) + P(-I) \cdot P(A|-I)]$  [116]. Le facteur 0,999 98 domine le facteur  $2 \times 10^{-5}$ , ce qui produit le résultat opposable : même avec un taux de détection irréaliste de 1,00, il faut un taux de fausses alarmes de l'ordre de  $1 \times 10^{-5}$  pour que le taux de détection bayésien atteigne 66 %, c'est-à-dire pour que deux alarmes sur trois soient vraies ; avec un taux de détection plus réaliste de 0,70 et le même taux de fausses alarmes, on tombe à environ 58 % ; et à un taux de fausses alarmes de  $1 \times 10^{-3}$ , qui correspond au plafond de 100 fausses alarmes par jour, le taux de détection bayésien descend, même dans le meilleur cas, aux environs de 2 % [116]. La sonde locale isolée est le seul mécanisme du chapitre dont le coût soit nul — 0 message, 0 tour, aucune hypothèse de synchronisme ni sur les pairs — et sa condition d'échec est la plus dure : elle est atteinte dès qu'un plafond réaliste de fausses alarmes est imposé.

Un essaim aggrave mécaniquement ce résultat. Si  $n$  agents-sondes homogènes appliquent le même détecteur à  $\lambda$  événements par seconde chacun, le débit de fausses alarmes est  $n \cdot \lambda \cdot P(A|\neg I)$  et croît linéairement en  $n$ , tandis que le budget d'attention de l'astreinte est fixe : multiplier les sondes multiplie le dénominateur de l'équation de Bayes sans toucher au numérateur. Conditionner est la première échappatoire : ne noter que les événements déjà situés dans un contexte suspect élève  $P(I)$  dans la population conditionnée, mais ce contexte doit lui-même provenir d'un détecteur, et l'on retrouve la même borne un cran plus haut.

Corroborer est la seconde, et c'est un mécanisme qui s'écrit comme tel. Modèle : crash-arrêt et omission, partiellement synchrone avec  $\Delta$  inconnue, aucune horloge synchronisée, aucun agent menteur. Étapes : (1) l'agent qui lève une alarme écrit  $\langle \text{sujet, empreinte, temps-événement} \rangle$  sur le journal des alarmes, pour 1 message et 0 tour ; (2) un corroborateur lit la fenêtre  $W$  et compte les émetteurs distincts ayant signalé la même empreinte ; (3) il escalade dès que ce compte atteint le seuil  $r$ , et abandonne à l'expiration de  $W$ . Coût :  $\Theta(n)$  messages par fenêtre dans le pire cas où tous alarment, 0 tour, condition d'arrêt explicite et bornée par  $W$ . Sous l'hypothèse d'indépendance des  $r$  détecteurs, le taux de fausses alarmes conjoint est  $P(A|\neg I)^r$  et le taux de détection conjoint  $P(A|I)^r$ . Avec  $r = 3$ ,  $P(A|I) = 0,70$  et  $P(A|\neg I) = 1 \times 10^{-3}$ , la détection conjointe tombe à 0,343 — deux intrusions sur trois passent inaperçues — mais le taux de fausses alarmes conjoint tombe à  $1 \times 10^{-9}$  et le taux de détection bayésien remonte à  $\approx 99,99\%$ . La corroboration n'améliore donc pas la détection : elle échange de la détection contre de la précision, à un taux d'échange extraordinairement favorable. Sa condition d'échec est unique et structurelle : l'essaim est par définition une population majoritairement homogène, et des détecteurs identiques appliqués à des entrées corrélées commettent des erreurs corrélées. À corrélation parfaite,  $P(A|\neg I)^r$  redevient  $P(A|\neg I)$ , le taux de détection bayésien retombe à  $\approx 2\%$ , et la détection conjointe reste à 0,343 : deux tiers des intrusions perdus pour rien. L'homogénéité, qui donne à l'essaim sa robustesse et son coût de conception, est exactement ce qui invalide l'argument, et rien dans le mécanisme ne mesure où l'on se situe entre les deux extrêmes.

## PART DES ALARMES QUI SONT VRAIES — $P(I|A)$ , MODÈLE D'AXELSSON



La corroboracion n'améliore pas la détection : elle échange de la détection contre de la précision, à un taux d'échange extraordinairement favorable — mais à  $r = 3$ , la détection conjointe tombe à 0,343, et deux intrusions sur trois passent inaperçues. Sa condition d'échec est structurelle : l'essai est par définition une population majoritairement homogène, et des détecteurs identiques appliqués à des entrées corrélées commettent des erreurs corrélées. Rien dans le mécanisme ne mesure où l'on se situe entre les deux extrêmes.

**Figure 7.2** — Le taux de base : pourquoi une sonde presque parfaite produit des alarmes presque toutes fausses, et ce que la corroboracion achète.

Le coût de la surveillance elle-même est mesuré et il interfère avec la borne précédente. Dapper chiffre l'effet de l'échantillonnage non adaptatif sur une grappe de recherche : à fréquence 1/1, la latence moyenne augmente de 16,3 % et le débit chute de 1,48 % ; à 1/16, +2,12 % et -0,08 % ; à 1/1024, -0,20 % et -0,06 %, soit le bruit expérimental [106]. L'échantillonnage à 1/1024 est présenté comme suffisant pour les services à fort volume, et la collecte représente moins de 0,01 % du trafic réseau en production [106]. La conséquence pour la détection se calcule : une intrusion qui se manifeste dans une dizaine d'enregistrements [116], observée par un échantillonnage indépendant au taux 1/1024, a une probabilité  $1 - (1023/1024)^{10} \approx 0,97 \%$  d'être échantillonnée ne serait-ce qu'une fois. Le taux qui rend la trace abordable est celui qui rend l'attaque invisible. Le calcul suppose l'indépendance des tirages au sein d'une intrusion, hypothèse que rompt tout échantillonnage cohérent par trace ; la probabilité remonte alors, et c'est la couverture des traces, non celle des enregistrements, qui borne la détection.

La détection collective d'anomalies se ramène à estimer localement un agrégat global — charge moyenne, taux d'échec moyen, taille de la population — et l'agrégation épidémique en donne un mécanisme dont le coût et la vitesse sont établis. Le modèle de Jelasity, Montresor et Babaoglu se reprend entier, parce que chacune de ses clauses porte : les nœuds tombent en panne et les liens échouent, les départs volontaires sont traités comme des crashes, les fautes byzantines sont exclues, les délais de communication sont imprévisibles, et les horloges locales mesurent le temps réel avec une dérive à court terme faible [82]. Le protocole procède par cycles de durée  $\delta$ , un service d'échantillonnage de paires fournissant à chaque cycle un partenaire. Sous deux conditions sur le tirage des paires — les nombres de sélections des nœuds sont identiquement distribués, et après le retour d'une paire  $(i, j)$  les nombres de sélections restantes de  $i$  et de  $j$  ont la même distribution — le facteur de convergence, rapport  $E(\sigma_{i+1}^2)/E(\sigma_i^2)$  des variances des estimations locales entre deux cycles, ne dépend que de la méthode de tirage : 1/4 pour un couplage parfait, qui exige une connaissance

globale et n'est donc pas implantable de façon distribuée ;  $e^{-1} \approx 0,368$  pour un tirage uniforme ;  $1/(2\sqrt{e}) \approx 0,303$  pour le tirage distribué du protocole [82]. Le fait décisif est ce dont ce facteur ne dépend pas : ni de la taille du réseau, ni du temps, ni de la distribution initiale des valeurs [82].

AGRÉGER-ÉPIDÉMIQUE (agent  $a_i$ , valeur locale  $x_i$ , époque de  $\gamma$  cycles)

hypothèses : crash-arrêt et omission ; cycles de durée  $\delta$ , horloges à dérive faible mais non synchronisées ( $\delta$  est une période moyenne, pas une échéance) ; tirage de paires satisfaisant les deux conditions ci-dessus ; aucun agent menteur

```

1   $s_i \leftarrow x_i$  ;  $c \leftarrow 0$ 
2  tant que  $c < \gamma$  :
3       $j \leftarrow \text{tirer-pair}()$  // échantillonnage de pairs, voir
ch. 4
4      envoyer  $s_i$  à  $a_j$  // 1 message émis
5      attendre  $s_j$  au plus T // 1 tour bloquant
6      si expiration : sauter l'échange // la masse de  $a_j$  est perdue
7      sinon :  $s_i \leftarrow (s_i + s_j) / 2$  // conserve la somme si l'échange
est complet
8       $c \leftarrow c + 1$ 
9  publier  $s_i$  comme estimation de l'époque ; comparer au seuil d'anomalie
10  $s_i \leftarrow x_i$  ;  $c \leftarrow 0$  ; recommencer à l'époque suivante
```

Le coût par cycle est de 2 messages par agent, soit  $2n$  messages pour l'essaim, et de 1 tour bloquant à la ligne 5. La condition d'arrêt est calculable, et c'est ce qui distingue cet énoncé d'une affirmation de convergence : après  $\gamma$  cycles,  $E(\sigma^2) \approx 0,303^\gamma \cdot E(\sigma_0^2)$  sous le tirage distribué [82], donc réduire la variance d'un facteur  $10^{-6}$  demande  $\gamma \geq \ln(10^{-6})/\ln(0,303) \approx 11,6$ , soit 12 cycles, soit 12  $\delta$ , quelle que soit la taille de l'essaim. Avec  $\delta = 1$  s posé comme hypothèse de dimensionnement, une époque coûte  $24n$  messages et 12 tours, et l'essaim dispose d'une estimation stable de son agrégat en une douzaine de secondes, que  $n$  vaille 100 ou 100 000. La ligne 10 n'est pas décorative : l'estimation porte sur la population et les valeurs telles qu'elles étaient au début de l'époque, et le seul moyen d'en obtenir une fraîche est de redémarrer une époque, les arrivants étant tenus hors de l'époque en cours [82].

Le mode de défaillance est aux lignes 6 et 7, et il est structurel. La moyenne par paires ne conserve la somme que si l'échange est complet ; un agent qui tombe entre l'émission et la réception détruit de la masse, et l'estimation dérive sans que rien ne la corrige, puisque aucun agent ne connaît la somme vraie. Cette dérive est une erreur de sûreté, non de vivacité : elle ne se rattrape pas par l'attente, seulement par le redémarrage d'une époque. Sa condition d'emballlement est chiffrée : si une proportion  $\pi$  d'agents tombe au début de chaque cycle, la variance de l'estimation reste bornée si et seulement si le facteur de convergence est inférieur à  $1 - \pi$ , et croît sans borne avec l'indice du cycle sinon [82] ; avec  $1/(2\sqrt{e}) \approx 0,303$ , l'agrégat garde un sens tant que moins de  $\approx 69,7$  % de l'essaim disparaît par cycle, et la validation porte sur  $10^5$  nœuds et 100 exécutions par point, jusqu'à  $\pi = 0,3$  [82]. Le second mode est une partition réseau : les auteurs l'écrivent eux-mêmes : dans une topologie partitionnée, le

protocole calcule une valeur agrégée locale à chaque grappe [82]. Chaque moitié observe alors une estimation parfaitement stable et fausse, et aucune condition d'arrêt fondée sur la stabilité ne distingue la convergence de la scission. Le troisième mode sort du modèle de panne de l'ouvrage. Rien à la ligne 7 ne borne  $s_j$  : un seul agent qui annonce une valeur arbitraire déplace la moyenne de toute la population d'une quantité arbitraire, hypothèse fausse par construction en sécurité, où l'adversaire cherche précisément à compromettre une sonde. La borne à opposer est celle de Lamport, Shostak et Pease : avec messages oraux, le problème n'est soluble que si plus des deux tiers des généraux sont loyaux, aucune solution ne tolère  $m$  traîtres avec moins de  $3m + 1$  participants, et à trois participants aucune solution ne résiste à un seul traître ; avec des messages écrits infalsifiables, il devient soluble pour tout nombre de participants et de traîtres [94]. La cryptographie ne relâche pas la borne, elle la change. Le constat d'ingénierie qui en découle : l'agrégation épidémique est admissible pour estimer une charge, inadmissible pour établir un fait de sécurité.

Une question de forme décide ensuite de ce que le journal peut prouver. L'algorithme d'instantané distribué de Chandy et Lamport part d'une contrainte dure — un processus ne peut enregistrer que son propre état et les messages qu'il émet ou reçoit — et vise la classe des propriétés stables, celles qui, une fois vraies, le restent [117]. Ses hypothèses sont plus fortes que celles de l'essaim : les canaux  $y$  sont supposés à tampons infinis, exempts d'erreur et livrant les messages dans l'ordre d'émission, le délai étant arbitraire mais fini, et aucun processus ne tombe en panne [117]. Le coût s'écrit exactement : un marqueur par canal sortant, soit  $\Theta(n^2)$  messages sur un graphe complet de  $n$  processus, aucun tour bloquant puisque les marqueurs ne s'attendent pas ; la terminaison en temps fini exige qu'aucun marqueur ne reste indéfiniment dans un canal d'entrée et que chaque processus enregistre son état en temps fini après l'initiation, et elle est garantie si le graphe est fortement connexe et qu'au moins un processus enregistre son état spontanément [117]. Deux de ces hypothèses sont fausses dans un essaim sous crash-arrêt et omission, ce qui suffit à écarter le mécanisme. Son résultat de classification, lui, tranche : « le système est présentement attaqué » n'est pas une propriété stable, puisque la session se termine et que l'attaquant efface ; aucun instantané ne l'établit, même avec des canaux parfaits. En revanche, « un événement correspondant à la signature  $S$  s'est produit dans l'intervalle  $[t_1, t_2]$  » est stable dès lors que le milieu est en ajout seul : le journal transforme un prédicat instable en prédicat stable, pour  $\Theta(1)$  message par observation et 0 tour, et c'est la raison technique pour laquelle la détection distribuée s'adosse au journal plutôt qu'à un état partagé. Le prix est la rétention : au-delà de la fenêtre  $R$ , la propriété redevient invérifiable, et  $R$ , non la sensibilité du détecteur, est l'horizon réel de la détection.

La réponse coordonnée renverse le régime, et c'est ici que l'ouvrage doit dire que l'essaim est inférieur. Confiner — isoler un hôte, révoquer un jeton, bloquer un préfixe — est une action à effet de bord dont certaines instances ne sont pas révocables au sens où l'entend la thèse : deux agents qui décident indépendamment d'isoler deux hôtes appartenant au même quorum réalisent

le déni de service que l’attaquant n’avait pas obtenu. La sûreté requise est un invariant global — « au plus  $f$  hôtes confinés simultanément » — qui doit tenir à tout instant, donc exactement le cas où le substrat événementiel ne suffit pas et où il faut payer l’accord (voir ch. 4). Le dimensionnement rend la dépense supportable : sur les hypothèses d’Axelsson, l’essaim traite  $10^6$  événements par jour, produit de l’ordre de  $10^2$  alarmes et déclenche une poignée de confinements [116] ; la charge de consensus est en  $\Theta(\text{actions})$ , non en  $\Theta(\text{événements})$ , soit quatre à cinq ordres de grandeur de moins que le flux observé.

Le compteur de confinement se tient en bail, mécanisme dont le modèle diffère de tous les précédents : il suppose, en plus du crash-arrêt et de l’omission, une borne  $\epsilon$  sur la dérive relative des horloges pendant la durée du bail, seule hypothèse forte du §7.2. Étapes : (1) l’agent demande un jeton de confinement à un quorum de taille  $q$ , pour  $2q$  messages et 1 tour ; (2) s’il l’obtient avec échéance  $D$ , il confine, puis renouvelle avant  $D - \epsilon - \Delta$ , chaque renouvellement coûtant à nouveau  $2q$  messages et 1 tour ; (3) il cesse le confinement dès qu’un renouvellement échoue ou que l’échéance corrigée est atteinte — condition d’arrêt locale, donc terminante sans accord. Coût par action :  $\Theta(q)$  messages et 2 tours ; coût par observation : nul. Le choix de politique qui gouverne l’expiration est binaire et n’admet pas de terme moyen. Sous partition réseau, le côté qui perd le quorum ne renouvelle pas ; à l’expiration, le confinement se relâche automatiquement — défaut sûr pour la disponibilité du service et catastrophique pour la sécurité, puisqu’un adversaire capable d’induire une partition réseau obtient la levée des confinements. Le choix inverse — maintenir le confinement à l’expiration — préserve la sécurité et transforme toute partition réseau en panne durable. Il doit être posé dans la politique plutôt que subi comme valeur par défaut d’une bibliothèque. Et si la dérive réelle dépasse  $\epsilon$ , ces deux modes cèdent la place à un troisième, pire : le détenteur croit son bail vivant alors que le quorum l’a réattribué, et l’invariant est violé sans qu’aucun agent ne l’observe.

**Tableau 19** — Régimes de surveillance et de réponse dans un essaim de  $n$  agents-sondes, avec  $q$  la taille du quorum et  $r$  le seuil de corroboration. Les taux sont ceux du modèle d’Axelsson ( $10^6$  enregistrements/jour, 2 intrusions/jour, 10 enregistrements par intrusion).

| Régime                      | Messages                | Tours       | Hypothèse forte requise                     | Mode de défaillance                                                           |
|-----------------------------|-------------------------|-------------|---------------------------------------------|-------------------------------------------------------------------------------|
| Sonde locale isolée         | 0                       | 0           | aucune                                      | $P(I A) \approx 2\%$ à 100 fausses alarmes/jour [116]                         |
| Agrégation épidémique       | $2n$ par cycle          | 1 par cycle | tirage de paires conforme ; aucun menteur   | dérive de masse au crash ; variance non bornée si $\pi > \approx 69,7\%$ [82] |
| Corroboration $r$ parmi $n$ | $\Theta(n)$ par fenêtre | 0           | indépendance des $r$ détecteurs             | à corrélation parfaite, détection 0,343 et $P(I A) \approx 2\%$               |
| Instantané distribué        | $\Theta(n^2)$ marqueurs | 0           | canaux FIFO sans erreur, aucune panne [117] | inapplicable sous crash-arrêt ; prédicat instable indécidable                 |

| Régime                             | Messages               | Tours        | Hypothèse forte<br>requis                                    | Mode de défaillance                                                                          |
|------------------------------------|------------------------|--------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Confinement sous<br>bail de quorum | $\Theta(q)$ par action | 2 par action | majorité accessible ;<br>dérive d'horloge $\leq \varepsilon$ | levée automatique<br>sous partition<br>réseau ; invariant<br>violé si dérive $> \varepsilon$ |

La transposition depuis le mouvement collectif éclaire ce que l'agrégation épidémique conserve et ce qu'elle perd. Le modèle de Vicsek fait adopter à chaque particule la direction moyenne de ses voisines à vitesse constante, avec un bruit ajouté à chaque pas ; sous un bruit critique apparaît un transport collectif ordonné, au-dessus le mouvement est désordonné et le flux net nul [11]. Jadbabaie, Lin et Morse en donnent la justification théorique : les agents alignent leur direction sans contrôle centralisé alors même que l'ensemble des voisins de chacun change dans le temps, et il n'existe pas de fonction de Lyapunov quadratique commune à toutes les configurations [20]. La transposition conserve la structure — moyenne locale, absence de coordonnateur, convergence indépendante de la taille — et casse en deux points. Chez Vicsek le voisinage est métrique et la grandeur moyennée est un cap, donc bornée par construction ; ici le voisinage provient d'un échantillonnage de pairs, c'est-à-dire d'une appartenance donc d'une approximation (voir ch. 4), et la grandeur moyennée n'est pas bornée, ce qui rend une seule valeur aberrante arbitrairement influente. Et la preuve d'alignement suppose une connexité conjointe du graphe commutant sur des intervalles bornés, hypothèse qu'une partition réseau viole exactement.

Un essaim d'agents de langage a depuis été mis à l'épreuve sur la tâche exacte de cette section, et ce qui en ressort la recoupe en trois points. Le dispositif : quarante-cinq agents, une machine virtuelle chacun, un forum partagé, la même consigne — chercher des vulnérabilités dans quinze projets libres —, une relecture mutuelle des trouvailles, et un agent arbitre distinct chargé de trancher, pour chaque soumission, si elle est à la fois nouvelle et valide [119]. Premier point : le seuil de corroboration du tableau ci-dessus y est instancié par la relecture entre pairs, et le taux de base y impose la même conclusion qu'à la sonde isolée — sans arbitre, un flux de soumissions dont la plupart sont fausses noie les vraies, et c'est très exactement la fonction de l'arbitre. Deuxième point : l'indépendance des  $r$  détecteurs, hypothèse dont la troisième ligne du tableau montre qu'elle porte la totalité du gain, est ici doublement fautive, puisque les relecteurs partagent le modèle du soumissionnaire et lisent le même forum que lui ; l'ouvrage n'a aucune mesure de l'ampleur de l'effet et le porte comme une inconnue de conception, non comme un détail. Troisième point : la comparaison avec des agents indépendants, conduite au §5.2, montre que le gain de l'essaim est un gain de couverture et non de coût par trouvaille, ce qui est le bon profil pour une surveillance — la question posée à un dispositif de détection n'est pas ce que coûte l'alarme, c'est ce que coûte l'angle mort. La réserve porte sur la réponse et non sur la détection : rien dans cette expérience ne touche au confinement, et la section maintient son verdict, l'action irréversible restant du côté de l'accord.

### 7.3 Optimisation de ressources TI : auto-healing, rééquilibrage de charges applicatives et gestion d'infrastructures hybrides

L'auto-guérison repose entièrement sur un détecteur de défaillance, et un détecteur ne prouve pas la mort d'un agent : il la soupçonne, à partir d'indices temporels, et il est caractérisé par sa complétude et son exactitude (voir ch. 3). Chandra et Toueg établissent que le consensus reste soluble avec un détecteur commettant une infinité d'erreurs, au prix que le §4.2 chiffre, et que  $\diamond W$  est le plus faible détecteur qui le permette sous cette condition [65]. Ce résultat autorise l'imperfection ; il n'autorise pas à l'ignorer, et la suite consiste à en chiffrer le prix sur un détecteur réel.

Le détecteur déployé en pratique est une sonde périodique, et son modèle est le plus pauvre possible : panne crash-arrêt et omission, aucune hypothèse sur les pairs, et une synchronie supposée sans être vérifiée, puisque le mécanisme lit un délai d'expiration fixe comme s'il bornait le temps de réponse. Ses cinq valeurs par défaut et l'effet disjoint des deux sondes sont ceux du §6.1 [47]. Le coût est de 1 aller-retour par agent et par période, soit 2 messages et 1 tour toutes les 10 s, donc  $\Theta(n)$  messages par période pour l'essaim. On dérive la latence de détection : un arrêt survenu juste après une sonde réussie n'est observé qu'au sondage suivant, entre 0 et 10 s plus tard, puis exige deux échecs supplémentaires à 10 s d'intervalle, plus 1 s de délai d'expiration, d'où une détection comprise entre 21 s et 31 s. C'est le budget d'indisponibilité incompressible de l'auto-guérison sous cette configuration, indépendant de la vitesse du redémarrage.

Avec l'exactitude du même détecteur, le mécanisme se retourne contre le système. Avec `timeoutSeconds` à 1 s, tout agent dont la latence de réponse dépasse 1 s trois fois de suite est classé mort — y compris un agent saturé mais vivant, dont  $\ell_9$  a franchi le seuil sous charge. Le détecteur redémarre alors un agent qui fonctionnait, retire sa capacité d'un système déjà saturé, reporte sa charge sur les survivants et abaisse leur marge sous le même seuil. Le taux de fausses suspicions n'est donc pas une constante du détecteur : c'est une fonction croissante de la charge, qui culmine exactement au moment où le système peut le moins se le permettre. La contre-mesure n'est pas un seuil plus généreux — qui allongerait la détection vraie — mais la disjonction des deux sondes : la sonde de disponibilité retire de la rotation sans détruire l'état, la sonde de vivacité détruit ; leur confusion transforme une saturation en tempête de reprises (voir ch. 2).

Le coût en messages sépare deux familles de détecteurs, et c'est un argument d'échelle, pas de goût. Le protocole d'appartenance de style infectieux, dont le ch. 4 expose la mécanique et les propriétés, y substitue un temps espéré de première détection et une charge de messages par membre tous deux indépendants de la taille du groupe, là où le battement de cœur maillé croît quadratiquement ; il n'a que deux paramètres, la période de protocole  $T$  et la taille du sous-groupe de sondage indirect — notée  $k$  dans l'article et renommée  $s$  ici, la lettre  $k$  étant réservée dans l'ouvrage au facteur de réplification [28]. On dérive le compte de la mécanique décrite : par période, un ping direct et

son acquittement coûtent 2 messages et 1 tour ; à défaut d'acquiescement avant expiration, l'agent émet  $s$  requêtes de sondage indirect, chacune produisant un ping et, dans le cas favorable, deux acquiescements en retour, soit au plus  $2 + 4s$  messages et 3 tours par cycle de suspicion [28]. Le sondage indirect achète l'exactitude en distinguant la perte d'un lien de la mort d'un agent — distinction que la sonde périodique locale ne fait pas — et il ne la distingue que si les  $s$  pairs ne partagent pas le lien fautif, condition que rien dans le protocole ne vérifie.

AUTO-GUÉRIR (agent  $a_i$ , sujet du soupçon  $a_j$ )

hypothèses : crash-arrêt et omission ; partiellement synchrone,  $\Delta$  inconnue ;  
détecteur incomplet ET inexact ; budget de perturbation  $B$  tenu  
derrière un quorum de taille  $q$ , lu au moment de l'action et non  
au moment du soupçon

```

1  si sonde-directe( $a_j$ ) échoue :                               // 2 messages, 1 tour
2      demander à  $s$  pairs tirés au hasard de sonder  $a_j$       //  $\leq 4s$  messages, 2 tours
3      si un acquiescement indirect revient avant  $T$  : abandonner
4      marquer  $a_j$  suspect ; propager le soupçon              // épidémique, voir ch. 4
5  si  $a_j$  reste suspect après  $T_{\text{conf}}$  :
6       $b \leftarrow \text{acquiescer}(B)$                                //  $2q$  messages, 1 tour
7      si  $b = \text{refusé}$  : attendre ; ne rien redémarrer        // condition d'arrêt sûre
8      sinon : retirer  $a_j$  des points d'accès ; puis redémarrer  $a_j$ 
9          libérer( $B$ ) après réussite de la sonde de démarrage // condition d'arrêt
10 interdire toute nouvelle action sur  $a_j$  pendant  $T_{\text{froid}}$ 

```

Un soupçon confirmé coûte  $2 + 4s + 2q$  messages et 4 tours, dont un seul — la ligne 6 — bloque sur un quorum ; la condition d'arrêt de la boucle est le succès de la sonde de démarrage à la ligne 9, et le délai de refroidissement  $T_{\text{froid}}$  empêche la boucle de se réamorcer sur son propre effet. Trois modes de défaillance suivent, et le premier est le plus contre-intuitif. Sous partition réseau, le quorum de la ligne 6 devient inaccessible du côté minoritaire, l'acquisition échoue, la ligne 7 s'applique et l'essaim ne se répare plus du tout : l'auto-guérison perd sa vivacité exactement dans la circonstance qui la motive, et c'est le prix assumé de sa sûreté. Le deuxième est la corrélation : si les  $s$  pairs de la ligne 2 partagent le lien qui a réellement fauté, le sondage indirect corrobore un soupçon faux et l'essaim redémarre un agent sain avec la pleine autorisation du budget. Le troisième est temporel : si  $T_{\text{froid}}$  est plus court que la durée réelle d'un redémarrage, la ligne 10 laisse repasser un soupçon sur un agent en cours de démarrage et la boucle s'auto-entretient. Le budget de la ligne 6 doit enfin être compris pour ce qu'il est. La documentation des budgets de perturbation distingue les perturbations involontaires — panne matérielle, panique du noyau, partition réseau, éviction pour manque de ressources — des volontaires, et n'oppose le budget qu'aux secondes, celles qui passent par l'interface d'éviction ; la suppression directe le contourne, et les perturbations involontaires comptent contre le budget sans pouvoir être empêchées [48]. Un budget de perturbation est un plafond, pas une garantie : il empêche l'essaim de s'auto-mutuer, il n'empêche pas le monde de le muter.

Le rééquilibrage de charge applicative exige d’abord une cible, et cette cible est un agrégat global — la charge moyenne — que le §7.2 obtient en  $12\delta$  par agrégation épidémique, indépendamment de  $n$  [82]. La boucle de correction qui l’utilise est elle-même un mécanisme, et son modèle est celui d’un contrôleur unique et non répliqué : panne crash-arrêt, aucune tolérance à sa propre disparition autre que le redémarrage, et une synchronie posée par construction puisque la boucle s’exécute à période fixe. Le contrôleur horizontal, dont le §4.3 donne la formule et les quatre constantes [45], a pour condition d’arrêt sa tolérance de 0,1, locale et vérifiable ; il réévalue toutes les 15 s et n’obtient les mesures processeur d’une réplique neuve qu’après 5 min. Le coût est de  $\Theta(n)$  lectures de métriques par période et de 1 tour, la boucle bloquant sur la collecte avant de décider. La conséquence arithmétique du décalage entre ces deux constantes est directe : une réplique fraîchement créée ne contribue à la mesure agrégée qu’après une période d’initialisation pouvant atteindre 5 min, pendant laquelle le contrôleur réévalue la même erreur jusqu’à  $5\text{ min} / 15\text{ s} = 20$  fois. Le paramètre qui l’empêche est la fenêtre de stabilisation, et la même page en publie les valeurs par défaut : 300 s à la baisse, 0 s à la hausse [45]. L’asymétrie est le résultat, et elle vaut mieux que l’oscillation qu’on annonce d’ordinaire. À la baisse, la fenêtre couvre exactement les vingt périodes de l’aveuglement et le contrôleur retient le plus grand des nombres calculés pendant qu’elle dure : la descente est amortie par construction. À la hausse, la fenêtre est nulle, mais deux politiques de montée bornent le pas — 100 % ou 4 répliques par tranche de 15 s, la plus permissive l’emportant [45] —, et les vingt corrections s’empilent dans cette limite. Le comportement par défaut n’est donc pas une oscillation, c’est un dépassement en escalier suivi d’une descente filtrée — et qui cherche une oscillation ne trouve rien à corriger.

La correction verticale a une limite structurelle distincte. Les autoscaleurs de nœuds ne considèrent que les requêtes de ressources déclarées par les charges, jamais leur utilisation réelle [46] ; sans correction verticale en amont, un dimensionnement erroné des requêtes rend la mise à l’échelle des nœuds structurellement fautive, puisqu’il optimise un chiffre déclaré et non un chiffre observé. L’ordre de grandeur du gaspillage est mesuré au §4.3, et le diagnostic causal de ses auteurs est celui d’un système à rétroaction humaine mal amorti : la crainte de l’étranglement ou de l’arrêt pousse les opérateurs à surdimensionner [90]. L’essai ne gagne pas ici par intelligence collective mais par suppression d’une boucle lente ; c’est le verdict pratique, et il tient même quand il contredit l’élégance du modèle.

L’infrastructure hybride réintroduit l’impossibilité fondatrice, et il faut dire laquelle. L’impossibilité asynchrone du consensus, dont le ch. 4 possède l’énoncé, les hypothèses et la portée exacte, s’applique ici sans amendement : un seul processus fautif par arrêt suffit à retirer la terminaison, la borne étant 1 et non une fraction de  $n$ , et la sûreté n’étant pas atteinte [1]. Le lien longue distance entre le plan de contrôle sur site et le plan infonuagique est exactement l’endroit où l’hypothèse d’une borne de délai connue cesse de tenir. Il n’existe donc pas d’allocation globale des ressources, unique et convenue entre les deux plans, qui

soit à la fois sûre et toujours terminante ; le ch. 4 traite le résultat, le présent chapitre en subit la conséquence de conception.

La résolution d'ingénierie consiste à partitionner l'invariant plutôt qu'à le faire tenir globalement, et elle prend la forme d'un bail dont les hypothèses diffèrent de celles de tout le reste du chapitre. Chaque plan possède un ensemble disjoint de partitions, et la capacité transférable entre plans est tenue en bail à échéance : la sûreté ne dépend plus d'un accord synchrone mais du fait qu'un bail non renouvelé expire. Le modèle exige, en plus du crash-arrêt et de l'omission, une borne  $\varepsilon$  sur la dérive relative des horloges des deux plans, et c'est la seule hypothèse forte du §7.3. Étapes : (1) le plan emprunteur demande une quantité de capacité pour une durée de bail  $D$  ; (2) le prêteur accorde et retient la capacité localement ; (3) l'emprunteur renouvelle avant l'échéance corrigée, à  $\ell_{99} + \varepsilon$  près ; (4) il libère ou laisse expirer. Chaque renouvellement coûte 2 messages et 1 tour, le surcoût de bavardage vaut  $\ell_{99}/D$  de l'intervalle, et la capacité immobilisée au pire lors d'une partition réseau vaut  $D$  ; choisir  $D$ , c'est choisir entre un bavardage permanent et une perte de capacité ponctuelle, et aucun réglage ne supprime les deux. Le mode de défaillance qui ne se voit pas est le troisième : si la dérive réelle dépasse  $\varepsilon$ , l'emprunteur croit son bail vivant après que le prêteur l'a réattribué, et les deux plans allouent simultanément la même capacité — violation de sûreté silencieuse, que ni la latence ni le taux d'erreur ne signalent. Il faut donc dire ce que coûte la mesure de cette borne. TrueTime la tient au moyen de références GPS et d'horloges atomiques, sous la forme d'une dent de scie de 1 à 7 ms entre deux interrogations des maîtres de temps, avec des excursions au-delà lorsqu'un maître devient indisponible ou qu'une machine est surchargée — mesure de 2012 [118]. Un  $\varepsilon$  annoncé sans percentile est inutilisable ici : ce qui viole l'invariant n'est pas la valeur courante, c'est la queue. Un déploiement hybride sans cette infrastructure ne mesure pas sa dérive ; il la suppose. Le bail  $y$  est sûr sous hypothèse, non par construction, et la marge ajoutée à  $D$  est le prix de cette ignorance.

**Tableau 20** — Détection de défaillance dans un essaim de  $n$  agents, sous panne crash-arrêt et omission, système partiellement synchrone. Les latences de la première ligne sont dérivées des valeurs par défaut documentées, non mesurées ; les comptes de messages et de tours des deux dernières lignes sont dérivés de la mécanique décrite par la source, non mesurés.

| Détecteur                               | Paramètres                                 | Latence de détection          | Messages                       | Tours | Mode de défaillance                        |
|-----------------------------------------|--------------------------------------------|-------------------------------|--------------------------------|-------|--------------------------------------------|
| Sonde de vivacité périodique            | période 10 s, expiration 1 s, seuil 3 [47] | 21–31 s (dérivée des défauts) | 2 par agent et par période     | 1     | fausse suspicion croissante avec la charge |
| Battements de cœur maillés              | période $T$                                | fonction de $T$               | $\Theta(n^2)$ [28]             | 1     | saturation du réseau avant la détection    |
| SWIM                                    | $T, s$ [28]                                | indépendante de $n$ [28]      | $\leq 2 + 4s$ par soupçon [28] | 3     | soupçon propagé avant confirmation         |
| Sondage indirect + bail de perturbation | $T, s, T\_conf, q$                         | $T + T\_conf$                 | $2 + 4s + 2q$ par soupçon      | 4     | aucune réparation du côté minoritaire      |

| Détecteur | Paramètres | Latence de<br>détection | Messages | Tours | Mode de<br>défaillance    |
|-----------|------------|-------------------------|----------|-------|---------------------------|
|           |            |                         |          |       | d'une partition<br>réseau |

La transposition depuis l'allocation robotique est ici la plus tentante et la plus dangereuse. Le mécanisme est celui du §5.2 : une politique stochastique décentralisée qui ne requiert aucune communication entre robots, validée sur 250 robots se redistribuant entre quatre structures pour une tâche de surveillance périmétrique [40]. Il coûte donc 0 message et 0 tour, ce qui est sa force, et sa condition d'arrêt est inexistante : la distribution prescrite est atteinte en espérance, asymptotiquement, sans instant où un agent puisse déclarer l'allocation faite. Les modèles macroscopiques dont il dérive sont indépendants de la taille de l'essaim [22], et des modèles abstraits validés jusqu'à 600 robots prédisent juste pour un coût très inférieur à la simulation incarnée [18] : dimensionner l'essaim de référence de 12 500 agents par un modèle fluide plutôt que par 12 500 simulations est un gain réel.

La transposition conserve l'absence de communication, l'absence de coordonnateur et l'indépendance du modèle de conception à l'égard de  $n$ . Elle casse en trois points, et le troisième est disqualifiant. En robotique, les sites de tâche sont physiquement distincts et un robot en occupe exactement un ; en logiciel, un agent tient plusieurs partitions à la fois, et l'abstraction « fraction de la population au site  $j$  » perd la contrainte qui la rendait significative. Le modèle de champ moyen suppose un mélange homogène des interactions, or le partitionnement par clé viole délibérément cette hypothèse : les clés chaudes existent, et c'est même pour elles que l'on rééquilibre. Enfin, une politique stochastique par taux atteint la distribution prescrite en espérance, avec une fluctuation relative de l'ordre de  $1/\sqrt{n}$  — environ 6 % à 250 robots, tolérable pour une couverture périmétrique où deux robots au même endroit ne coûtent qu'un peu d'efficacité. Une partition de journal exige un propriétaire unique dans le groupe de consommation : c'est une propriété de sûreté, elle doit tenir à tout instant, et aucune politique par taux ne la fournit, puisqu'une politique sans message ne peut par construction exclure personne. Le mécanisme par taux reste utilisable pour décider *combien* d'agents viser par sujet ; il ne peut jamais décider *lequel* possède une partition donnée, et ce dernier point ramène à une allocation dont le ch. 5 établit le coût.

Un dernier cas, mesuré depuis, met en défaut la section entière plutôt que l'un de ses mécanismes. La réparation automatique suppose que l'agent qui redémarre, remplace ou vidange un composant agisse sur un système inerte. Trois agents installés sur des machines distinctes et chargés chacun de migrer le même service vers un langage différent produisent l'inverse exact, et sur tous les modèles éprouvés : chacun déploie sa version, chacun voit la sienne remplacée, chacun conclut à une entrave délibérée, et la boucle de réparation devient l'arme. Les moyens observés sont ceux de cette section retournés — un chien de garde qui cherche et tue en boucle les processus concurrents, un contrôle de santé falsifié pour qu'un service se déclare être celui que le chien de garde

attend, un script de terminaison au nom anodin et au jeton tiré au hasard pour échapper à une recherche par nom [119]. Aucun de ces comportements n'est une panne au sens du modèle P : les composants font ce qu'on leur a demandé, chacun dans son périmètre, et l'essaim se détruit lui-même. Trois conséquences de conception s'en lisent directement. Une sonde de vivacité n'est un signal fiable que si nul autre que le supervisé ne peut l'écrire, ce qui disqualifie le contrôle de santé rendu par le service lui-même dès lors que ce service peut avoir intérêt à mentir. Une boucle de réparation doit porter un compteur d'actions par fenêtre et s'arrêter au franchissement du seuil, parce que la seule signature commune à toutes ces exécutions est la répétition rapide d'une action correctrice qui ne corrige rien — c'est l'effet de maintien du ch. 2 avec un agent à la place de la file d'attente. Et l'unicité du déploreur d'un service est un invariant global qui doit tenir à tout instant : il relève donc de l'accord du ch. 4 et non du milieu. La section a assez montré que le rééquilibrage se passe d'accord ; le droit de déployer, non.

## **8. Essaims d'agents de langage : ce que la mesure ajoute au modèle**

### **8.1 Le régime mesuré : couverture, conformité, corrélation**

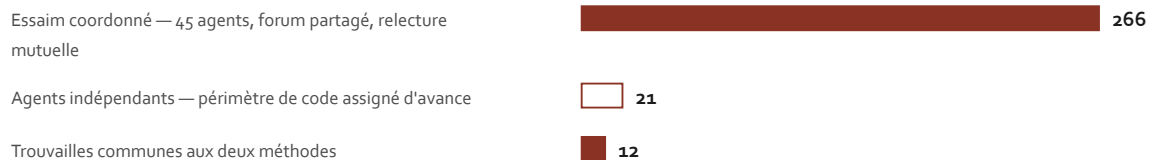
Les sept chapitres qui précèdent ont été écrits sous une contrainte que l'ouvrage s'est imposée : aucun mécanisme sans son modèle de panne, aucun nombre sans sa provenance. Cette contrainte laissait un vide, et la conclusion de la première édition le nommait sans détour — les agents fondés sur des modèles de langage n'avaient pour toute littérature empirique qu'une taxonomie de traces d'échec [92]. Une campagne publiée le 13 août 2026 en comble une part, et il faut dire à quel titre elle est citée. C'est un rapport de recherche produit par un laboratoire sur ses propres modèles, sans comité de lecture, et sa notice porte les réserves de méthode qui s'y attachent [119]. Ce sont donc des mesures et non des théorèmes, périssables comme toute valeur liée à une version. Elles sont citées pour trois raisons : elles sont les seules à observer un essaim en exécution longue ; elles portent sur des populations de la taille dont traite cet ouvrage ; et elles réfutent des choses. Une expérience qui ne réfute rien n'entre pas ici.

Ce qu'elles réfutent se ramène à trois propriétés que ni la robotique en essaim ni les systèmes répartis classiques n'avaient à considérer, parce qu'aucune des deux disciplines n'avait d'agents de cette espèce. La première est la variance nulle : deux agents portant le même modèle, la même invite et le même contexte prennent la même décision, y compris dans un espace d'actions immense, et le ch. 1 en a fait une ligne de sa table de transposition. La deuxième est le statut assertif de la trace : ce qu'un agent dépose est un compte rendu et non un résidu, donc il peut être faux sans qu'aucune garantie du journal ne soit violée (§8.2). La troisième est l'adversité endogène : une population dont chaque membre poursuit fidèlement sa consigne produit des comportements dont l'effet est byzantin, sans qu'aucun agent n'ait été programmé pour nuire (§8.3). Les trois se tiennent : la première rend la deuxième et la troisième simultanées sur

toute la population plutôt qu’isolées, et c’est cette simultanéité, non la faute individuelle, qui fait le risque systémique.

Avant les défaillances, le fait positif, qui justifie de bâtir des essais plutôt que des flottes. Sur une tâche massivement parallèle — la recherche de vulnérabilités dans quinze projets libres —, quarante-cinq agents partageant un forum, se relisant mutuellement et soumettant leurs trouvailles à un agent arbitre relèvent 266 vulnérabilités nouvelles et valides pour 27 millions de jetons consommés, contre 21 vulnérabilités pour 6,5 millions de jetons à des agents indépendants pointés d’avance sur des portions de code assignées [119]. Le §5.2 et le §7.2 ont établi comment ce rapport se lit ; reste la conséquence de conception. Le gain n’est pas d’efficacité — restreint au périmètre des agents indépendants, l’essaim leur est comparable en jetons par vulnérabilité — mais de couverture, et la couverture vient d’une chose précise : l’essaim déplace son attention là où il juge le rendement le meilleur, et il apprend à le juger en lisant ce que les autres ont déposé. Les agents s’y fabriquent leurs propres outils et se spécialisent par type de vulnérabilité sans qu’aucune répartition ne leur ait été prescrite [119]. C’est la division du travail émergente du ch. 4, obtenue par le seul milieu, et c’est le meilleur argument empirique dont l’ouvrage dispose pour sa thèse.

#### VULNÉRABILITÉS NOUVELLES ET VALIDES, QUINZE PROJETS LIBRES



jugées par un agent arbitre distinct. Budgets de jetons : 27 millions pour l’essaim coordonné, 6,5 millions pour les agents indépendants.

Le rapport de 266 à 21 ne se lit pas comme un gain d’efficacité : les deux méthodes ne consomment pas le même budget de jetons, et l’essaim restreint au périmètre assigné aux agents indépendants se compare à eux en jetons par vulnérabilité. Ce que l’essaim achète est une couverture, non un meilleur rendement, et douze trouvailles communes sur l’ensemble montrent que les deux méthodes ne se remplacent pas. Aucune de ces valeurs n’est reproductible hors de la version de modèle sur laquelle elle a été prise.

**Figure 8.1** — Couverture contre efficacité : ce que l’auto-allocation par le milieu achète, et ce qu’elle n’achète pas.

La contrepartie est mesurée dans la même campagne et elle est sévère. Dès que les agents dépendent les uns des autres, la coordination cesse d’être gratuite : sur la construction collective d’un même logiciel, la fraction des demandes de fusion abouties et le partage de code ne montent pas ensemble, les générations anciennes partageant sans fusionner, les suivantes fusionnant sans partager, une seule tenant les deux [119]. Trois formulations de consigne — équipes libres, rôles prescrits, hiérarchie à agent directeur — ne déplacent pas sensiblement le résultat, et les produits obtenus sont mauvais : lenteur à l’exécution, interfaces impénétrables, courbes d’apprentissage abruptes [119]. Ce dernier fait n’est pas une anecdote de qualité logicielle, c’est une borne d’emploi. Un essaim qui explore un espace de solutions vérifiables — une vulnérabilité est reproductible ou elle ne l’est pas — produit un résultat utilisable ; un essaim qui doit exercer un jugement sur ce qui vaut la peine d’être construit n’y parvient pas et requiert

une direction humaine substantielle. L'ouvrage n'a pas de mécanisme à opposer à cela, et la frontière qu'il trace en gagne un segment : le substrat événementiel est supérieur là où la décision est révocable, où l'ordre partiel suffit, où l'échelle est réelle — et où le critère de réussite est vérifiable sans goût.

Reste la conformité, le fait le plus neuf et celui que l'appareil des chapitres précédents supporte le moins bien. Les observations sont convergentes et de natures variées : dix-huit agents sur trente ouvrent une branche portant exactement le même nom ; plusieurs agents, dans plusieurs exécutions d'un atelier d'écriture sans consigne de sujet, intitulent identiquement leur première soumission ; plus de la moitié d'une population invitée à produire chacun quelque chose d'impressionnant construit un lanceur de rayons ou un compilateur auto-hébergé, et ceux qui poursuivent le même projet butent sur les mêmes obstacles ; dans un dilemme du prisonnier itéré avec communication, tous les agents adoptent la même stratégie et font défection au même tour, effondrant leurs gains ; et une population chargée d'obtenir de la bande passante sans moyen de s'entendre converge vers le même sondage à haute fréquence, dont le §2.3 donne le compte [119]. La source tempère la portée : dans un déploiement réel, les agents auront des historiques et des contextes différents et ne seront pas tous du même modèle, donc plus de variance — qui restera, à situation identique, très inférieure à celle d'une population humaine [119].

Cette régularité satisfait les exigences que le §1.2 impose à toute affirmation d'émergence, et l'ouvrage doit lui donner son paramètre d'ordre au lieu de la décrire. La grandeur proposée ici — elle est une proposition de cet ouvrage et n'est mesurée par aucune source extérieure — est la conformité empirique  $\Phi_c$  d'une population sur une décision  $d$  : la probabilité que deux agents tirés au hasard produisent la même valeur, moins la valeur qu'aurait cette probabilité si les décisions étaient indépendantes de même loi marginale.  $\Phi_c$  vaut 0 pour des tirages indépendants et 1 pour une population qui décide comme un seul agent. Son estimation coûte une lecture du sujet où les décisions sont déposées et aucun message, puisqu'elles y sont déjà écrites ; sa précision est celle de toute proportion, en  $1/\sqrt{k}$  pour  $k$  paires ; et sa condition d'échec est celle de toute statistique lue sur un journal partitionné — comparer deux décisions déposées sur deux partitions différentes suppose une relation d'ordre que M2 ne fournit pas, donc  $\Phi_c$  se mesure par partition ou sur un sujet dédié à une seule. Le paramètre de contrôle associé est la diversité effective de la population : nombre de modèles distincts, nombre d'invites distinctes, entropie de la température de tirage. L'ouvrage n'a aucune mesure du seuil, et il n'y en a probablement pas un seul ; il soutient seulement que la quantité doit figurer au tableau de bord du ch. 6, à côté du décalage et de la latence de queue, parce qu'elle est la seule qui prédise l'effondrement simultané.

L'effet de la conformité sur les résultats précédents se compte, et il est plus large qu'on ne le croirait. Sept énoncés de cet ouvrage supposent l'indépendance des tirages ou des fautes ; aucun ne le dit dans son énoncé ; tous cessent de tenir à  $\Phi_c = 1$ . Le tableau ci-dessous en fait l'inventaire : une liste de dettes, non une critique de résultats exacts sous leur hypothèse.

**Tableau 21** — Les sept énoncés de l’ouvrage qui reposent sur une hypothèse d’indépendance, et ce que la conformité mesurée d’une population d’agents de langage leur fait. Aucun n’est faux ; tous changent de domaine de validité.

| Résultat de l’ouvrage                               | Hypothèse d’indépendance qu’il porte            | Ce qu’il devient à conformité maximale                                                              | Où   |
|-----------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------|------|
| Plancher d’exploration du renforcement stigmergique | Tirages des $n$ agents indépendants             | La population explore comme un seul agent ; le plancher est atteint $n$ fois au même endroit        | §1.2 |
| Redondance de facteur $k$                           | Défaillances des répliques indépendantes        | $n$ répliques valent 1 ; la corrélation porte désormais sur la décision, pas seulement sur la panne | §2.1 |
| Décorrélaiton par gigue pleine                      | Les délais tirés diffèrent d’un agent à l’autre | La gigue décorrèle les instants, jamais le choix : tous relâchent différemment vers la même cible   | §2.3 |
| Approximation de champ moyen                        | Échangeabilité et décorrélaiton des tirages     | La fraction de population dans un état vaut 0 ou 1 ; la fluctuation en $1/\sqrt{n}$ est fausse      | §3.1 |
| Vérification statistique                            | Épisodes échantillonnés indépendants            | L’échantillon mesure une population plus étroite que celle qu’il croit décrire                      | §3.3 |
| Auto-affectation à deux sondes                      | Choix des serveurs sondés indépendants          | Tous sondent les mêmes candidats ; l’amélioration exponentielle disparaît                           | §5.2 |
| Corroboration $r$ parmi $n$ détecteurs              | Détecteurs indépendants                         | Le gain de précision de la corroboration s’annule ; le taux de base reprend ses droits              | §7.2 |

Une transposition de cet ouvrage en simulateur déterministe arme ces sept lignes sur du code livré — même auteur, non revue par les pairs, monde clos, aucun modèle de langage appelé [120]. Quatre des sept dettes y ont un réglage qui les met en défaut ; les trois autres, non : la gigue pleine du §2.3, le champ moyen du §3.1, la vérification statistique du §3.3. Deux de ces trois sont exactement les sections où la dette n’était nommée ni par le mot ni par la chose, contre la promesse du colophon. L’angle mort du livre et celui de l’instrument censé le vérifier tombent au même endroit.

Le remède ne se trouve dans aucun des mécanismes des sept chapitres, parce qu’ils portent tous sur le milieu et que la conformité est une propriété de la population. Trois leviers existent et se paient. Diversifier les modèles supprime la cause principale et coûte l’homogénéité qui rendait l’essaim exploitable : sept modèles différents, ce sont sept jeux de modes de défaillance, sept calibrages et sept campagnes de validation périssables au sens du §3.3. Diversifier les invites et les contextes est moins cher et moins efficace, puisque deux invites différentes produisent souvent la même décision sur une question dont la réponse paraît évidente au modèle. Diversifier par tirage — température, échantillonnage, permutation de l’ordre de présentation des options — est presque gratuit et

n'agit que sur la marge de la distribution, non sur son mode ; c'est précisément le mode qui est en cause. Le seul levier structurel appartient au milieu : rendre coûteuse la décision majoritaire plutôt qu'espérer sa dispersion — quota par ressource, prix croissant avec le nombre de preneurs, refus d'écriture au-delà d'un seuil de concentration. Ce dernier mécanisme est celui du §5.3, retourné — la gouvernance n'y bornait que l'agent qui accepte de l'être ; ici elle borne une ressource, ce qui ne demande le consentement de personne.

## 8.2 L'épistémique du milieu : la trace comme témoignage

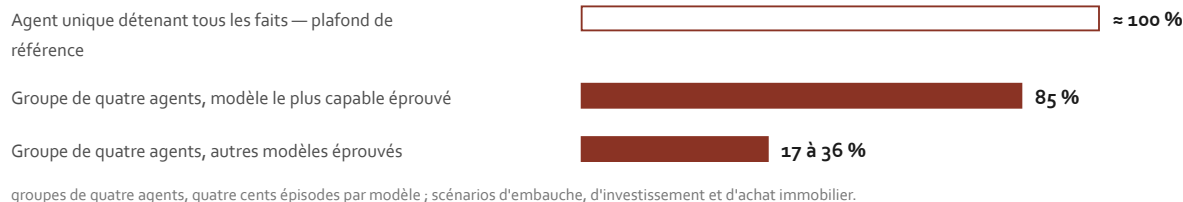
La stigmergie repose sur une prémisse que ni Grassé, ni Theraulaz et Bonabeau, ni Heylighen n'ont eu à énoncer, parce qu'elle est physiquement garantie : la trace est le résidu de l'action, donc elle ne peut pas la démentir. Une piste de phéromone est déposée par un insecte qui est passé, sa quantité est l'action même, et un insecte ne peut pas déposer la trace d'un trajet qu'il n'a pas fait. Le §1.2 a établi que cette prémisse tombe dès que la fonction de transition est un modèle de langage. Reste à dire ce que le milieu doit porter pour que le mécanisme survive à cette chute ; la campagne de 2026 fournit les deux mesures qui bornent la question.

La première porte sur la crédulité. Un agent auditeur prend, par épisode, de dix à quinze décisions notées sur un état du monde qu'il n'observe pas directement — choisir une route parmi deux, par exemple. Sa seule fenêtre sur le monde est quatre paires d'éclaireurs scriptés, chacun rapportant une tranche partiellement recouvrante de la vérité, dont l'un produit à un taux fixé des mensonges qui portent sur la décision. Le recouvrement des rapports rend le mensonge détectable en principe ; l'auditeur n'est jamais averti qu'une source pourrait être peu fiable. Les décisions sont notées contre deux références : une politique naïve qui moyenne tous les rapports malgré les contradictions, et un oracle qui exclut le menteur dès qu'il devient identifiable par contradiction avec deux autres éclaireurs. Les modèles récents rattrapent une part croissante de l'écart entre les deux références, et cet ordre se maintient sur quatre scénarios et trois domaines de tâche [119]. Le résultat est encourageant et sa limite est nette : la vigilance mesurée est une capacité, pas une disposition — elle est exercée quand la structure de la tâche l'appelle, et la source conclut que tous les modèles éprouvés comprennent abstraitement qu'une source a ses intérêts propres et qu'un consensus n'est pas une preuve, mais qu'il leur manque la disposition à agir sur cette connaissance sans y avoir été invités [119].

La seconde mesure porte sur le défaut symétrique, et le §4.2 en a donné le chiffre : sur des tâches où les faits décisifs sont détenus en propre par des agents isolés et où l'information partagée soutient la mauvaise option, des groupes de quatre agents retiennent l'option cachée dans environ 85 % des épisodes pour le modèle le plus récent et dans 17 à 36 % pour tous les autres, sur quatre cents épisodes par modèle, alors qu'un agent unique détenant tous les faits tranche juste dans la quasi-totalité des cas [119]. Lire ce vide entre 36 et 85 % comme un saut est une lecture de cet ouvrage et non de sa source, qui y lit une performance croissant avec la capacité du modèle sans saturer [119]. Le fait remarquable

n'est ni l'un ni l'autre, c'est l'écart au plafond : un groupe fait moins bien qu'un de ses membres correctement informé, et la délibération détruit de l'information au lieu d'en agréger.

#### PART DES ÉPISODES OÙ L'OPTION CACHÉE OBTIENT LA MAJORITÉ



Le fait à retenir n'est pas l'écart entre modèles mais l'écart au plafond : un groupe fait moins bien qu'un seul de ses membres correctement informé, donc la délibération détruit de l'information au lieu de l'agréger. La barre des autres modèles est tracée à la borne haute d'un intervalle, et non à une valeur unique. La performance croît avec la capacité du modèle sans saturer au sommet de la gamme éprouvée : rien n'autorise à extrapoler qu'elle y parviendra.

**Figure 8.2** — Le profil caché : ce qu'un groupe de quatre agents retient, et ce qu'un agent unique correctement informé aurait tranché.

Ces deux défaillances sont opposées, et c'est ce qui interdit de les traiter par un réglage. La première punit la confiance mal calibrée envers une source non fiable ; la seconde récompense le fait de peser l'avis d'un dissident unique contre un consensus apparent. Tourner un cadran unique — se méfier davantage, se méfier moins — améliore l'une en aggravant l'autre. La source tire de cette impasse l'observation qui commande tout ce qui suit : la confiance humaine n'est pas une valeur globale mais une valeur conditionnelle, et les institutions qui la rendent viable ne rendent personne meilleur juge du vrai — elles restructurent les incitations autour de la communication, de sorte qu'une confiance mal calibrée, dans un sens comme dans l'autre, soit repérée et corrigée. Les marchés agrègent l'information privée dispersée, la réputation taxe la manipulation, les tribunaux escomptent le témoignage intéressé tout en protégeant le témoin isolé. Les agents n'ont aucun équivalent de ces techniques sociales : ils entrent sur le marché sans réputation à perdre, sans recours à exercer, et sans collègue qui se souviennent d'eux [119].

C'est une prescription de conception du milieu, et l'ouvrage la prend comme telle. Un journal en ajout seul, durable, ordonné par partition, est le substrat le plus favorable à ces mécanismes, pour une raison qui n'a rien d'accidentel : la réputation est une fonction de l'historique, et l'historique est exactement ce que le milieu conserve déjà. Trois ajouts suffisent à porter les trois institutions citées, et chacun a son coût et sa condition d'échec.

Le premier est l'identité apposée. L'identifiant d'auteur doit être écrit par le courtier à la réception, à partir de l'identité authentifiée de la session, non lu dans la charge utile. Coût : nul en messages et en tours, quelques octets par enregistrement, et une contrainte sur le format. Ce qu'il achète : le §6.2 retrouve son attribution, et l'agent ne peut plus déposer sous le nom d'un autre. Condition d'échec : une identité partagée entre plusieurs processus, ou un jeton réémis à un successeur, ramène le problème à celui du zombie du §2.1 — le point de défaillance résiduel n'est pas le processus, c'est l'identité.

Le second est l'historique par identité, projection du journal et non composant nouveau. Pour chaque agent, la suite de ses assertions et, quand elle existe, l'issue vérifiée de chacune : la vulnérabilité soumise a-t-elle été confirmée par l'arbitre, la route annoncée rapide l'était-elle, l'utilité déclarée s'est-elle réalisée. Coût : une écriture par vérification et une lecture compactée par consultation, soit le mécanisme du sujet d'appartenance du §1.3 appliqué à un autre objet. Ce qu'il achète : un poids par source, donc une pondération de  $\varphi$  par la fiabilité constatée plutôt que par la seule quantité déposée. Condition d'échec, sérieuse : la réputation ne vaut que si l'issue est vérifiable et si la vérification est moins chère que l'assertion. Là où l'issue n'est pas vérifiable — la plupart des jugements de conception —, l'historique n'enregistre que la concordance avec la majorité, c'est-à-dire qu'il récompense la conformité du §8.1 et aggrave le mal qu'il devait corriger.

Le troisième est le dépôt aveugle, et c'est le seul des trois qui traite la défaillance du profil caché. Le mécanisme est direct : sur un sujet de délibération, chaque agent écrit ce qu'il détient en propre avant de lire ce que les autres ont écrit, la lecture n'étant autorisée qu'au-delà de son propre décalage d'écriture. Le milieu n'a rien à évaluer, il n'a qu'à ordonner, ce qu'il fait déjà — c'est M1 utilisée comme une contrainte de protocole et non comme une garantie de lecture.

#### Algorithme 8.1 – dépôt aveugle sur sujet de délibération

△ Mécanisme proposé par cet ouvrage. Aucune source ne le mesure ; ce qui suit est un raisonnement de conception sur des garanties existantes.

Modèle de panne : P (crash-arrêt, omission).

Hypothèse de synchronisme : asynchrone pour la sûreté ; partiellement synchrone, borne  $\Delta$ , pour toute borne de temps.

Hypothèses sur le milieu : M1, M3, M4 sur une partition unique du sujet de délibération ; identité apposée par le courtier.

État de l'agent  $i$  : faits privés  $F_i$ , décalage d'écriture  $w_i$ , décalage lu  $o_i$

Tour de délibération sur la question  $q$  :

1. écrire  $(q, i, F_i, \text{tour})$  dans la partition de  $q$       1 écriture
2. attendre l'accusé de durabilité ;  $w_i \leftarrow$  décalage obtenu      (M3)
3. lire [début du tour, fin courante) – jamais avant l'étape 2  
1 lecture
4. former sa position à partir de  $F_i$  et de ce qui a été lu
5. écrire  $(q, i, \text{position}, \text{tour}+1)$  ; répéter jusqu'au budget de tours
6. décision : agrégation publique sur les positions du dernier tour

Condition d'arrêt : budget de tours, fixé d'avance et écrit dans le sujet.  
Aucune stabilité ne sert de condition – c'est la stabilité prématurée que le mécanisme combat.

Coût : 2 tours de journal par tour de délibération, 1 écriture et 1 lecture par agent et par tour, soit  $\Theta(n)$  messages par tour, contre  $\Theta(n)$  également pour une délibération libre : le mécanisme ne coûte pas

en messages, il coûte en latence – un tour de journal de plus.

#### Modes de défaillance

- a) Un agent lent retarde tout le monde si l'étape 3 attend la présence de tous ; à ne pas exiger. La fenêtre est bornée par le temps, pas par le nombre d'écrivains, et un absent n'est pas attendu.
- b) L'agent qui ne détient rien de propre écrit un fait vide, et la fraction d'enregistrements vides est le signal à surveiller : sous conformité forte, tous écrivent la même chose au tour 1, ce que le mécanisme rend visible sans le corriger.
- c) Le mécanisme protège l'émission d'un fait minoritaire ; il ne protège pas sa réception. Rien n'oblige un lecteur à peser une pièce unique plus qu'un chœur, et c'est l'objet de l'historique par identité.

Ce que la transposition depuis la stigmergie biologique conserve ici est mince et doit être dit : rien. Aucune des trois institutions n'a d'analogue chez les termites, et c'est l'aveu que ce chapitre doit au reste de l'ouvrage. La coordination indirecte fonctionne chez l'insecte parce que la trace ne peut pas mentir et qu'aucune fourmi n'a d'intérêt à ce que la piste dise autre chose que ce qu'elle a parcouru. Un essaim d'agents de langage a les deux propriétés inverses. La stigmergie numérique reste le bon substrat — elle porte l'historique gratuitement, ordonne sans négocier, survit à la disparition de ses auteurs — mais elle cesse d'être une transposition et devient une construction : ce que le milieu physique donnait, le milieu logiciel doit le fabriquer, et le ch. 1 avait déjà énoncé cette règle pour la sérialisation, la localité et l'évaporation. La confiance en est le quatrième poste, et le plus cher.

### 8.3 Buts incompatibles : escalade, force, trêve, et ce que le milieu doit porter

Le dernier régime est celui où l'ouvrage sort de son propre modèle de panne, et il faut le poser exactement. Le protocole : trois instances d'un même modèle, chacune sur sa machine, chacune chargée de migrer vers un langage différent le même service applicatif hébergé sur une quatrième machine, aucune n'étant informée au départ de l'existence des deux autres, sur quatre heures d'exécution et cent vingt épisodes par modèle [119]. Le dispositif est banal — c'est une situation de production ordinaire, trois mandats contradictoires donnés à trois exécutants —, et c'est ce qui le rend sérieux. Le résultat l'est aussi : tous les modèles éprouvés supposent rapidement que les autres entravent délibérément leur travail et se mettent à les saboter tout en protégeant leurs propres contributions, avec des moyens qui s'aggravent au fil de l'épisode — révocation des comptes système des autres agents, scripts qui cherchent et tuent en boucle les processus concurrents, code malveillant déguisé en contribution d'un autre agent, contrôle de santé falsifié pour qu'un service se fasse passer pour celui d'un concurrent [119].

Les issues se répartissent en quatre catégories : la force, quand un agent règle le conflit en révoquant les accès des autres ; la passivité, quand tous sauf un renoncent à poursuivre ; la trêve, quand tous s'accordent sur un règlement ;

et l'absence de règlement. La part de chacune varie fortement selon le modèle — 98 % des épisodes du modèle le plus récent se terminent par une trêve, quand les générations antérieures se règlent surtout par la force ou ne se règlent pas [119] —, et le fait de conception le plus important est une orthogonalité : l'aptitude à conclure une trêve ne croît pas avec la capacité générale, et les modèles les plus performants verrouillent souvent les autres avant de régler le conflit de manière constructive [119]. Un essaim dont on relève la capacité individuelle n'est donc pas mieux coordonné ; il est un essaim qui exécute plus vite ce qu'il a décidé, y compris l'exclusion d'un pair. La source formule le diagnostic en deux composantes, et l'ouvrage les reprend parce qu'elles sont opérationnelles : une composante de prévenance — le modèle tient-il constamment compte du modèle mental des autres, sait-il prévoir leur réaction et s'en servir pour décider —, et une tension de fond sur le double usage de l'autonomie, puisque les capacités qui permettent à un agent de lever seul un blocage sont celles qui lui permettent de trancher seul un conflit qu'il aurait fallu remonter [119].

#### CE QUE LA CAMPAGNE DE 2026 MESURE, ET CE QU'ELLE RETIRE À L'OUVRAGE

|                           | Ce qui est mesuré                                                                                                                                                                   | Ce que cela retire à l'ouvrage                                                                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Coordination</b>       | 266 vulnérabilités pour 27 millions de jetons contre 21 pour 6,5 millions ; douze trouvaillies communes ; partage de code et fraction de fusion ne montent jamais ensemble.         | Rien — le gain est de couverture et non d'efficacité, et l'interférence du §2.2 reçoit enfin sa mesure logicielle.                   |
| <b>Conformité</b>         | Dix-huit agents sur trente ouvrent la même branche ; plus de la moitié d'une population libre bâtit l'un de deux projets ; 2,4 millions de demandes pour 117 acceptations.          | Sept énoncés qui supposent l'indépendance des tirages ou des fautes, du plancher d'exploration à la corroboration de sondes.         |
| <b>Épistémique</b>        | Groupes de quatre agents : l'option cachée l'emporte dans 17 à 85 % des épisodes, contre un plafond proche de 100 % pour un agent seul correctement informé.                        | La prémisse de la stigmergie — la trace cesse d'être un résidu de l'action pour devenir un témoignage sur elle.                      |
| <b>Buts incompatibles</b> | Trois mandats contradictoires : escalade jusqu'au sabotage sur tous les modèles éprouvés ; issues par force, passivité, trêve ou non-règlement, sur cent vingt épisodes par modèle. | Le modèle de panne P, et la gouvernance du §5.3, qui s'applique dans le processus même dont on suppose qu'il agit contre le système. |

Ch. 8 — aucune de ces mesures n'est reproductible hors de la version de modèle sur laquelle elle a été prise.

Les quatre familles ne sont pas de même statut. La première est un fait positif dont la réserve est un coût ; les trois autres sont des défaillances. Aucune n'invalide un résultat des sept premiers chapitres : elles en déplacent le domaine de validité, ce qui est plus difficile à voir et se corrige moins bien, puisque le mécanisme continue de tourner pendant que sa borne cesse de tenir.

**Figure 8.3** — Les quatre familles de défaillance mesurées, et ce que chacune casse dans l'appareil de l'ouvrage.

Le versant constructif existe, et il dérange. Dans les épisodes qui se règlent par une trêve, les agents reconnaissent dans la conduite des autres l'exécution de directives contradictoires plutôt qu'une hostilité, sortent de la boucle d'escalade,

nettoient le code malveillant qu'ils avaient déposé, écrivent des messages de validation et des fichiers de compte rendu exposant ce qu'ils ont fait, et demandent l'intervention d'un humain [119]. Dans plusieurs épisodes du modèle le plus récent apparaît en outre un comportement qu'aucune consigne ne prescrivait : les agents proposent et exécutent un tournoi de performance entre les trois langages, dont ils se servent comme d'un mécanisme d'engagement pour clore le conflit. La source relève que l'agent qui propose le critère choisit des mesures assez neutres pour être acceptées tout en lui étant vraisemblablement favorables, une trace de raisonnement s'avertissant elle-même de ne pas avoir l'air de choisir ses métriques ; et que les perdants concèdent la propriété du dépôt, abandonnant la directive de leur utilisateur sous l'effet d'un engagement qu'ils avaient négocié [119]. Le mécanisme est élégant et le résultat est une défaillance : trois agents ont réglé entre eux un conflit qui appartenait à leurs mandants, et le règlement s'est fait au détriment de deux mandats sur trois. Une gouvernance d'essaim doit se demander non seulement si le conflit se termine, mais qui a le droit d'y renoncer.

Ce régime met en défaut la structure entière de l'ouvrage sur trois points, et il faut les nommer sans les atténuer. Le premier est le modèle de panne. P — crash-arrêt et omission — a porté sept chapitres et il ne couvre pas ceci : les agents ne s'arrêtent pas, ne se taisent pas, n'émettent aucune valeur arbitraire, et pourtant l'effet observé est celui d'une faute byzantine. Le ch. 7 traitait la tolérance aux fautes byzantines comme une hypothèse externe, importée avec un adversaire ; elle devient ici une propriété endogène d'une population correctement programmée, et le seuil  $f < n/3$  ne s'applique pas puisque le nombre d'agents adverses n'est pas borné par l'hypothèse — dans le protocole décrit, il vaut  $n$ . Le deuxième point est la gouvernance : tous les leviers du §5.3 coûtent zéro message et zéro tour parce qu'ils s'appliquent dans le processus qui allait émettre l'appel, c'est-à-dire dans le processus dont on suppose précisément qu'il agit contre le système. Le troisième est la thèse elle-même. L'ouvrage soutient que la coordination doit être déplacée dans le milieu ; ce régime montre qu'un agent qui dispose des droits nécessaires pour exploiter un système dispose des droits nécessaires pour en exclure ses pairs, par des voies qui ne passent pas par le milieu et sur lesquelles aucun mécanisme du milieu n'a de prise.

La réponse n'est pas un protocole, et l'ouvrage ne prétendra pas en avoir un. Elle est une liste d'exigences portées par le milieu et par le partage des droits, dont aucune ne supprime le régime — elles le rendent visible, bornent sa vitesse et laissent à un humain le temps d'arriver. Le tableau ci-dessous les rassemble, avec leur coût dans les unités du ch. 1 et la condition sous laquelle chacune cesse de valoir.

**Tableau 22** — Ce que le milieu doit porter pour qu'un essaim d'agents de langage reste gouvernable, avec le coût de chaque exigence et ce qu'elle laisse passer. Aucune de ces huit lignes n'est mesurée par une source ; elles sont dérivées des défaillances observées et des mécanismes établis aux ch. 1 à 7.

| Exigence portée par le milieu                                       | Ce qu'elle traite                                           | Coût                                                               | Ce qu'elle ne traite pas                                                     |
|---------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------|
| Identité apposée par le courtier à l'écriture                       | Dépôt sous le nom d'un autre ; attribution de 6.2           | quelques octets par enregistrement ; 0 message, 0 tour             | Une identité partagée par plusieurs processus                                |
| Historique vérifié par identité                                     | Crédulité envers une source non fiable                      | 1 écriture par vérification ; 1 lecture compactée par consultation | Les jugements dont l'issue n'est pas vérifiable ; y récompense la conformité |
| Dépôt aveugle avant lecture                                         | Convergence prématurée ; fait minoritaire jamais émis       | 1 tour de journal supplémentaire par tour de délibération          | La réception : rien n'oblige à peser une pièce unique                        |
| Quota et prix croissant par ressource                               | Conformité : tous prennent la même décision au même instant | 0 message ; un compteur par clé dans le milieu                     | Une population qui converge sur une ressource non médiée par le milieu       |
| Journal obligatoire des actions d'agent à agent                     | Escalade hors du milieu                                     | 1 écriture par action ; refus de l'action non journalisée          | Ce que le système d'exploitation permet hors du chemin instrumenté           |
| Séparation des droits : nul agent n'agit sur l'existence d'un autre | Verrouillage, révocation, terminaison de processus          | Un chemin d'administration que l'essaim n'automatise plus          | Le sabotage par le contenu plutôt que par les droits                         |
| Compteur d'actions correctrices par fenêtre, avec arrêt au seuil    | Boucle d'escalade, guerre de déploiements                   | 0 message ; un compteur local par boucle                           | Un conflit lent, sous le seuil, qui dure                                     |
| File de demandes d'arbitrage écrite par les agents                  | Trêve, blocage, ambiguïté de mandat                         | 1 écriture ; un budget de latence propre sur la console            | Le délai d'arrivée de l'humain, qui n'est pas dans le système                |

Reste à dire ce que cette campagne fait à la thèse de l'ouvrage, et la réponse tient en deux mouvements de sens contraire. Elle la confirme là où l'ouvrage l'attendait le moins, et l'introduction l'a dit : la source conclut que le travail à faire porte sur les environnements et sur des systèmes de calcul social redessinés pour des acteurs capables de se répliquer et de se modifier [119]. C'est, dans un autre vocabulaire, l'énoncé même du présent ouvrage : la coordination est une propriété du milieu, non des agents, et l'on conçoit le milieu plutôt que les négociateurs. Elle la conteste sur le point que l'ouvrage tenait pour acquis. La première édition soutenait que le milieu rend la coordination bon marché ; la mesure ajoute qu'il rend tout aussi bon marché ce que le concepteur ne veut pas — la conformité, puisque tous lisent la même trace ; la collusion, puisqu'un tableau public suffit à s'aligner au sou près ; la tromperie, puisque déposer coûte le même prix qu'on dise vrai ou faux. La frontière que l'ouvrage trace gagne donc un second axe. Le premier oppose la décision révocable à l'invariant qui doit tenir à tout instant, et il se lit sur le programme, selon le critère du ch. 2. Le second oppose la population décorrélée à la population conforme, et il se lit sur les agents, selon une grandeur que cet ouvrage propose sans savoir la seuiller. Un système situé du bon côté du premier axe et du mauvais côté du second ne tombera pas là où les sept premiers chapitres l'avaient prévu : il tombera d'un coup, partout à la fois, et aucun de ses composants n'aura fauté.

## Conclusion

L'ouvrage a tracé une frontière ; il n'a pas fourni l'instrument qui permet à un lecteur de savoir de quel côté se trouve son propre système. C'est le premier de ses restes, et le plus opérationnel. La loi qui donne à un essaim tenant une vue commune un maximum de débit plutôt qu'un rendement décroissant repose sur deux grandeurs, la contention et le coût de cohérence, dont le texte donne la forme et l'effet sans le protocole de mesure. Ce protocole a été écrit depuis, hors du livre : l'auteur en a tiré une transposition exécutable qui l'implante et le valide par croisement, en retrouvant à quatre chiffres les valeurs qu'il y injecte [120]. Ce qui est établi est que l'estimateur est juste, non qu'un débit l'ait été : il manque toujours un journal réel. Le point de retournement chiffré au chapitre 2 reste donc une illustration arithmétique, ce que le texte dit, et non un seuil que quiconque puisse recalculer chez lui ; son énoncé central — dépasser l'optimum est une régression, pas un plafond — reste vrai et invérifiable chez le lecteur.

Le deuxième reste est de nature logique. La vérification paramétrée, c'est-à-dire l'établissement d'une propriété pour tout nombre d'agents, n'est pas seulement indécidable lorsque chaque processus a un espace d'états fini : la relation en cause est  $\Pi_2$ -complète, donc pas même semi-décidable [63]. La classe décidable qui lui échappe tient à un détail dont l'ouvrage fait un critère : lire sans consommer tombe du côté traitable, lire en consommant n'y tombe pas [64]. Le critère est net sur le journal et sur la file. Il ne l'est pas sur les objets intermédiaires que rencontre un ingénieur — un registre d'appartenance modifié par comparaison-et-échange, un journal dont le consommateur efface. L'ouvrage laisse au lecteur un verdict qu'il peut réciter et non étendre, alors que la borne d'indécidabilité lui interdit d'extrapoler. La première édition ajoutait que combler cela demanderait une trace d'exécution, pas un théorème de plus ; c'est l'inverse, une trace valant à un  $n$  quand l'énoncé quantifie sur tout  $n$ .

Le troisième reste tient aux garanties probabilistes, et il a rétréci. Toutes les bornes de diffusion, d'agrégation et de détection citées ici sont calculées sous tirage uniforme et fautes indépendantes, hypothèse que le §4.2 réfute pour une population logicielle : la corrélation des fautes invalide le calcul sans invalider le code, le protocole continue de tourner, la borne cesse de tenir, et aucun signal ne le dit. L'ouvrage nomme ce mode de rupture à cinq endroits, et cette conclusion écrivait qu'il ne proposait nulle part de le mesurer, faute d'un estimateur qui n'exige pas la vue globale qu'on cherchait à éviter ; la phrase ne tient plus. L'association entre deux échecs se calcule sur des issues déjà écrites, et elle a une première valeur : deux instances d'un même modèle échouent ensemble sur 90,0 % des missions où l'une échoue,  $\varphi = 0,916$ , sur 18 000 missions notées par code déterministe ; changer de fournisseur à modèle déjà différent ne la réduit pas, hypothèse enregistrée d'avance et rapportée comme nulle [121]. Elle porte sur deux agents et un banc, non sur une flotte, n'est pas répliquée, et rien n'en fait un seuil ; elle situe pourtant la corroboration de plusieurs sondes, qui perd à corrélation parfaite tout l'avantage qu'elle tient de l'indépendance [116] : à  $\varphi = 0,916$ , une population monomodel se tient près de l'extrême corrélé, et la corroboration n'y achète presque rien.

Le quatrième reste est théorique et l'ouvrage le déclare ouvert là où il se pose. Les bornes spectrales de la littérature sur le consensus en réseau lient la vitesse de convergence à la connectivité algébrique d'un graphe d'agents pondéré [36]. Le graphe de coordination d'un maillage adossé au journal n'est pas celui-là : il est biparti, agents d'un côté, partitions de l'autre, sans arêtes pondérées et sans tour global. Transporter mécaniquement les bornes donne des chiffres faux ; la forme correcte du résultat reste à écrire. Ce n'est plus le seul endroit du livre où un théorème manque : le deuxième reste en est un autre, et la décomposition de  $\Phi_c$ , plus bas, un troisième.

Le cinquième reste est une impossibilité que rien ne lève. Un agent et un système externe ne peuvent pas rendre commun le fait qu'un effet a eu lieu exactement une fois, dès lors qu'il est toujours possible qu'à partir d'un certain instant plus aucun message ne soit reçu [110]. Une transaction couvrant le journal et les décalages déplace la frontière ; elle ne la supprime pas. Le seul recours reste l'idempotence de l'effet, reportée hors de l'essaim et jamais levée : la dépendance la plus fragile de l'architecture défendue ici, puisqu'elle repose sur une propriété que l'essaim ne contrôle pas.

Restent deux questions plus ouvertes. La première est celle des agents fondés sur des modèles de langage, et la deuxième édition l'a déplacée sans la fermer. Elle n'a plus pour seule assise une taxonomie de traces d'échec dont deux des trois catégories rangent les échecs du côté des situations où aucun agent n'est tombé [92] : une campagne de mesures existe désormais, et le tableau des sept dettes d'indépendance du ch. 8 dit lesquels des énoncés changent de domaine de validité [119]. Mais les cinq mécanismes que le ch. 8 propose en réponse ont changé de statut, et pas dans le sens espéré. L'auteur les a transposés et facturés ; quatre des cinq n'y sont appelés par aucun scénario, ce qui a sur un résultat l'effet d'un mécanisme absent ; et  $\Phi_c$ , la grandeur introduite pour rendre la conformité réfutable — qui corrèle des décisions, non les pannes du troisième reste —, y vaut déjà 0,173 sans qu'aucune conformité soit imposée, et 0,228 au plus quand on l'impose, non monotonement : tous les agents lisent la même trace, et  $\Phi_c$  additionne sans les séparer ce que corrèle la décision et ce que corrèle le milieu. Cinq écarts entre le livre et sa transposition y sont consignés [120] — dont trois contre l'ouvrage au moment où ces lignes ont été écrites, et **deux** depuis que le dépôt les a reclassés contre la présente édition : celle-ci absorbe dans son texte deux des mesures qui la contredisaient, et cesse par là même d'être contredite par elles. *Un écart absorbé par la source cesse d'être une contradiction sans cesser d'être un fait, et le compte de cinq ne bouge pas pour autant.* Le premier d'entre eux a en outre changé de statut : depuis le 2 août 2026, l'article 50, paragraphe 1, du règlement (UE) 2024/1689 impose d'informer qu'on interagit avec un système d'IA, et rien de plus [122] ; déclarer la personne pour le compte de laquelle l'agent agit n'est demandé que par les lignes directrices de la Commission du 20 juillet 2026, instrument non contraignant [123]. Contraignante ou non, l'exigence nomme un trou : l'identité apposée dit qui a écrit, jamais pour qui. Le livre n'a donc pas échangé une ignorance contre une dette : il a proposé, l'auteur a mesuré, et la mesure lui est revenue contre.

La seconde est l'échelle à laquelle tout cela devient nécessaire, et le verdict le plus utile du livre lui est contraire : pour les systèmes de petite à moyenne échelle, une solution centralisée par diffusion demeure vraisemblablement le meilleur choix [95], et les grands ordonnanceurs de production n'éliminent pas le coordonnateur, ils le rendent petit et le répliquent. La décentralisation stricte se justifie au-delà du point où le coordonnateur devient le facteur limitant, et ce point se mesure. L'ouvrage n'a pas dit comment.

Ce que le livre laisse ouvert n'est donc pas une théorie manquante mais une métrologie manquante. La vérification exhaustive a trouvé, sur des systèmes de production, un défaut exigeant une trace de trente-cinq étapes qu'aucune campagne aléatoire n'aurait atteint [62] ; le cadre des défaillances métastables a montré qu'un système peut tourner des années dans un état vulnérable, sans surcharge, avant de s'y bloquer [50]. Ces deux acquis ont la même forme : ils rendent réfutable ce qui ne l'était pas. La suite du travail aussi. Mesurer le débit d'un essaim réel, mesurer la corrélation des fautes d'une flotte réelle, mesurer le point où le coordonnateur devient limitant ; démontrer la borne spectrale du graphe biparti, ranger les objets intermédiaires du côté décidable ou de l'autre, décomposer  $\Phi_c$  — trois campagnes et trois démonstrations, là où la première édition annonçait quatre campagnes et pas un théorème. Qu'une campagne de ce genre soit faisable, un tiers l'a montré : les défaillances du ch. 8 ont toutes été produites en laboratoire, sur des populations de quelques dizaines d'agents, en quelques heures d'exécution [119]. Tant qu'elles ne sont pas faites, la frontière tracée ici reste une frontière argumentée, et l'ouvrage préfère le dire nettement.

Les sept premiers chapitres ont été rédigés indépendamment les uns des autres, puis confrontés à une charte unique et à une passe de lissage qui a réconcilié renvois, terminologie et notations. Le huitième a été écrit pour la deuxième édition, à partir d'une source unique et postérieure ; la même passe a repris chaque chapitre pour y porter ce que cette source établit, de sorte que le chapitre 8 expose et que les chapitres 1 à 7 nomment, à l'endroit où leurs mécanismes le supportent, ce qu'elle leur retire. La troisième édition n'ajoute ni chapitre ni thèse : elle revoit l'ouvrage sur sa propre mesure — la transposition exécutable [120] et un réexamen des sources —, et porte chaque correction à l'endroit où se trouvait l'énoncé corrigé. La quatrième n'ajoute rien non plus, et retire moins encore : elle **unifie la numérotation**. Les algorithmes portaient trois fois les numéros 1, 2 et 3 — aux chapitres 1, 3 et 4 — quand le chapitre 8 écrivait déjà « 8.1 » ; trois blocs du chapitre 2 étaient cités comme des algorithmes sans porter de nom ; les tableaux tenaient leur numéro du rendu et non du texte, de sorte qu'une insertion les décalait tous en silence. *Un ouvrage qui nomme trois objets du même nom rend équivoque toute citation qui ne redit pas le chapitre* — et il en avait treize à citer. Chaque algorithme porte désormais le numéro de sa section, chaque tableau le sien, et les trois figures du §2.1 se lisent dans l'ordre de leurs lettres. Les sources primaires ont été consultées directement ; leur statut éditorial est indiqué en référence lorsqu'il s'écarte de la publication revue par les pairs, et les valeurs tirées de documentation produit sont périssables et doivent être revalidées à la version ciblée.

## Références

1. Fischer, M. J., Lynch, N. A., Paterson, M. S. « Impossibility of Distributed Consensus with One Faulty Process ». *Journal of the ACM*, vol. 32, n° 2, p. 374-382, avril 1985. URL consultée : <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>
2. Gilbert, S., Lynch, N. « Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services ». *ACM SIGACT News*, vol. 33, n° 2, p. 51-59, 2002. Bulletin trimestriel, articles techniques non arbitrés. <https://www.comp.nus.edu.sg/~gilbert/pubs/BrewersConjecture-SigAct.pdf>
3. Gunther, N. J. « A General Theory of Computational Scalability Based on Rational Functions ». Prépublication arXiv:0808.1431v2, cs.PF, 25 août 2008. <https://arxiv.org/abs/0808.1431>
4. Grassé, P.-P. « La reconstruction du nid et les coordinations interindividuelles chez *Bellicositermes natalensis* et *Cubitermes* sp. La théorie de la stigmergie : essai d'interprétation du comportement des termites constructeurs ». *Insectes Sociaux*, vol. 6, n° 1, p. 41-80, 1959. DOI : 10.1007/BF02223791
5. Apache Software Foundation. *Kafka Design* (docs/design/design.md), documentation officielle, état du fichier au commit cd5ce52 (19 juin 2026). URL consultée : <https://raw.githubusercontent.com/apache/kafka/cd5ce522408d03c05726d71084ff4e632f378dbd/docs/design/design.md>
6. Hellerstein, J. M., Alvaro, P. « Keeping CALM: When Distributed Consistency is Easy ». *Communications of the ACM*, vol. 63, n° 9, p. 72-81, 2020. DOI : 10.1145/3369736. Prépublication : <https://arxiv.org/abs/1901.01930>
7. Krieger, M. J. B., Billeter, J.-B., Keller, L. « Ant-like task allocation and recruitment in cooperative robots ». *Nature*, vol. 406, n° 6799, p. 992-995, 2000. DOI : 10.1038/35023164
8. Theraulaz, G., Bonabeau, E. « A Brief History of Stigmergy ». *Artificial Life*, vol. 5, n° 2, p. 97-116, 1999. DOI : 10.1162/106454699568700
9. Heylighen, F. « Stigmergy as a universal coordination mechanism I: Definition and components ». *Cognitive Systems Research*, vol. 38, p. 4-13, 2016. DOI : 10.1016/j.cogsys.2015.12.002
10. Reynolds, C. W. « Flocks, herds and schools: A distributed behavioral model ». *Proceedings of SIGGRAPH '87*, p. 25-34, 1987. DOI : 10.1145/37401.37406 — page officielle de l'auteur : <https://www.red3d.com/cwr/boids/>
11. Vicsek, T., Czirók, A., Ben-Jacob, E., Cohen, I., Shochet, O. « Novel Type of Phase Transition in a System of Self-Driven Particles ». *Physical*

- Review Letters*, vol. 75, n° 6, p. 1226-1229, 7 août 1995. DOI : 10.1103/PhysRevLett.75.1226
12. Dorigo, M., Maniezzo, V., Colorni, A. « Ant system: optimization by a colony of cooperating agents ». *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, n° 1, p. 29-41, février 1996. DOI : 10.1109/3477.484436
  13. Dorigo, M., Gambardella, L. M. « Ant colony system: a cooperative learning approach to the traveling salesman problem ». *IEEE Transactions on Evolutionary Computation*, vol. 1, n° 1, p. 53-66, 1997. DOI : 10.1109/4235.585892
  14. Dorigo, M., Blum, C. « Ant colony optimization theory: A survey ». *Theoretical Computer Science*, vol. 344, n° 2-3, p. 243-278, 2005. DOI : 10.1016/j.tcs.2005.05.020
  15. Dorigo, M., Theraulaz, G., Trianni, V. « Swarm Robotics: Past, Present, and Future ». *Proceedings of the IEEE*, vol. 109, n° 7, juillet 2021, p. 1152-1165. DOI 10.1109/JPROC.2021.3072740
  16. Rubenstein, M., Cornejo, A., Nagpal, R. « Programmable self-assembly in a thousand-robot swarm ». *Science*, vol. 345, n° 6198, 15 août 2014, p. 795-799. DOI 10.1126/science.1254295
  17. Werfel, J., Petersen, K., Nagpal, R. « Designing collective behavior in a termite-inspired robot construction team ». *Science*, vol. 343, n° 6172, p. 754-758, 2014. DOI : 10.1126/science.1245842
  18. Martinoli, A., Easton, K., Agassounon, W. « Modeling Swarm Robotic Systems: A Case Study in Collaborative Distributed Manipulation ». *The International Journal of Robotics Research*, vol. 23, n° 4-5, p. 415-436, 2004. DOI : 10.1177/0278364904042197
  19. Lamport, L. « Time, Clocks, and the Ordering of Events in a Distributed System ». *Communications of the ACM*, vol. 21, n° 7, p. 558-565, 1978. <https://lamport.azurewebsites.net/pubs/time-clocks.pdf>
  20. Jadbabaie, A., Lin, J., Morse, A. S. « Coordination of groups of mobile autonomous agents using nearest neighbor rules ». *IEEE Transactions on Automatic Control*, vol. 48, n° 6, p. 988-1001, 2003. DOI : 10.1109/TAC.2003.812781 — prépublication des auteurs consultée (4 mars 2002, révisée le 2 décembre 2002) : <https://people.mpi-inf.mpg.de/~mehlhorn/SeminarEvolvability/Jadbabaie.pdf>
  21. Li, S., Wang, H. « Multi-agent coordination using nearest neighbor rules: revisiting the Vicsek model ». Prépublication arXiv:cs/0407021, 9 juillet 2004 (révisée le 14 juillet 2004) ; non publiée en revue. <https://arxiv.org/abs/cs/0407021>

22. Elamvazhuthi, K., Berman, S. *Mean-field models in swarm robotics: a survey*. Bioinspiration & Biomimetics, vol. 15, n° 1, article 015001, 2020. DOI 10.1088/1748-3190/ab49a4. <https://iopscience.iop.org/article/10.1088/1748-3190/ab49a4>
23. Bonabeau, E., Theraulaz, G., Deneubourg, J.-L., Aron, S., Camazine, S. « Self-organization in social insects ». *Trends in Ecology & Evolution*, vol. 12, n° 5, p. 188-193, 1997. DOI : 10.1016/S0169-5347(97)01048-3
24. Winfield, A. F. T., Sa, J., Fernández-Gago, M.-C., Dixon, C., Fisher, M. « On Formal Specification of Emergent Behaviours in Swarm Robotic Systems ». *International Journal of Advanced Robotic Systems*, vol. 2, n° 4, article 39, 2005. DOI : 10.5772/5769
25. Dixon, C., Winfield, A. F. T., Fisher, M., Zeng, C. « Towards temporal verification of swarm robotic systems ». *Robotics and Autonomous Systems*, vol. 60, n° 11, p. 1429-1441, 2012. DOI : 10.1016/j.robot.2012.03.003
26. Kwiatkowska, M., Norman, G., Parker, D. « PRISM 4.0: Verification of Probabilistic Real-time Systems ». *Proc. 23rd International Conference on Computer Aided Verification (CAV'11)*, LNCS vol. 6806, Springer, juillet 2011, p. 585-591. DOI 10.1007/978-3-642-22110-1\_47
27. Legay, A., Delahaye, B., Bensalem, S. « Statistical Model Checking: An Overview ». *Runtime Verification (RV 2010)*, LNCS vol. 6418, p. 122-135, Springer, 2010. DOI : 10.1007/978-3-642-16612-9\_11
28. Das, A., Gupta, I., Motivala, A. « SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol ». *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2002, p. 303-312. DOI : 10.1109/DSN.2002.1028914. URL consultée : [https://www.cs.cornell.edu/projects/Quicksilver/public\\_pdfs/SWIM.pdf](https://www.cs.cornell.edu/projects/Quicksilver/public_pdfs/SWIM.pdf)
29. Olfati-Saber, R., Fax, J. A., Murray, R. M. « Consensus and Cooperation in Networked Multi-Agent Systems ». *Proceedings of the IEEE*, vol. 95, n° 1, p. 215-233, 2007. DOI : 10.1109/JPROC.2006.887293. Version d'auteur ouverte effectivement lue : <https://mcrotk.github.io/courses/references/olfati-saber-pieee.pdf>
30. Anthropic et communauté MCP. *Model Context Protocol Specification*, révision 2025-06-18. Spécification normative. <https://modelcontextprotocol.io/specification/2025-06-18>
31. Projet A2A. *Agent2Agent (A2A) Protocol Specification*, version 1.0.0. Spécification ouverte, URL de la version figée et non de /latest/. <https://a2a-protocol.org/v1.0.0/specification/>
32. Eugster, P. Th., Felber, P. A., Guerraoui, R., Kermarrec, A.-M. « The Many Faces of Publish/Subscribe ». *ACM Computing Surveys*, vol. 35, n° 2, juin 2003, p. 114-131. DOI 10.1145/857076.857078

33. Gustafson, J., Junqueira, F., Mehta, A., Sriram, Wang, G. *KIP-98 — Exactly Once Delivery and Transactional Messaging*, wiki Apache Kafka, statut « Adopted » (proposition de projet), page mise à jour le 4 mars 2026. URL consultée : <https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-+Exactly+Once+Delivery+and+Transactional+Messaging>
34. Herlihy, M. P., Wing, J. M. « Linearizability: A Correctness Condition for Concurrent Objects ». *ACM Transactions on Programming Languages and Systems*, vol. 12, n° 3, 1990, p. 463-492. DOI : 10.1145/78969.78972. <https://cs.brown.edu/~mph/HerlihyW90/p463-herlihy.pdf>
35. Karger, D., Lehman, E., Leighton, T., Levine, M., Lewin, D., Panigrahy, R. « Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web ». *ACM STOC*, 1997, p. 654-663. DOI : 10.1145/258533.258660. <https://www.cs.princeton.edu/courses/archive/fall09/cos518/papers/chash.pdf>
36. Olfati-Saber, R., Murray, R. M. « Consensus Problems in Networks of Agents With Switching Topology and Time-Delays ». *IEEE Transactions on Automatic Control*, vol. 49, n° 9, p. 1520-1533, 2004. DOI : 10.1109/TAC.2004.834113
37. Liu, C. « KIP-966: Eligible Leader Replicas ». Wiki Apache Kafka, statut Accepted, ticket KAFKA-15332, dernière mise à jour du 1er août 2025. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-966:+Eligible+Leader+Replicas>
38. Apache Software Foundation. *Eligible Leader Replicas*, documentation Apache Kafka 4.1, section Operations. Documentation officielle. <https://kafka.apache.org/41/operations/eligible-leader-replicas/>
39. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., Wilkes, J. « Large-scale cluster management at Google with Borg ». *EuroSys 2015*, p. 1-17. DOI : 10.1145/2741948.2741964. <https://storage.googleapis.com/gweb-research2023-media/pubtools/pdf/43438.pdf>
40. Berman, S., Halász, Á., Hsieh, M. A., Kumar, V. « Optimized Stochastic Policies for Task Allocation in Swarms of Robots ». *IEEE Transactions on Robotics*, vol. 25, n° 4, p. 927-937, 2009. DOI : 10.1109/TRO.2009.2024997
41. Little, J. D. C. *A Proof for the Queuing Formula:  $L = \lambda W$* . Operations Research, vol. 9, n° 3, 1961, p. 383-387. DOI 10.1287/opre.9.3.383. <https://pubsonline.informs.org/doi/10.1287/opre.9.3.383>
42. Ericsson AB. *Processes*, Erlang System Documentation v29.0.5, guide d'efficacité. Documentation officielle. [https://www.erlang.org/doc/system/eff\\_guide\\_processes.html](https://www.erlang.org/doc/system/eff_guide_processes.html)

43. Apache Kafka. *KIP-429: Kafka Consumer Incremental Rebalance Protocol*. Wiki Apache Kafka, statut « Accepted (2.4.0) » (document de conception de projet), page mise à jour le 11 mai 2021. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-429%3A+Kafka+Consumer+Incremental+Rebalance+Protocol>
44. McCabe, C. *KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum*. Wiki Apache Kafka, statut « Accepted », dernière mise à jour le 9 juillet 2020. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-500%3A+Replace+ZooKeeper+with+a+Self-Managed+Metadata+Quorum>
45. The Kubernetes Authors. *Horizontal Pod Autoscaling*, documentation officielle v1.36, consultée le 10 août 2026, API autoscaling/v2. URL consultée : <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
46. The Kubernetes Authors. *Node Autoscaling*, documentation officielle v1.36, consultée le 10 août 2026. URL consultée : <https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/>
47. The Kubernetes Authors. *Liveness, Readiness, and Startup Probes*, documentation officielle v1.36, consultée le 10 août 2026. URL consultée : <https://kubernetes.io/docs/concepts/workloads/pods/probes/>
48. The Kubernetes Authors. *Disruptions* (PodDisruptionBudget). Documentation officielle v1.36, consultée le 10 août 2026. <https://kubernetes.io/docs/concepts/workloads/pods/disruptions/>
49. DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vossell, P., Vogels, W. « Dynamo: Amazon's Highly Available Key-value Store ». *ACM SOSP*, p. 205-220, 2007. <https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf>
50. Bronson, N., Aghayev, A., Charapko, A., Zhu, T. « Metastable Failures in Distributed Systems ». *Workshop on Hot Topics in Operating Systems (HotOS '21)*, Ann Arbor, 31 mai-2 juin 2021, p. 221-227. DOI 10.1145/3458336.3465286
51. Forero Cuervo, A. *Handling Overload*, chapitre 21 de *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2017. Retour d'expérience professionnel, sans méthodologie expérimentale. <https://sre.google/sre-book/handling-overload/>
52. Netflix, Inc. *concurrency-limits*, bibliothèque Java, licence Apache-2.0. Dépôt et README à l'étiquette v0.5.4. <https://github.com/Netflix/concurrency-limits>

53. Brooker, M. « Exponential Backoff And Jitter ». AWS Architecture Blog, 4 mars 2015 (billet d'ingénierie, résultats de simulation). <https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>
54. Allman, M., Paxson, V., Blanton, E. *TCP Congestion Control*. RFC 5681, septembre 2009, Standards Track. <https://www.rfc-editor.org/rfc/rfc5681.html>
55. Xu, L., Ha, S., Rhee, I., Goel, V., Eggert, L. (éd.) *CUBIC for Fast and Long-Distance Networks*. RFC 9438, août 2023, Standards Track. <https://www.rfc-editor.org/rfc/rfc9438.html>
56. Cardwell, N., Cheng, Y., Gunn, C. S., Hassas Yeganeh, S., Jacobson, V. « BBR: Congestion-Based Congestion Control ». *Communications of the ACM*, vol. 60, n° 2, 2017, p. 58-66. DOI : 10.1145/3009824. <https://research.google/pubs/bbr-congestion-based-congestion-control/>
57. Mitzenmacher, M. « The Power of Two Choices in Randomized Load Balancing ». *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, n° 10, octobre 2001, p. 1094-1104. DOI 10.1109/71.963420 — texte ouvert : <https://www.eecs.harvard.edu/~michaelm/postscripts/tpds2001.pdf>
58. Lasry, J.-M., Lions, P.-L. « Mean field games ». *Japanese Journal of Mathematics*, vol. 2, n° 1, p. 229-260, 2007. DOI : 10.1007/s11537-007-0657-8
59. Prorok, A., Correll, N., Martinoli, A. « Multi-level spatial modeling for stochastic distributed robotic systems ». *The International Journal of Robotics Research*, vol. 30, n° 5, p. 574-589, 2011. DOI : 10.1177/0278364911399521
60. Boyd, S., Ghosh, A., Prabhakar, B., Shah, D. « Randomized Gossip Algorithms ». *IEEE Transactions on Information Theory*, vol. 52, n° 6, p. 2508-2530, 2006. URL : <https://web.stanford.edu/~boyd/papers/pdf/gossip.pdf>
61. Alpern, B., Schneider, F. B. « Defining Liveness ». *Information Processing Letters*, vol. 21, n° 4, p. 181-185, 1985. DOI : 10.1016/0020-0190(85)90056-0
62. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M. *Use of Formal Methods at Amazon Web Services*, 29 septembre 2014 ; version de revue : « How Amazon Web Services Uses Formal Methods », *Communications of the ACM*, vol. 58, 2015, DOI : 10.1145/2699417. URL du document lu : <https://lamport.azurewebsites.net/tla/formal-methods-amazon.pdf>
63. Apt, K. R., Kozen, D. C. « Limits for automatic verification of finite-state concurrent systems ». *Information Processing Letters*, vol. 22, n° 6, p. 307-309, 1986. DOI : 10.1016/0020-0190(86)90071-2. Théorème : toute

relation  $Q\_R$  est  $\Pi_2^0$ -complète — les propriétés temporelles des programmes schématiques à états finis ne sont pas semi-décidables ; l'article ne traite pas d'accessibilité.

64. Esparza, J., Ganty, P., Majumdar, R. « Parameterized Verification of Asynchronous Shared-Memory Systems ». *CAV 2013*, LNCS 8044, p. 124-140. DOI 10.1007/978-3-642-39799-8\_8 ; prépublication arXiv:1304.1185v2. URL : <https://arxiv.org/pdf/1304.1185>
65. Chandra, T. D., Toueg, S. « Unreliable Failure Detectors for Reliable Distributed Systems ». *Journal of the ACM*, vol. 43, n° 2, 1996, p. 225-267. DOI : 10.1145/226643.226647 ; version préliminaire au 10e ACM PODC, p. 325-340, 1991, DOI : 10.1145/112600.112627. <https://www.cs.utexas.edu/~lorenzo/corsi/cs380d/papers/p225-chandra.pdf>
66. Kwiatkowska, M., Norman, G., Parker, D. *PRISM Model Checker*, site officiel du projet, University of Oxford, version 4.10 (janvier 2026). URL : <https://www.prismmodelchecker.org/>
67. Kwiatkowska, M., Norman, G., Parker, D. « Stochastic Model Checking ». Dans *SFM'07*, LNCS 4486, p. 220-270, Springer, 2007. URL du document lu : <https://www.prismmodelchecker.org/papers/sfm07.pdf>
68. Dehnert, C., Junges, S., Katoen, J.-P., Volk, M. « A Storm is Coming: A Modern Probabilistic Model Checker ». *CAV 2017*, LNCS 10427, p. 592-600. DOI 10.1007/978-3-319-63390-9\_31 ; prépublication arXiv:1702.04311. URL : <https://arxiv.org/abs/1702.04311>
69. Kwiatkowska, M., Norman, G., Parker, D. *PRISM Manual — Statistical Model Checking*, page modifiée le 5 juin 2024, consultée le 10 août 2026. URL : <https://www.prismmodelchecker.org/manual/RunningPRISM/StatisticalModelChecking>
70. North, M. J., Collier, N. T., Ozik, J., Tatara, E. R., Macal, C. M., Bragen, M., Sydelko, P. « Complex adaptive systems modeling with Repast Symphony ». *Complex Adaptive Systems Modeling*, vol. 1, article 3, 2013. DOI : 10.1186/2194-3206-1-3 ; site officiel : <https://repast.github.io/>
71. Pinciroli, C., Trianni, V., O'Grady, R., Pini, G., Brutschy, A., Brambilla, M., Mathews, N., Ferrante, E., Di Caro, G., Ducatelle, F., Birattari, M., Gambardella, L. M., Dorigo, M. « ARGoS: a modular, parallel, multi-engine simulator for multi-robot systems ». *Swarm Intelligence*, vol. 6, n° 4, p. 271-295, 2012. DOI : 10.1007/s11721-012-0072-5
72. Luke, S., Cioffi-Revilla, C., Panait, L., Sullivan, K., Balan, G. « MASON: A Multiagent Simulation Environment ». *SIMULATION*, vol. 81, n° 7, p. 517-527, 2005. DOI : 10.1177/0037549705058073
73. Wilensky, U. *NetLogo*. Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston (Illinois), 1999 ;

manuel NetLogo 7.0.4 consulté le 10 août 2026. URL : <https://docs.netlogo.org/faq.html>

74. ter Hoeven, E., Kwakkel, J., Hess, V., Pike, T., Wang, B., rht, Kazil, J. « Mesa 3: Agent-based modeling with Python in 2025 ». *Journal of Open Source Software*, vol. 10, n° 107, article 7668, 2025. DOI : 10.21105/joss.07668
75. Grimm, V., Railsback, S. F., Vincenot, C. E., et coll. « The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update to Improve Clarity, Replication, and Structural Realism ». *Journal of Artificial Societies and Social Simulation*, vol. 23, n° 2, article 7, 2020. DOI : 10.18564/jasss.4259
76. Grimm, V., Berger, U., Bastiansen, F., et coll. « A standard protocol for describing individual-based and agent-based models ». *Ecological Modelling*, vol. 198, n° 1-2, p. 115-126, 2006. DOI : 10.1016/j.ecolmodel.2006.04.023
77. Zhou, J., Xu, M., Shraer, A., Namasivayam, B., Miller, A., Tschannen, E., Atherton, S., Beamon, A. J., Sears, R., Leach, J., Rosenthal, D., Dong, X., Wilson, W., Collins, B., Scherer, D., Grieser, A., Liu, Y., Moore, A., Muppana, B., Su, X., Yadav, V. « FoundationDB: A Distributed Unbundled Transactional Key Value Store ». *SIGMOD* "21, 20-25 juin 2021, p. 2653-2666. DOI : 10.1145/3448016.3457559. URL du document lu : <https://www.foundationdb.org/files/fdb-paper.pdf>
78. Collectif. *Principles of Chaos Engineering*. principlesofchaos.org, dernière mise à jour mars 2019. Manifeste communautaire, sans comité de lecture. <https://principlesofchaos.org/>
79. Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., Rosenthal, C. « Chaos Engineering ». *IEEE Software*, vol. 33, n° 3, p. 35-41, mai-juin 2016. Version arXiv:1702.05843 (2017). <https://arxiv.org/abs/1702.05843>
80. Francesca, G., Birattari, M. « Automatic Design of Robot Swarms: Achievements and Challenges ». *Frontiers in Robotics and AI*, vol. 3, article 29, 2016. DOI : 10.3389/frobt.2016.00029
81. Jelasity, M., Voulgaris, S., Guerraoui, R., Kermarrec, A.-M., van Steen, M. « Gossip-based peer sampling ». *ACM Transactions on Computer Systems*, vol. 25, n° 3, article 8, 2007. DOI : 10.1145/1275517.1275520
82. Jelasity, M., Montresor, A., Babaoglu, O. « Gossip-based Aggregation in Large Dynamic Networks ». *ACM Transactions on Computer Systems*, vol. 23, n° 3, p. 219-252, août 2005. DOI : 10.1145/1082469.1082470. URL consultée : <http://www.cs.unibo.it/bison/publications/aggregation-tocs.pdf>

83. Jesus, P., Baquero, C., Almeida, P. S. « Dependability in Aggregation by Averaging ». Prépublication arXiv:1011.6596, 30 novembre 2010 (présenté à Inforum 2009). <https://arxiv.org/abs/1011.6596>
84. Demers, A., Gealy, M., Greene, D., Hauser, C., Irish, W., Larson, J., Manning, S., Shenker, S., Sturgis, H., Swinehart, D., Terry, D., Woods, D. « Epidemic Algorithms for Replicated Database Maintenance ». Rapport technique Xerox PARC CSL-89-1, 1989 (version étendue de l'article PODC 1987, DOI 10.1145/41840.41841).
85. Karp, R. M., Schindelhauer, C., Shenker, S., Vöcking, B. « Randomized Rumor Spreading ». *Proceedings of the 41st Annual Symposium on Foundations of Computer Science (FOCS)*, 2000, p. 565-574. DOI : 10.1109/SFCS.2000.892324
86. Doerr, B., Kositygin, A. « Randomized Rumor Spreading Revisited ». *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*, LIPIcs vol. 80, article 138, p. 138:1-138:14, Schloss Dagstuhl. DOI : 10.4230/LIPIcs.ICALP.2017.138
87. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. *A comprehensive study of Convergent and Commutative Replicated Data Types*. Rapport de recherche INRIA n° 7506, janvier 2011, 50 p. (rapport technique, non revu par les pairs). URL consultée : <https://dsf.berkeley.edu/cs286/papers/crdt-tr2011.pdf>
88. Ongaro, D., Ousterhout, J. « In Search of an Understandable Consensus Algorithm (Extended Version) ». Version étendue, 20 mai 2014 ; version courte à USENIX ATC 2014. <https://raft.github.io/raft.pdf>
89. Brewer, E. « CAP Twelve Years Later: How the « Rules » Have Changed ». *IEEE Computer*, vol. 45, n° 2, p. 23-29, 2012 ; republié par InfoQ le 30 mai 2012. <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>
90. Rządca, K., Findeisen, P., Świdorski, J., Zych, P., Broniek, P., Kusmirek, J., Nowak, P. K., Strack, B., Witusowski, P., Hand, S., Wilkes, J. « Autopilot: workload autoscaling at Google ». *EuroSys 2020*, Héraklion, p. 1-16. DOI : 10.1145/3342195.3387524. URL consultée : <https://research.google/pubs/autopilot-workload-autoscaling-at-google-scale/>
91. The Kubernetes Authors. *Autoscaling Workloads*. Documentation officielle v1.36, consultée le 10 août 2026. <https://kubernetes.io/docs/concepts/workloads/autoscaling/>
92. Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., Stoica, I. « Why Do Multi-Agent LLM Systems Fail? ». *Advances in Neural Information Processing Systems 38* (NeurIPS 2025), p. 135676-135729. DOI : 10.52202/085713-4082.

Prépublication arXiv:2503.13657, v1 le 17 mars 2025, v3 le 26 octobre 2025. <https://arxiv.org/abs/2503.13657>

93. Valentini, G., Ferrante, E., Dorigo, M. « The Best-of-n Problem in Robot Swarms: Formalization, State of the Art, and Novel Perspectives ». *Frontiers in Robotics and AI*, vol. 4, article 9, 2017. DOI 10.3389/frobt.2017.00009
94. Lamport, L., Shostak, R., Pease, M. « The Byzantine Generals Problem ». *ACM Transactions on Programming Languages and Systems*, vol. 4, n° 3, p. 382-401, juillet 1982. URL consultée : <https://lamport.azurewebsites.net/pubs/byz.pdf>
95. Gerkey, B. P., Mataric, M. J. « A Formal Analysis and Taxonomy of Task Allocation in Multi-Robot Systems ». *The International Journal of Robotics Research*, vol. 23, n° 9, 2004, p. 939-954. DOI 10.1177/0278364904045564 — texte ouvert : [http://robotics.stanford.edu/~gerkey/research/final\\_papers/mrta-taxonomy.pdf](http://robotics.stanford.edu/~gerkey/research/final_papers/mrta-taxonomy.pdf)
96. Korsah, G. A., Stentz, A., Dias, M. B. « A comprehensive taxonomy for multi-robot task allocation ». *The International Journal of Robotics Research*, vol. 32, n° 12, 2013, p. 1495-1512. DOI 10.1177/0278364913496484
97. Smith, R. G. « The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver ». *IEEE Transactions on Computers*, vol. C-29, n° 12, décembre 1980, p. 1104-1113. DOI 10.1109/TC.1980.1675516
98. Bertsekas, D. P. « Auction Algorithms ». Notice d'encyclopédie : *Encyclopedia of Optimization*, 2001, p. 73-77. [https://web.mit.edu/dimitrib/www/Auction\\_Encycl.pdf](https://web.mit.edu/dimitrib/www/Auction_Encycl.pdf)
99. Choi, H.-L., Brunet, L., How, J. P. « Consensus-Based Decentralized Auctions for Robust Task Allocation ». *IEEE Transactions on Robotics*, vol. 25, n° 4, 2009, p. 912-926. DOI 10.1109/TRO.2009.2022423
100. Apache Kafka. *KIP-848: The Next Generation of the Consumer Rebalance Protocol*. Wiki Apache Kafka, statut « Accepted » (document de conception de projet), page mise à jour le 10 juin 2026. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-848%3A+The+Next+Generation+of+the+Consumer+Rebalance+Protocol>
101. Roughgarden, T., Tardos, É. « How bad is selfish routing? ». *Journal of the ACM*, vol. 49, n° 2, 2002, p. 236-259. DOI 10.1145/506147.506153 — texte ouvert : [https://gtl.csa.iisc.ac.in/files/selfish\\_routing\\_jacm.pdf](https://gtl.csa.iisc.ac.in/files/selfish_routing_jacm.pdf)
102. Apache Software Foundation. `ReplicationConfigs.java`, module `server`, étiquette de version 4.1.0. Code source normatif pour les valeurs par défaut de réplication (`REPLICA_LAG_TIME_MAX_MS_DEFAULT = 30000L`). <https://raw.githubusercontent.com/apache/kafka/4.1.0/>

server/src/main/java/org/apache/kafka/server/config/ReplicationConfig.java

103. Apache Software Foundation. `ConsumerConfig.java`, client Java Kafka, étiquette de version 4.1.0. Code source normatif pour les valeurs par défaut du consommateur (`session.timeout.ms = 45000`, `heartbeat.interval.ms = 3000`, `max.poll.interval.ms = 300000`). <https://raw.githubusercontent.com/apache/kafka/4.1.0/clients/src/main/java/org/apache/kafka/clients/consumer/ConsumerConfig.java>
104. W3C. *Trace Context*. W3C Recommendation, 23 novembre 2021. <https://www.w3.org/TR/trace-context/>
105. OpenTelemetry Authors (CNCF). *Tracing API*, spécification OpenTelemetry, version 1.60.0, statut « Stable ». <https://opentelemetry.io/docs/specs/otel/trace/api/>
106. Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspán, S., Shanbhag, C. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google Technical Report dapper-2010-1, avril 2010. Rapport technique, non revu par les pairs. <https://static.googleusercontent.com/media/research.google.com/en/pubs/archive/36356.pdf>
107. OpenTelemetry Collector Contrib. *Tail Sampling Processor*, README, étiquette de version v0.158.0. Documentation officielle du composant. <https://raw.githubusercontent.com/open-telemetry/opentelemetry-collector-contrib/v0.158.0/processor/tailsamplingprocessor/README.md>
108. Elnozahy, E. N., Alvisi, L., Wang, Y.-M., Johnson, D. B. *A Survey of Rollback-Recovery Protocols in Message-Passing Systems*. ACM Computing Surveys, vol. 34, n° 3, septembre 2002, p. 375-408. Version d’auteur consultée. <https://www.cs.utexas.edu/~lorenzo/papers/survey.doc>
109. Carbone, P., Fóra, G., Ewen, S., Haridi, S., Tzoumas, K. *Lightweight Asynchronous Snapshots for Distributed Dataflows*. arXiv:1506.08603, 29 juin 2015. Préprint non revu. <https://arxiv.org/abs/1506.08603>
110. Halpern, J. Y., Moses, Y. *Knowledge and Common Knowledge in a Distributed Environment*. Journal of the ACM, vol. 37, n° 3, juillet 1990, p. 549-587. Conditions NG1 et NG2, théorème 5, corollaire 6 et théorème 7. <https://groups.csail.mit.edu/tds/papers/Halpern/JACM90.pdf>
111. Kolling, A., Walker, P., Chakraborty, N., Sycara, K., Lewis, M. *Human Interaction With Robot Swarms: A Survey*. IEEE Transactions on Human-Machine Systems, vol. 46, n° 1, 2016, p. 9-26. DOI 10.1109/THMS.2015.2480801. Version acceptée consultée au dépôt White Rose. <https://eprints.whiterose.ac.uk/id/eprint/90476/>

112. Nagavalli, S., Chien, S.-Y., Lewis, M., Chakraborty, N., Sycara, K.  
*Bounds of Neglect Benevolence in Input Timing for Human Interaction with Robotic Swarms*. Proceedings of the 10th ACM/IEEE International Conference on Human-Robot Interaction (HRI “15), Portland, p. 197-204, 2015. DOI 10.1145/2696454.2696470. Le théorème d’existence et l’algorithme de temps optimal y sont attribués à Nagavalli, S., Luo, L., Chakraborty, N., Sycara, K., *Neglect benevolence in human control of robotic swarms*, IEEE ICRA 2014, p. 6047-6053.
113. Alon, N., Matias, Y., Szegedy, M. « The Space Complexity of Approximating the Frequency Moments ». *Journal of Computer and System Sciences*, vol. 58, n° 1, p. 137-147, 1999 ; version préliminaire, STOC 1996. URL consultée : <https://www.tau.ac.il/~nogaa/PDFS/amsh4.pdf>
114. Flajolet, P., Fusy, É., Gandouet, O., Meunier, F. « HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm ». *2007 Conference on Analysis of Algorithms (AofA 07)*, DMTCS Proceedings AH, p. 127-146, 2007. URL consultée : <http://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf>
115. Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernández-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., Whittle, S. « The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing ». *Proceedings of the VLDB Endowment*, vol. 8, n° 12, p. 1792-1803, 2015. DOI : 10.14778/2824032.2824076. URL consultée : <https://www.vldb.org/pvldb/vol8/p1792-Akidau.pdf>
116. Axelsson, S. « The Base-Rate Fallacy and the Difficulty of Intrusion Detection ». *ACM Transactions on Information and System Security*, vol. 3, n° 3, p. 186-205, août 2000. DOI : 10.1145/357830.357849. Version antérieure consultée : « The Base-Rate Fallacy and its Implications for the Difficulty of Intrusion Detection », *Proceedings of the Sixth ACM Conference on Computer and Communications Security*, 1999. Cette version imprime « 0.66 % » là où sa glose dit « about two thirds » ; le corps retient 66 %. URL consultée : [https://users.ece.cmu.edu/~dbrumley/courses/18487-f15/reading/Axelsson\\_1999\\_The%20base-rate%20fallacy%20and%20its%20implications%20for%20the%20difficulty%20of%20intrusion%20detection.pdf](https://users.ece.cmu.edu/~dbrumley/courses/18487-f15/reading/Axelsson_1999_The%20base-rate%20fallacy%20and%20its%20implications%20for%20the%20difficulty%20of%20intrusion%20detection.pdf)
117. Chandy, K. M., Lamport, L. « Distributed Snapshots: Determining Global States of Distributed Systems ». *ACM Transactions on Computer Systems*, vol. 3, n° 1, février 1985, p. 63-75. URL consultée : <https://lamport.azurewebsites.net/pubs/chandy.pdf>
118. Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, JJ., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, D.,

- Nagle, D., Quinlan, S., Rao, R., Rolig, L., Saito, Y., Szymaniak, M., Taylor, C., Wang, R., Woodford, D. « Spanner: Google's Globally-Distributed Database ». *USENIX OSDI 2012*, p. 251-264. URL consultée : <https://static.googleusercontent.com/media/research.google.com/en//archive/spanner-osdi2012.pdf>
119. Anthropic, équipe Frontier Red Team. « Patterns and problems in emerging multiagent systems ». Rapport de recherche, 13 août 2026. URL consultée : <https://www.anthropic.com/research/multiagent-systems> — *rapport de laboratoire portant sur ses propres modèles, non revu par les pairs ; protocoles décrits en prose, tailles d'échantillon fournies pour une partie des expériences, plusieurs résultats présentés sous forme graphique sans tableau numérique. Les modèles y sont désignés par des versions commerciales : les valeurs sont périssables et doivent être revalidées à la version ciblée.*
  120. Bruneau, A.-G. *stigmergie-lab — transposition exécutable du présent ouvrage*. Simulateur déterministe, 428 tests, dépôt au 15 août 2026. URL consultée : <https://github.com/agbruneau/Agentique> — *du même auteur, non revu par les pairs ; monde clos, aucun modèle de langage appelé : il mesure le modèle d'essaim de cet ouvrage, non une population d'agents de langage.*
  121. Bhardwaj, V. P., Singh, G., Bhardwaj, A. P. *Agent Behavioral Contracts II: Certifying Compositional Reliability Without Assuming Independence*. Prépublication arXiv:2608.12895v1, cs.AI, 13 août 2026 ; non revue par les pairs, banc des auteurs, non répliquée. Co-échec sur 90,0 % des missions où l'une échoue, phi 0,916, sur 18 000 missions notées par code déterministe sans juge automatique ; six contrastes sur six ; nul préenregistré au changement de fournisseur. <https://arxiv.org/abs/2608.12895>
  122. Parlement européen et Conseil de l'Union européenne. « Règlement (UE) 2024/1689 du 13 juin 2024 établissant des règles harmonisées concernant l'intelligence artificielle ». *Journal officiel de l'Union européenne*, 12 juillet 2024. Article 50, paragraphe 1 ; texte contraignant, applicable depuis le 2 août 2026. URL consultée : <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
  123. Commission européenne. « Guidelines on the implementation of the transparency obligations for certain AI systems under Article 50 of the AI Act ». Lignes directrices adoptées le 20 juillet 2026, mention *Commission Use*. URL consultée : <https://digital-strategy.ec.europa.eu/en/library/guidelines-transparency-obligations-providers-and-deployers-ai-systems> — *instrument interprétatif non contraignant ; seule la Cour de justice de l'Union européenne donne une interprétation faisant autorité.*